Programowanie Układów FPGA

Wstęp

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

Zakres przedmiotu

- Główne zagadnienia tematyczne
  - Rozwój układów cyfrowych
  - Ogólna charakterystyka układów FPGA
  - Przegląd cech układów FPGA
  - Zagadnienia dotyczące projektowania dla układów cyfrowych
  - Język opisu sprzętu
    - Verilog
  - Języki opisu sprzętu wysokiego poziomu
    - SystemVerilog

Wymagana wiedza

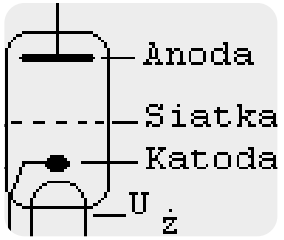
- Wymagana podstawowa wiedza z zakresu układów cyfrowych!
  - Funkcje logiczne, tabele prawdy, bramki logiczne, przerzutniki...
- Literatura uzupełniająca

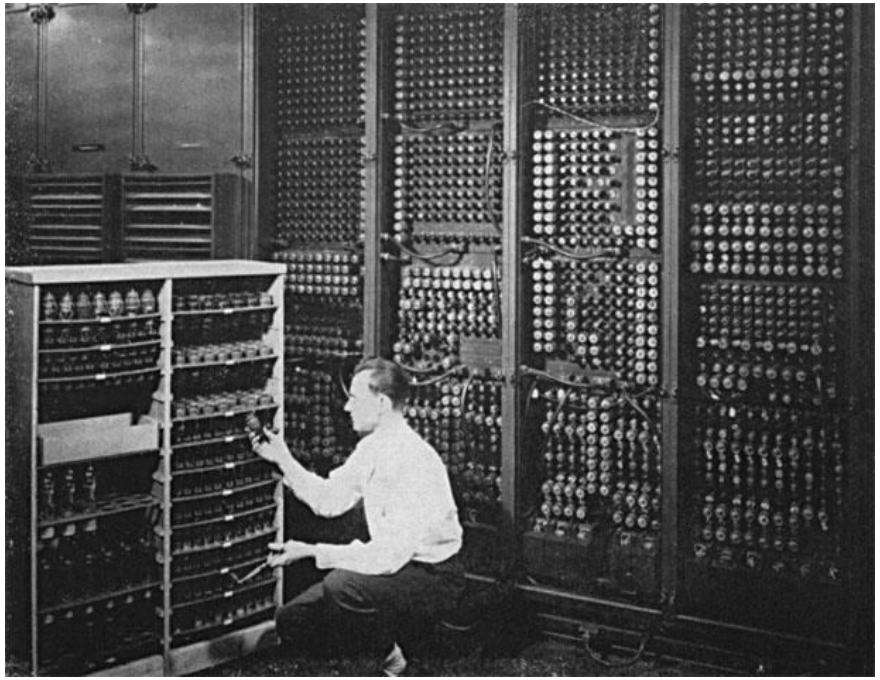
- [1] J. Pasierbiński, P. Zbysiński, „***Układy programowalne w praktyce***"
- [2] J. Pasierbiński, P. Zbysiński, „***Układy programowalne, pierwsze kroki***"
- [3] K. Skahill, „***Projektowanie programowalnych układów logicznych, Język VHDL***"
- [4] Praca zbiorowa pod redakcją J. Kalisza, "Język VHDL w praktyce"
- [5] T. Łuba, M. Markowski, B. Zbierzchowski, „***Kompilatory układów logicznych***"
- [6] P. Pirsach, „***Architectures for Digital Signal Processing***"
- [7] T. Łuba, M. Markowski „***Specjalizowane układy cyfrowe w strukturach PLD i FPGA***"

Rozwój układów cyfrowych

- Technologia lampowa
  - Wczesne lata 50-te
  - Pierwsza technologia użyta do budowy układów cyfrowych
  - Pierwsze komputery
    - ENIAC, EDVAC, EDSAC, BINAC, UNIVAC, WHIRLWIND, E-101, G-15, LGP-30
  - Debugging





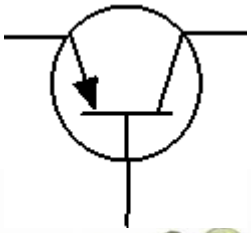


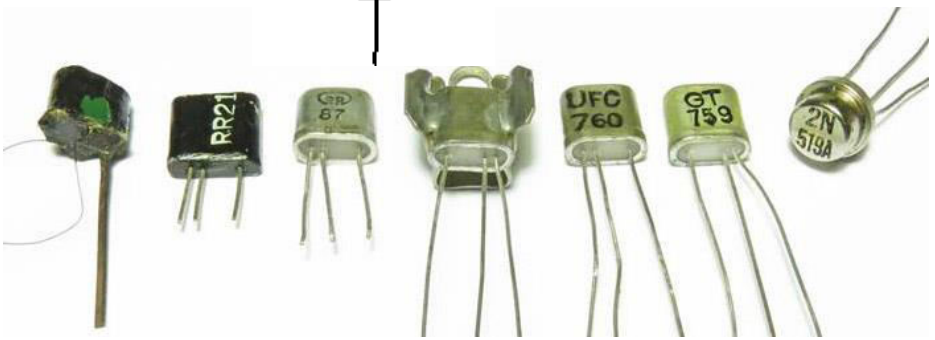


Rozwój układów cyfrowych

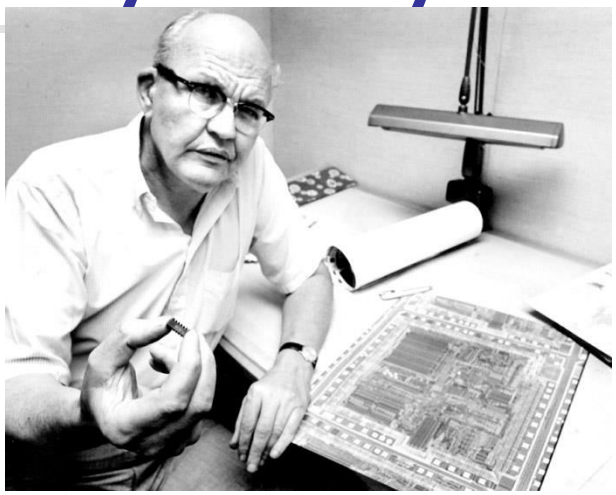
- Tranzystory
  - 1947 - laboratoria Bella
    - John Bardeen, Walter Brattain, William Shockley
  - Podstawa do budowy szybszych, efektywniejszych i oszczędniejszych w pobór energii maszyn liczących
  - Technologia wykorzystywana w latach 50-tych i 60-tych



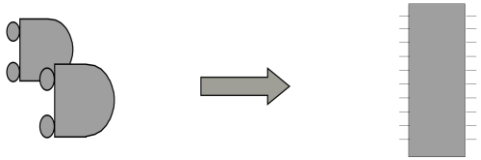




Rozwój układów cyfrowych



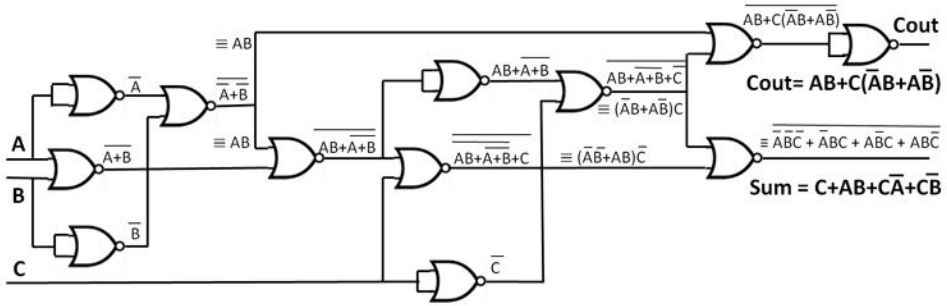
- 1959 - pierwszy układ scalony
  - Pojedyncze tranzystory
  - Texas Instruments
  - Jack Kilby



- 1960-1965 - SSI (Small Scale Integration)
  - Układy o małej skali integracji
    - od kilku do kilkunastu elementów w jednym układzie
- 1965 – Układy MSI (Medium Scale Integration)
  - Układy o średniej skali integracji
  - Upakowanie ~100 bramek/układ
- 1971 – Układy LSI (Large Scale Integration)
  - Układy o dużej skali integracji
  - Przekroczono próg 1000 bramek/układ
  - Pierwsze mikrokontrolery i mikroprocesory

Rozwój układów cyfrowych

- Układy
  - Coraz większe układy (>1000 bramek)
  - Większe prawdopodobieństwo błędów
  - Coraz większe wolumeny produkcyjne
- Projektowanie
  - Rysowanie schematów bramek



  - Ręczna minimalizacja funkcji logicznych
    - Tabele Karnaugh
- Narzędzia CAD
  - Pierwsze komputery osobiste

Rozwój układów cyfrowych

- 1980 – VLSI (Very-Large Scale Integration)
  - Układy o bardzo dużej skali integracji
  - 100 tyś bramek/układ
- W praktyce niemożliwe projektowanie „ręczne”
- Niezbędne nowe metody projektowania i weryfikacji
  - Przed uruchomieniem linii produkcyjnej!



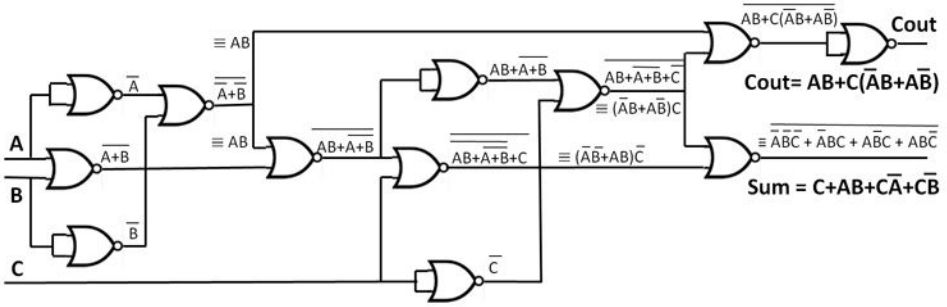
Nowe metody projektowania

- 1. Komputery osobiste
  - Narzędzia symulacyjne
    - Możliwość sprawdzenia wektorów testowych
    - Możliwość obserwacji przebiegów i znalezienia błędów
    - Ograniczone możliwości obliczeniowe
      - Wtedy: Komputer - 1 MHz, Symulowany układ - 100 tyś bramek
      - Teraz: Komputer - 1 GHz, Symulowany układ - 100 mln bramek
    - Symulacja 1 sekunda działania układu
      = kilkanaście godzin pracy symulatora

- 2. Programowalne Układy Cyfrowe

Programowalne Układy Cyfrowe

- Układ cyfrowy o programowalnej strukturze
  - Po wyprodukowaniu jest „czysty”
  - Po zaprogramowaniu realizuje zadaną strukturę bramek
  - Zbliżone działanie do finalnego układu cyfrowego



  - Nie procesor/mikrokontroler
    - Procesor ma stałą strukturę
    - Procesor wykonuje program

```
hello.c
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("hello, world\n");
6      return 0;
7  }
```

Rozwój układów programowalnych

- 1975 - pierwszy układ PLA (Signetics/Philips)
- 1978 - pierwszy układ PAL (Monolitic Memories/AMD)
- 1984 - Monolitic Memories wprowadza GAL 22V10
- 1985 - Xilinx wprowadza pierwszy układ FPGA
- 1990 - pierwsze układy prototyp dzisiejszych układów CPLD
- 1992 - pierwszy układ programowalny w systemie (isp)
- 1995 - pierwsze układy GAL o czasie propagacji 3,5 ns

- PAL, GAL, CPLD, PLED
  - matryce 1 000 – 500 000 elementów
- FPGA (Field Programmable Gate Array)
  - matryce ~ 30 000 000 elementów, częstotliwość pracy ~ 0,5 GHz

Nomenklatura układów scalonych

- IC – Integrated Circuit (układ scalony)
- ASIC – Application Specific Integrated Circuits
  - Układ scalony zaprojektowany do realizacji z góry określonego, specyficznego zadania
- Full Custom
  - Sposób projektowania ASIC
  - Specyfikowany jest rozkład każdego tranzystora i ich połączeń
- Semi Custom
  - Ograniczenia technologicze i projektowe
  - Tańsze i szybsze w realizacji
  - GateArray, StandardCell
- User Programmable Devices – układy programowalne
  - Układ scalony którego działanie określa „użytkownik” (inżynier dostarczający „program”)
  - PLD, CPLD, FPGA

Porównanie układów scalonych

	ASIC *full-custom*	*ASIC semi-custom*	*Układ programowalny*
Debugowanie	**Tylko symulacja**	**Tylko symulacja**	**Symulacja + uruchomienie**
Czas projektowania struktury krzemowej	**długi (nawet do 1 roku)**	**długi (kilka miesięcy)**	**krótki (kilka minut-godzin)**
Kosz pojedynczego układu	**Bardzo duży (nawet 100 000 razy droższy)**	**Bardzo duży (nawet 100 000 razy droższy**	**Tani**
Koszt układu w produkcji wielkoseryjnej	**tani**	**tani**	**duży (ok 1000 razy droższy niż układ full-custom)**
Prawdopodobieństwo niepowodzenia projektu	**„51 %”**	**„49 %”**	**„0 %”**

Porównanie układów scalonych

	ASIC *full-custom*	*ASIC semi-custom*	*Układ programowalny*
Debugowanie	**Tylko symulacja**	**Tylko symulacja**	**Symulacja + uruchomienie**
Czas projektowania struktury krzemowej	**długi (nawet do 1 roku)**	**długi (kilka miesięcy)**	**krótki (kilka minut-godzin)**
Kosz pojedynczego układu	**Bardzo duży (nawet 100 000 razy droższy)**	**Bardzo duży (nawet 100 000 razy droższy**	**Tani**
Koszt układu w produkcji wielkoseryjnej	**tani**	**tani**	**duży (ok 1000 razy droższy niż układ full-custom)**
Prawdopodobieństwo niepowodzenia projektu	**„51 %”**	**„49 %”**	**„0 %”**

Porównanie układów scalonych

	ASIC *full-custom*	*ASIC semi-custom*	*Układ programowalny*
Debugowanie	**Tylko symulacja**	**Tylko symulacja**	**Symulacja + uruchomienie**
Czas projektowania struktury krzemowej	**długi (nawet do 1 roku)**	**długi (kilka miesięcy)**	**krótki (kilka minut-godzin)**
Kosz pojedynczego układu	**Bardzo duży (nawet 100 000 razy droższy)**	**Bardzo duży (nawet 100 000 razy droższy**	**Tani**
Koszt układu w produkcji wielkoseryjnej	**tani**	**tani**	**duży (ok 1000 razy droższy niż układ full-custom)**
Prawdopodobieństwo niepowodzenia projektu	**„51 %”**	**„49 %”**	**„0 %”**

Porównanie układów scalonych

	ASIC *full-custom*	*ASIC semi-custom*	*Układ programowalny*
Debugowanie	**Tylko symulacja**	**Tylko symulacja**	**Symulacja + uruchomienie**
Czas projektowania struktury krzemowej	**długi (nawet do 1 roku)**	**długi (kilka miesięcy)**	**krótki (kilka minut-godzin)**
Kosz pojedynczego układu	**Bardzo duży (nawet 100 000 razy droższy)**	**Bardzo duży (nawet 100 000 razy droższy**	**Tani**
Koszt układu w produkcji wielkoseryjnej	**tani**	**tani**	**duży (ok 1000 razy droższy niż układ full-custom)**
Prawdopodobieństwo niepowodzenia projektu	**„51 %”**	**„49 %”**	**„0 %”**

Porównanie układów scalonych

	ASIC *full-custom*	*ASIC semi-custom*	*Układ programowalny*
Debugowanie	**Tylko symulacja**	**Tylko symulacja**	**Symulacja + uruchomienie**
Czas projektowania struktury krzemowej	**długi (nawet do 1 roku)**	**długi (kilka miesięcy)**	**krótki (kilka minut-godzin)**
Kosz pojedynczego układu	**Bardzo duży (nawet 100 000 razy droższy)**	**Bardzo duży (nawet 100 000 razy droższy**	**Tani**
Koszt układu w produkcji wielkoseryjnej	**tani**	**tani**	**duży (ok 1000 razy droższy niż układ full-custom)**
Prawdopodobieństwo niepowodzenia projektu	**„50 %”**	**„49 %”**	**„0 %”**


XILINX®

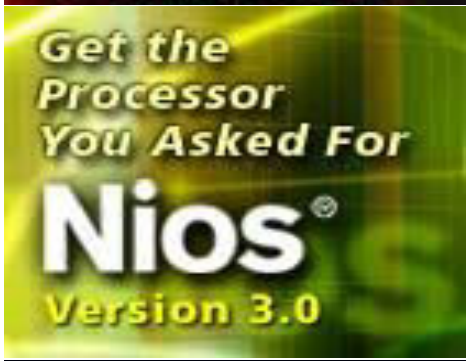

LATTICE
SEMICONDUCTOR.

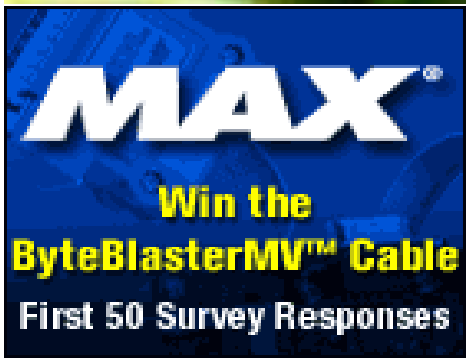

ALTERA®
MEASURABLE ADVANTAGE™

Altera/Intel

High-Density FPGAs	Excalibur Embedded Processor Solutions
Stratix™ (1.5 V, 10-Mbit RAM, DSP, LVDS) ***Stratix GX*** (1.5 V, 3.125-Gbps Transceivers) ***APEX™ II*** (1.5 V, 1-Gbps LVDS, Copper) ***APEX 20K*** APEX 20KC (1.8 V, LVDS, Copper) APEX 20KE (1.8 V, LVDS) APEX 20K (2.5 V) ***Mercury™*** (1.8 V, 1.25-Gbps CDR)	***About Excalibur™*** ***Excalibur Nios®***
High-Volume FPGAs	**High-Volume CPLDs**
Cyclone™ (1.5 V, Memory Interfaces, PLLs) ***ACEX® 1K*** (2.5 V)	***MAX® 3000A*** (3.3 V)
Mid-Density FPGAs	**High-Performance CPLDs**
FLEX 10K® FLEX 10KE (2.5 V) FLEX 10KA (3.3 V) FLEX 10K (5.0 V)	***MAX 7000*** MAX 7000B (2.5 V, SSTL, GTL+) MAX 7000AE (3.3 V) MAX 7000S (5.0 V)







Xilinx

Artix/Kintex/Virtex™- 6,7, UltraSCALE, UltraSCALE+

…

Virtex™-5 FPGA Family
Virtex-5 LX Platform logic
Virtex-5 LXT Platform
Virtex-5 SXT Platform

Virtex-4 FPGA Family
Virtex-4 LX Platform
Virtex-4 SX Platform
Virtex-4 FX Platform

Spartan™-3 Generation FPGA Families
World´s Lowest Cost FPGAs
Spartan-3A DSP Platform - DSP optimized
Spartan-3AN Platform - Non volatile
Spartan-3A Platform - I/O optimized
Spartan-3E Platform - Logic optimized
Spartan-3 Platform - For highest density and pin-count apps.









Lattice

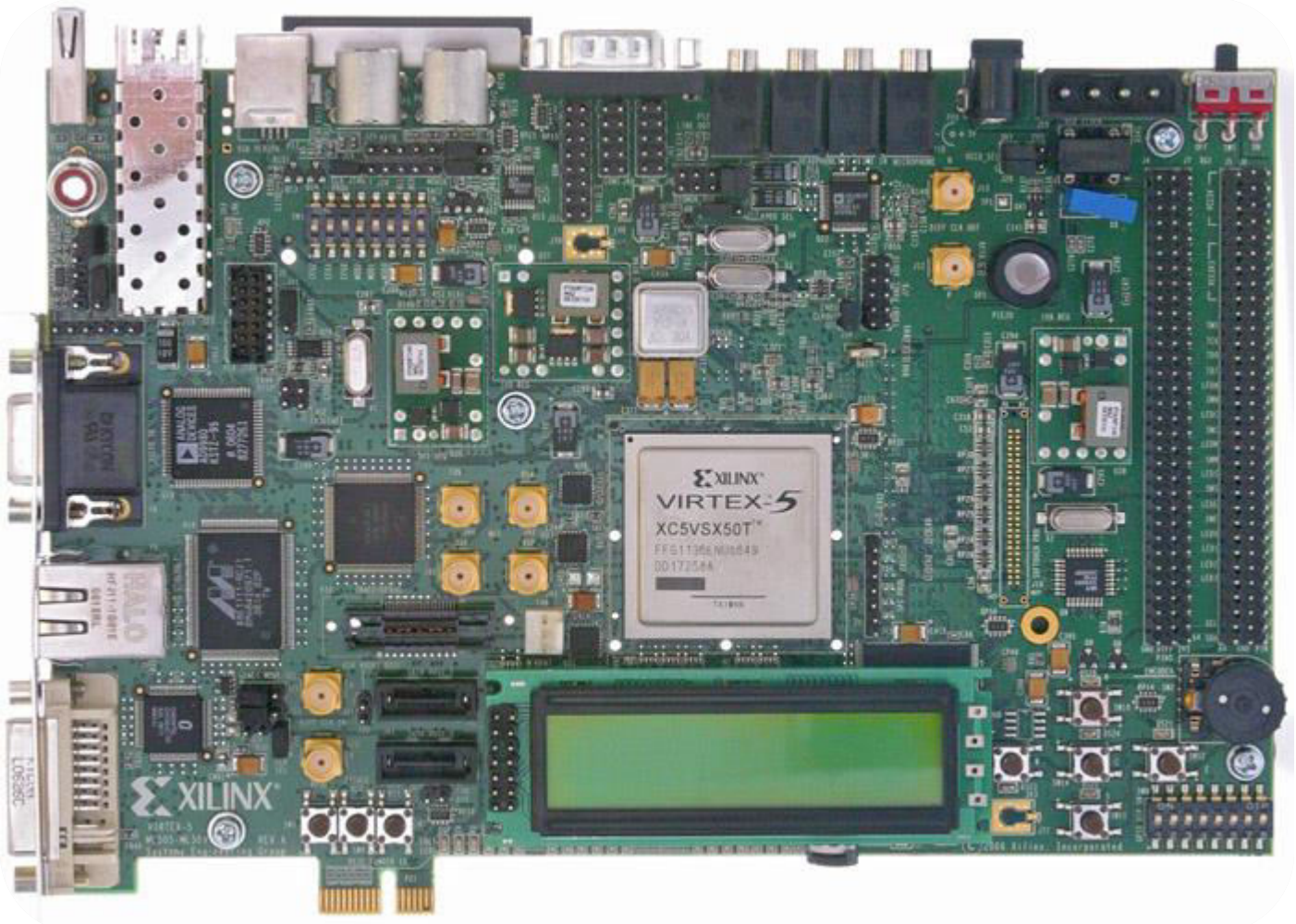
- ECP5
- ECP3
- ECP2
- MachXO3
- MachXO2
  - Mach XO2-4000
  - Mach XO2-7000
- ispMach
- Crosslink







Płyty rozwojowe



Nowe metody projektowania

- W praktyce niemożliwe projektowanie „ręczne”
- Niezbędne nowe metody projektowania i weryfikacji



- 1. Komputery osobiste
  - Narzędzia symulacyjne
- 2. Programowalne Układy Cyfrowe

Nowe metody projektowania

- Pierwsze komputery PC (lata 80) umożliwiły rozwój i pełne wykorzystanie układów programowalnych
  - Algorytmy optymalizacji układów
  - Programy wspomagające proces projektowania
  - Łatwo dostępne (i tanie) maszyny obliczeniowe



- Inspiracja językami programowania
  - C, Pascal, Fortran
- Języki opisu sprzętu
  - Verilog, VHDL, System Verilog…

Języki

	Klasyczne języki programowania	***Języki opisu sprzętu***
Przykłady	C, C++, Pascal, Java …	Verilog, VHDL, SystemVerilog
Co opisują	Kolejne kroki algorytmu	Elementy struktury sprzętowej
Stworzenie wersji produkcyjnej	Kompilator Kompilacja: kilka sek. – kilka minut	Syntezer Synteza: kilka minut – kilka godzin
Wprowadzenie poprawek	Szybkie	Powolne: Dekoncentracja programisty
Debugowanie	Proste: Kompilacja + uruchomienie + praca krokowa	Trudne: Symulacja (setki przebiegów) Synteza + uruchomienie (mało inf. zwrotnej)
Czas programowania	40%	40%
Czas pisania testów	40%	80%
Czas debugowania	40%	200%

Języki

	Klasyczne języki programowania	***Języki opisu sprzętu***
Przykłady	C, C++, Pascal, Java …	Verilog, VHDL, SystemVerilog
Co opisują	Kolejne kroki algorytmu	Elementy struktury sprzętowej
Stworzenie wersji produkcyjnej	Kompilator Kompilacja: kilka sek. – kilka minut	Syntezer Synteza: kilka minut – kilka godzin
Wprowadzenie poprawek	Szybkie	Powolne: Dekoncentracja programisty
Debugowanie	Proste: Kompilacja + uruchomienie + praca krokowa	Trudne: Symulacja (setki przebiegów) Synteza + uruchomienie (mało inf. zwrotnej)
Czas programowania	40%	40%
Czas pisania testów	40%	80%
Czas debugowania	40%	200%

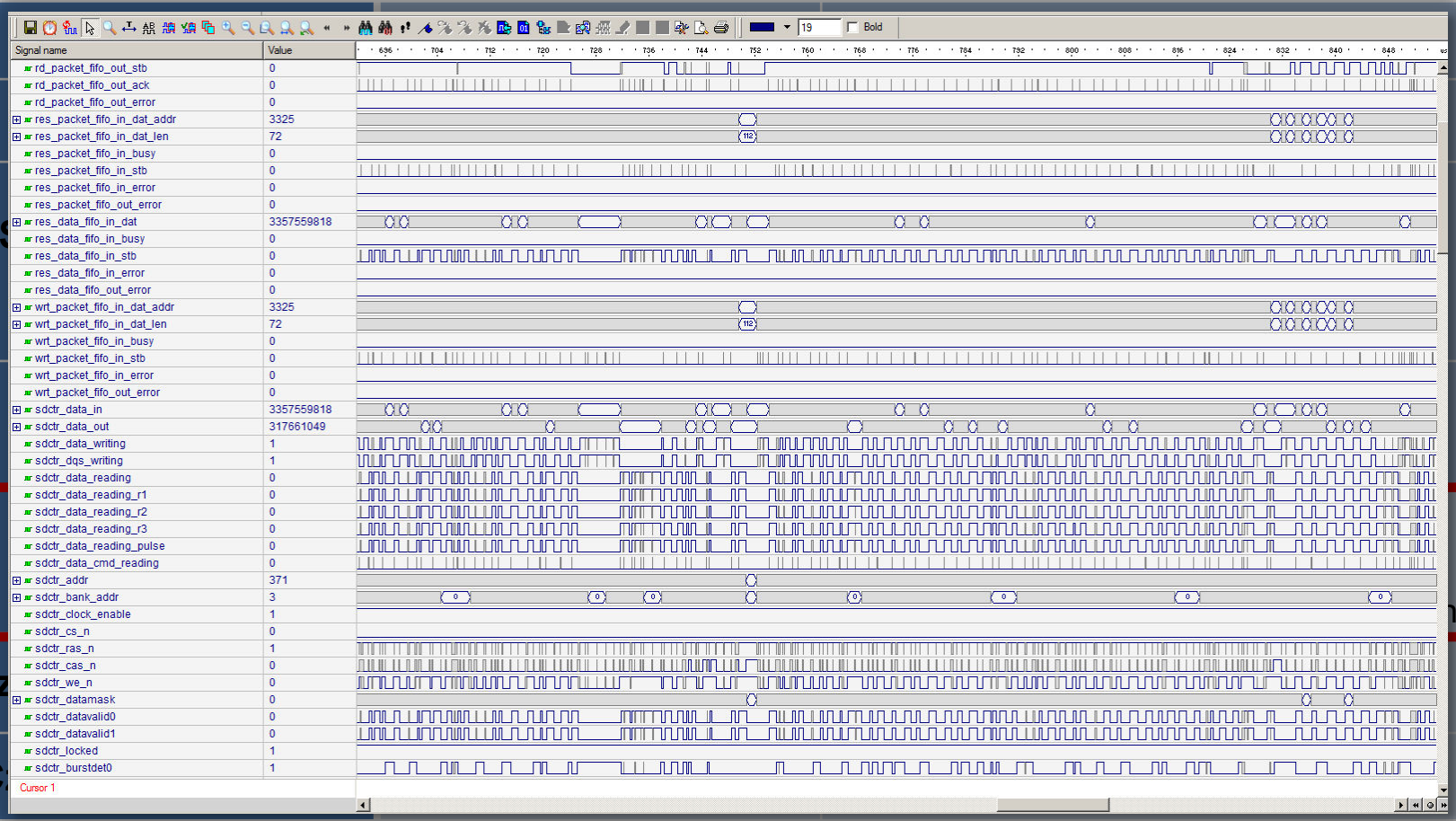
Języki

	Klasyczne języki programowania	***Języki opisu sprzętu***
Przykłady	C, C++, Pascal, Java …	Verilog, VHDL, SystemVerilog
Co opisują	Kolejne kroki algorytmu	Elementy struktury sprzętowej
Stworzenie wersji produkcyjnej	Kompilator Kompilacja: kilka sek. – kilka minut	Syntezer Synteza: kilka minut – kilka godzin
Wprowadzenie poprawek	Szybkie	Powolne: Dekoncentracja programisty
Debugowanie	Proste: Kompilacja + uruchomienie + praca krokowa	Trudne: Symulacja (setki przebiegów) Synteza + uruchomienie (mało inf. zwrotnej)
Czas programowania	40%	40%
Czas pisania testów	40%	80%
Czas debugowania	40%	200%

Języki

	Klasyczne języki programowania	***Języki opisu sprzętu***
Przykłady	C, C++, Pascal, Java …	Verilog, VHDL, SystemVerilog
Co opisują	Kolejne kroki algorytmu	Elementy struktury sprzętowej
Stworzenie wersji produkcyjnej	Kompilator Kompilacja: kilka sek. – kilka minut	Syntezer Synteza: kilka minut – kilka godzin
Wprowadzenie poprawek	Szybkie	Powolne: Dekoncentracja programisty
Debugowanie	Proste: Kompilacja + uruchomienie + praca krokowa	Trudne: Symulacja (setki przebiegów) Synteza + uruchomienie (mało inf. zwrotnej)
Czas programowania	40%	40%
Czas pisania testów	40%	80%
Czas debugowania	40%	200%

Języki

	Klasyczne języki programowania	Języki opisu sprzętu
		
Czas debugowania	40%	200%

Języki

	Klasyczne języki programowania	***Języki opisu sprzętu***
Przykłady	C, C++, Pascal, Java …	Verilog, VHDL, SystemVerilog
Co opisują	Kolejne kroki algorytmu	Elementy struktury sprzętowej
Stworzenie wersji produkcyjnej	Kompilator Kompilacja: kilka sek. – kilka minut	Syntezer Synteza: kilka minut – kilka godzin
Wprowadzenie poprawek	Szybkie	Powolne: Dekoncentracja programisty
Debugowanie	Proste: Kompilacja + uruchomienie + praca krokowa	Trudne: Symulacja (setki przebiegów) Synteza + uruchomienie (mało inf. zwrotnej)
Czas programowania	40%	40%
Czas pisania testów	40%	80%
Czas debugowania	40%	200%

Systemy projektowe

- Programy do syntezy i symulacji dla układów programowalnych
  - Diamond (Lattice)
  - Synplify (Synplicity)
  - ModelSim (MentorGraphics)
  - Quartus (MentorGraphics)
  - ActiveHDL (Aldec)
  - ISE (Xilinx)
  - MaxPlus (Altera/Intel)
  - Leonardo Spectrum (MentorGraphics)
  - FPGA Compiler
  - Verilator (open source)

Dziękuję za uwagę

- Pytania?

Verilog
Język opisu sprzętu

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

Języki opisu sprzętu

- Hardware Description Language (HDL)
  - Język VHDL
    - początek 1987
    - język uniwersalny, pozwala symulować wszelkie zjawiska fizyczne np. matryce DLP
  - Język Verilog
    - początek 1995
    - pomyślany jako język modelowania (symulowania) układów cyfrowych
  - Inne „żyjące” i rozwijające się języki
    - ~~SystemC (C++)~~
    - SystemVerilog (Verilog)
    - MyHDL (Python)
    - MiGen (Python)

Verilog – podstawowe informacje

- Stworzony 1984 przez Phila Moorby i Prabhu'a Goel'a w firmie Gateway Design Automation
  - Stworzony dla potrzeb symulacji
  - Składania opisująca co, kiedy ma się wydarzyć
  - Dostosowany dla potrzeb projektowania i syntezy
- W 1990 firma została kupiona przez Cadence Design Systems
- W 1995 został poddany standaryzacji przez IEEE (Verilog-95)
- W roku 2001 dodano znaczące rozszerzenia do języka (Verilog-2001)
  - Od tego momentu Verilog był już wspierany przez większość oprogramowania do projektowania sprzętu (narzędzia EDA)
- W 2005 kolejna rozszerzona wersja (Verilog-2005)
- W 2005/2006 wprowadzono nowy standard SystemVerilog

Verilog – podstawowe informacje

- Co opisuje Verilog?
  - Jak realizować symulację danego układu na podstawie kodu źródłowego
    - Zmienne
    - Zdarzenia
    - Przebiegi czasowe
  - Jak zrealizować strukturę sprzętową działającą tak samo jak kod źródłowy
    - Moduły, podmoduły, rejestry, liczniki, maszyny stanu, multipleksery, liczniki, bramki
    - Syntezer stosuje domyślanie się/wnioskowanie (inferring)
      - X = A & B → bramka AND
      - Y = S ? A : B → multiplekser (A,B - wejścia, S – sterowanie)

Verilog – podstawowe informacje

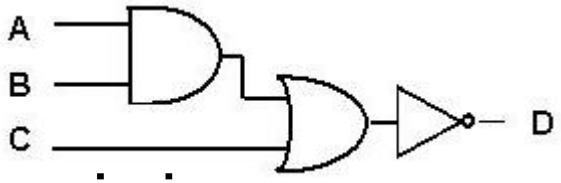
- Ogólnie - wiele sposobów na opisanie „tego samego” układu
  - Opis behawioralny
    - Opis jak układ działa

```
D = !( (A&B) | C )
```

    - Jakie algorytmy są wykorzystywane
    - Wymaga wnioskowania przez syntezer (inferring)
    - Niekoniecznie opisuje coś syntezowalnego!

```
$display(„Hello world”)
```

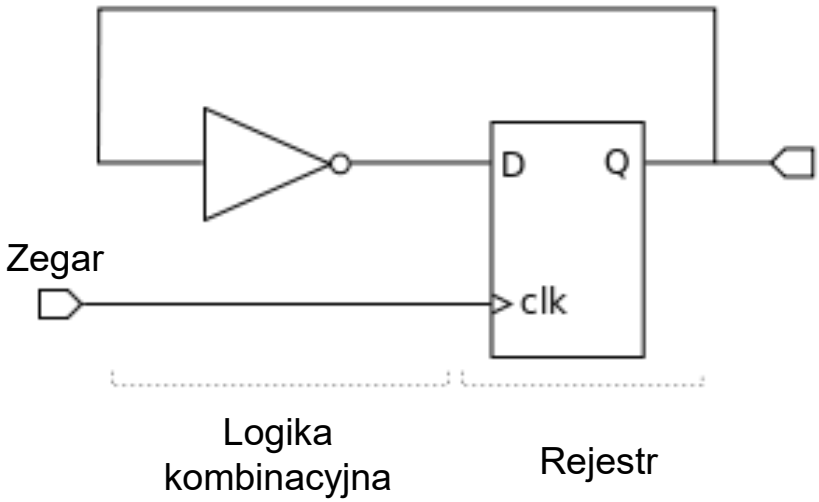
  - Opis strukturalny
    - Z jakich pod-modułów się składa



    - Jakie sygnały przepływają pomiędzy nimi
    - Także opis tekstowy!

Register Transfer Level - RTL

- Sposób opis układu na poziomie rejestrów
  - Rejestry
    - Przerzutnik (Flip-flop-D)
    - Zatrzask (Latch)
  - Logika kombinacyjna
    - Funkcje logiczne
    - Bramki
  - Połączenia między nimi
- Sposób opisu w Verilogu
- Wynik syntezy



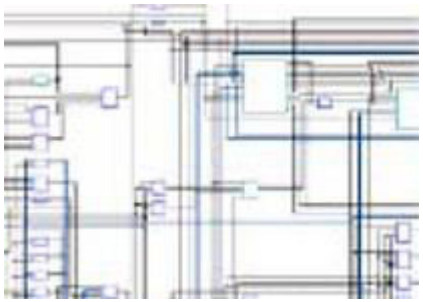
Potok projektowania

Hardware Description Language

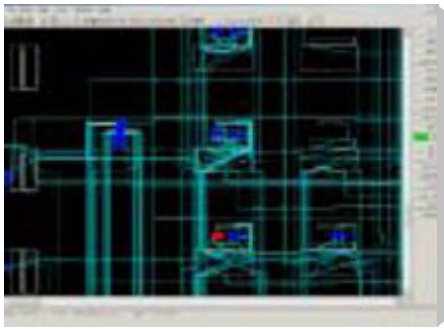




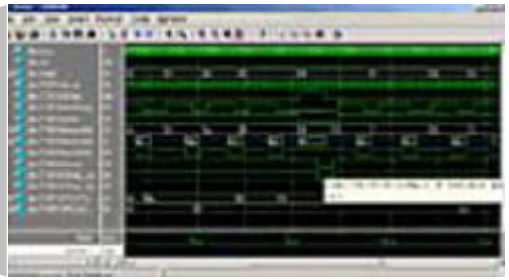
Symulacja behawioralna



Synteza logiki RTL (Register Transfer Level)



Mapping & Routing (Physical Layout)



Symulacja funkcjonalna + czasowa



Programowanie układu FPGA



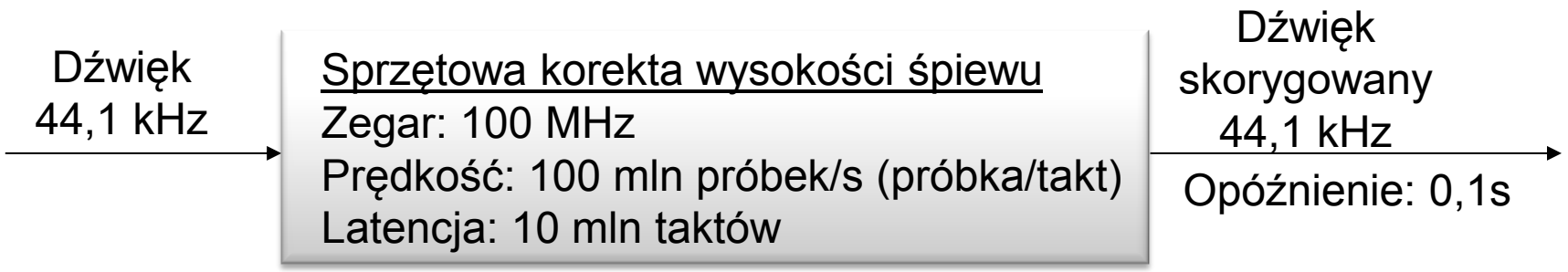
Testowanie na układzie FPGA

Projektowanie w HDL

- Zasady dobrego projektowania
  - Dobrze zdefiniowane zadanie układu
  - Praca ze schematem (ogólnym) układu
  - Ujednolicone interfejsy wejścia/wyjścia
  - Układy synchroniczne 
  - Równoległa praca wielu modułów
  - Optymalizacja racjonalna

Projektowanie w HDL

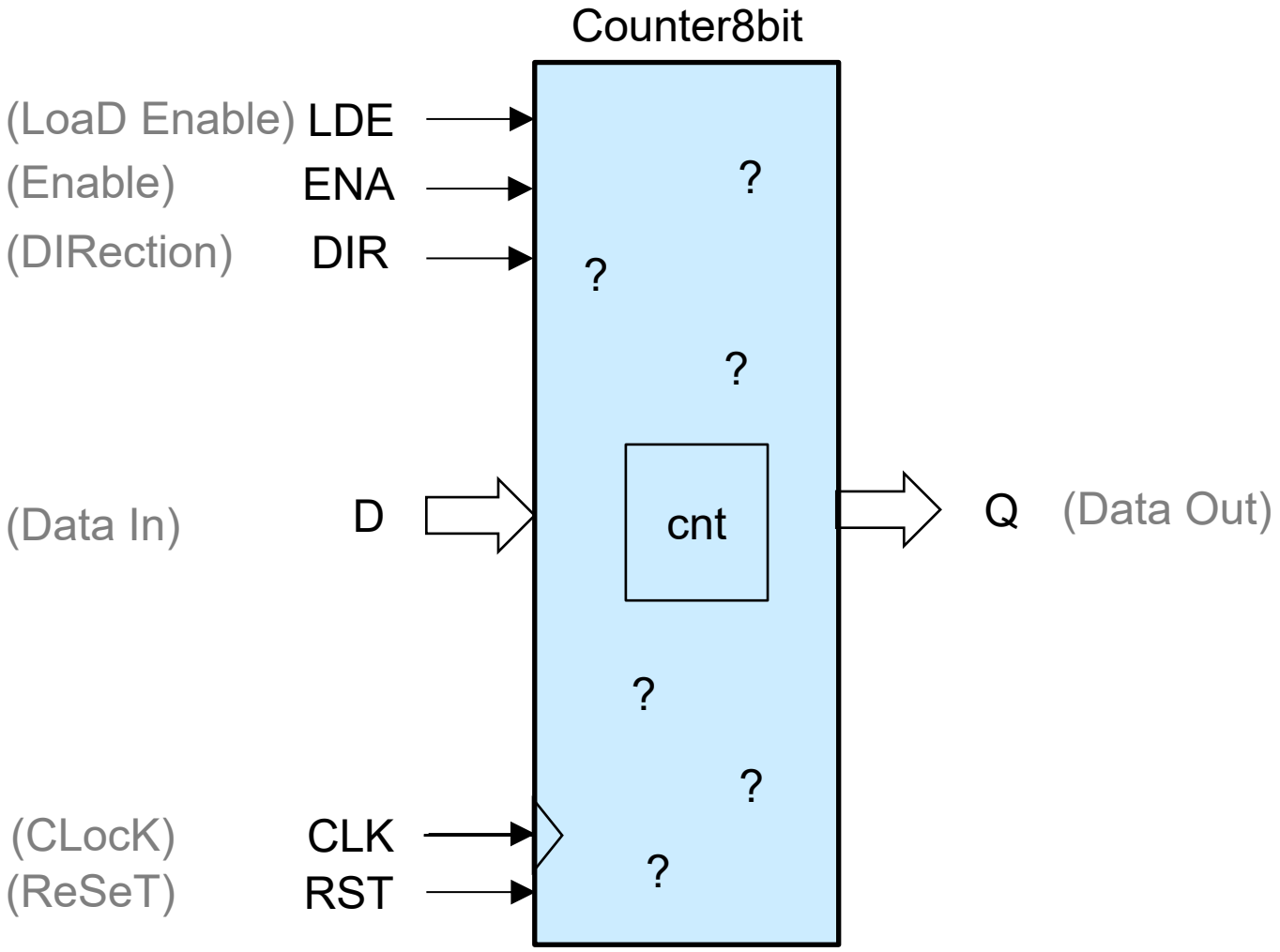
- Zasady dobrego projektowania
  - Optymalizacja
    - Wielkość
    - Zużycie energii
    - Prędkość (liczba taktów↔czas↔zegar [MHz]), latencja



  - Nasz moduł jest częścią całości (systemu)
  - Tylko jeśli i tam gdzie jest potrzebna

Składnia języka Verilog

Verilog – pierwszy układ



Verilog – pierwszy układ

```
module Counter8bit
(
 input  wire       CLK, // clock
 input  wire       RST, // reset

 input  wire       ENA, // enable
 input  wire       LDE, // load enable
 input  wire       DIR, // direction

 input  wire [7:0] D,
 output wire [7:0] Q
);
  reg [7:0] cnt;

  always @(posedge CLK or posedge RST)
  if (RST)                  cnt <= 0;
  else if (LDE)             cnt <= D;
  else if (ENA && DIR)      cnt <= cnt - 1;
  else if (ENA)             cnt <= cnt + 1;
  else begin // { in C++
            //   code
  end       // } in C++

  assign Q = cnt;

endmodule
```

Verilog – pierwszy układ

```
module Counter8bit
(
 input  wire       CLK, // clock
 input  wire       RST, // reset

 input  wire       ENA, // enable
 input  wire       LDE, // load enable
 input  wire       DIR, // direction

 input  wire [7:0] D,
 output wire [7:0] Q
);
  reg [7:0] cnt;

  always @(posedge CLK or posedge RST)
  if (RST)                  cnt <= 0;
  else if (LDE)             cnt <= D;
  else if (ENA && DIR)      cnt <= cnt - 1;
  else if (ENA)             cnt <= cnt + 1;
  else begin // { in C++
            //   code
  end       // } in C++

  assign Q = cnt;

endmodule
```

Składnia języka Verilog

- Bardzo zbliżona do języków C i Pascal
- Stworzona z myślą o symulacji
- Różne sposoby opisu układu skutkują różnymi strukturami sprzętowymi
- Umożliwia tworzenie konstrukcji których nie można fizycznie wykonać w sprzęcie
- Język wciąż rozwijany

Składnia języka Verilog

- Znaki białe
  - Spacja, enter, tabulator
- Komentarze
  - // …
  - /* … … */
- Czułość na wielkość liter
  - Wszystkie słowa kluczowe są pisane małymi literami
  - Abc to inna zmienna niż abc
    - Nie nadużywać, trzymać się konwencji

Składnia języka Verilog

- Nazwy
  - A…Z, a…z, _
  - Wewnątrz nazwy dodatkowo: liczby, $
- Liczby
  - Reprezentacja binarna
  - Domyślna wielkość – 32 bity
- Możliwe stany
  - 0, 1
  - x – wartość nieznana
  - z – wysoka impedancja

```
is_Available_$100
Is_available_100$
```

```
100$_is_available_
$100_Is_Available
```

Składnia języka Verilog

- Liczby
  - Możliwość zapisu w różnych systemach
    - `15` (w.dziesiętna:15, 32 bity)
    - `'hFF` (w.dziesiętna:255, 32 bity) // 0xFF w C++
    - `8'hFF` (w.dziesiętna:255, 8 bitów)
    - `6'b100011` (w.dziesiętna:35, 6 bitów)
    - `6'b10_00_11` (w.dziesiętna:35, 6 bitów)
    - `6'b10_00_1x` (w.dziesiętna:34 lub 35, 6 bitów)
  - Co jeśli podamy za mało bitów?
    - `40'b101`

Składnia języka Verilog

- Liczby
  - Brakujące bity są dopełniane w zależności od stanu na najwyższym bicie
    - 0,1 → dopełnienie zerami
    - x → dopełnienie „x”-ami
    - z → dopełnienie „z”-ami

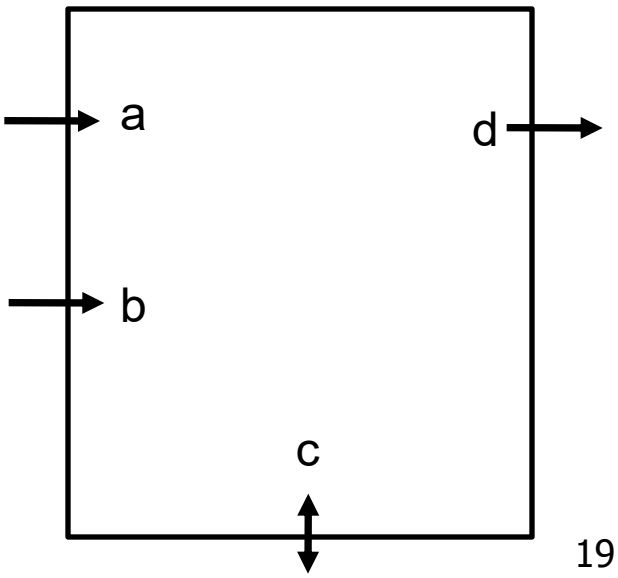
    - `8'b001` → `8'b00000001`
    - `8'b101` → `8'b00000101`
    - `8'bx01` → `8'bxxxxxx01`
    - `8'bz01` → `8'bzzzzzz01`

Moduły

- Moduły
  - Bloki tworzące opis układu
  - Odpowiednik klas C++
  - Mogą być wykorzystywane wielokrotnie
  - Komunikują się przez porty

C++	Verilog
Klasa (class)	Moduł (module)
Obiekt klasy	Instancja modułu

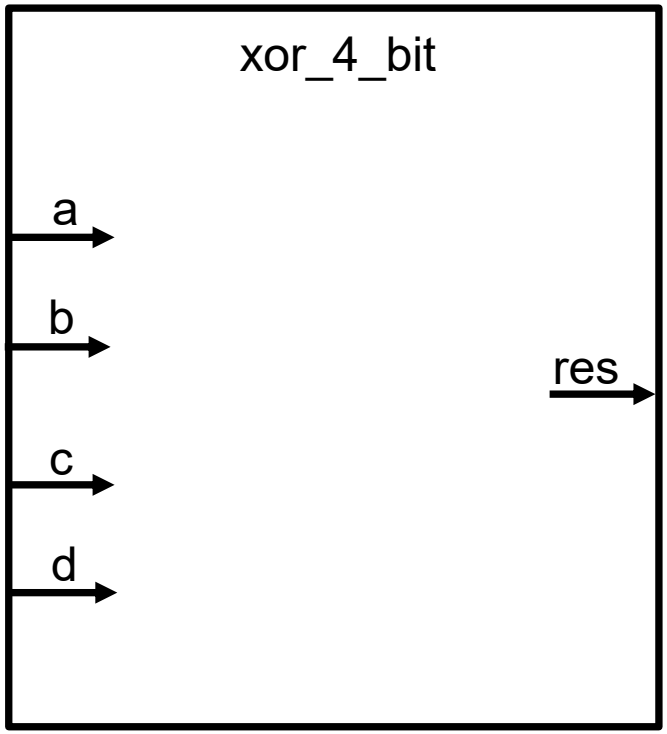
```
module nazwa ( /* porty */ )
... // wnętrze modułu
endmodule
```



- Rodzaje portów (wejścia/wyjścia)
  - input - wejście
  - output - wyjście
  - inout – wejście (x) lub wyjście
- Szerokości portów
  - Domyślnie – 1 bit
  - input [szerokość bitowa] nazwa;

4 bitowy moduł xor

- Zaprojektujmy moduł xor
  - 4 wejścia
  - 1 wyjście



Wstawienie instancji modułu

```
module xor_4_bit  (a,b,c,d, res);

 input a;
 input b;
 input c;
 input d;

 output res;

 wire sum_ab,  sum_cd;


 xor_2_bit s0 (a,b, sum_ab);
 xor_2_bit s1 (c,d, sum_cd);
 xor_2_bit s3 (sum_ab, sum_cd, res);

endmodule
```

Wstawienie instancji modułu

```
module xor_4_bit   (a,b,c,d, res);

 input a;
 input b;
 input c;
 input d;

 output res;

 wire sum_ab,   sum_cd;

 xor_2_bit s0 (a,b, sum_ab);
 xor_2_bit s1 (c,d, sum_cd);
 xor_2_bit s3 (sum_ab, sum_cd, res);

endmodule
```

Nazwa modułu którego instancję wstawiamy

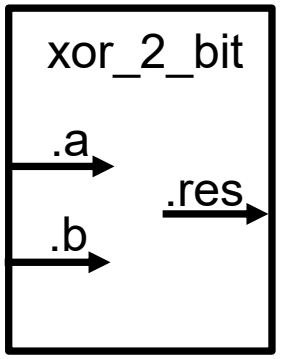
Nazwy instancji modułów które wstawiamy

Podłączenie portów instancji

Założenia?

- Wierzymy że gdzieś „tam”, istnieje moduł:

```
module xor_2_bit  (a,b, res);
...
endmodule
```



- Co jeśli argumenty:
  - Są w innej kolejności?
  - Ich kolejność zostanie w „międzyczasie” zmieniona?
  - Zostaną w ogóle przez kogoś zmienione?

Wstawienie instancji modułu

```
module xor_4_bit  (a,b,c,d, res);

 input a;
 input b;
 input c;
 input d;

 output res;

 wire sum_ab,  sum_cd;


 xor_2_bit s0 (a,b, sum_ab);
 xor_2_bit s1 (c,d, sum_cd);
 xor_2_bit s3 (sum_ab, sum_cd, res);

endmodule
```

Wstawienie instancji modułu

```
module xor_4_bit   (a,b,c,d, res);

 input a;
 input b;
 input c;
 input d;

 output res;

 wire sum_ab,   sum_cd;


 xor_2_bit s0 (.a(a      ), .b(b      ), .res(sum_ab) );
 xor_2_bit s1 (.a(c      ), .b(d      ), .res(sum_cd) );
 xor_2_bit s3 (.a(sum_ab), .b(sum_cd), .res(res    ) );

endmodule
```

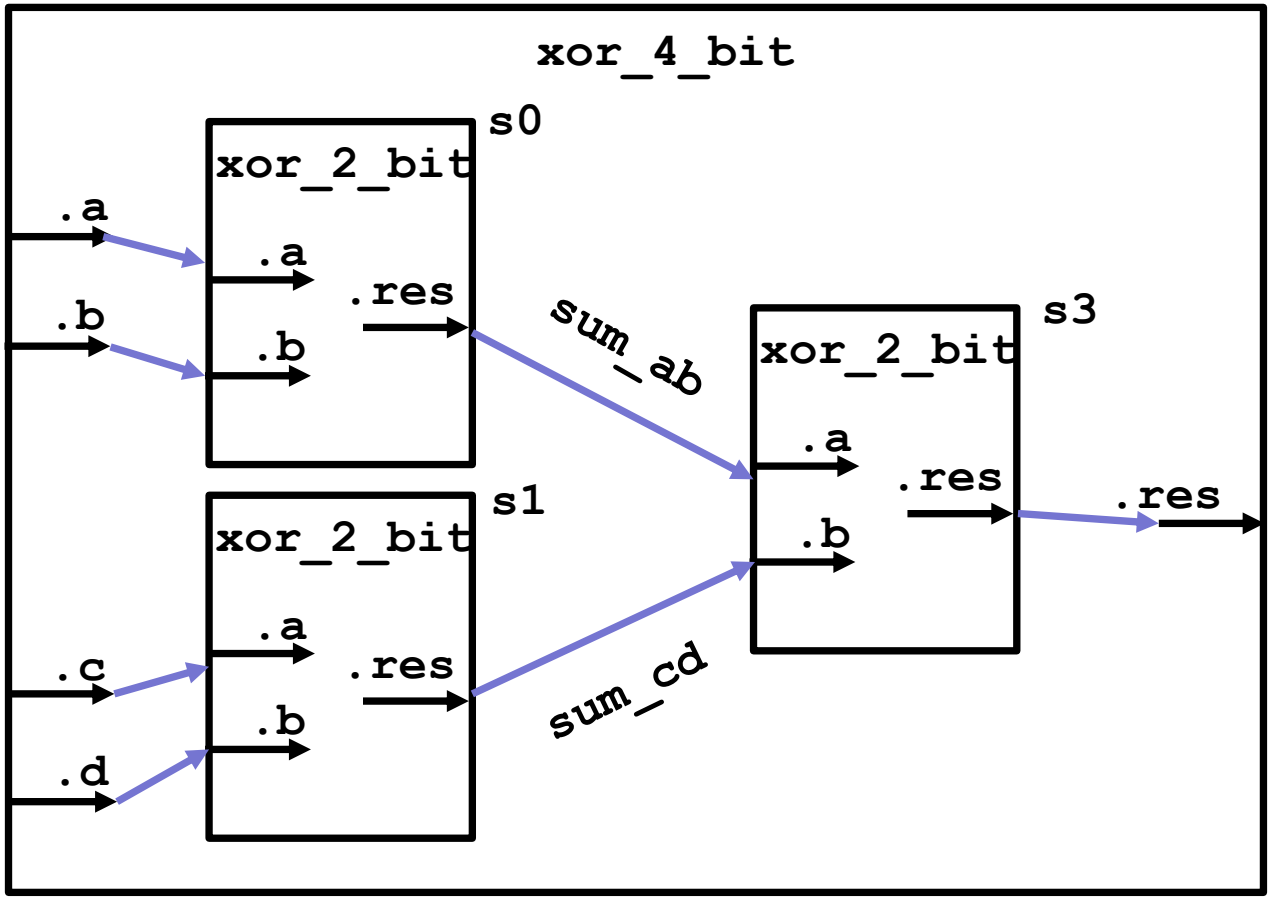
Porty wewnętrzne pod-modułu xor_2_bit

lokalne sygnały w aktualnym module

Wstawienie instancji modułu

```
input a,b,c,d;    output res;     wire sum_ab,  sum_cd;

 xor_2_bit s0(.a(a),       .b(b),       .res(sum_ab) );
 xor_2_bit s1(.a(c),       .b(d),       .res(sum_cd) );
 xor_2_bit s3(.a(sum_ab), .b(sum_cd), .res(res)    );
```



Szyny danych

- Typy danych
  - wire
  - reg
- Szerokości
  - wire|reg nazwa; // 1 bit
  - wire|reg [szerokość bitowa] nazwa;
- Pamięci
  - reg [szerokość bitowa] nazwa[zakres_adresacji];

Dane - przykłady

- wire a;
  - 1-bitowa szyna danych
- wire [15:0] b;
  - 16-bitowa szyna danych
- reg [31:0] c;
  - rejestr 32-bitowy
- reg [7:0] mm[0:127]; // <- uwaga: odwrotnie!!
  - Pamięć o słowach 8-bitowych, 128 komórek

Operatory

- Arytmetyczne

  + - * / %

- Porównawnia

  < > <= >=

- Równości

  == !=

  === !== uwzględniając x i z

- Kolor pomarańczowy
  - problemy z syntezą rzeczywistego sprzętu

Operatory == oraz ===

a	b	wynik a==b	komentarz	wynik a===b
100	001	false	różne	false
101	101	true	takie same wartości	true
100	01x	false	niezależnie od "x" są różne	false
100	10x	x	zależnie od "x" mogą być równe	false
10x	10x	x	zależnie od "x"-ów mogą być równe lub nie	true
10z	10x	x	zależnie od "x" i "z" mogą być równe lub nie (przy odczycie „z” jest czytane jako „x”)	false

Operatory

- Przesunięcie bitowe

  << >>

- Wycinanie bitów
  - wire [7:0] a;
    wire [2:0] b = a[7:5];
- Łączenie bitów { … , …} np.
  - {a, b[1:0]}
    - Wynikowy wektor bitowy: (8 bitów a) + (2 bity z b) = 10 bitów
  - {3'b101, 4'hF} = 7'b101_1111
    - Wynikowy wektor bitowy: (3 bity) + (4 bity) = 7 bitów

Operatory

- ! oraz ^ - NOT, negacja
- && oraz & - AND, iloczyn
- || oraz | - OR, suma
- Istotna różnica:
  - Logiczne (logical)

    ! && || wejście/wyjście: 1 bit!
  - Bitowe (bit-wise/binary)

    ~ & | wejście/wyście takie samo jak wejście

    ^ - xor

    ^~ lub alternatywnie ~^ - xnor

Operatory

- Operatory bitowe vs logiczne

		Przykłady				
Wartości bitowe	a	001	110	000	000	000
	b	010	111	101	010	000
Wyniki bitowe	a & b	000	110	000	000	000
	a | b	011	111	101	010	000
Wartości logiczne	Czy a!=0 ?	true	true	false	false	false
	Czy b!=0 ?	true	true	true	true	false
Wyniki logiczne	a && b	true	true	false	false	false
	a || b	true	true	true	true	false

Operatory

- Redukcja

  & ~& | ~| ^ ~^

  - Zaznaczamy nawiasami np.
    - (& 4'b1001) daje 1'b0
    - (& 4'b1111) daje 1'b1

  - Na przykład zamiast

    ```
    wire [7:0] a;
    wire b = a[7] & a[6] & a[5] …… & a[0];
    ```

  - piszemy

    ```
    wire b = (&a);
    ```

Operatory

- Replikacja

  {n{m}}

  - Powtarza **n** razy daną **m**
  - **Np. {4{2'b01}} = 8'b01010101**

- Wyrażenie warunkowe (multiplexer)
  - warunek ? wart.jeśli.prawda : wart.jeśli.fałsz
  - Zapisy tożsame np..
    - if (a>b) wiekszy_ab=a; else wiekszy_ab=b;
    - wiekszy_ab = (a>b) ? a : b;

Operatory

- Przypisania podłączające/kombinatoryczne (kabelki wire)

  assign = podłączenie sygnału np. assign a=b;

- Przypisania proceduralne

  = przypisanie blokujące/sekwencyjne np. a = b;

  <= przypisanie nieblokujące np. a <= b;

Operatory

- Opóźnienia
  - #liczba - przerywa obliczenia na zadaną liczbę jednostek czasowych symulatora

```
a = #10 b; // weź b, poczekaj #10, zapisz do a
#10 a = b; // weź b, zapisz do a, poczekaj #10

#10 a = #10 b*c;
// weź b, przemnóż przez c, poczekaj #10,
// zapisz do a, #10
```

Kolejność wykonywania działań

- ! ~ * / %
- + - << >>
- < > <= >= == != === !==
- & !& ^ ^~ | ~|
- && ||
- ? :

- = <=

Operowanie na danych

- Połączenia kombinatoryczne (jawne) - wire
  - assign res = a+c;
  - …lub od razu podczas definiowania:
  - wire res = a+c;
- Bloki proceduralne - reg
  - reg clk, data;
    begin
      clk = 0;
      data = 1;
      $display(”%d %d”, clk, data); // printf(..)
    end

Bloki proceduralne

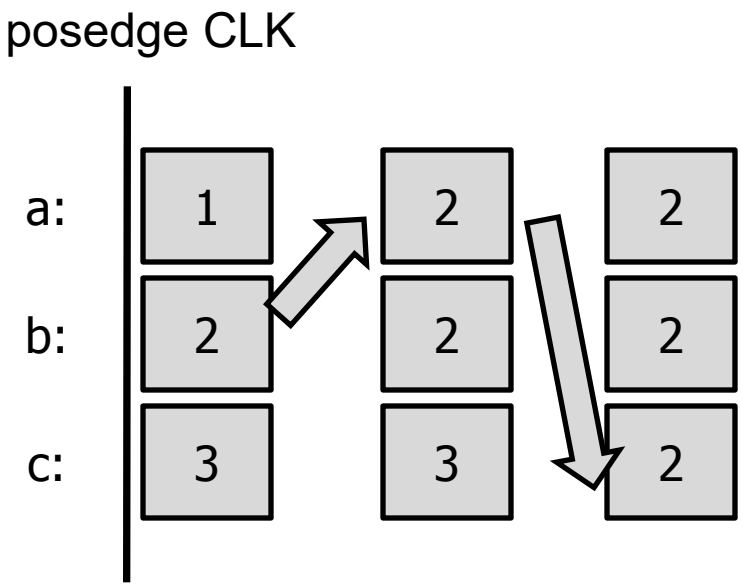
- Kiedy są wykonywane?
  - initial – początek symulacji

```
reg clk, data;
initial
begin
  $monitor("clk=%d data=%d", clk, data);
  // printf(..) z monitorowaniem wartości
  clk=0;
  data=1;
end
```

```
clk=x data=x
clk=0 data=x
clk=0 data=1
```

Bloki proceduralne

- Kiedy są wykonywane?
  - always @ (lista zdarzeń)
    - zawsze przy wystąpieniu określonych warunków
    - **posedge**
    - negedge
    - edge
  - reg [7:0] a,b,c;
    always @(posedge CLK)
    begin
      a = b;
      c = a;
    end



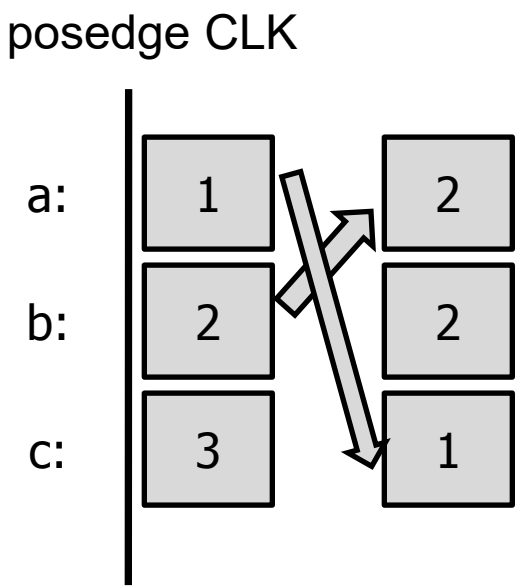
UWAGA!!

zbliżamy się do końca myślenia
w sposób sekwencyjny

Bloki proceduralne

- Przypisanie nieblokujące

```
always @(posedge CLK)
begin
  a <= b;
  c <= a;
end
```

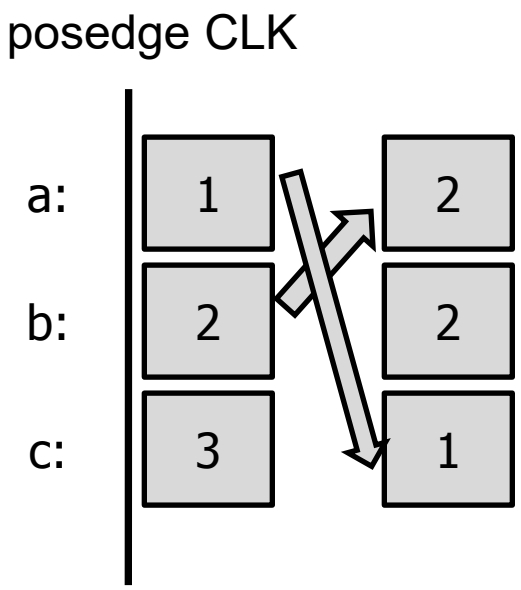


- Jest syntezowalne
- Wymagane w opisie syntezowalnego sprzętu
- Wymagane na laboratoriach
  - Wyjątek: testbench, moduły testujące

Bloki proceduralne

- Przypisanie nieblokujące

```
always @(posedge CLK)
begin
  a <= b;
  c <= a;
end
```

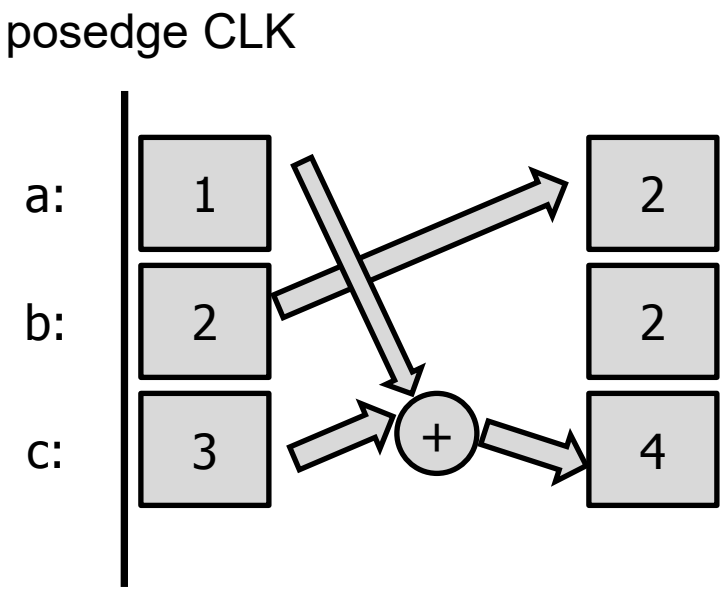


- Wykonywanie
  - weź b, czyli 2
  - zaplanuj zapis a=2
  - weź a, czyli 1
  - zaplanuj zapis c=1
  - P.S. zostaw b bez zmian

Bloki proceduralne

- Przypisanie nieblokujące

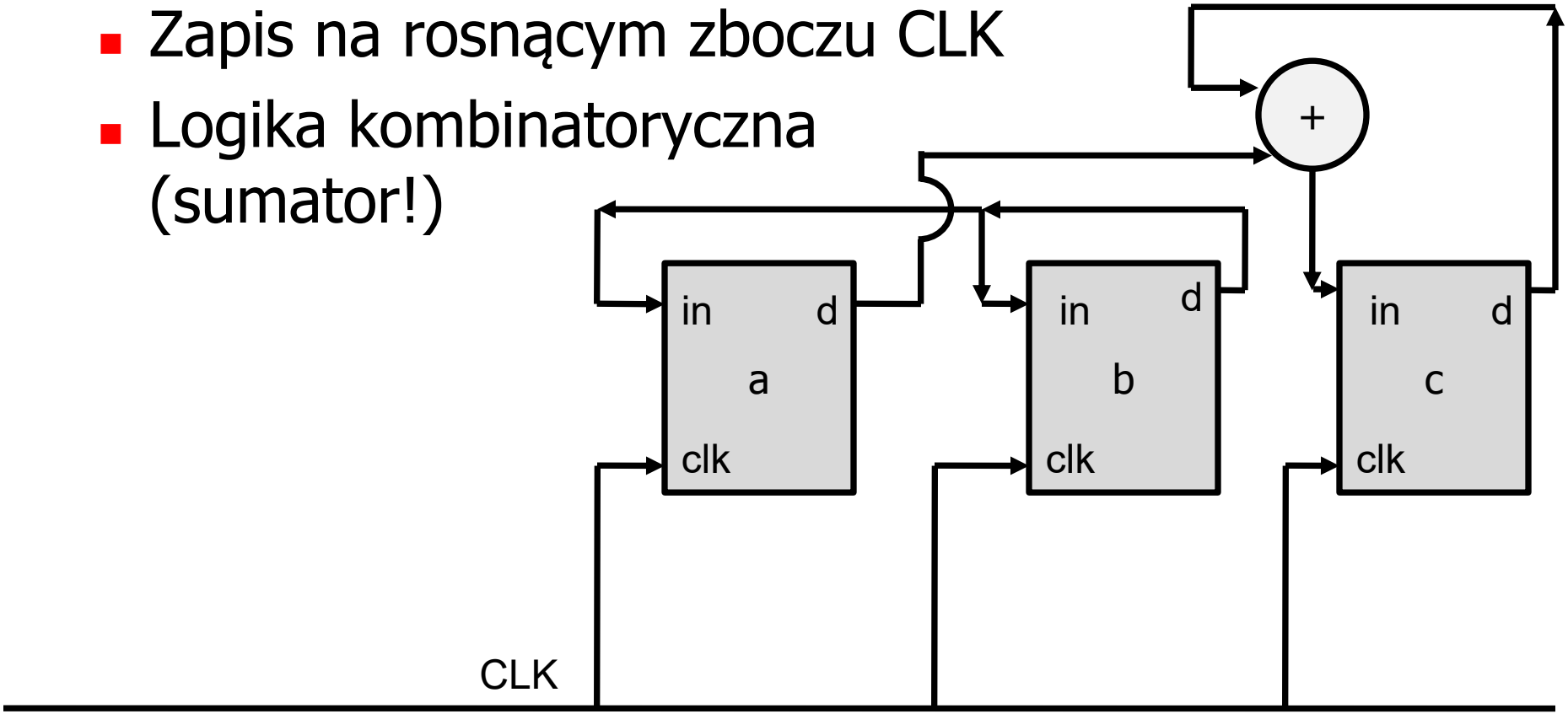
```
always @(posedge CLK)
begin
  a <= b;
  c <= a+c;
end
```



- Wykonywanie
  - weź b, czyli 2
  - zaplanuj zapis a=2
  - weź a (czyli 1) oraz c (czyli 3) i wylicz 1+3=4
  - zaplanuj zapis c=4
  - P.S. zostaw b bez zmian

Opis sprzętu

- Co opisaliśmy?
  - 3 przerzutniki (flip-flop-D)
  - Zapis na rosnącym zboczu CLK
  - Logika kombinatoryczna (sumator!)



Bloki proceduralne

- Opis przerzutnika z resetem

```
always @(posedge CLK or posedge RST)
if (RST) begin
  a <= 0; // reset
end else begin
  a <= ...; // normalna praca układu
end
```

Konstrukcje warunkowe

- if (warunek) ..... ;
- if (warunek) ..... ; else ....;
- case (zmienna) // w C++: switch(zmienna)
  Wartość1: ....; // w C++: case Wartość1: break
  Wartość2: ....; // w C++: case Wartość2: break
  endcase
- brak potrzeby break
- casex traktuje x jako „don't care"
- casez traktuje z jako „don't care"

Konstrukcje warunkowe

- Przykład:

```
wire [3:0] liczba;
casex(liczba)
 4'b1111: ...; // wartość 15
 4'b00xx: ...; // wartości 00,01,10,11
 4'bx001: ...; // wartości 0001 oraz 1001
endcasex
```

- Uwaga: kolejność czytania może stanowić problem!

Pętle

- repeat (liczba wykonań)
- while (warunek_wykonywania) …
- for (początek;warunek;krok) …
- forever

- W większości przypadków NIE są syntezowalne

Przykłady

- Generowanie sygnału zegara

```
reg clk;
initial
begin
  #1 clk = 0;
  forever begin
    #5 clk = ! clk;
  end
end
```

Przykłady

- Generowanie sygnału zegara (metoda alternatywna)

```
reg clk;

initial clk  <= 1;

always #20  clk <= ~clk;
```

Verilog

- Verilog posiada znacznie szersze możliwości
- Znaczna część przydatna jedynie dla symulacji
- Składnia użyteczna przy opisie fizycznych syntezowalnych układów
  - Nie jest ograniczona przez sam język
  - lecz przez: narzędzia syntezy, fizyczny sprzęt

  - Dzielenie, modulo, $display…

Dziękuje za uwagę

- Pytania?

Układy FPGA: cz. 1

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

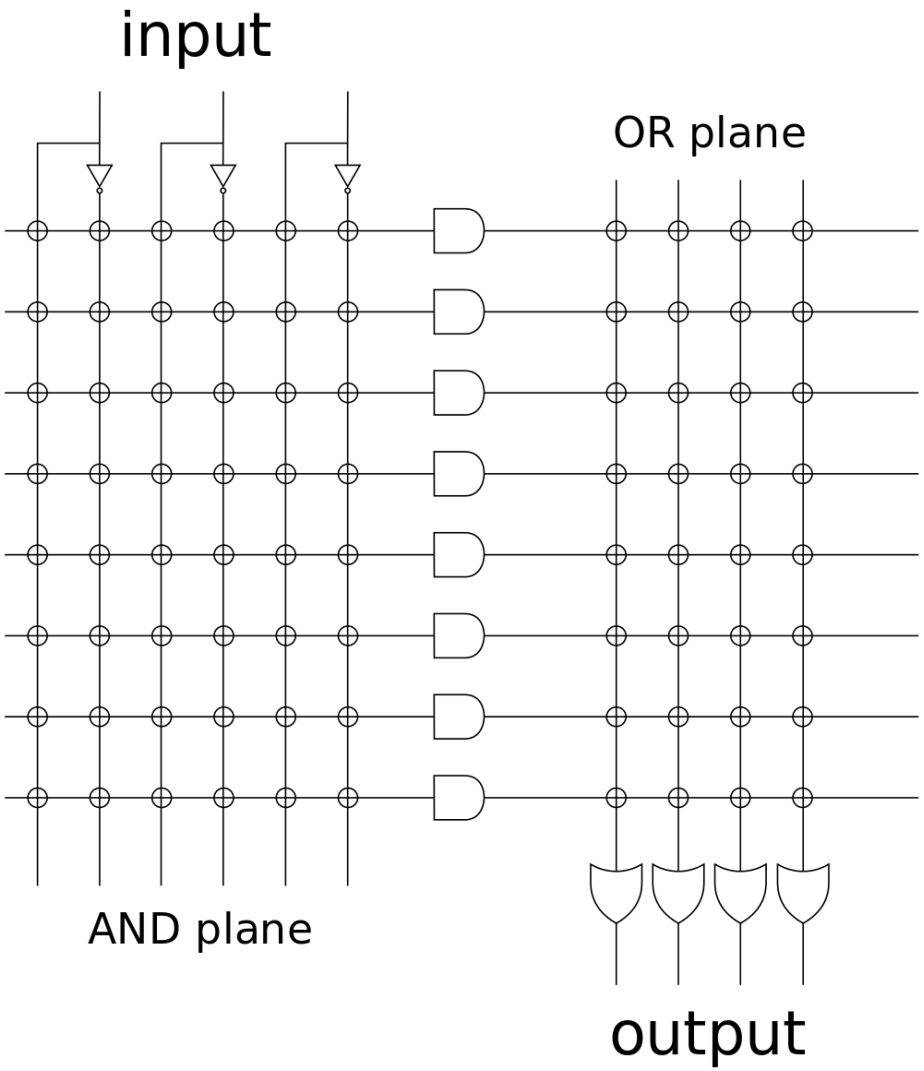
Układy programowalne

- PLD (Programmable Logic Device)
  - Układ elektroniczny o programowalnej strukturze
    - Nie procesor (ma stałą strukturę)
    - PLD – struktura zależy od zaprogramowania
  - Typy układów PLD
    - SPLD (Simple PLD) - PLA, PAL, PLE, GAL…
    - CPLD (Complex PLD)
    - FPGA (Field Programmable Gate Array)
- Application Specific Integrated Cicuits (ASIC)
  - Nieprogramowalne – np. pamięć DDR, Sprzętowy kodek dźwięku, ARM, Intel Core i7

Historia układów programowalnych

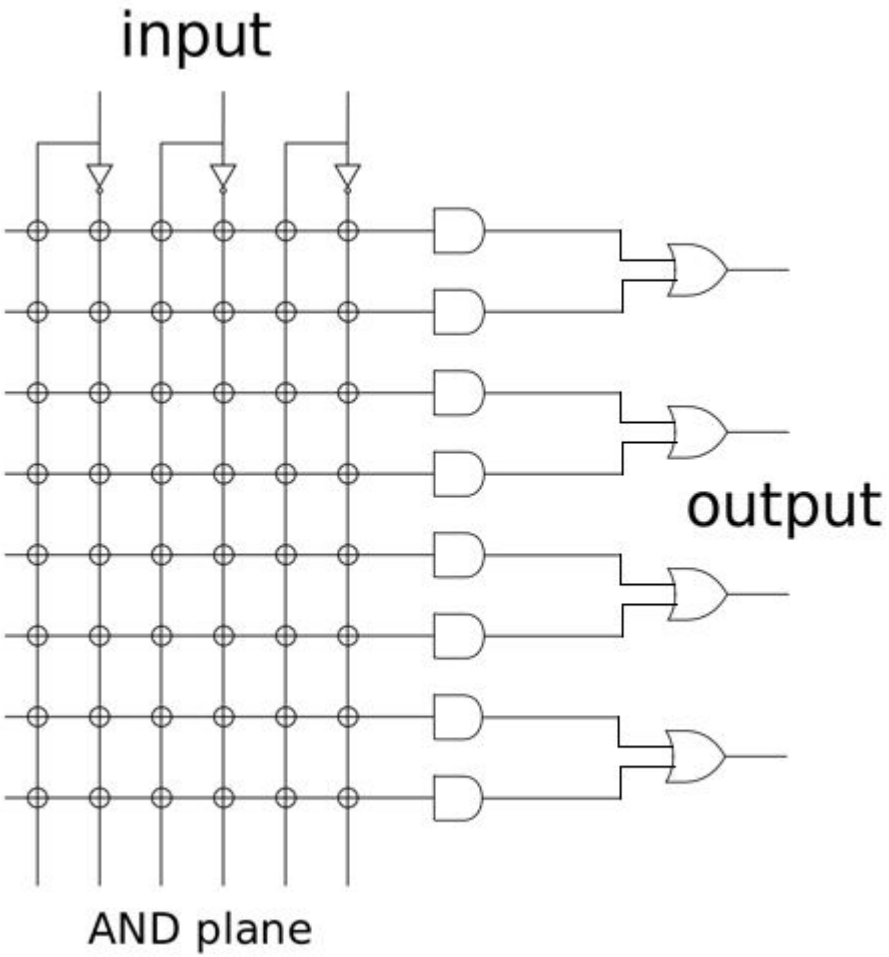
Programmable Logic Array (PLA)

- Dwie matryce
- Mała wielkość
- Układ powolny
- Np.
  - 906114-01 PLA
    - użyte w Commodore 64



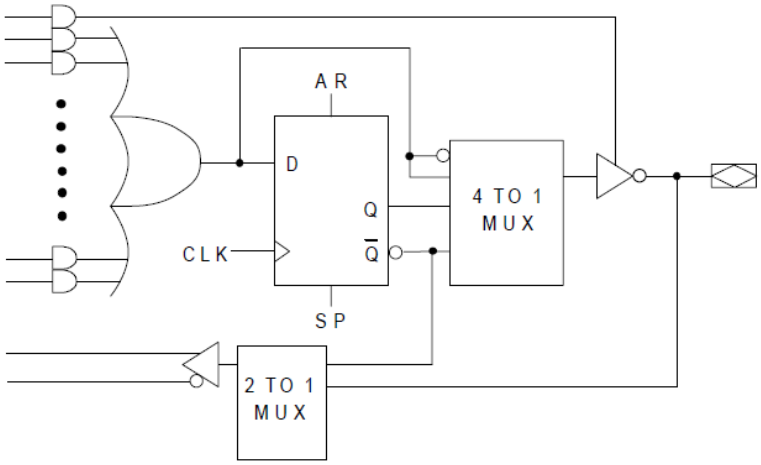
Programmable Array Logic (PAL)

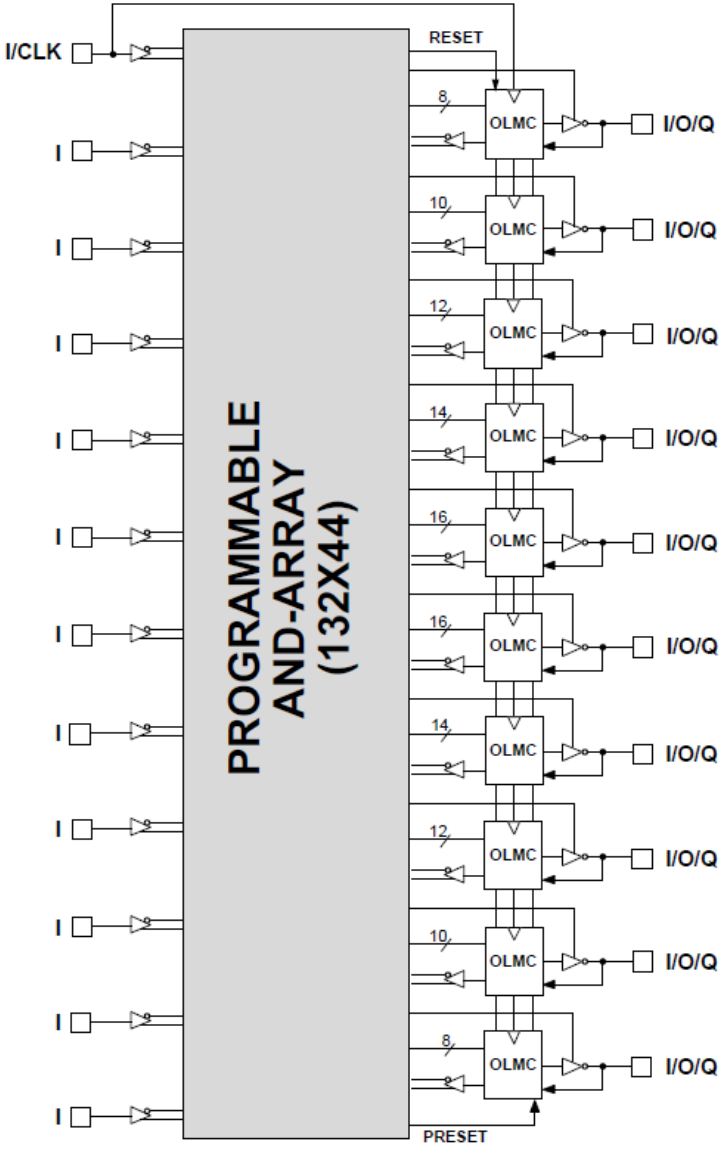
- Jedna matryca
  - Macierz OR na stałe
- Większe
- Rodzaje
  - Kombinacyjne
  - Rejestrowe
  - z makrokomórkami
- Np.
  - PAL 16R4
  - AMD 22V10



Generic/Gate Array Logic (GAL)

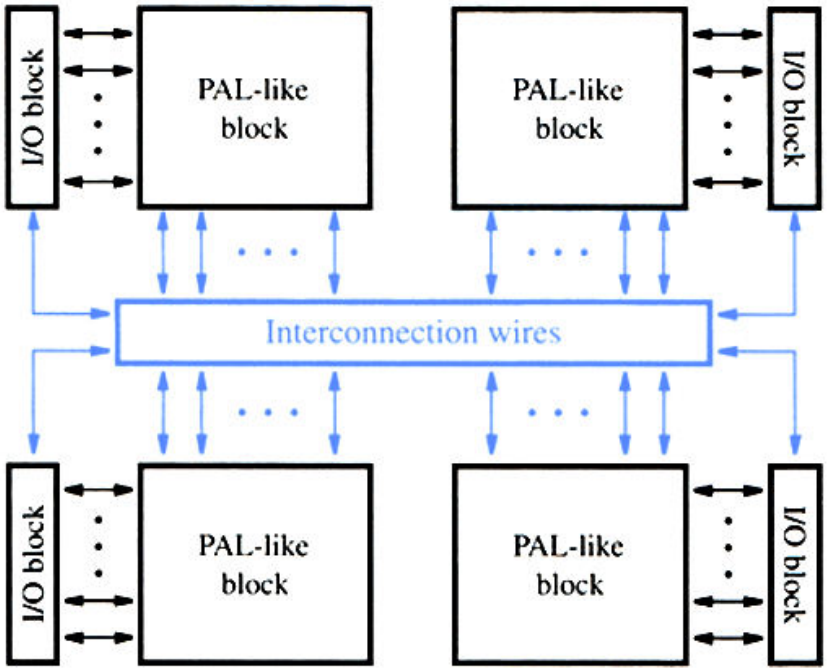
- **Reprogramowalne**
- Większe
  - Odpowiednik wielu układów PAL
  - Zbudowane z komórek
  - Rejestry wewnątrz
- Np. GAL16V8D-15LJ, GAL22V10



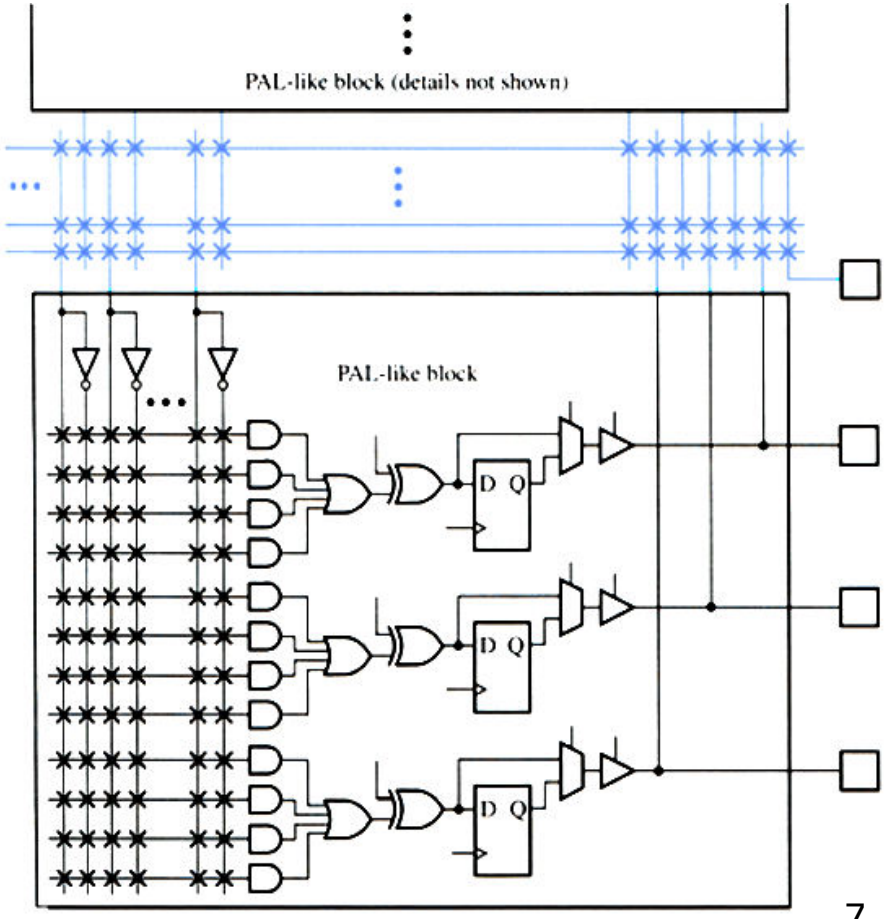


Complex Programmable Logic Device (CPLD)

- Pomiędzy GAL a FPGA
- Zbudowane z makrokomórek
- Pamięć stała
- Verilog!



Structure of a complex programmable logic device (CPLD)

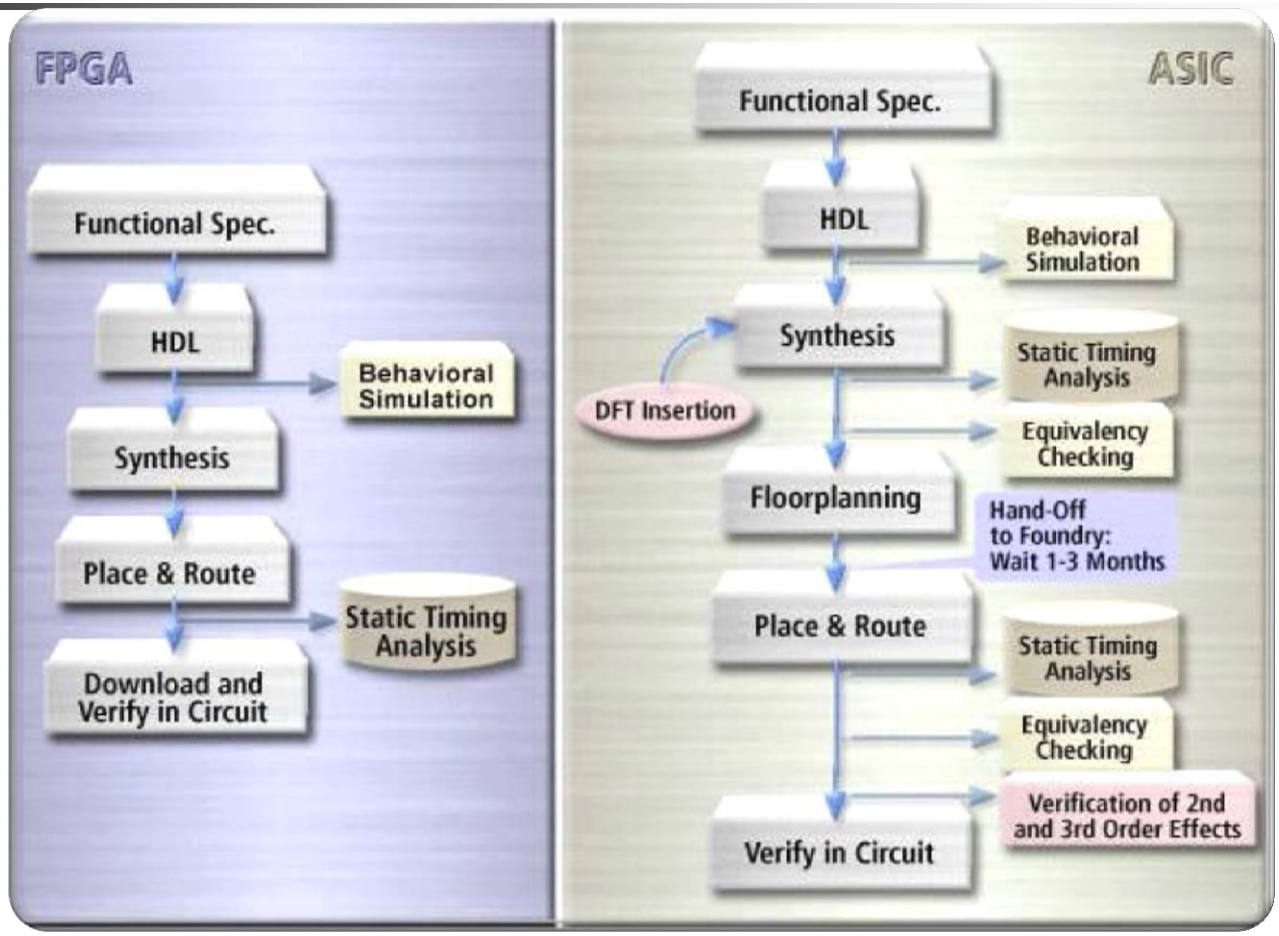


A section of CPLD

Field Progammable Gate Array (FPGA)

- Cechy
  - Symetryczna budowa
  - Duża pojemność (miliony bramek)
  - Łatwe opanowanie struktury układu (komórka podstawowa)
  - Duża ilość pinów I/O do komunikacji z zewnętrzem układu
- Zalety układów FPGA w porównaniu do układów ASIC
  - Szybkie projektowanie i prototypowanie
  - Możliwość zmiany/poprawy struktury układu w dowolnym memencie
  - Uproszczone projektowania obwodów drukowanych (PCB) wynikające z możliwości umiejscowienia pinów I/O w dowolnej konfiguracji
- Wady układów FPGA w porównaniu do układów ASIC
  - Większe zużycie energii
  - Wolniejszy maksymalny zegar taktujący układ
  - Wysoki koszt jednostkowy układu

FPGA vs. ASIC

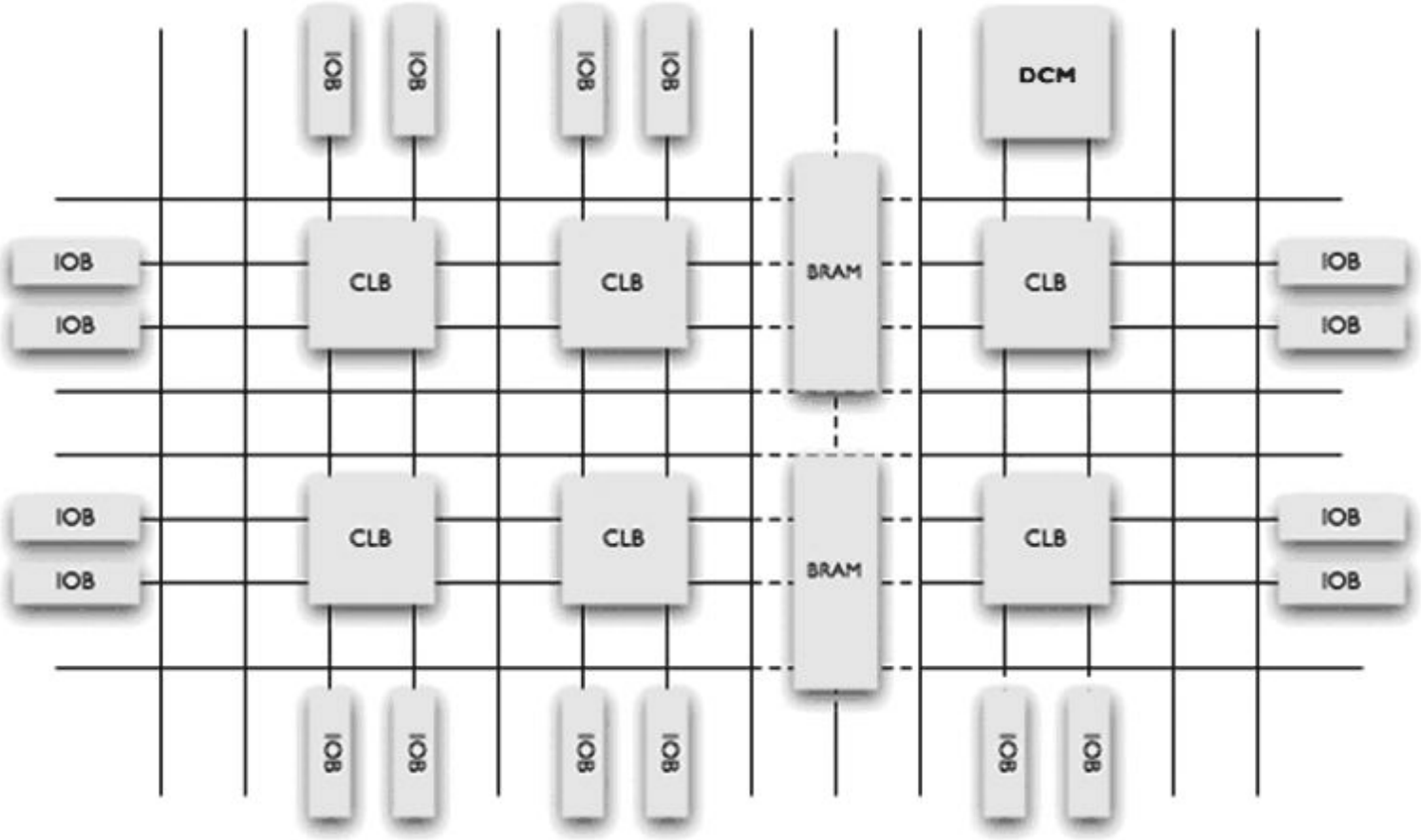


Field
Progammable
Gate
Array

Application
Specific
Integrated
Cicuits

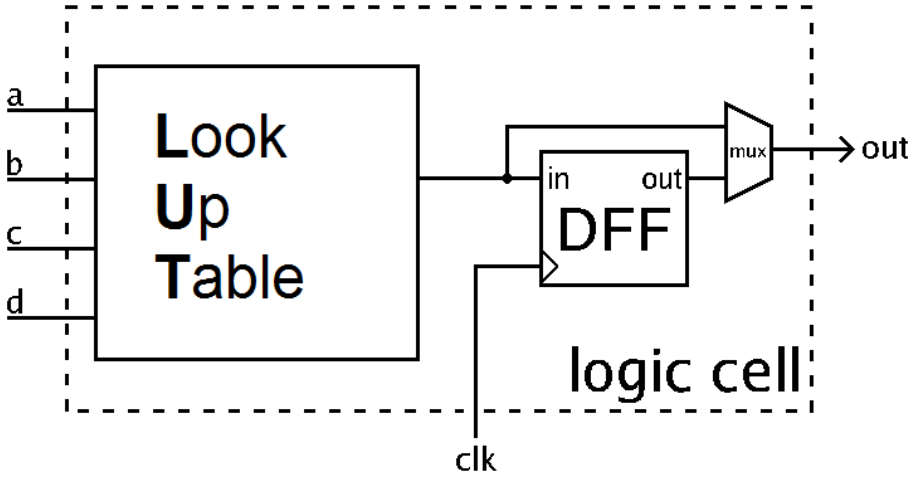
Układy FPGA – ogólna struktura

FPGA - Ogólna struktura



CLB (Configurable Logic Block)

- Komórka logiczna
  - Realizuje dowolne funkcje logiczne (kombinacyjne)
    - NIE używamy bramek
      - Nie używamy minimalizacji
        - De Karnaugh
        - Quine McCluskey
    - Bezpośrednie użycie tablic prawdy
      - LUT (Look Up Table)



  - Umożliwia zapisanie wyniku w przerzutniku/zatrzasku
  - Czasami posiada dodatkową funkcjonalność
    - Wsparcie implementacji licznika/sumatora
    - Rejestr przesuwny
    - Pamięć RAM/ROM

Tablice prawdy funkcji logicznych

Indeks {b,a}	Wejście		Wyjście
	b	**a**	**?**
0	0	0	0
1	0	1	0
2	1	0	0
3	1	1	1

Tablice prawdy funkcji logicznych

Indeks {b,a}	Wejście		Wyjście
	b	a	AND
0	0	0	0
1	0	1	0
2	1	0	0
3	1	1	1

Tablice prawdy funkcji logicznych

Indeks {b,a}	Wejście		Wyjście
	b	**a**	**?**
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1

Implementacja funkcji logicznych

Indeks {b,a}	Wejście		Wyjście
	b	a	OR
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1

Implementacja funkcji logicznych

Indeks {b,a}	Wejście		Wyjście		
	b	a	AND	OR	XOR
0	0	0	0	0	0
1	0	1	0	1	1
2	1	0	0	1	1
3	1	1	1	1	0

Większe tablice prawdy

Indeks {c,b,a}	Wejście			Wyjście
	c	b	a	F(c,b,a)
0	0	0	0	.
1	0	0	1	.
2	0	1	0	.
3	0	1	1	.
4	1	0	0	.
5	1	0	1	.
6	1	1	0	.
7	1	1	1	.

Jeszcze większe tablice prawdy

Indeks {d,c,b,a}	Wejście				Wyjście
	d	c	b	a	F(d,c,b,a)
0	0	0	0	0	.
1	0	0	0	1	.
2	0	0	1	0	.
3	0	0	1	1	.
4	0	1	0	0	.
5	0	1	0	1	.
6	0	1	1	0	.
7	0	1	1	1	.
8	1	0	0	0	.
9	1	0	0	1	.
10	1	0	1	0	.
11	1	0	1	1	.
12	1	1	0	0	.
13	1	1	0	1	.
14	1	1	1	0	.
15	1	1	1	1	.

Przykładowa tablica prawdy

Indeks {d,c,b,a}	Wejście				Wyjście
	d	c	b	a	F(d,c,b,a)
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	1
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	1

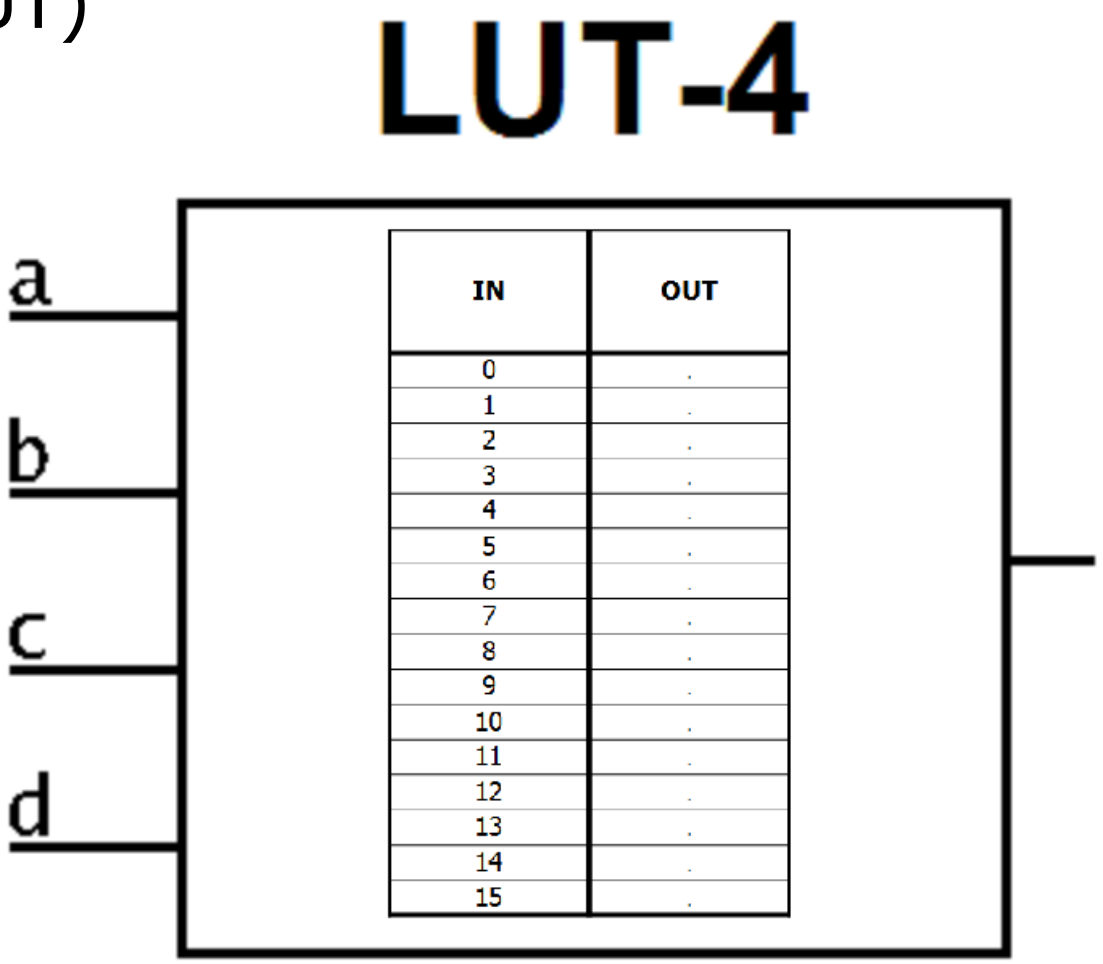
Przykład w C++

- Tablica z wartościami stałymi
- Odczytujemy wartość wyjściową spod indeksu określonego przez argumenty wejściowe

```
void get_value(int d,c,b,a) // 0 or 1
{
  int idx = (d<<3) | (c<<2) | (b<<1) | a; //0..15
  const int LUT[16] = {
    0, // [0]  = b0000
    1, // [1]  = b0001
    1, // [2]  = b0010
    1, // [3]  = b0011
    0, // [4]  = b0100
    0, // [5]  = b0101
    1, // [6]  = b0110
    0, // [7]  = b0111
    1, // [8]  = b1000
    0, // [9]  = b1001
    1, // [10] = b1010
    1, // [11] = b1011
    1, // [12] = b1100
    0, // [13] = b1101
    0, // [14] = b1110
    1  // [15] = b1111
  };
  return LUT[idx];
}
```

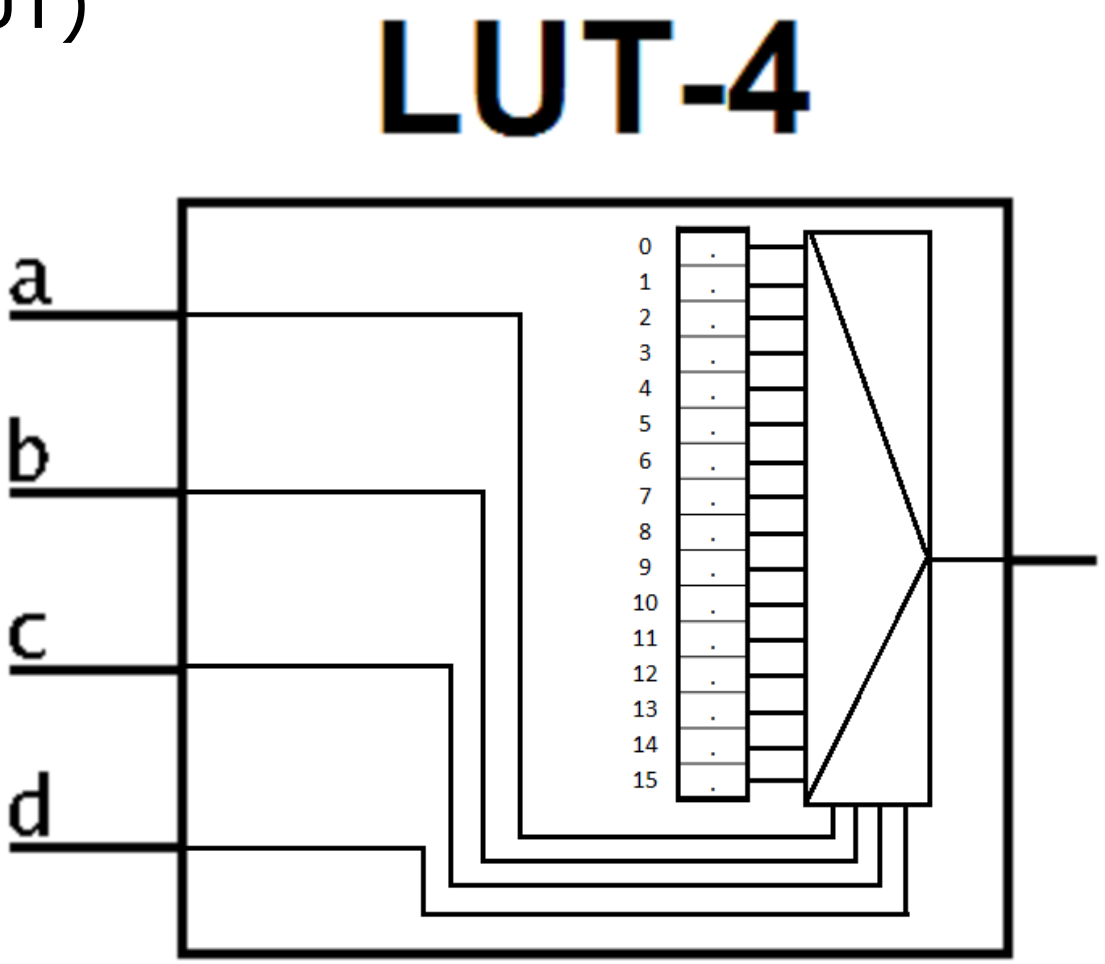
Implementacja tablic prawdy

- Look Up Table (LUT)
- Implementuje tablicę prawdy
- Dla danego indeksu {d,c,b,a} odczytuje wartość f(d,c,b,a)
- Pamięć ROM czytająca adres {d,c,b,a}



Implementacja tablic prawdy

- Look Up Table (LUT)
- Implementuje tablicę prawdy
- Dla danego indeksu
  {d,c,b,a}
  odczytuje wartość
  f(d,c,b,a)
- Multiplekser sterowany przez
  {d,c,b,a}



Jaki rozmiar bazowy LUT?

- Jaki rozmiar LUT powinien być używany w układzie FPGA?
  - Jaki minimalny?
  - Jaki maksymalny?
  - Jaki optymalny?

- Przydanej wielkości LUT
  - Jak zaimplementować większe funkcje logiczne?

Jaki rozmiar bazowy LUT? 1?

- Możliwe wyjścia:
  - Bez zmian
  - Negacja
  - Nie da się zrobić z tego funkcji N-wejściowej
- **Za mało!**

Wejście: a	Wyjście: a	Wyjście: ~a
0	0	1
1	1	0

Jaki rozmiar bazowy LUT? 2?

- Możliwe wyjścia
  - Wszystkie 1 wejściowe
  - Wszystkie bramki 2 wejściowe
- **System funkcjonalnie pełny**
  - NOT AND OR
    - Suma lub iloczyn - może być wyeliminowana dzięki prawu De Morgana
  - NAND
  - NOR
  - **LUT-2**

Indeks {b,a}	Wejście		Wyjście				
	b	a	~a	&	|	^	...
0	0	0	1	0	0	0	...
1	0	1	0	0	1	1	...
2	1	0	1	0	1	1	...
3	1	1	0	1	1	0	...

Jaki rozmiar bazowy LUT? 2?

- Możliwe wyjścia
  - Wszystkie 1 wejściowe
  - Wszystkie bramki 2 wejściowe
- **System funkcjonalnie pełny**
  - Ale wymaga minimalizacji dla większych funkcji
    - de Karnaugh
    - Quine McCluskey
  - **Brak możliwości wykorzystania tablic prawdy wprost**

Indeks {b,a}	Wejście		Wyjście				
	b	a	~a	&	|	^	...
0	0	0	1	0	0	0	...
1	0	1	0	0	1	1	...
2	1	0	1	0	1	1	...
3	1	1	0	1	1	0	...

Jaki rozmiar bazowy LUT? 3?

- Możliwe wyjścia
  - Wszystkie funkcje logiczne 1-3 wejściowe
- **System funkcjonalnie pełny**
- Jak zbudować większe funkcje logiczne?

Indeks {d,c,b,a}	Wejście			Wyjście
	c	b	a	F(d,c,b,a)
0	0	0	0	D0
1	0	0	1	D1
2	0	1	0	D2
3	0	1	1	D3
4	1	0	0	D4
5	1	0	1	D5
6	1	1	0	D6
7	1	1	1	D7

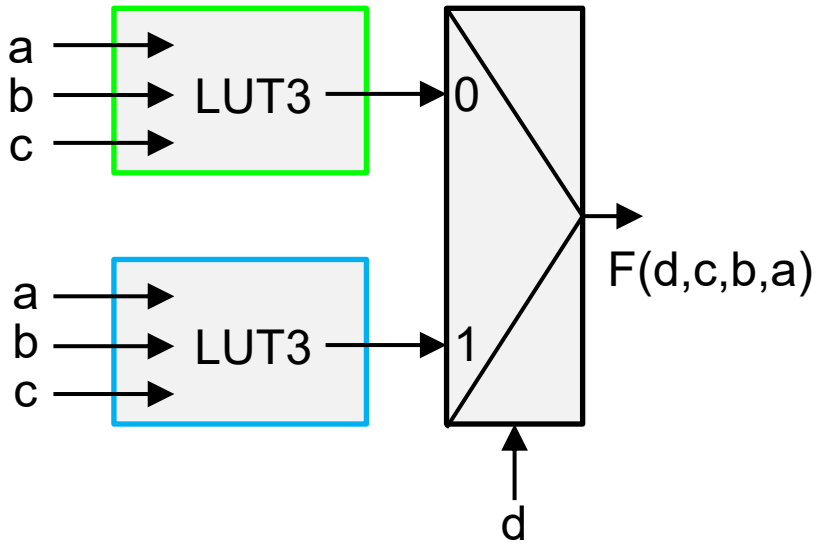
Jaki rozmiar bazowy LUT? 3?

- Jak zbudować LUT4 z LUT3 ?
  - Komórki 0..7 → LUT3
    - Daje poprawną odpowiedź gdy d=0
    - Możliwe że kłamie, gdy d=1
  - Komórki 8..15 → LUT3
    - Daje poprawną odpowiedź gdy d=1
    - Możliwe że kłamie, gdy d=0
  - Poprawny wynik?
    - Jeśli d=0: weź zielony
    - Jeśli d=1: weź niebieski
    - Multiplekser sterowany przez d

Indeks {d,c,b,a}	Wejście				Wyjście
	d	c	b	a	F(d,c,b,a)
0	0	0	0	0	D0
1	0	0	0	1	D1
2	0	0	1	0	D2
3	0	0	1	1	D3
4	0	1	0	0	D4
5	0	1	0	1	D5
6	0	1	1	0	D6
7	0	1	1	1	D7
8	1	0	0	0	D8
9	1	0	0	1	D9
10	1	0	1	0	D10
11	1	0	1	1	D11
12	1	1	0	0	D12
13	1	1	0	1	D13
14	1	1	1	0	D14
15	1	1	1	1	D15

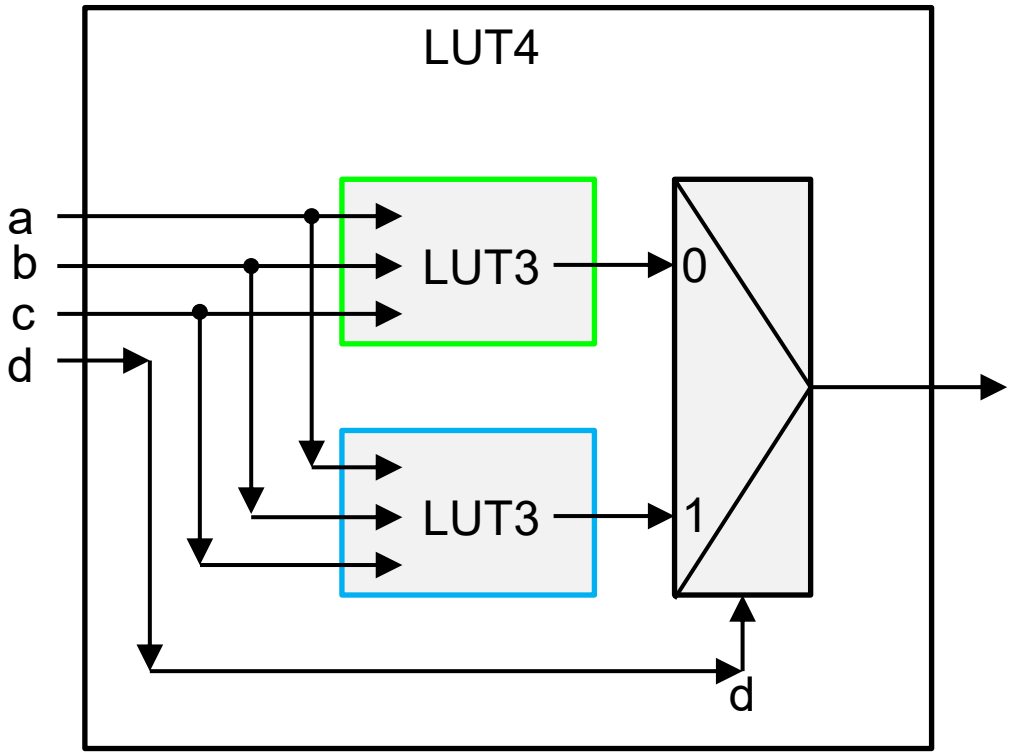
Jaki rozmiar bazowy LUT? 3?

- Jak zbudować LUT4 z LUT3 ?
  - Komórki 0..7 → LUT3
    - Daje poprawną odpowiedź gdy d=0
    - Możliwe że kłamie, gdy d=1
  - Komórki 8..15 → LUT3
    - Daje poprawną odpowiedź gdy d=1
    - Możliwe że kłamie, gdy d=0
  - Poprawny wynik?
    - Jeśli d=0: weź zielony
    - Jeśli d=1: weź niebieski
    - Multiplekser sterowany przez d



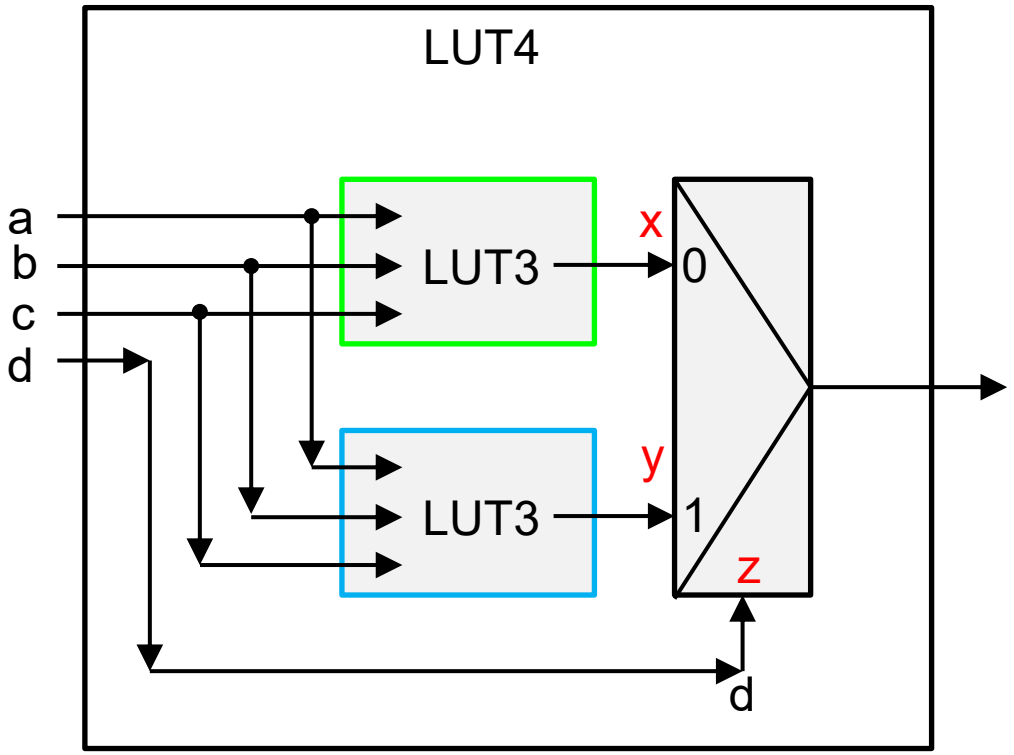
Jaki rozmiar bazowy LUT? 3?

- Jak zbudować LUT4 z LUT3 ?
  - 2 × LUT3
  - 1 × Multiplekser



Jaki rozmiar LUT? 3?

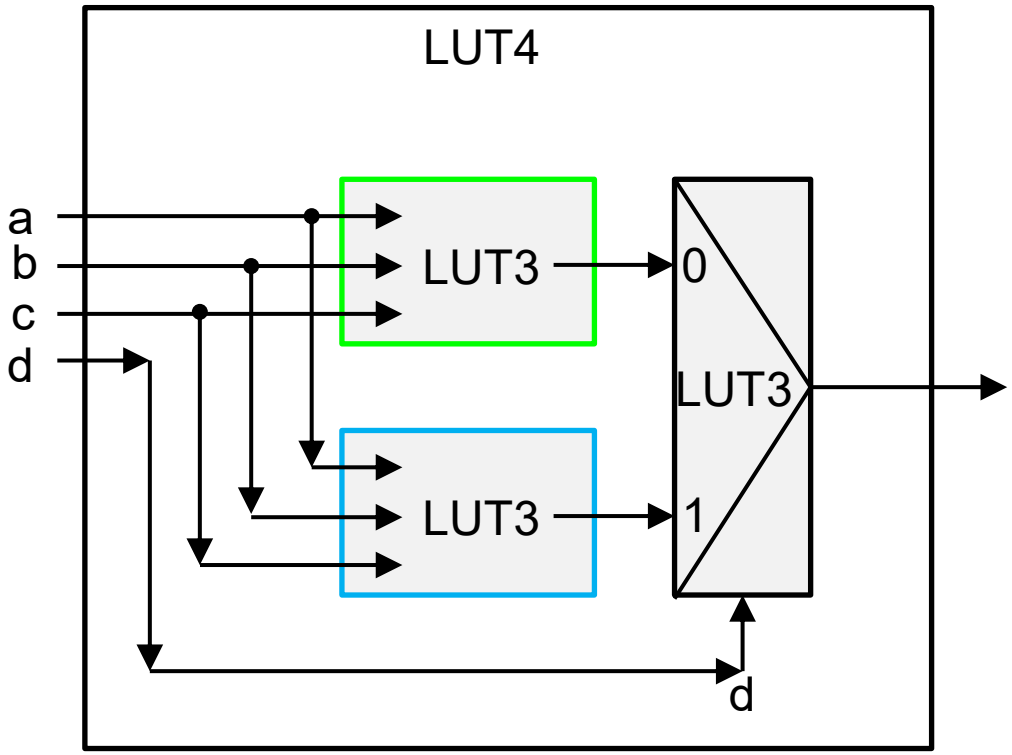
- Jak zbudować LUT4 z LUT3 ?
  - 2 × LUT3
  - 1 × Multiplekser
    - Też LUT3 !!!



Indeks {d,c,b,a}	Wejście			Wyjście
	z	y	x	F(z,y,x)
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

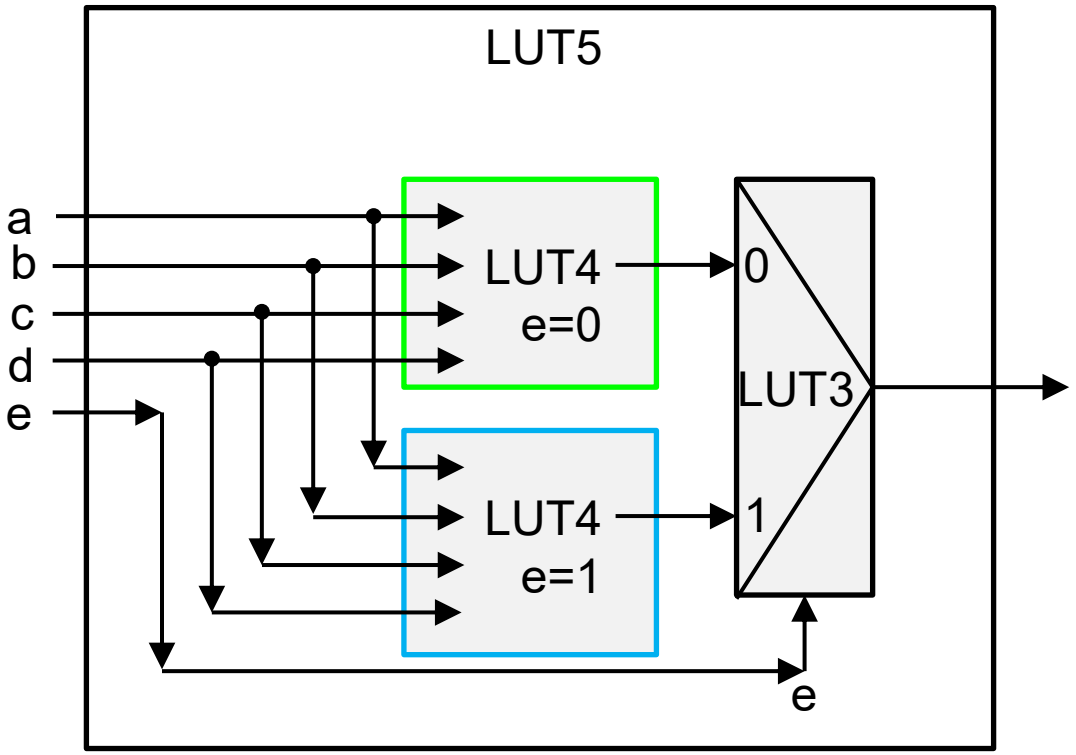
Jaki rozmiar bazowy LUT? 3?

- Jak zbudować LUT4 z LUT3 ?
  - 2 × LUT3
  - 1 × LUT3 jako mux
  - Głębokość
    - +1 poziom logiki



Jaki rozmiar bazowy LUT? 3? 4?

- Jak zbudować LUT5 z LUT4 ?
  - 2 × LUT4
  - 1 × LUT3 jako mux
  - Głębokość
    - +1 poziom logiki



Jaki rozmiar bazowy LUT? 3,4,5?

- Jak zbudować LUT_{N+1} z LUT_N ?
  - $2 \times LUT_N$
  - $1 \times LUT_3$ jako mux
  - Głębokość
    - +1 poziom logiki
  - Więcej poziomów logiki

⇓

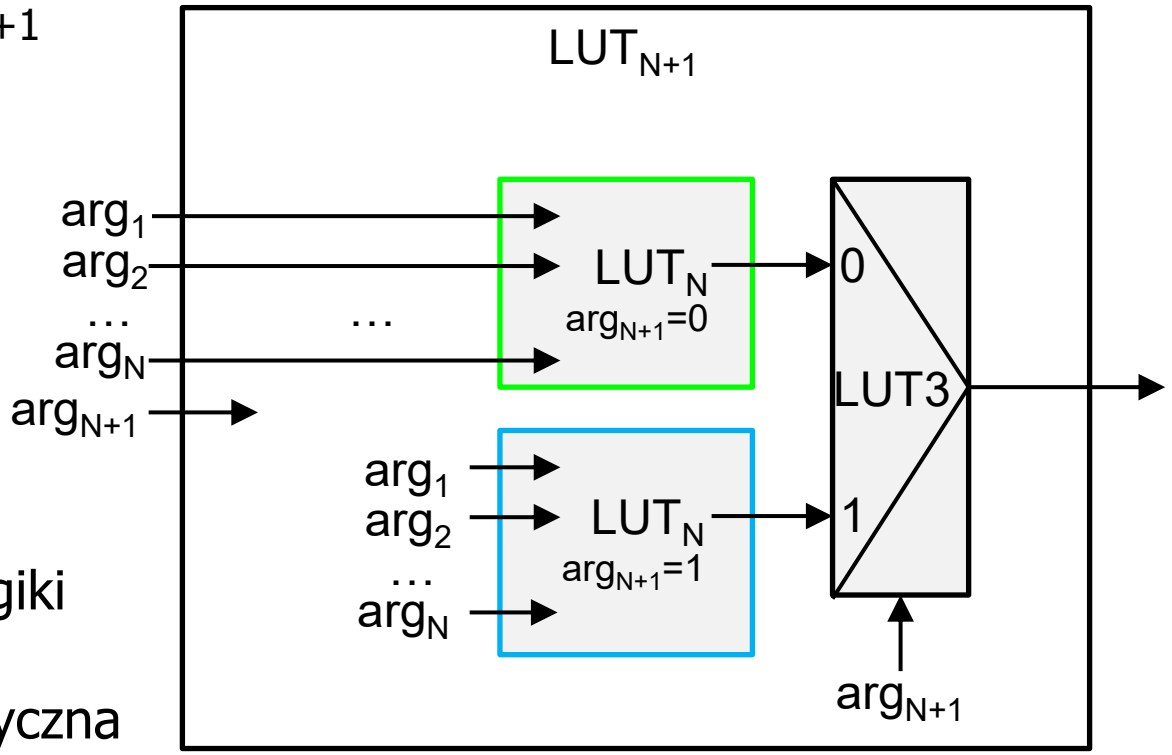
Dłuższa ścieżka krytyczna

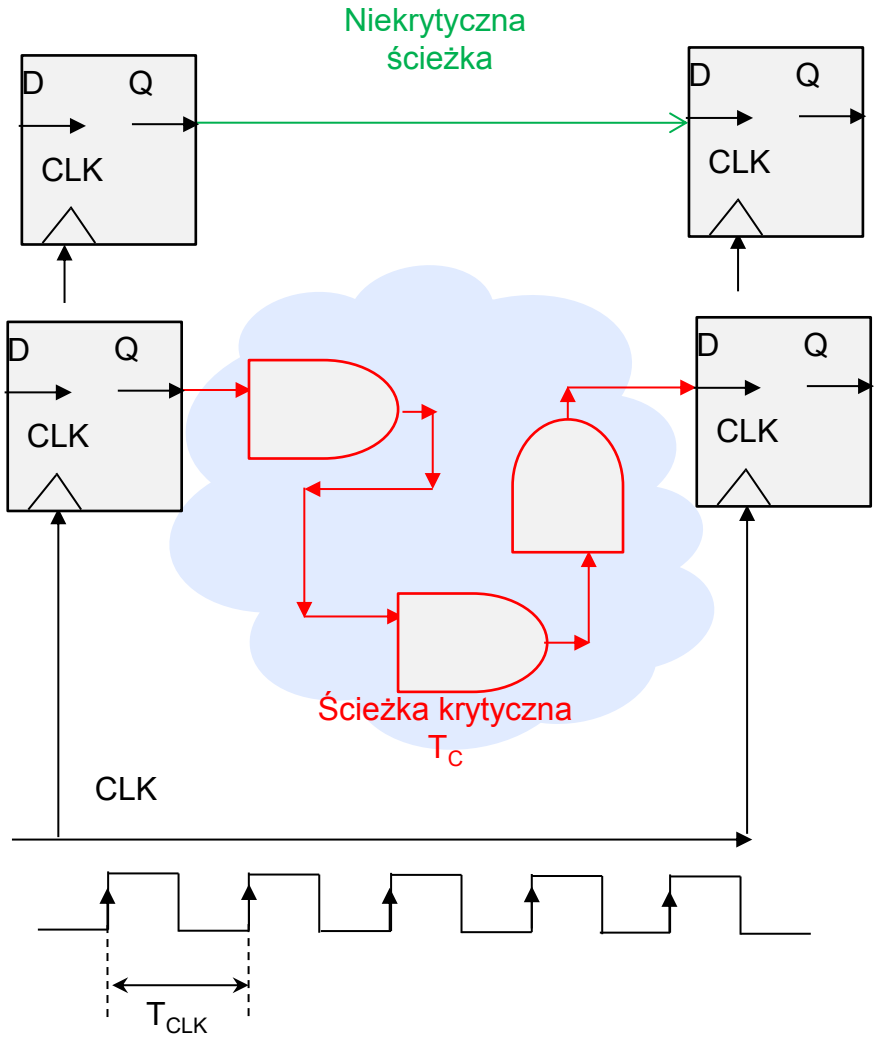
⇓

Niższa częstotliwość sygnału taktującego

⇓

Niższa prędkość





$$T_{CLK}[ns] > T_c[ns]$$

$$f_{CLK}[GHz] < \frac{1}{T_c[ns]}$$

Przykład:

$$T_c = 10\text{ns}$$

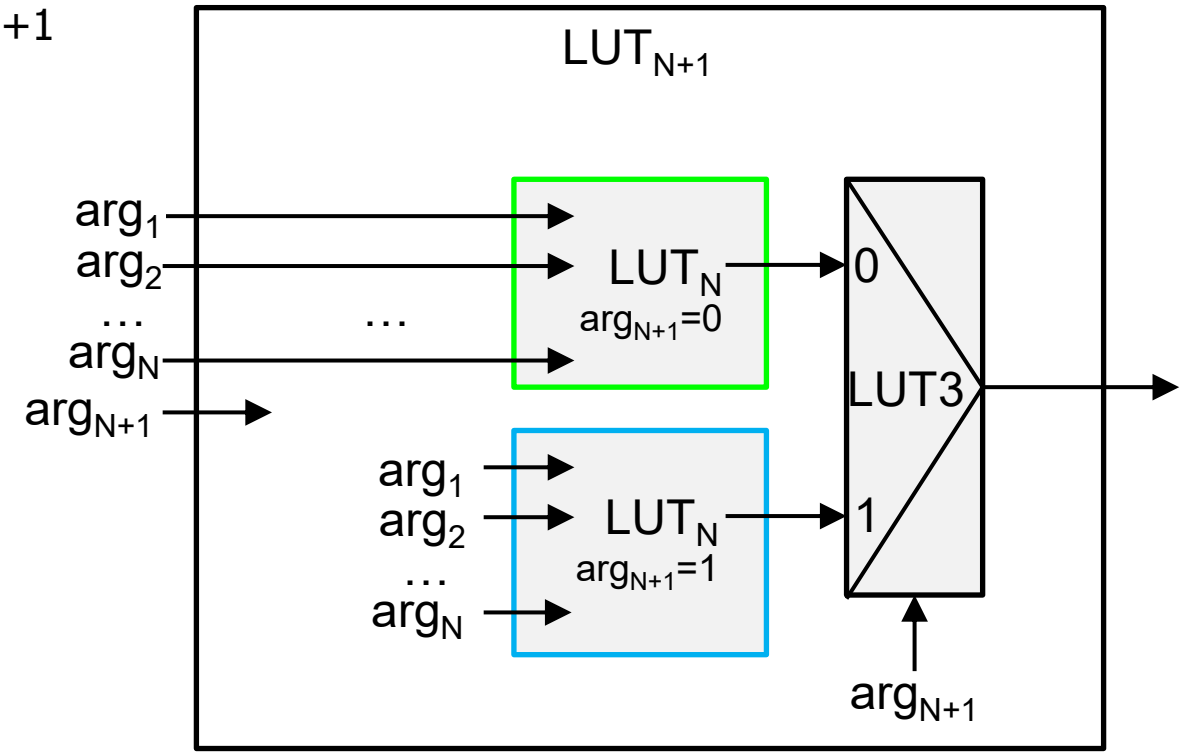
$$f_{CLK} < \frac{1}{10\text{ns}}$$

$$f_{CLK} < 0{,}1\text{GHz}$$

$$f_{CLK} < 100\text{MHz}$$

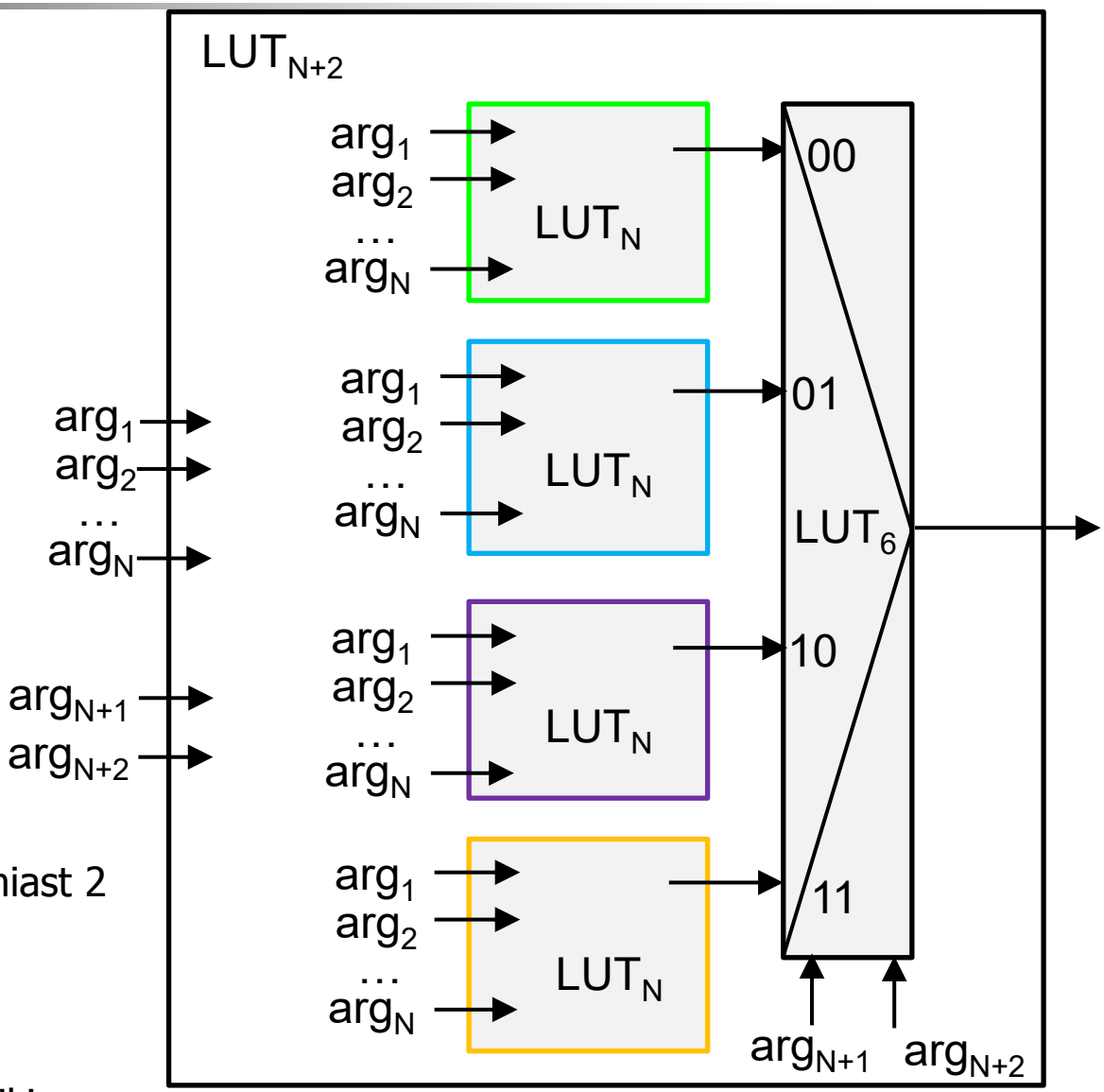
Jaki rozmiar bazowy LUT? 3,4,5?

- Jak zbudować LUT_{N+1} z LUT_N ?
  - $2 \times LUT_N$
  - $1 \times LUT_3$ jako mux
  - Głębokość
    - +1 poziom logiki
  - Czy coś dałby nam LUT_4 jako mux?
    - Nie
  - LUT_5 ?
    - Nie
  - LUT_6 ?



Jaki rozmiar bazowy LUT? 6?

- Jak zbudować LUT_{N+2} z LUT_N ?
  - $4 \times LUT_N$
  - $1 \times LUT_6$ jako mux
  - Głębokość
    - +1 poziom logiki
  - Czy coś dałby nam LUT_6 jako mux?
    - Tak!
    - Możemy składać 4 elementy bazowe zamiast 2
    - Dodajemy 2 wejścia zamiast 1
    - Tracimy tylko 1 MUX
    - Tylko +1 poziomów logiki



Wielkość realizacji funkcji logicznej

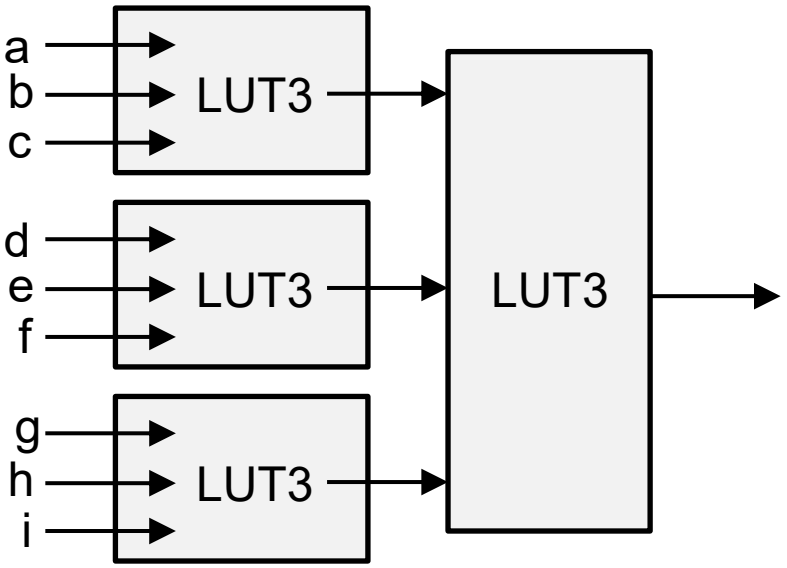
Bazowy LUT / Arg. funkcji logicznej	LUT3		LUT4		LUT5		LUT6	
	Zużytych elem.	Poziomy logiki	Zużytych elem.	Poziomy logiki	Zużytych elem.	Poziomy logiki	Zużytych elem.	Poziomy logiki
3	1	1	1	1	1	1	1	1
4	3	2	1	1	1	1	1	1
5	7	3	3	2	1	1	1	1
6	15	4	7	3	3	2	1	1
7	31	5	15	4	7	3		
8	63	6	31	5	15	4	5	2
9	127	7	63	6	31	5		
10	255	8	127	7	63	6	21	3
+1	×2 + 1	+1	×2 + 1	+1	×2 + 1	+1		
+2	×4 + 3	+2	×4 + 3	+2	×4 + 3	+2	×4 + 1	+1

Wielkość realizacji funkcji logicznej

- Przedstawione liczby są pesymistyczne
  - Wielokrotnie funkcja logiczna jest „prosta”
  - Możliwa minimalizacja
    - Mniejsza liczba elementów
    - Mniejsza liczba poziomów logiki
  - Musimy jednak taką pesymistyczną prognozę brać pod uwagę

Wielkość realizacji funkcji logicznej

- Prognoza optymistyczna
  - Musimy mieć tyle wejść ile argumentów ma nasza funkcja logiczna
  - Wyniki musimy zebrać wyniki pośrednie do jednego bitu
  - Przykład:
    - 9-bitowa funkcja OR
    - Sprawdza czy którykolwiek bit jest równy 1
    - Realizacja na LUT3:
      $3 \times LUT_3 + 1 \times LUT_3$ na zebranie wyników pośrednich
      $= 4 \times LUT_3$ oraz 2 poziomy logiki
    - Zamiast (wg. tabelki):
      $127 \times LUT_3$ oraz 7 poziomów logiki



Jaki rozmiar bazowy LUT?

- Klient vs Producent FPGA vs Klient
  - W interesie klienta jest mieć
    - Jak największe tablice LUT
    - Najtańszy układ FPGA
  - Producent FPGA musi znaleźć złoty środek
    - Typowe wielkości funkcji logicznych
      - Proste warunki STB, ACK, READY - 2-3 argumenty
      - Proste multipleksery - 3-6 argumentów
      - Komparator liczby 8-bit. ze stałą - 8 argumentów
      - Komparator dwóch liczb 8-bit. - 16 argumentów
      - Sumator pełny 1-bitowy - 3 argumenty
    - Praktyczność wielkości LUT
      - Poniżej LUT4 jest więcej kłopotu z doprowadzeniem sygnałów wejściowych do LUT niż zysku z liczenia w nim
      - Powyżej LUT6 spora redundancja – typowe funkcje używają 2-4 argumentów a nie 6+

Dziękuje za uwagę

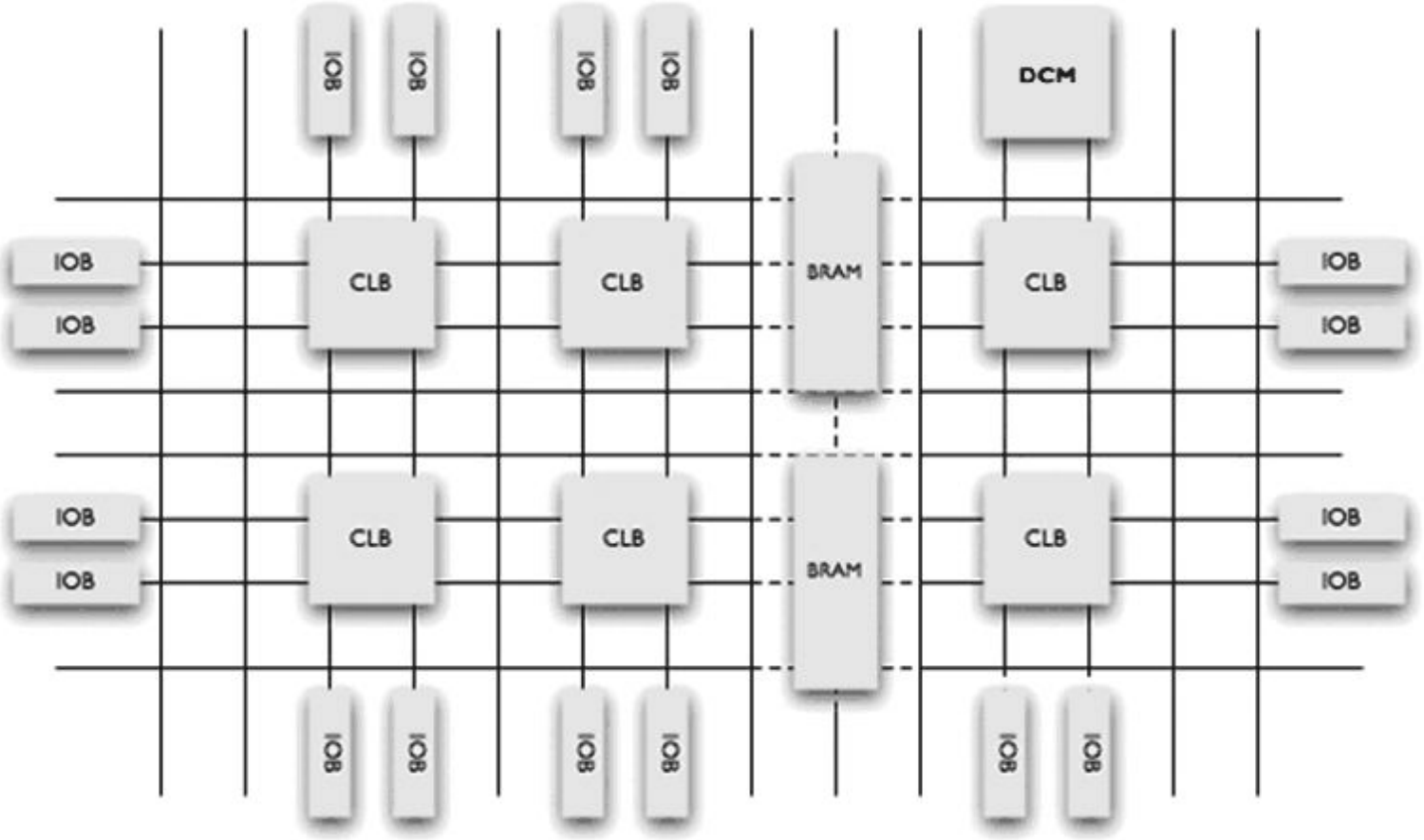
- Pytania?

Układy FPGA: cz. 2

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

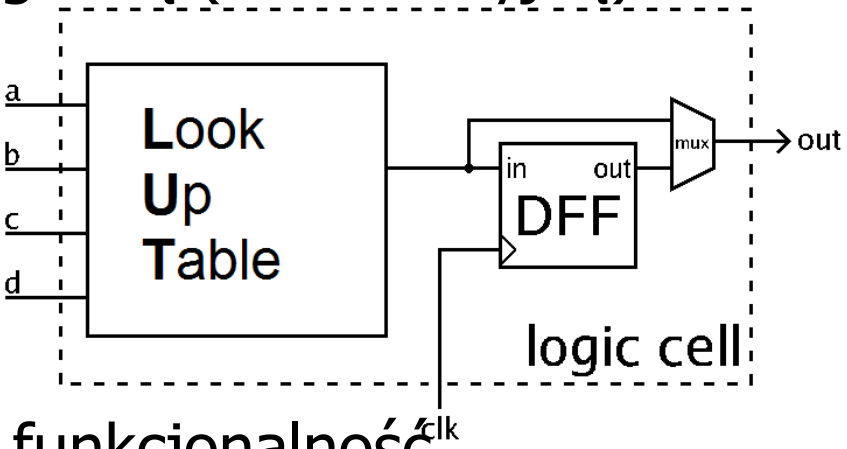
Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

FPGA - Ogólna struktura



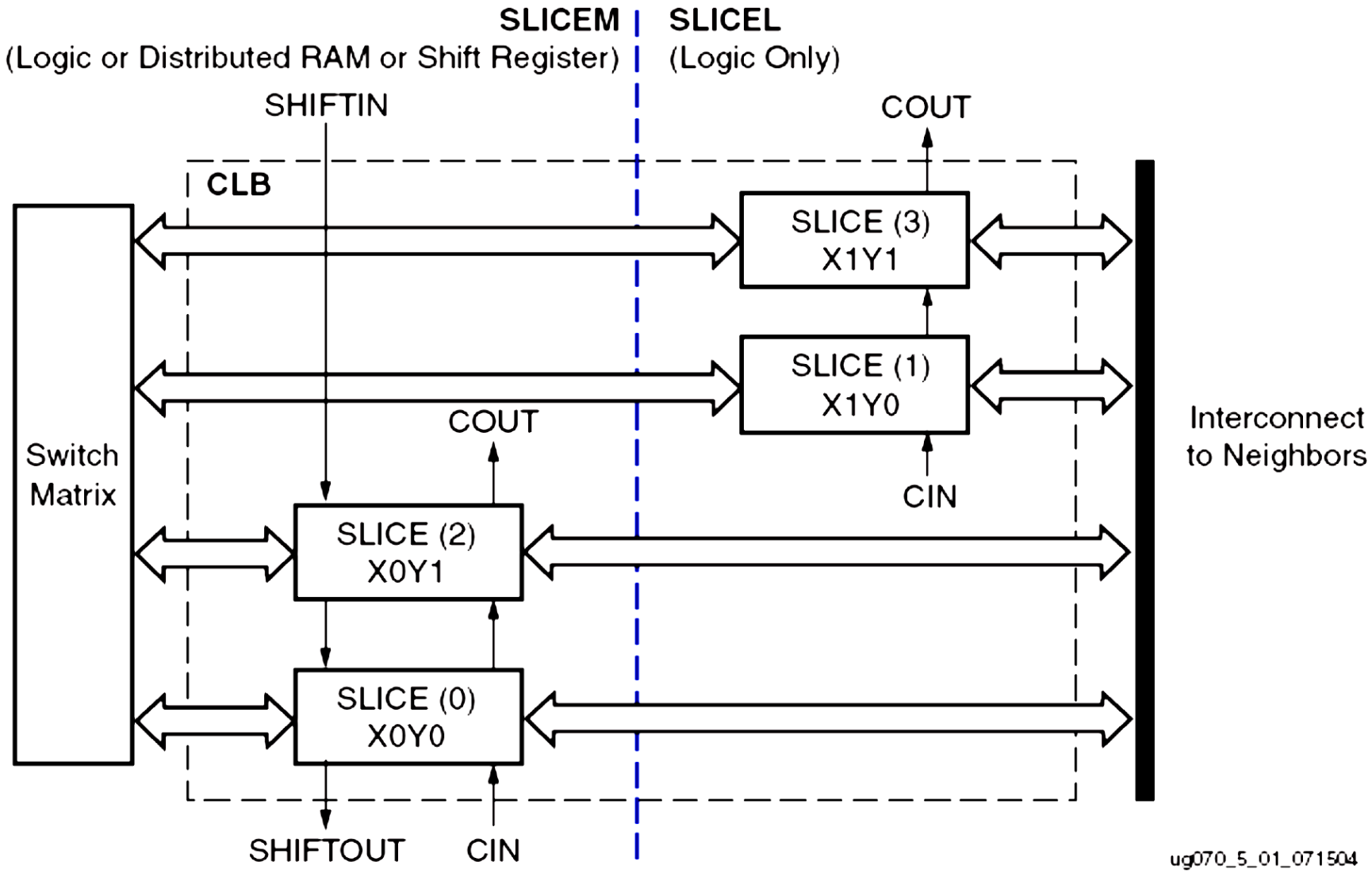
CLB (Configurable Logic Block)

- W środku znajdują się tablice LUT
  - Realizuje dowolną funkcję logiczną (kombinacyjną)
    - Bezpośrednie użycie tablic prawdy (Look Up Table)
      - LUT4 lub LUT6
  - Umożliwia zapisanie wyniku w rejestrze/zatrzasku

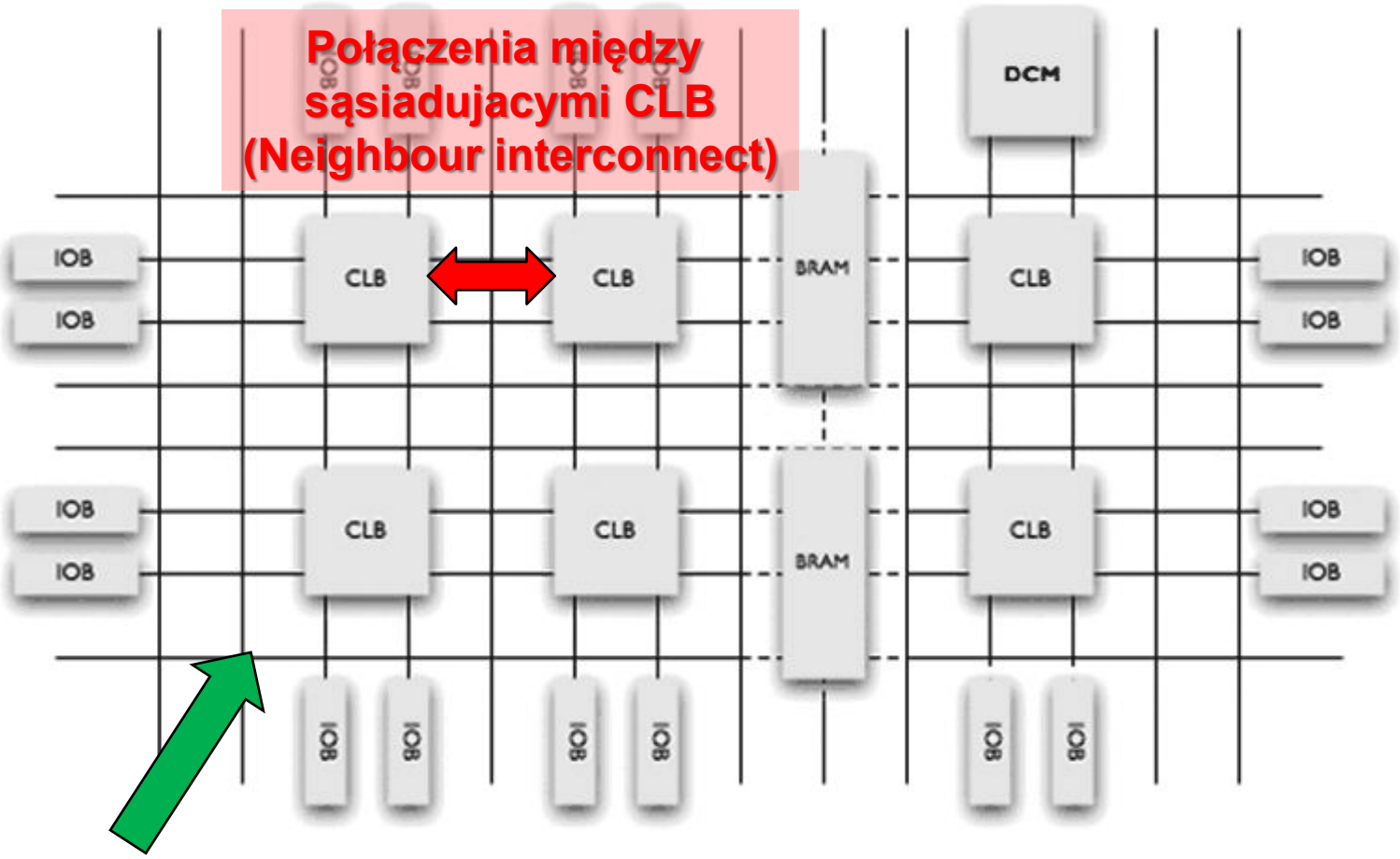


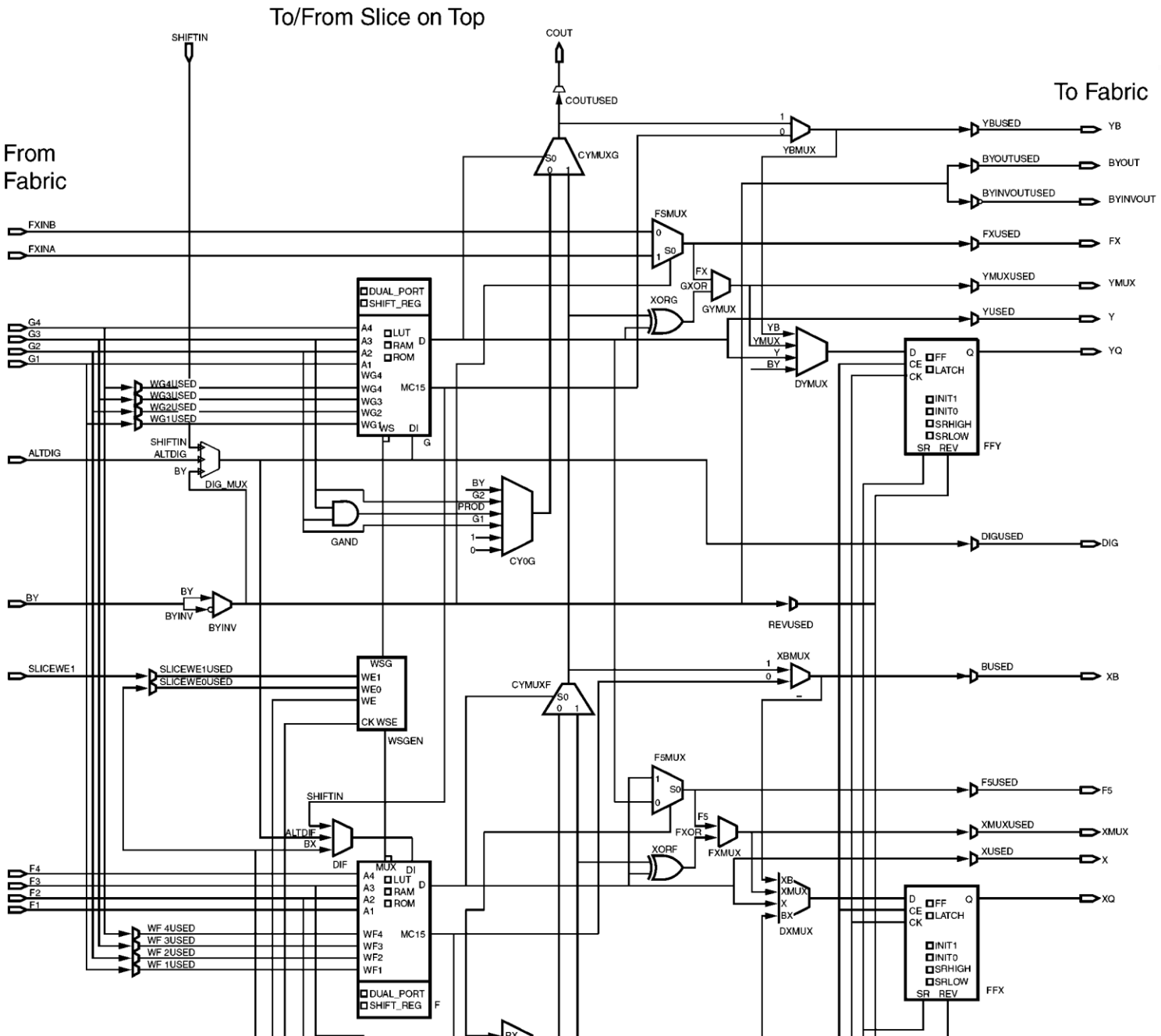
  - Czasami posiada dodatkową funkcjonalność
    - Wsparcie implementacji licznika/sumatora
    - Rejestr przesuwny
    - Pamięć RAM/ROM
  - Pojedyncza komórka CLB zawiera wiele tablic LUT
  - Szczegóły zależą od konkretnego układu/producenta
    - Xilinx, Lattice, Altera/Intel…

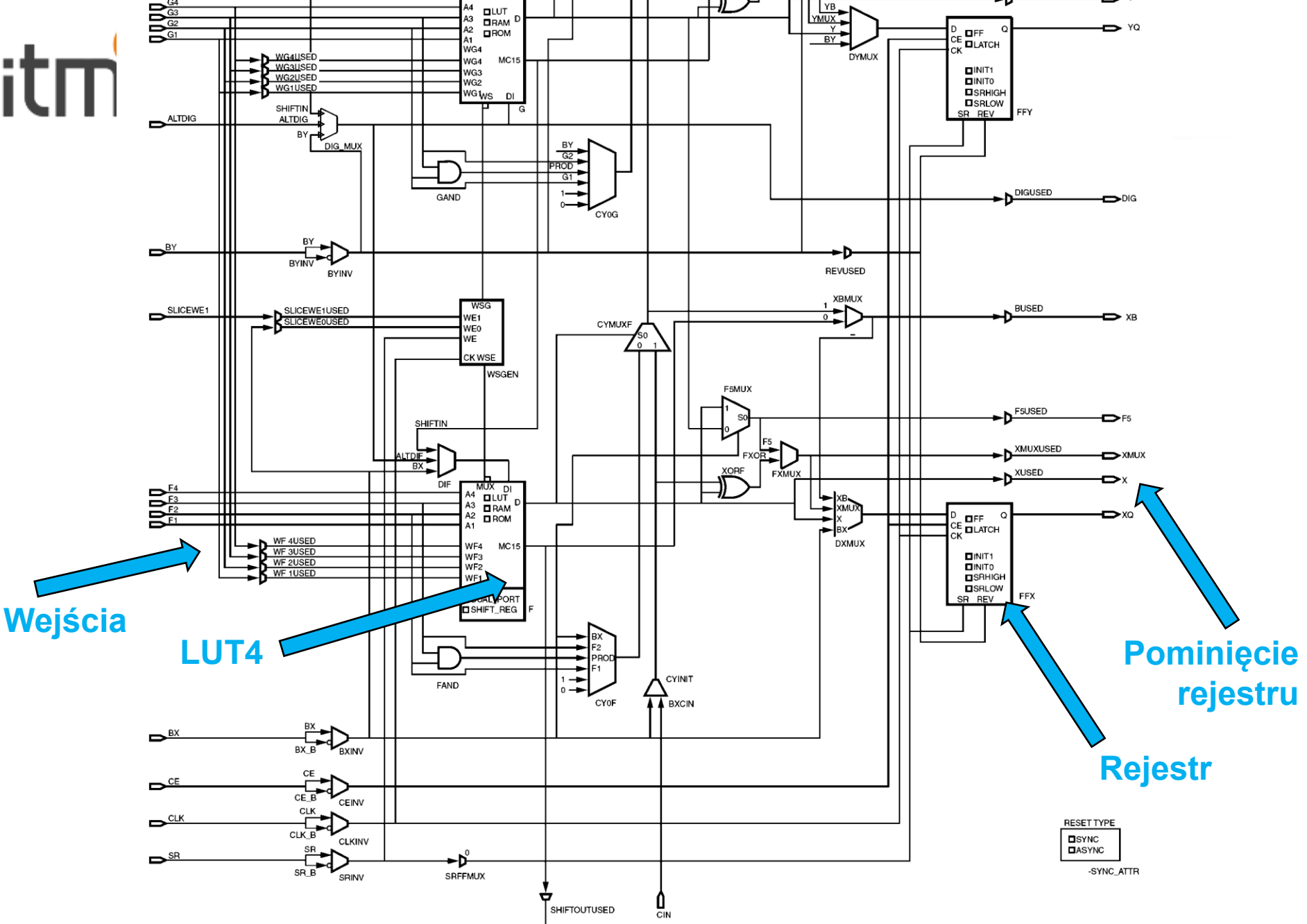
XILINX – zasoby asymetryczne



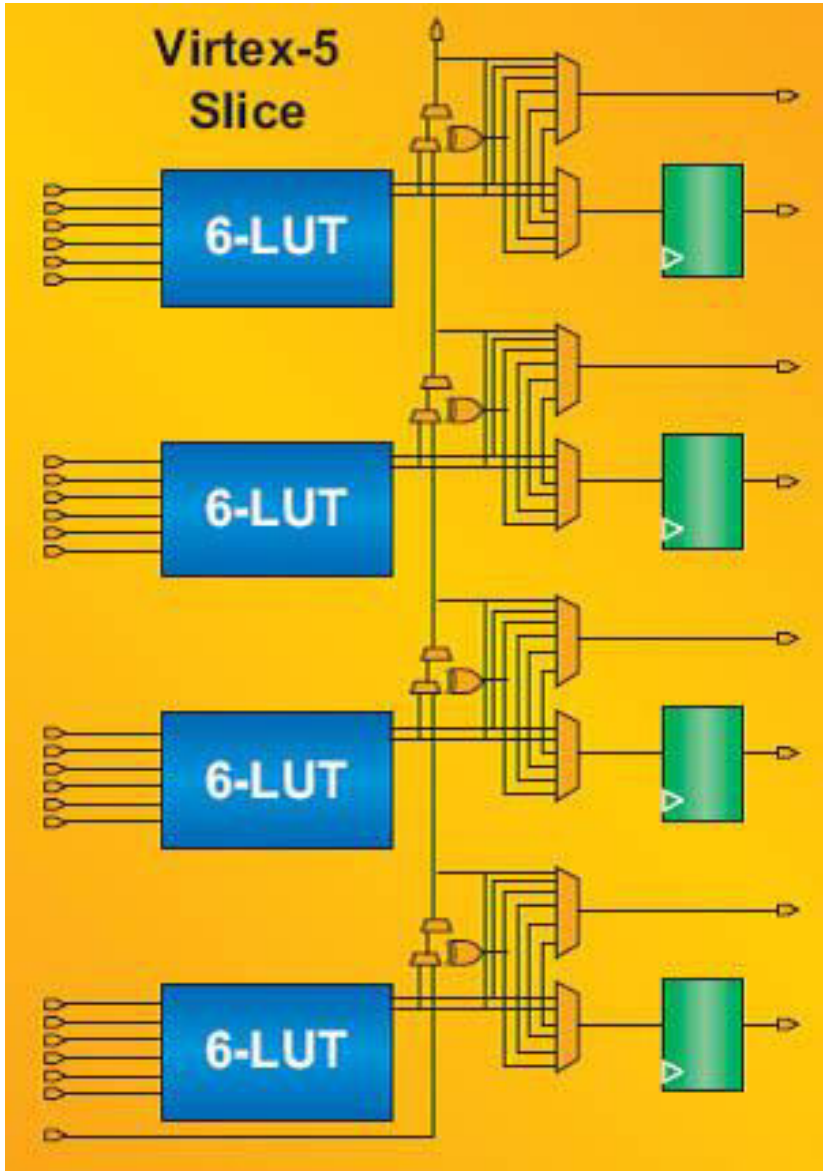
FPGA - Ogólna struktura

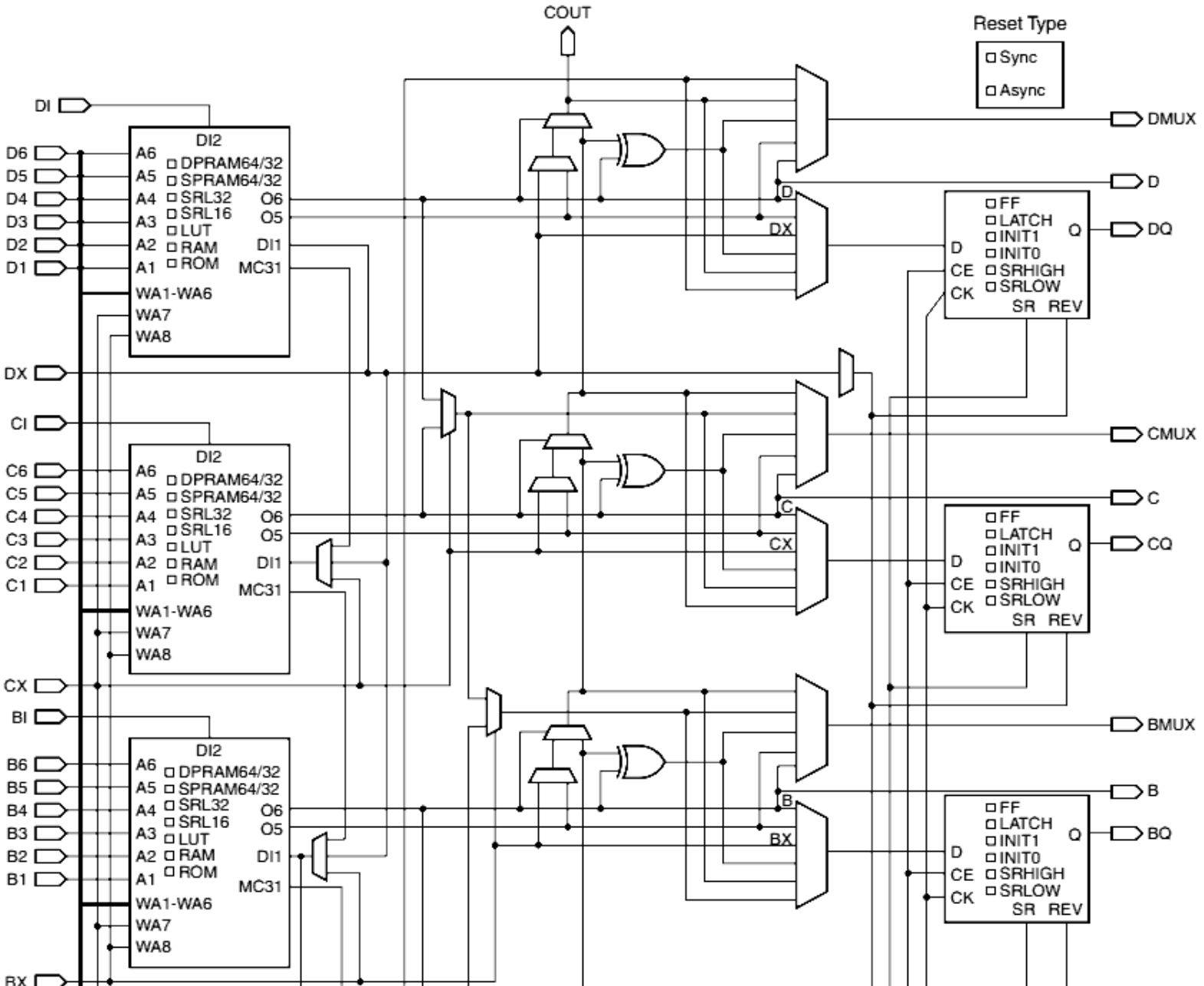


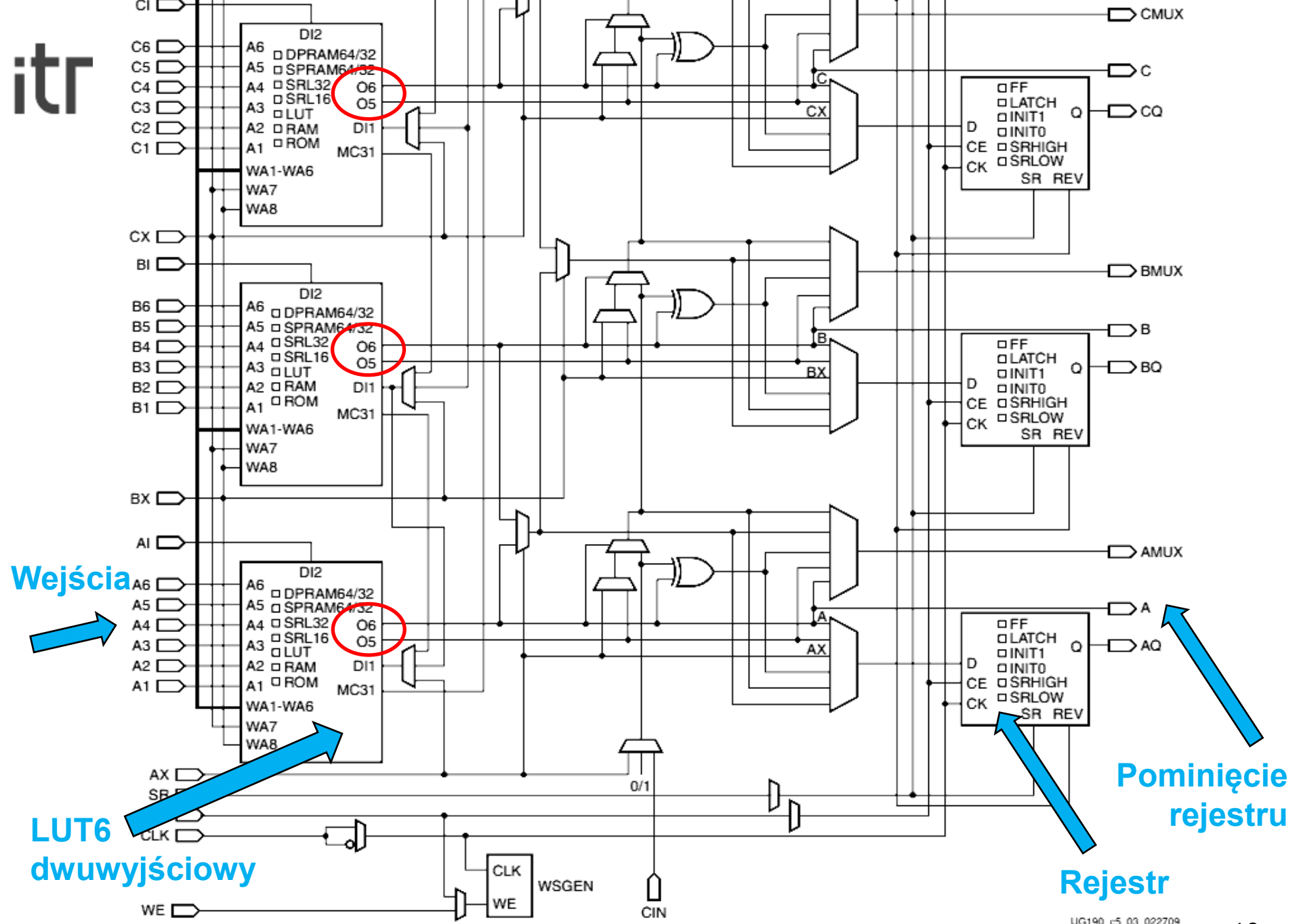

To/From Slice on Top
From Fabric
To Fabric
SHIFTIN
COUT
COUTUSED
CYMUXG
YBMUX
YBUSED
YB
BYOUTUSED
BYOUT
BYINVOUTUSED
BYINVOUTUSED
FXINB
FXINA
FSMUX
FXUSED
FX
GXOR
XORG
GYMUX
YMUXUSED
YMUX
YUSED
Y
YB
YMUX
Y
BY
DYMUX
YQ
G4
G3
G2
G1
DUAL_PORT
SHIFT_REG
LUT
RAM
ROM
MC15
WG4USED
WG3USED
WG2USED
WG1USED
WS
DI
G
FFY
INIT1
INIT0
SRHIGH
SRLOW
SR
REV
ALTDIG
SHIFTIN
ALTDIG
BY
DIG_MUX
GAND
BY
G2
PROD
G1
CY0G
DIGUSED
DIG
BY
BYINV
BYINV
REVUSED
SLICEWE1
SLICEWE1USED
SLICEWE0USED
WSG
WE1
WE0
WE
CK
WSE
WSGEN
CYMUXF
XBMUX
BUSED
XB
F5MUX
F5USED
F5
SHIFTIN
ALTDIF
BX
DIF
F5
FXOR
XORF
FXMUX
XMUXUSED
XMUX
XUSED
X
F4
F3
F2
F1
MUX
DI
WF4USED
WF3USED
WF2USED
WF1USED
WF4
WF3
WF2
WF1
DUAL_PORT
SHIFT_REG
F
XB
XMUX
X
BX
DXMUX
XQ
FFX


Wejścia
LUT4
Pominięcie rejestru
Rejestr
Np. Xilinx Spartan 3
To/From Slice on Bottom
UG070_5_02_071504

XILINX – Virtex - LUT6




COUT
Reset Type
Sync
Async
DI
D6
D5
D4
D3
D2
D1
DI2
A6
A5
A4
A3
A2
A1
DPRAM64/32
SPRAM64/32
SRL32
SRL16
LUT
RAM
ROM
O6
O5
DI1
MC31
WA1-WA6
WA7
WA8
DX
CI
C6
C5
C4
C3
C2
C1
CX
BI
B6
B5
B4
B3
B2
B1
BX
D
DX
C
CX
B
BX
FF
LATCH
INIT1
INIT0
SRHIGH
SRLOW
CE
CK
SR
REV
Q
DMUX
D
DQ
CMUX
C
CQ
BMUX
B
BQ



Np. Xilinx Virtex 5

Czemu LUT6 dwuwyjściowy?

- Typowe wielkości funkcji logicznych
  - Proste warunki STB, ACK, READY - 2-3 argumenty
  - Proste multipleksery - 3-6 argumentów
  - Komparator liczby 8-bit. ze stałą - 8 argumentów
  - Komparator dwóch liczb 8-bit. - 16 argumentów
  - Sumator pełny 1-bitowy - 3 argumenty
- Bardzo często mamy mniejsze funkcje logiczne niż 6 wejściowe
  - LUT6 byłby redundantny
  - Można to wykorzystać!

Czemu LUT6 dwuwyjściowy?

- LUT6
  - 6 wejść
    - 64 kombinacje wejść
  - Skoro używamy mniejszej funkcji to używamy tylko kawałka pamięci
    - Np. 4 wejścia
      - 16 komórek LUT używanych
        16/64 = ¼ = 25%, czyli 75% się marnuje

Indeks	Wejście						Wyjście
{f,e,d,c,b,a}	f	e	d	c	b	a	F(f,e,d,c,b,a)
0	0	0	0	0	0	0	D0
1	0	0	0	0	0	1	D1
2	0	0	0	0	1	0	D2
3	0	0	0	0	1	1	D3
4	0	0	0	1	0	0	D4
5	0	0	0	1	0	1	D5
6	0	0	0	1	1	0	D6
7	0	0	0	1	1	1	D7
8	0	0	1	0	0	0	D8
9	0	0	1	0	0	1	D9
10	0	0	1	0	1	0	D10
11	0	0	1	0	1	1	D11
12	0	0	1	1	0	0	D12
13	0	0	1	1	0	1	D13
14	0	0	1	1	1	0	D14
15	0	0	1	1	1	1	D15
16	0	1	0	0	0	0	D16
…	…	…	…	…	…	…	…
32	1	0	0	0	0	0	D32
33	1	0	0	0	0	1	D33
…	…	…	…	…	…	…	…
48	1	1	0	0	0	0	D48
49	1	1	0	0	0	1	D49
…	…	…	…	…	…	…	…
59	1	1	1	0	1	1	D59
60	1	1	1	1	0	0	D60
61	1	1	1	1	0	1	D61
62	1	1	1	1	1	0	D62
63	1	1	1	1	1	1	D63

Czemu LUT6 dwuwyjściowy?

- LUT6
  - 6 wejść
    - 64 kombinacje wejść
  - Skoro używamy mniejszej funkcji to używamy tylko kawałka pamięci
    - Np. 3 wejścia
      - 8 komórek LUT używanych
        8/64 = 1/8 = 12,5%, czyli 87,5% się marnuje

Indeks	Wejście						Wyjście
{f,e,d,c,b,a}	f	e	d	c	b	a	F(f,e,d,c,b,a)
0	0	0	0	0	0	0	D0
1	0	0	0	0	0	1	D1
2	0	0	0	0	1	0	D2
3	0	0	0	0	1	1	D3
4	0	0	0	1	0	0	D4
5	0	0	0	1	0	1	D5
6	0	0	0	1	1	0	D6
7	0	0	0	1	1	1	D7
8	0	0	1	0	0	0	D8
9	0	0	1	0	0	1	D9
10	0	0	1	0	1	0	D10
11	0	0	1	0	1	1	D11
12	0	0	1	1	0	0	D12
13	0	0	1	1	0	1	D13
14	0	0	1	1	1	0	D14
15	0	0	1	1	1	1	D15
16	0	1	0	0	0	0	D16
…	…	…	…	…	…	…	…
32	1	0	0	0	0	0	D32
33	1	0	0	0	0	1	D33
…	…	…	…	…	…	…	…
48	1	1	0	0	0	0	D48
49	1	1	0	0	0	1	D49
…	…	…	…	…	…	…	…
59	1	1	1	0	1	1	D59
60	1	1	1	1	0	0	D60
61	1	1	1	1	0	1	D61
62	1	1	1	1	1	0	D62
63	1	1	1	1	1	1	D63

Czemu LUT6 dwuwyjściowy?

- LUT3
  - 3 wejścia *a,b,c*
    - 8 indeksów używanych
  - *d,e,f* nie są używane
    - Mogą mieć dowolne wartości
    - Należy:

Indeks	Wejście						Wyjście
{f,e,d,c,b,a}	f	e	d	c	b	a	F(f,e,d,c,b,a)
0	0	0	0	0	0	0	D0
1	0	0	0	0	0	1	D1
2	0	0	0	0	1	0	D2
3	0	0	0	0	1	1	D3
4	0	0	0	1	0	0	D4
5	0	0	0	1	0	1	D5
6	0	0	0	1	1	0	D6
7	0	0	0	1	1	1	D7
8	0	0	1	0	0	0	D8
9	0	0	1	0	0	1	D9
10	0	0	1	0	1	0	D10
11	0	0	1	0	1	1	D11
12	0	0	1	1	0	0	D12
13	0	0	1	1	0	1	D13
14	0	0	1	1	1	0	D14
15	0	0	1	1	1	1	D15
16	0	1	0	0	0	0	D16
…	…	…	…	…	…	…	…
32	1	0	0	0	0	0	D32
33	1	0	0	0	0	1	D33
…	…	…	…	…	…	…	…
48	1	1	0	0	0	0	D48
49	1	1	0	0	0	1	D49
…	…	…	…	…	…	…	…
59	1	1	1	0	1	1	D59
60	1	1	1	1	0	0	D60
61	1	1	1	1	0	1	D61
62	1	1	1	1	1	0	D62
63	1	1	1	1	1	1	D63

Czemu LUT6 dwuwyjściowy?

- LUT3
  - 3 wejścia *a,b,c*
    - 8 indeksów używanych
  - *d,e,f* nie są używane
    - Mogą mieć dowolne wartości
    - Należy:
      a) podpiąć zero d,e,f=0

Indeks	Wejście						Wyjście
{f,e,d,c,b,a}	f	e	d	c	b	a	F(f,e,d,c,b,a)
0	0	0	0	0	0	0	D0
1	0	0	0	0	0	1	D1
2	0	0	0	0	1	0	D2
3	0	0	0	0	1	1	D3
4	0	0	0	1	0	0	D4
5	0	0	0	1	0	1	D5
6	0	0	0	1	1	0	D6
7	0	0	0	1	1	1	D7
8	0	0	1	0	0	0	D8
9	0	0	1	0	0	1	D9
10	0	0	1	0	1	0	D10
11	0	0	1	0	1	1	D11
12	0	0	1	1	0	0	D12
13	0	0	1	1	0	1	D13
14	0	0	1	1	1	0	D14
15	0	0	1	1	1	1	D15
16	0	1	0	0	0	0	D16
…	…	…	…	…	…	…	…
32	1	0	0	0	0	0	D32
33	1	0	0	0	0	1	D33
…	…	…	…	…	…	…	…
48	1	1	0	0	0	0	D48
49	1	1	0	0	0	1	D49
…	…	…	…	…	…	…	…
59	1	1	1	0	1	1	D59
60	1	1	1	1	0	0	D60
61	1	1	1	1	0	1	D61
62	1	1	1	1	1	0	D62
63	1	1	1	1	1	1	D63

Czemu LUT6 dwuwyjściowy?

- LUT3
  - 3 wejścia *a,b,c*
    - 8 indeksów używanych
  - *d,e,f* nie są używane
    - Mogą mieć dowolne wartości
    - Należy:
      b) powielić wypełnienie
      tutaj regularnie co 8 indeksów, D0..D7 dla różnych *d,e,f*

Indeks	Wejście						Wyjście
{f,e,d,c,b,a}	f	e	d	c	b	a	F(f,e,d,c,b,a)
0	0	0	0	0	0	0	D0
1	0	0	0	0	0	1	D1
2	0	0	0	0	1	0	D2
3	0	0	0	0	1	1	D3
4	0	0	0	1	0	0	D4
5	0	0	0	1	0	1	D5
6	0	0	0	1	1	0	D6
7	0	0	0	1	1	1	D7
8	0	0	1	0	0	0	D0
9	0	0	1	0	0	1	D1
10	0	0	1	0	1	0	D2
11	0	0	1	0	1	1	D3
12	0	0	1	1	0	0	D4
13	0	0	1	1	0	1	D5
14	0	0	1	1	1	0	D6
15	0	0	1	1	1	1	D7
16	0	1	0	0	0	0	D0
...	...	...	...	...	...	...	...
32	1	0	0	0	0	0	D0
33	1	0	0	0	0	1	D1
...	...	...	...	...	...	...	...
48	1	1	0	0	0	0	D0
49	1	1	0	0	0	1	D1
...	...	...	...	...	...	...	...
59	1	1	1	0	1	1	D3
60	1	1	1	1	0	0	D4
61	1	1	1	1	0	1	D5
62	1	1	1	1	1	0	D6
63	1	1	1	1	1	1	D7

Czemu LUT6 dwuwyjściowy?

- LUT3
  - 3 wejścia *a,b,c*
- Drugi LUT 3
  - *d,e,f*
  - Trzeba poukładać

Wejście			Wyjście
f	e	d	F_2(f,e,d)
0	0	0	Q0
0	0	1	Q1
0	1	0	Q2
0	1	1	Q3
1	0	0	Q4
1	0	1	Q5
1	1	0	Q6
1	1	1	Q7

Indeks	Wejście						Wyjście
{f,e,d,c,b,a}	f	e	d	c	b	a	F_1(f,e,d,c,b,a)
0	0	0	0	0	0	0	D0
1	0	0	0	0	0	1	D1
2	0	0	0	0	1	0	D2
3	0	0	0	0	1	1	D3
4	0	0	0	1	0	0	D4
5	0	0	0	1	0	1	D5
6	0	0	0	1	1	0	D6
7	0	0	0	1	1	1	D7
8	0	0	1	0	0	0	D0
9	0	0	1	0	0	1	D1
10	0	0	1	0	1	0	D2
11	0	0	1	0	1	1	D3
12	0	0	1	1	0	0	D4
13	0	0	1	1	0	1	D5
14	0	0	1	1	1	0	D6
15	0	0	1	1	1	1	D7
16	0	1	0	0	0	0	D0
...	...	...	...	...	...	...	...
32	1	0	0	0	0	0	D0
33	1	0	0	0	0	1	D1
...	...	...	...	...	...	...	...
48	1	1	0	0	0	0	D0
49	1	1	0	0	0	1	D1
...	...	...	...	...	...	...	...
59	1	1	1	0	1	1	D3
60	1	1	1	1	0	0	D4
61	1	1	1	1	0	1	D5
62	1	1	1	1	1	0	D6
63	1	1	1	1	1	1	D7

Czemu LUT6 dwuwyjściowy?

Wejście			Wyjście
f	e	d	F_2(f,e,d)
0	0	0	Q0
0	0	1	Q1
0	1	0	Q2
0	1	1	Q3
1	0	0	Q4
1	0	1	Q5
1	1	0	Q6
1	1	1	Q7

Wyjście2	Indeks	Wejście						Wyjście1
F_2(f,e,d,c,b,a)	{f,e,d,c,b,a}	f	e	d	c	b	a	F_1(f,e,d,c,b,a)
Q0	0	0	0	0	0	0	0	D0
Q0	1	0	0	0	0	0	1	D1
Q0	2	0	0	0	0	1	0	D2
Q0	3	0	0	0	0	1	1	D3
Q0	4	0	0	0	1	0	0	D4
Q0	5	0	0	0	1	0	1	D5
Q0	6	0	0	0	1	1	0	D6
Q0	7	0	0	0	1	1	1	D7
Q1	8	0	0	1	0	0	0	D0
Q1	9	0	0	1	0	0	1	D1
Q1	10	0	0	1	0	1	0	D2
Q1	11	0	0	1	0	1	1	D3
Q1	12	0	0	1	1	0	0	D4
Q1	13	0	0	1	1	0	1	D5
Q1	14	0	0	1	1	1	0	D6
Q1	15	0	0	1	1	1	1	D7
Q2	16	0	1	0	0	0	0	D0
...	...	...	...	...	...	...	...	...
Q4	32	1	0	0	0	0	0	D0
Q4	33	1	0	0	0	0	1	D1
...	...	...	...	...	...	...	...	...
Q6	48	1	1	0	0	0	0	D0
Q6	49	1	1	0	0	0	1	D1
...	...	...	...	...	...	...	...	...
Q7	59	1	1	1	0	1	1	D3
Q7	60	1	1	1	1	0	0	D4
Q7	61	1	1	1	1	0	1	D5
Q7	62	1	1	1	1	1	0	D6
Q7	63	1	1	1	1	1	1	D7

Czemu LUT6 dwuwyjściowy?

Wejście			Wyjście
c	b	a	$F_1(c,b,a)$
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4
1	0	1	D5
1	1	0	D6
1	1	1	D7

Wejście			Wyjście
f	e	d	$F_2(f,e,d)$
0	0	0	Q0
0	0	1	Q1
0	1	0	Q2
0	1	1	Q3
1	0	0	Q4
1	0	1	Q5
1	1	0	Q6
1	1	1	Q7

Wyjście2	Indeks	Wejście						Wyjście1
$F_2(f,e,d,c,b,a)$	{f,e,d,c,b,a}	f	e	d	c	b	a	$F_1(f,e,d,c,b,a)$
Q0	0	0	0	0	0	0	0	D0
Q0	1	0	0	0	0	0	1	D1
Q0	2	0	0	0	0	1	0	D2
Q0	3	0	0	0	0	1	1	D3
Q0	4	0	0	0	1	0	0	D4
Q0	5	0	0	0	1	0	1	D5
Q0	6	0	0	0	1	1	0	D6
Q0	7	0	0	0	1	1	1	D7
Q1	8	0	0	1	0	0	0	D0
Q1	9	0	0	1	0	0	1	D1
Q1	10	0	0	1	0	1	0	D2
Q1	11	0	0	1	0	1	1	D3
Q1	12	0	0	1	1	0	0	D4
Q1	13	0	0	1	1	0	1	D5
Q1	14	0	0	1	1	1	0	D6
Q1	15	0	0	1	1	1	1	D7
Q2	16	0	1	0	0	0	0	D0
...	...	...	...	...	...	...	...	...
Q4	32	1	0	0	0	0	0	D0
Q4	33	1	0	0	0	0	1	D1
...	...	...	...	...	...	...	...	...
Q6	48	1	1	0	0	0	0	D0
Q6	49	1	1	0	0	0	1	D1
...	...	...	...	...	...	...	...	...
Q7	59	1	1	1	0	1	1	D3
Q7	60	1	1	1	1	0	0	D4
Q7	61	1	1	1	1	0	1	D5
Q7	62	1	1	1	1	1	0	D6
Q7	63	1	1	1	1	1	1	D7

Czemu LUT6 dwuwyjściowy?

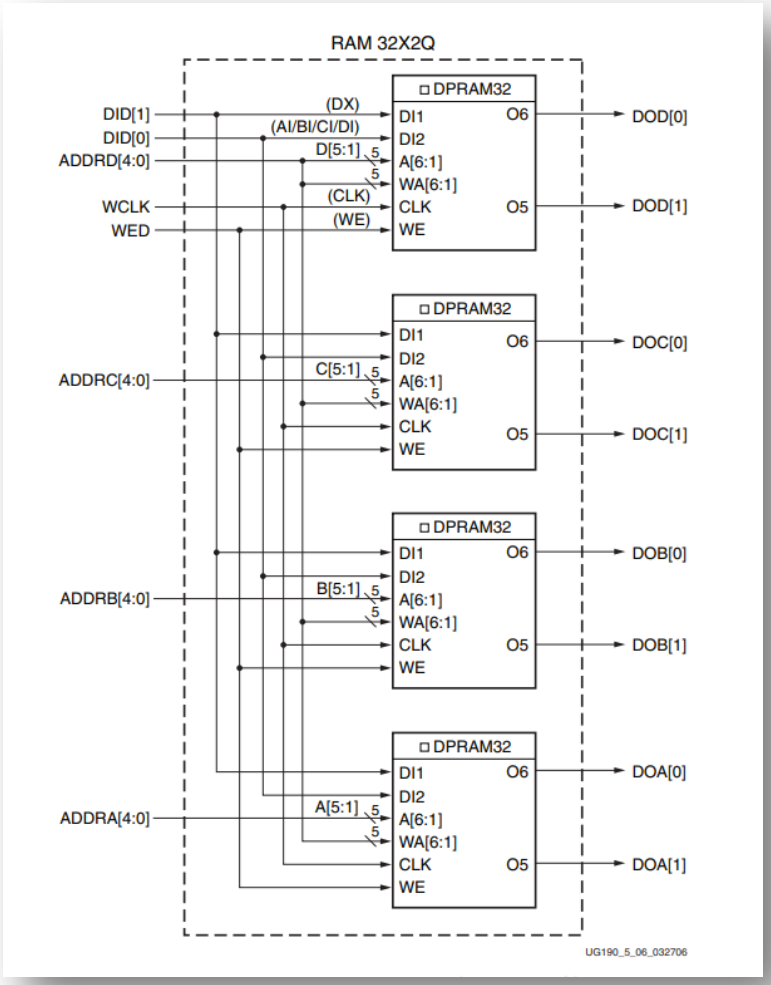
- 2×LUT3 (8+8) jest tańsze
  niż LUT6 dwuwyjściowy (64+64)
- Czemu nie zrobić 2×LUT3 ?
  - Gdyż nie wiemy czy akurat LUT3 !
  - Możliwe przykładowe (!!!) konfiguracje
    - 3+3 np. F_1(c,b,a) F_2(f,e,d)
    - 2+4 np. F_1(b,a) F_2(f,e,d,c)
    - 1+5 np. F_1(a) F_2(f,e,d,c,b)
    - …**ale niektóre argumenty mogą się pokrywać!**
    - 6+6 np. F_1(f,e,d,c,b,a) F_2(f,e,d,c,b,a)
    - 6+5 np. F_1(f,e,d,c,b,a) F_2(f,e,d,c,b)
    - 5+5 np. F_1(e,d,c,b,a) F_2(f,e,d,c,b)

Czemu LUT6 dwuwyjściowy?

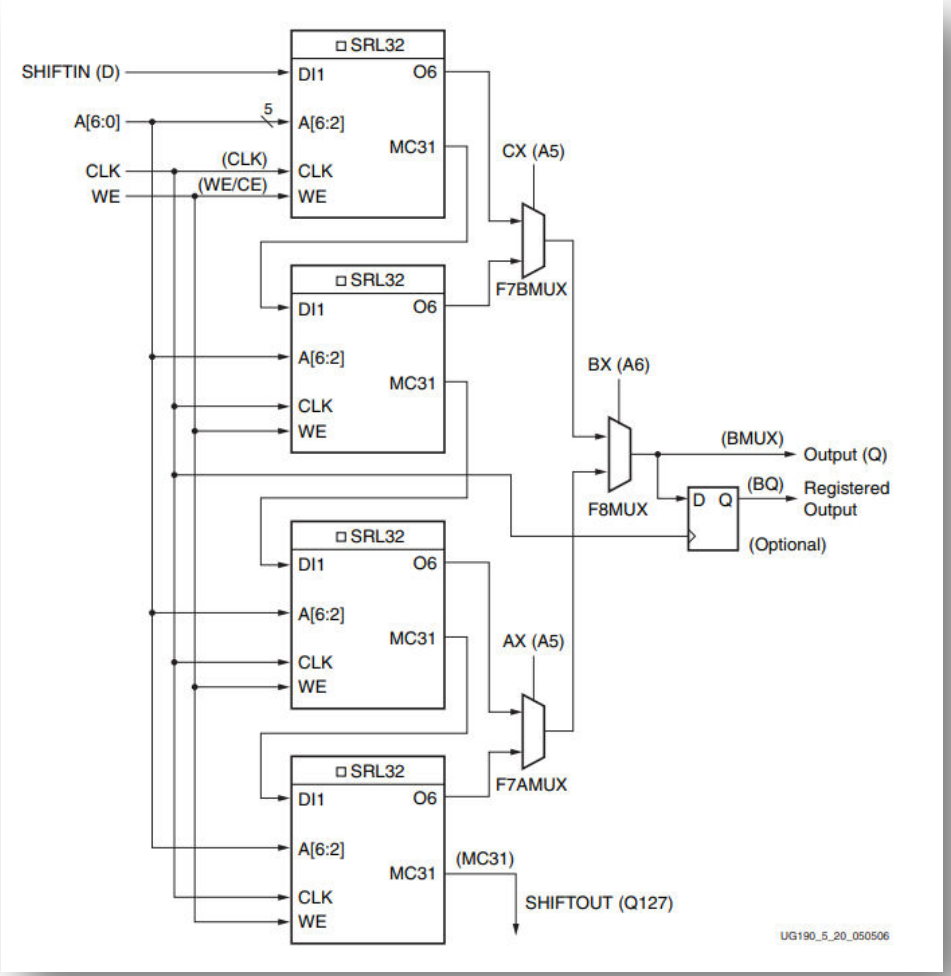
- Zalety LUT-ów dwuwyjściowych
  - Koszt dwóch wyjść jest mniejszy niż $\times 2$
    - Współdzielone doprowadzenia, reprogramowanie itp.
  - W praktyce wiele funkcji współdzieli argumenty
    - Doprowadzane są tylko raz przez macierz rutującą
  - Elastyczność
- Wady LUT-ów dwuwyjściowych
  - Większy koszt
  - Ryzyko zmarnotrawienia drugiego wyjścia
- Są stosowane w bardziej złożonych układach FPGA (większych i szybszych)

XILINX – LUT jako RAM lub SRL

Distributed RAM

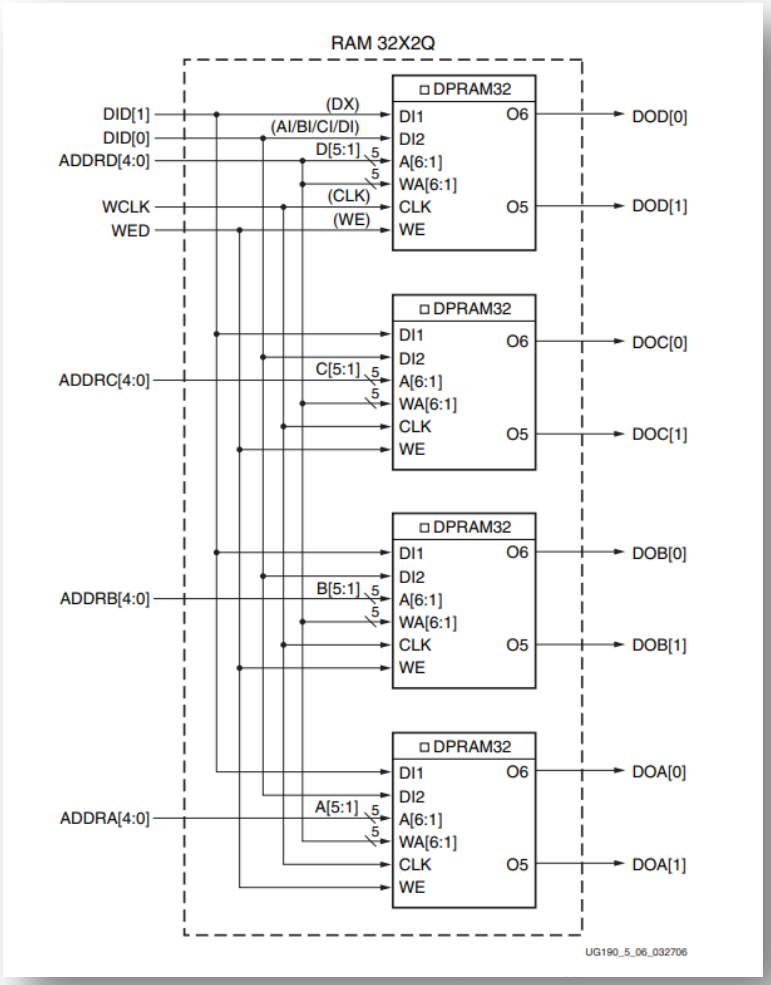


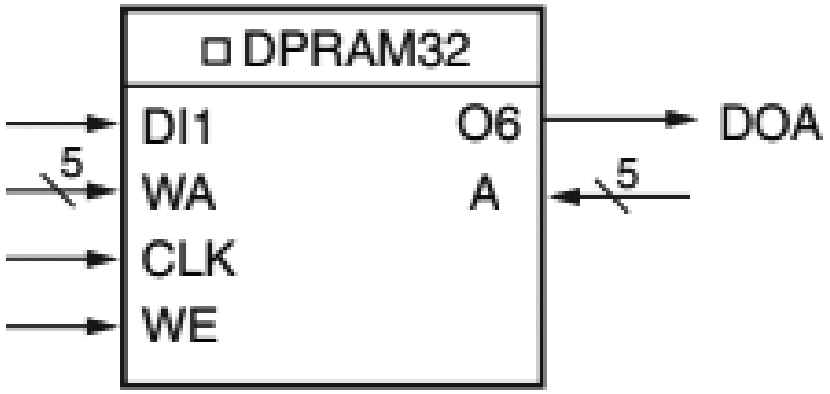
Shift register



XILINX – LUT jako RAM

Distributed RAM



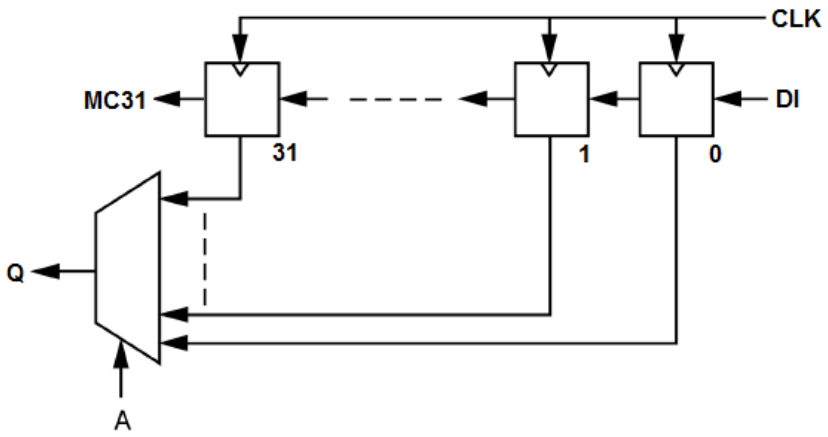



```
reg RAM[0:31];

always @(posedge CLK)
if (WE) RAM[WA] <= DI;

assign DO = RAM[A];
```

XILINX – LUT jako SRL

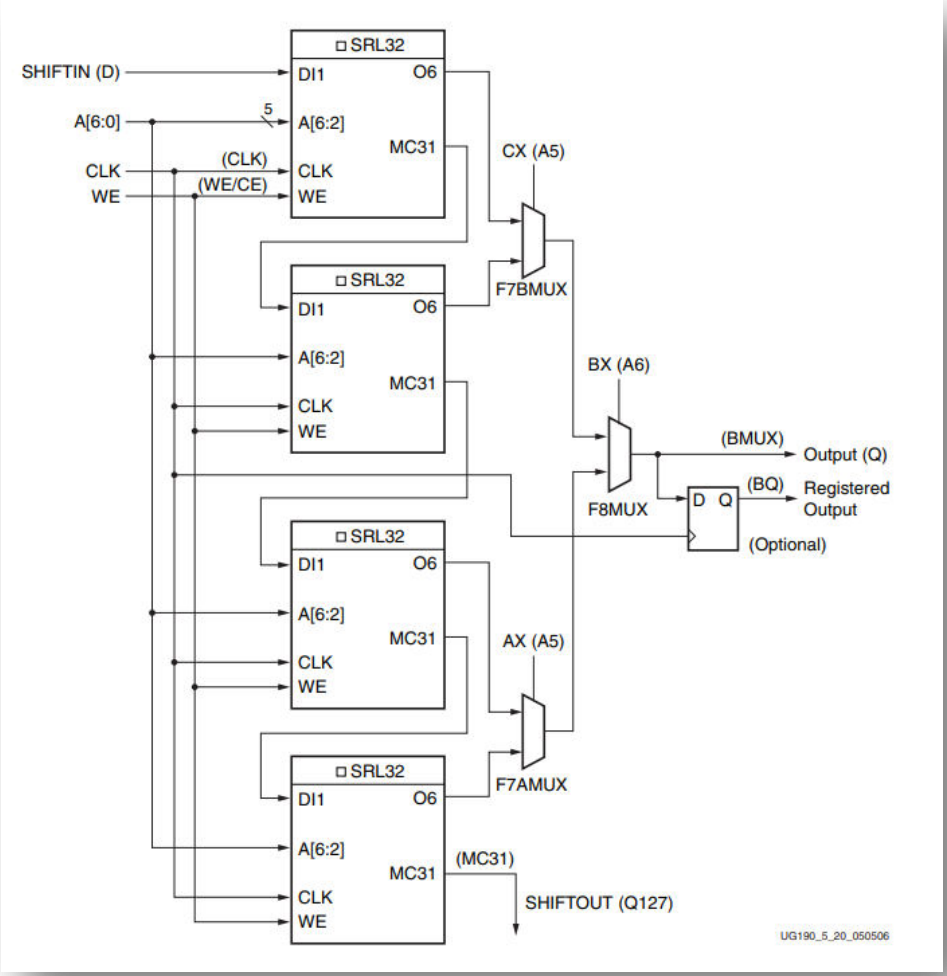



```
reg [31:0] data;

assign Q    = data[A];
assign MC31 = data[31];

always @(posedge CLK)
if (WE) begin
  data    <= data << 1; // przesunięcie
  data[0] <= DI;        // nadpisanie [0]
end
```

Shift register

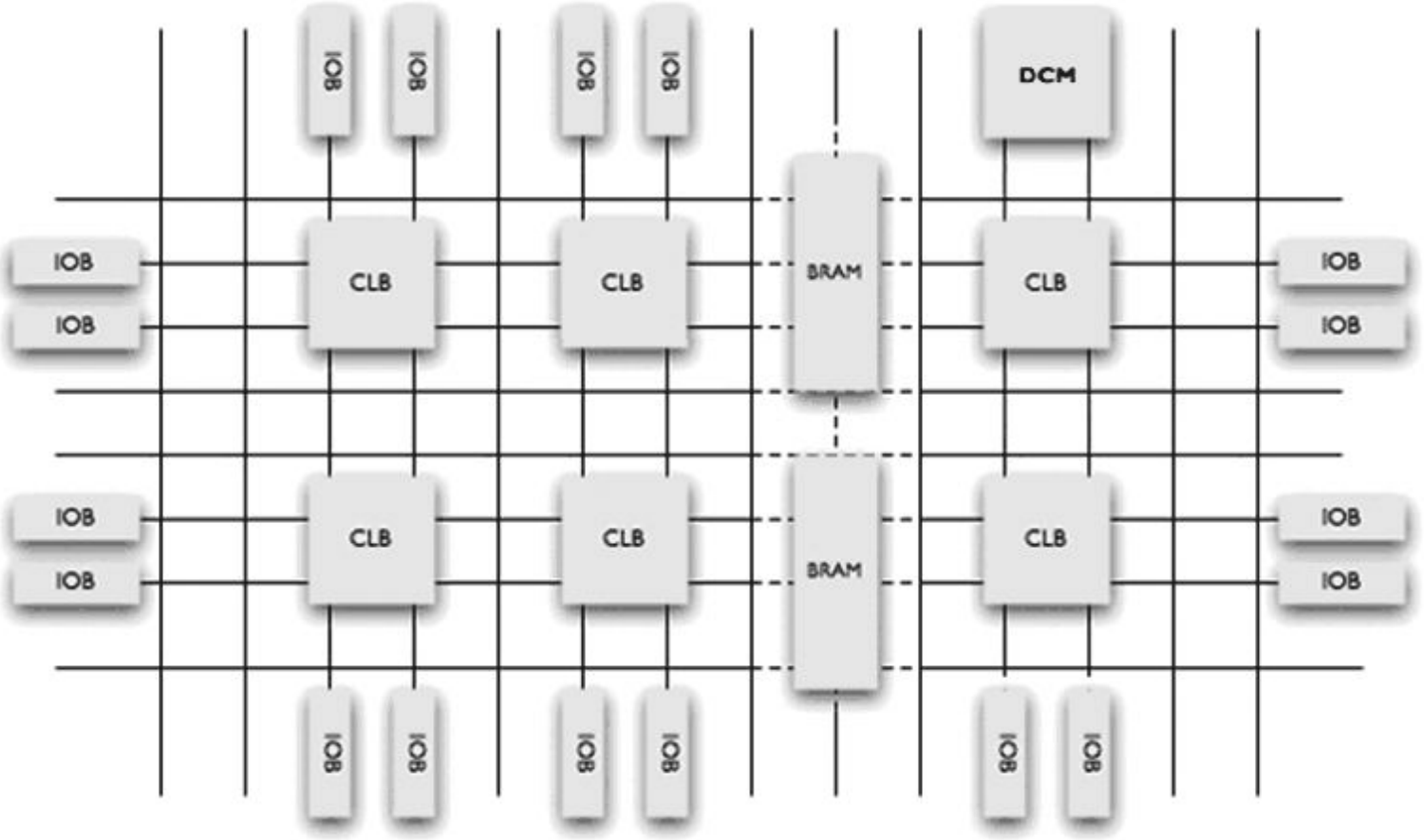


Bloki specjalizowane w układach FPGA

Wbudowane bloki funkcjonalne

- Configurable Logic Block (CLB)
- Block RAM, (BRAM /RAMB), 1KB to 4KB
- Digital Signal Processing (DSP)
- Digital Clock Manager (DCM)
  - Delay-Locked Loop (DLL), Phase-Locked Loop (PLL)
- Oscilator
- Input/Output Buffers (IOB)
- Serializer/Deserializer (SerDes), GTP
- SYS_MONITOR
- ARM Core, Microblaze

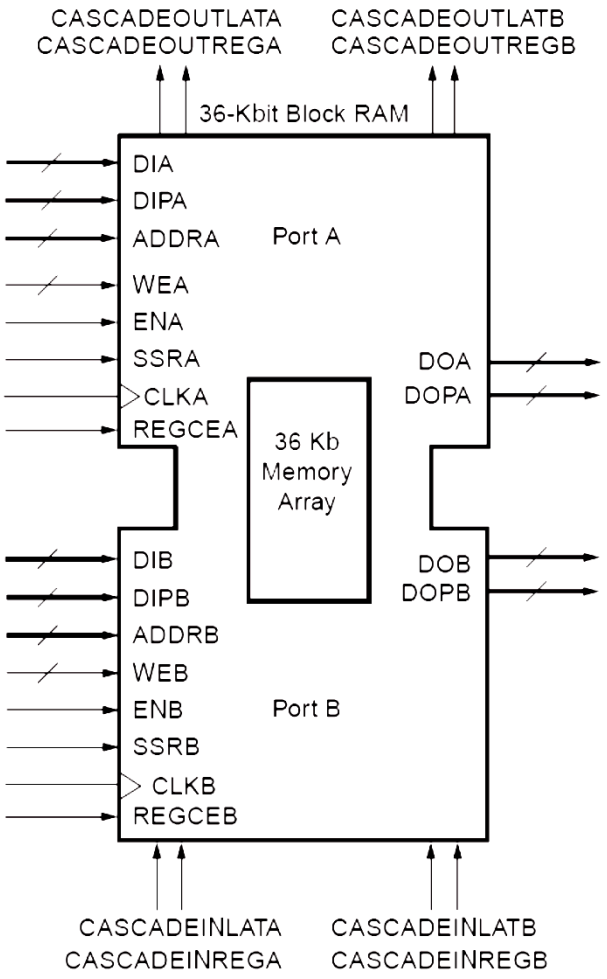
FPGA - Ogólna struktura



Pamięć blokowa Block RAM

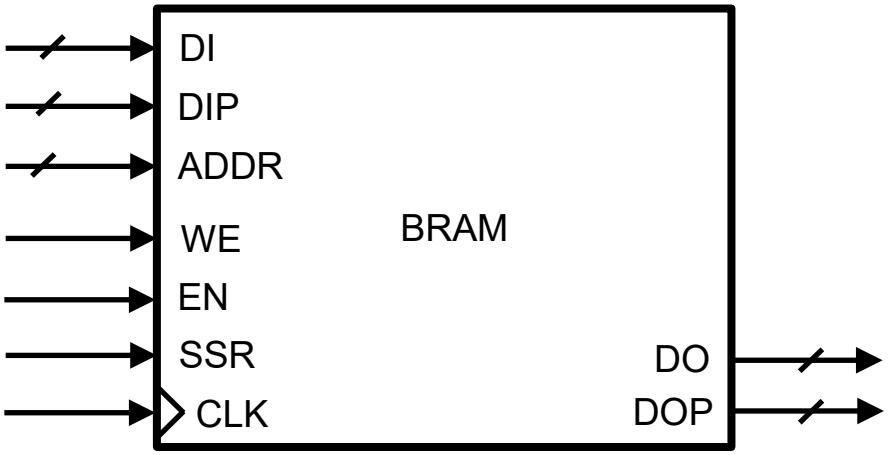
XILINX – Memory Block (BRAM)

- W wybranych miejscach układu FPGA
  - Np. w pojedynczych kolumnach
  - Kilkanaście-kilkaset bloków
- Większa pojemność
  - 9, 36, 72 kbit
- Dwa porty: A,B
  - Zapis/odczyt
  - Dwie domeny częstotliwościowe!



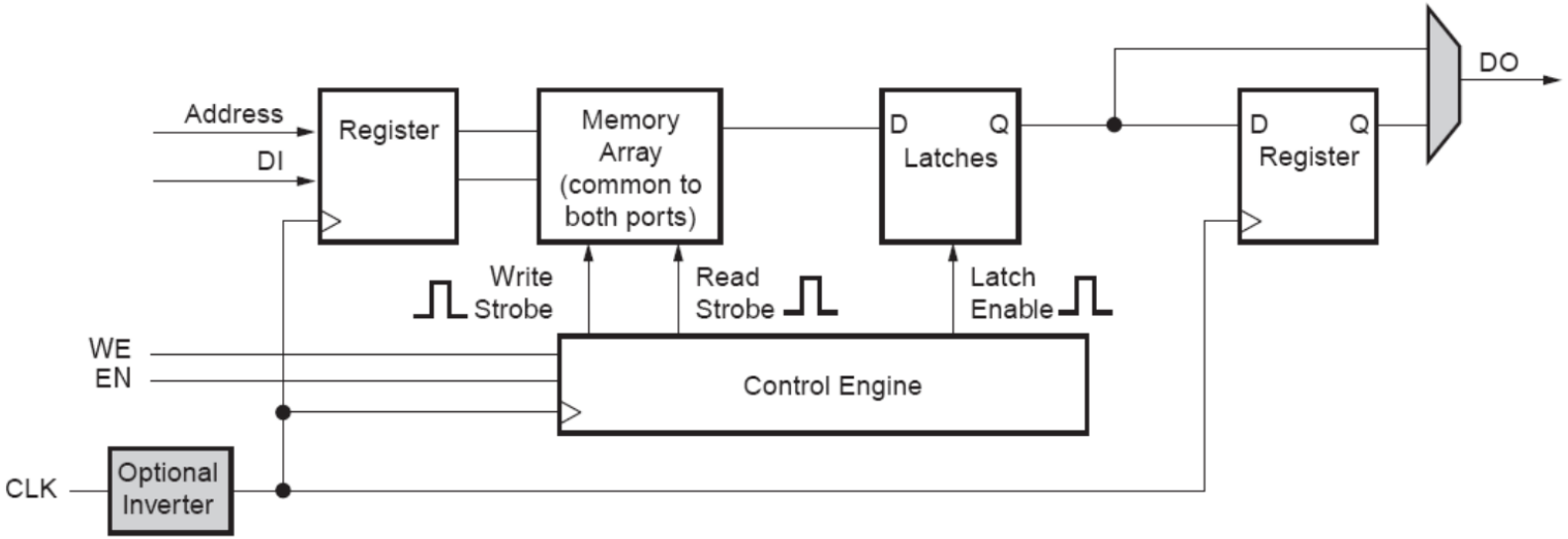
BRAM – typowy port

- Zapis i odczyt spod adresu ADDR
  - Jeśli EN jest aktywny
- Zapis:
  - DI, DIP (bity parzystości)
  - Jeśli WE
- Odczyt
  - DO, DOP (bity parzystości)
  - Poprzez rejestry wyjściowe (opóźnienie)
- Zegar + Reset
  - SSR – reset tylko rejestrów wyjściowych (nie RAM!)
  - CLK - Wszystko synchronicznie
    - w każdym porcie niezależnie

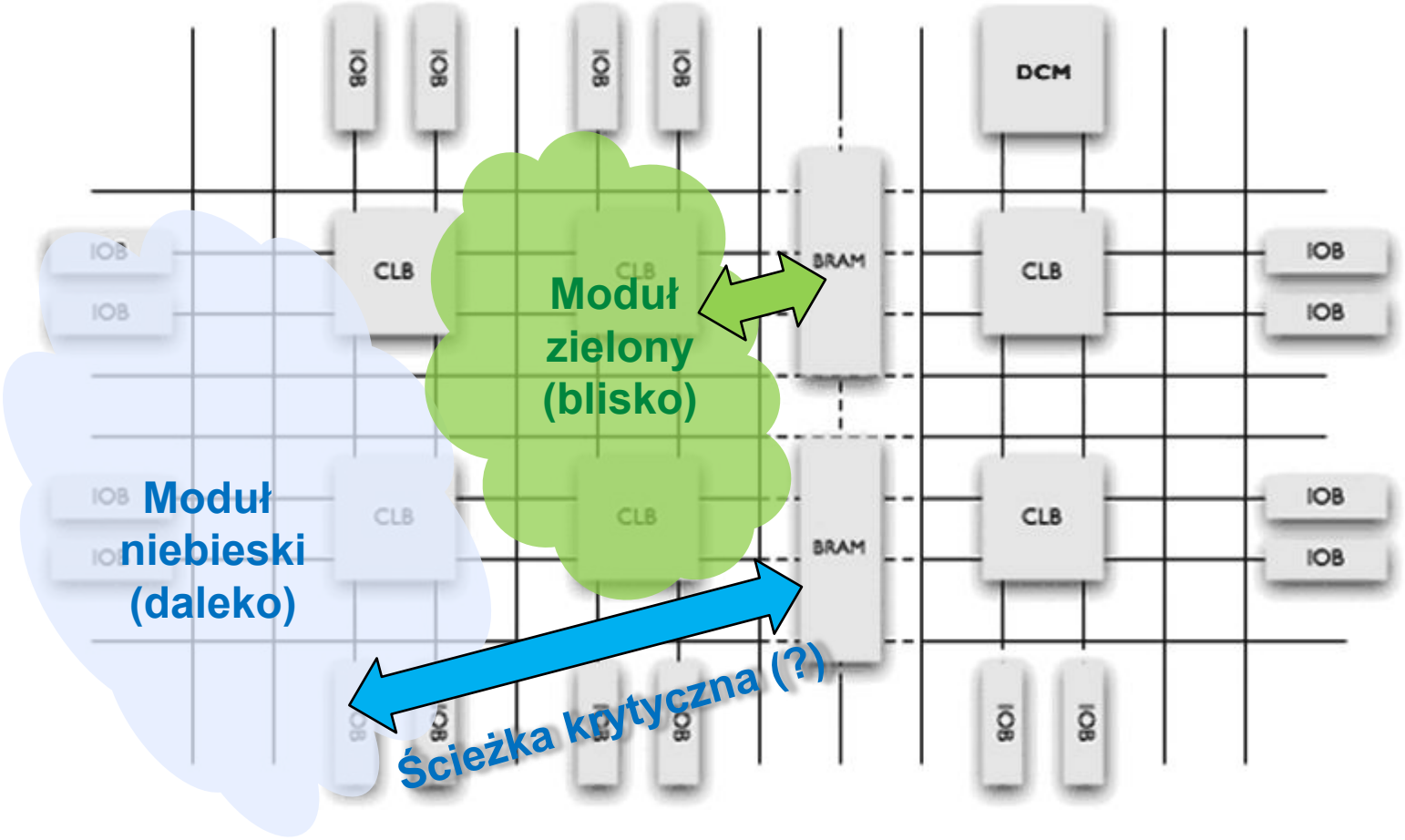


XILINX – BRAM - opóźnienia

- W przeciwieństwie do pamięci rozproszonej odczyt i zapis zajmuje kilka taktów
  - Zapis – 1 takt
  - Odczyt – 1-2 takty
    - Czemu rejestr jest opcjonalny?



To reg or not to reg?



XILINX – RAMB parameters

- Możliwość wybory szerokości słów na każdym porcie (A i B) niezależnie

Primitive	Settings		Effective Read Width	Effective Write Width
	Read Width	Write Width		
RAMB36	1, 2, or 4	9, 18, or 36	Same as setting	8, 16, or 32
RAMB36	9, 18, or 36	1, 2, or 4	8, 16, or 32	Same as setting
RAMB36	1, 2, or 4	1, 2, or 4	Same as setting	Same as setting
RAMB36	9, 18, or 36	9, 18, or 36	Same as setting	Same as setting

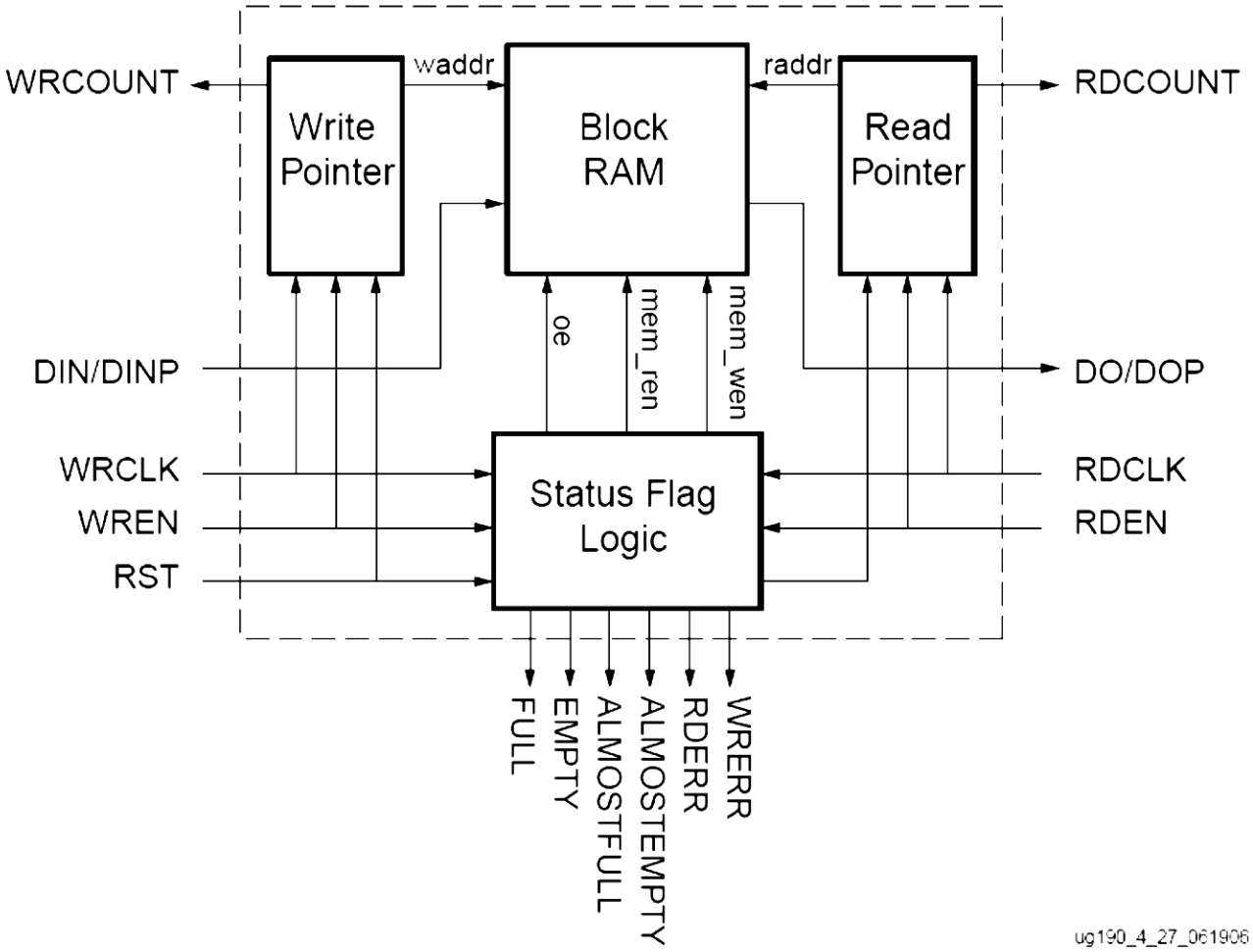
Primitive	Maximum Bit Width	Number of Byte-wide Write Enables
RAMB36	36	4
RAMB36SDP	72	8
RAMB18	18	2
RAMB18SDP	36	4

XILINX – szerokości słów RAMB

- Słowa są rozszerzone o bity „parzystości”
  - Bit znaku
  - Parzystość
  - Bit ważności
  - Dowolna wartość

Szerokość słowa	Bity parzystości	Sumaryczna szerokość	Liczba słów	Bitów adresu	Pojemność
1	-	1	32768	15	1 ×32768 = 32kbit
2	-	2	16384	14	2 ×16384 = 32kbit
4	-	4	8192	13	4 × 8192 = 32kbit
8	1	9	4096	12	(8+1) × 4096 = 36kbit
16	2	18	2048	11	(16+2) × 2048 = 36kbit
32	4	36	1024	10	(32+4) × 1024 = 36kbit
64	8	72	512	9	(64+8) × 512 = 36kbit

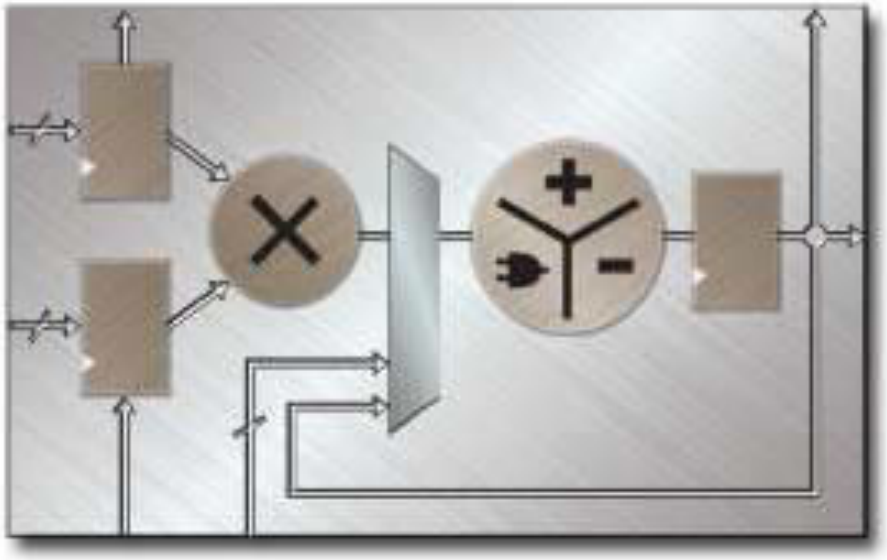
XILINX – RAMB jako FIFO



Blok przetwarzania sygnałów
Digital Signal Processing (DSP)

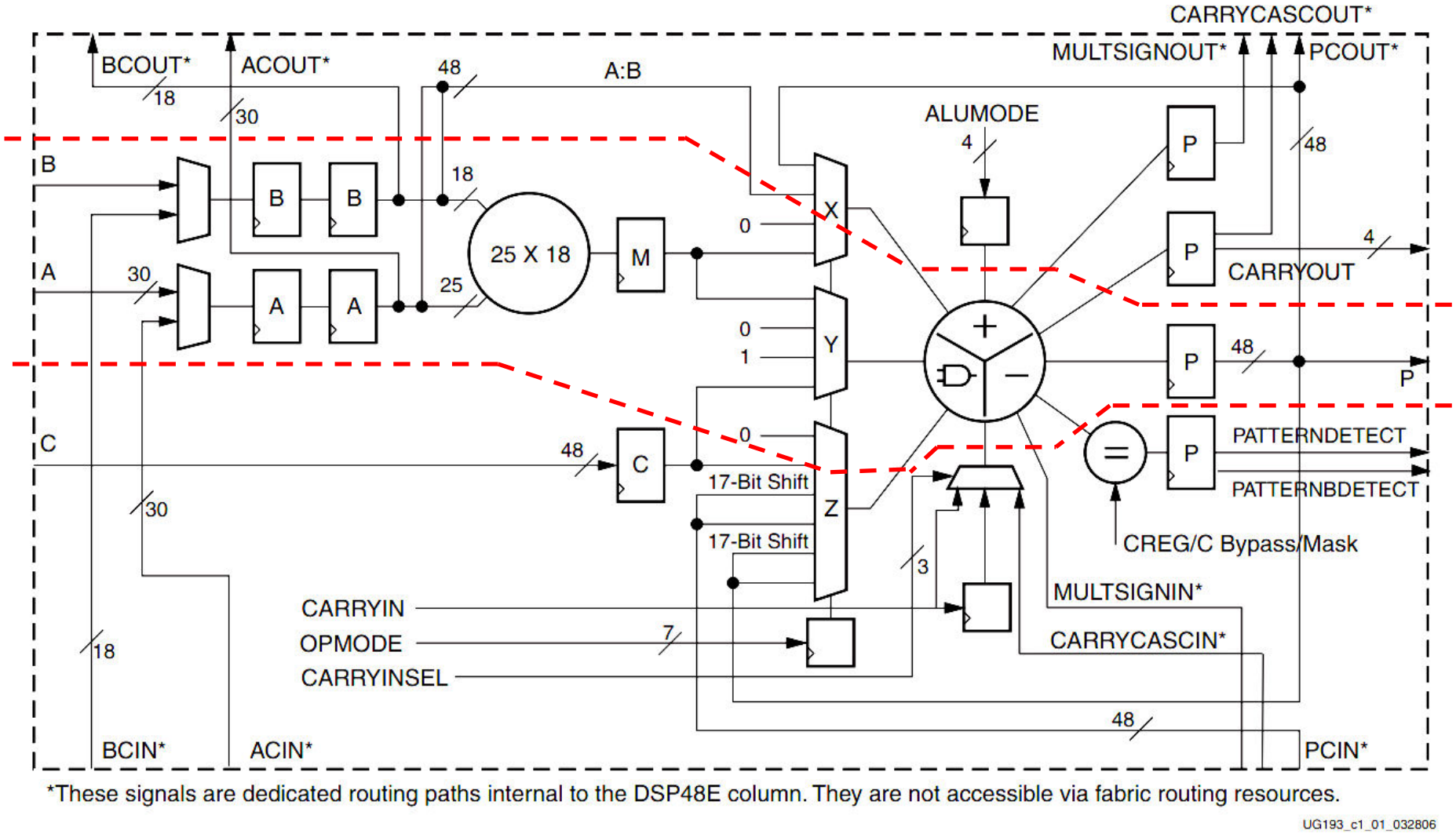
XILINX – DSP

- Digital Signal Processing
  - Mnożenie
  - Dodawanie
  - Odejmowanie
  - Porównywanie



- Czemu nie z pomocą zwykłej logiki na LUT?
  - Złożoność ⇒ Ścieżki krytyczne ⇒ Niskie taktowanie
  - DSP - Szybciej, wydajniej

XILINX – struktura DSP



*These signals are dedicated routing paths internal to the DSP48E column. They are not accessible via fabric routing resources.

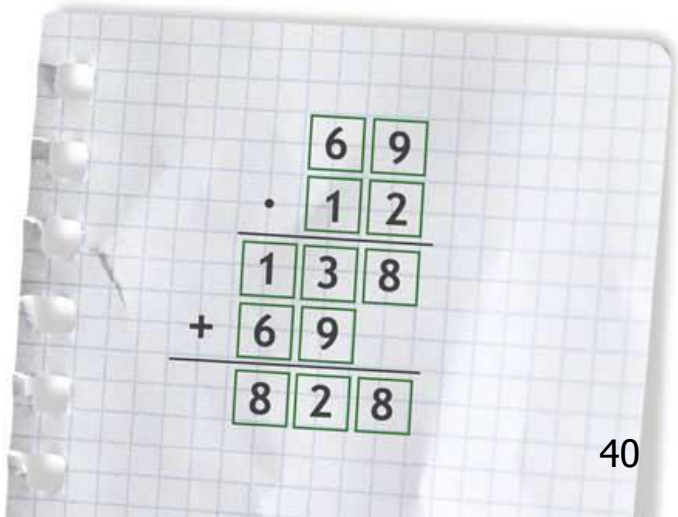
UG193_c1_01_032806

XILINX – czemu 25×18 ?

- Typowe mnożenia które chcemy robić
  - Mnożenie całkowite – ze znakiem i bez znaku
    - int8 × int8, int16 × int16, int32 × int32
  - Mnożenie liczby zmiennoprzecinkowej (IEEE754) przez całkowitą
    - float × int8, float × int16, float × int32
  - Mnożenie liczby zmiennoprzecinkowej przez zmiennoprzecinkową (IEEE754)
    - float × float

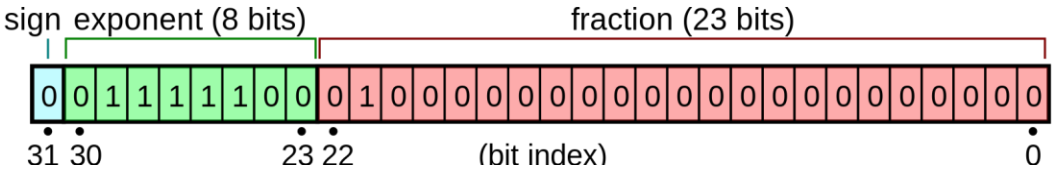
XILINX – czemu 25×18 ?

- 25 × 18 realizuje mnożenie ze znakiem
  - Czyli maksymalnie 24 × 17 bez znaku
- Obsługiwane wprost
  - int8 × int8 ze znakiem
  - int8 × int8 bez znaku, jako int9 × int9 ze znakiem
  - int16 × int16 ze znakiem
    - int16 × int16 bez znaku, jako int17 × int17 ze znakiem
- Obsługiwane przez wiele bloków DSP (mnożenie pisemne)
  - int32 × int16 – potrzebne 2 DSP
  - int32 × int32 – potrzebne 4 DSP



XILINX – czemu 25×18 ?

- 25 × 18 realizuje mnożenie ze znakiem
  - Czyli maksymalnie 24 × 17 bez znaku
- Liczby zmiennoprzecinkowe (IEEE 754)
  - 1 bit znaku (Sign)
  - 8 bit wykładnika (EXPonent)
  - 23 bity mantysy (MANtisa / fraction)
  - Tak jak w zapisie potęgowym
    - liczba $= \pm MAN \cdot 10^{EXP}$ np. $-1{,}237 \cdot 10^3$
  - Tylko że binarnie, i w mantysie zapisujemy tylko część ułamkową
    - liczba $= \pm(1, MAN) \cdot 2^{EXP}$ np. $1{,}0010101 \cdot 2^{44}$



XILINX – czemu 25×18 ?

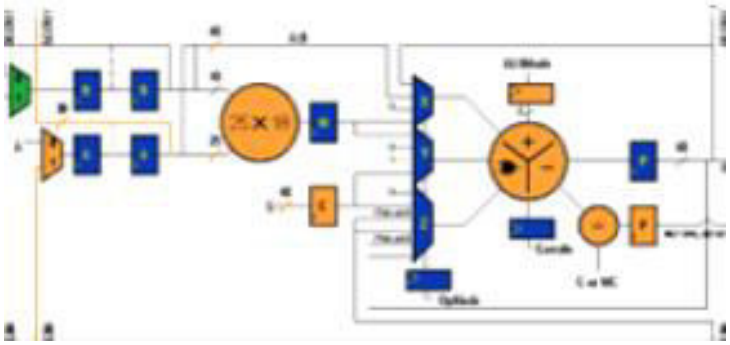
- 25 × 18 realizuje mnożenie ze znakiem
  - Czyli maksymalnie 24 × 17 bez znaku
- Mnożenie zmiennoprzecinkowe (IEEE 754)
  - $\mathrm{A} = S_A \cdot (1, MAN_A) \cdot 2^{EXP_A}$
  - $\mathrm{B} = S_B \cdot (1, MAN_B) \cdot 2^{EXP_B}$
  - $\mathrm{A} \cdot \mathrm{B} = S_A \cdot (1, MAN_A) \cdot S_B \cdot (1, MAN_B) \cdot 2^{EXP_A + EXP_B}$
- 23 bity Mantysy
  + 1 bit na ukrytą jedynkę
  + 1 bit na znak
  = 25 bitów!
- Do pełni szczęścia potrzebujemy mnożenia 25 × 25

XILINX – czemu 25×18 ?

- 25 × 18 realizuje mnożenie ze znakiem
  - Czyli maksymalnie 24 × 17 bez znaku
- Obsługiwane wprost
  - float × int8 ze znakiem
  - float × int8 bez znaku, jako float × int9 ze znakiem
  - float × int16 ze znakiem
  - float × int16 bez znaku, jako float × int17 ze znakiem
- Obsługiwane przez wiele bloków DSP (mnożenie pisemne)
  - float × int32 – potrzebne 2 bloki DSP
  - float × float – potrzebne 2 bloki DSP

XILINX – funkcjonalność DSP

- 25 x 18 multiplier
- Cascading A and B input
- Independent C input and C register with independent reset and clock enable.
- CARRYCASCIN and CARRYCASCOUT internal cascade signals to support 96-bit accumulators/adders/subtracters in two DSP48E slices
- MULTSIGNIN and MULTSIGNOUT internal cascade signals with special OPMODE setting to support a 96-bit MACC extension
- Single Instruction Multiple Data (SIMD) Mode for three-input adder/subtracter which precludes use of multiplier in first stage
  - Dual 24-bit SIMD adder/subtracter/accumulator with two separate CARRYOUT signals
  - Quad 12-bit SIMD adder/subtracter/accumulator with four separate CARRYOUT signals



XILINX – funkcjonalność DSP

- 48-bit logic unit
  - Multiply and Add, Mutiply and Accumulate
  - Bit-wise logic operations - two-input AND, OR, NOT, NAND, NOR, XOR, and XNOR
  - Logic unit mode dynamically selectable via ALUMODE
- Pełna szerokość mnożenia 25 × 18
  - 25+18 = 43 bity (będące wynikiem mnożenia)
  - 48 (w akumulatorze) – 43 = 5 (dodatkowych bitów)
  - $2^5 = 32$
  - Możemy zakumulować wyniki 32 kolejnych mnożeń
- Mniejsze mnożenie np. 20 × 16
  - 20+16 = 36 bity (będące wynikiem mnożenia)
  - 48 (w akumulatorze) – 36 = 12 (dodatkowych bitów)
  - $2^{12} = 4096$
  - Możemy zakumulować wyniki 4096 kolejnych mnożeń

```
reg signed [24:0] A[0:31];
reg signed [17:0] B[0:31];

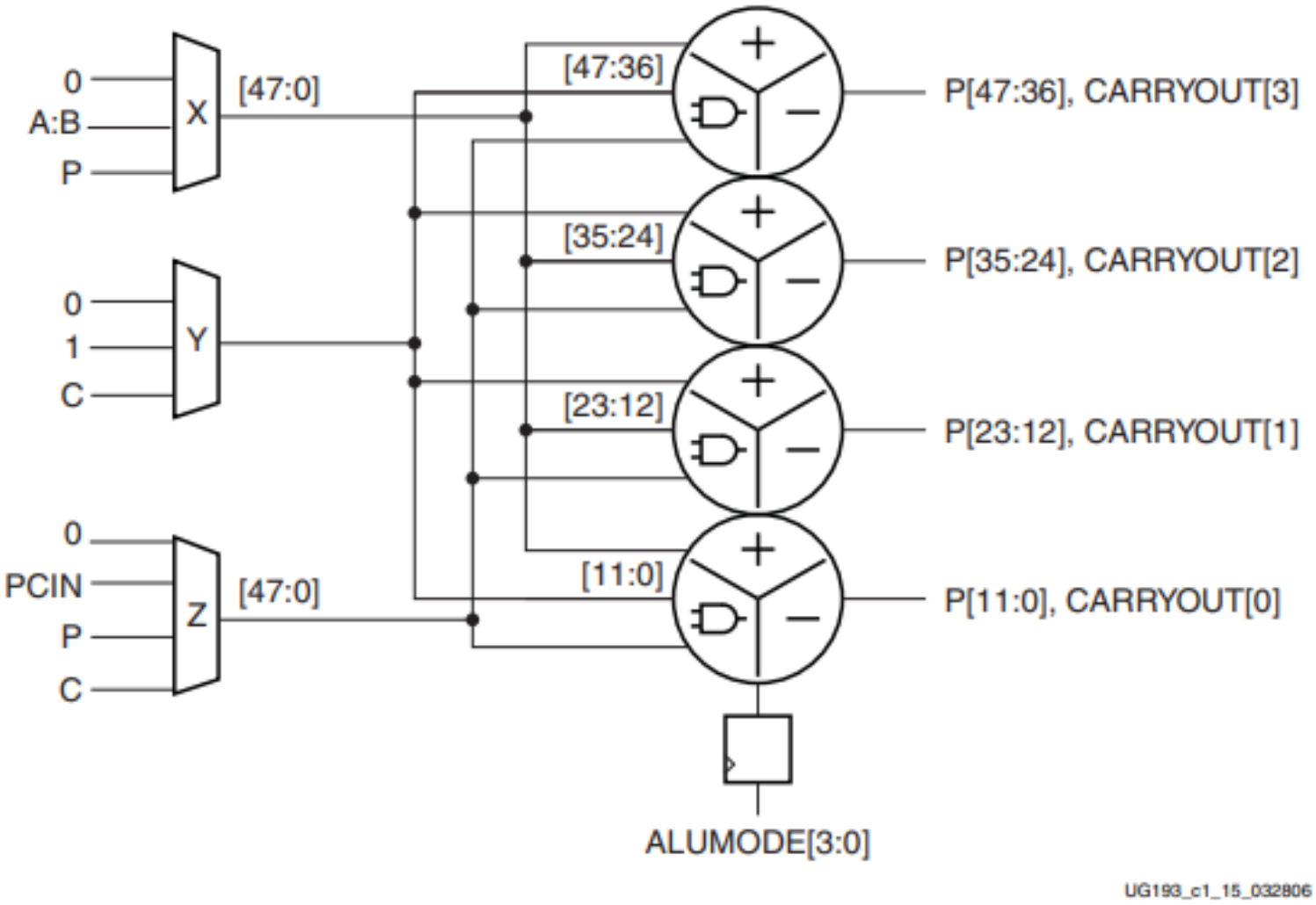
reg signed [47:0] acc;

for (i=0; i<32; ++i)
  acc += A[i] * B[i];
```

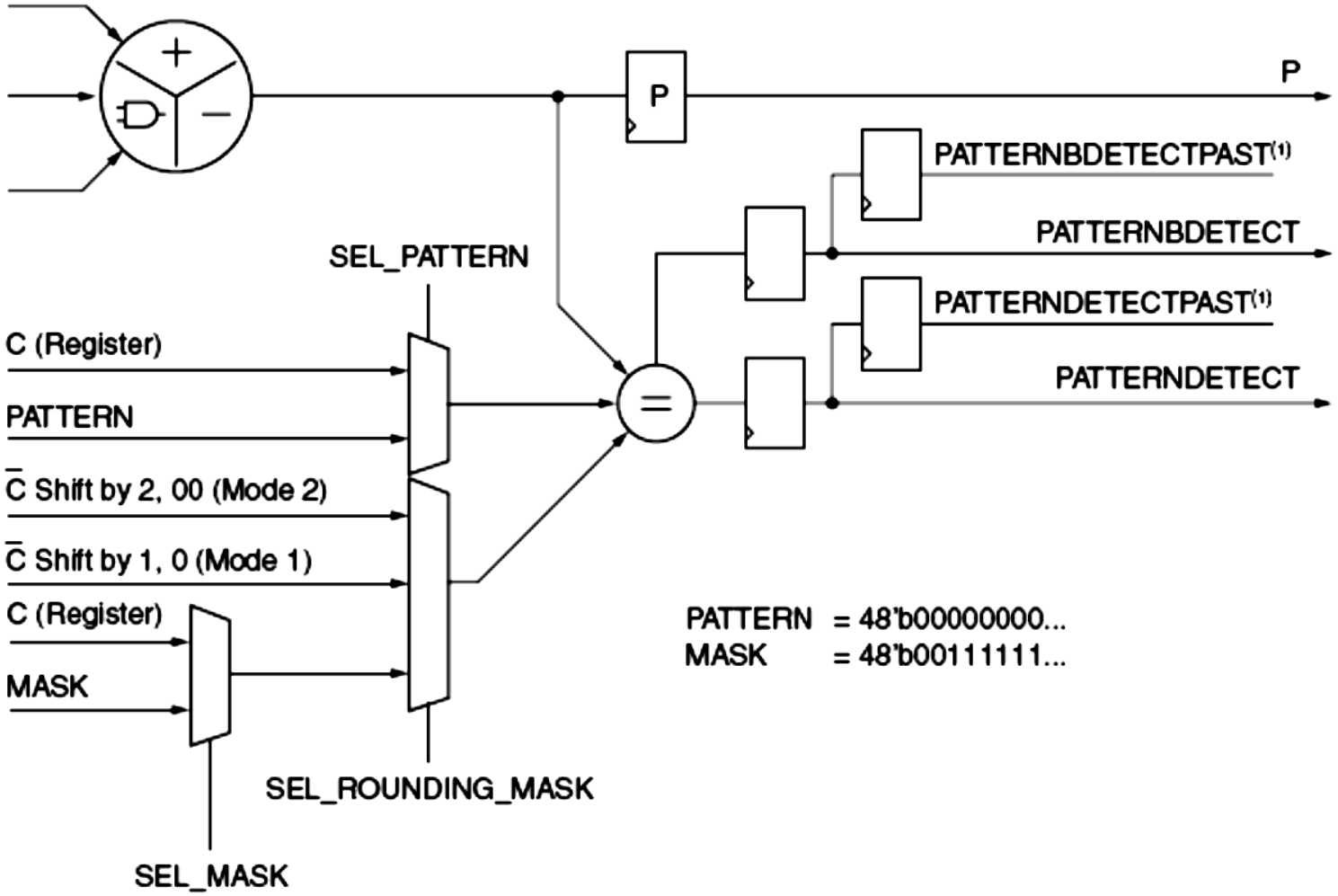
XILINX – funkcjonalność DSP

- 48-bit logic unit
  - Multiply and Add, Mutiply and Accumulate
  - Bit-wise logic operations - two-input AND, OR, NOT, NAND, NOR, XOR, and XNOR
  - Logic unit mode dynamically selectable via ALUMODE
- Pattern detector
  - Overflow/underflow support
  - Convergent rounding support
  - Terminal count detection support and auto resetting
- Cascading 48-bit P bus supports internal low-power adder cascade
  - The 48-bit P bus allows for 12-bit/QUAD or 24-bit/DUAL SIMD adder cascade support
- Optional 17-bit right shift to enable wider multiplier implementation
- Dynamic user-controlled operating modes
  - 7-bit OPMODE control bus provides X, Y, and Z multiplexer select signals

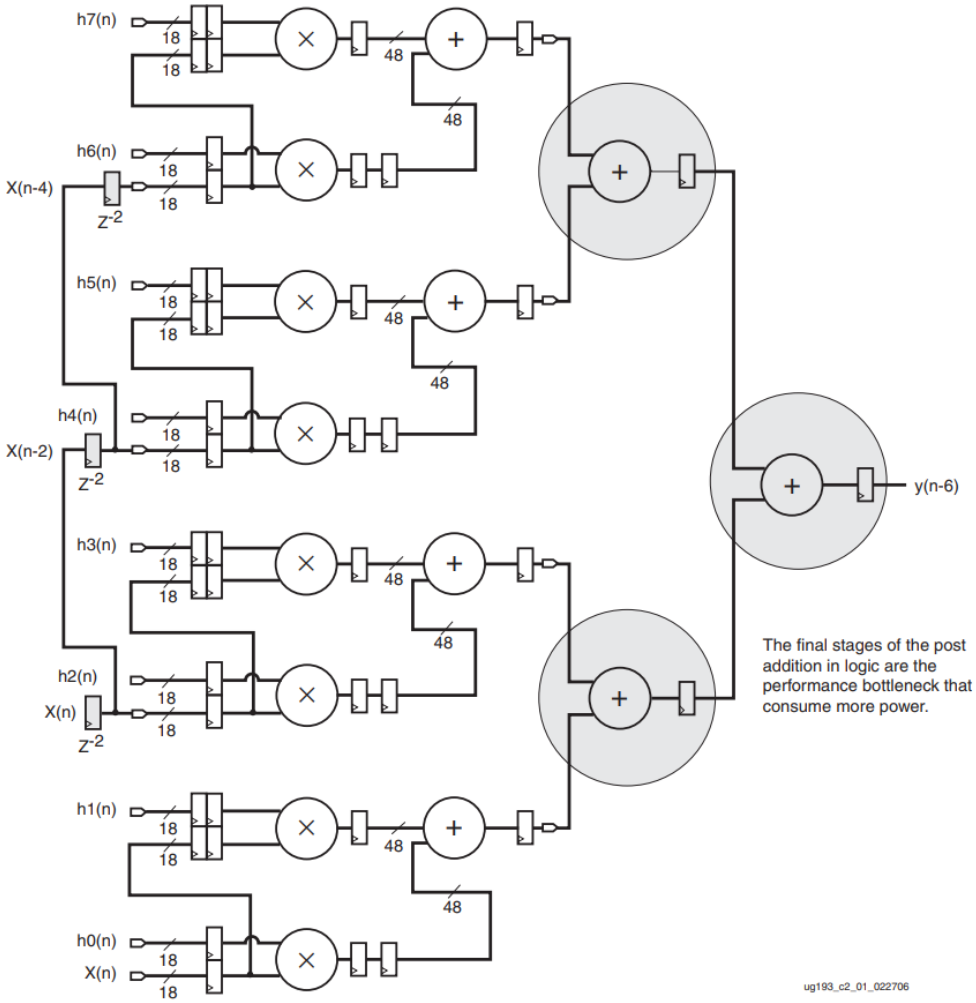
XILINX – DSP – dodatkowa funkcjonalność



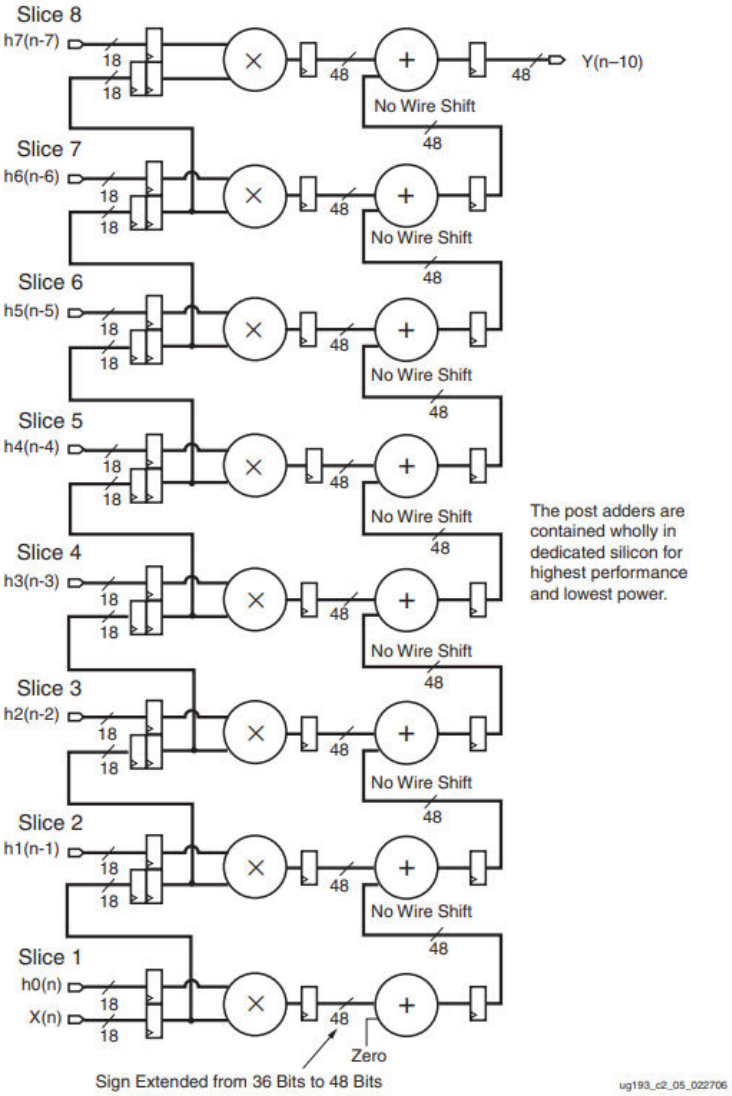
XILINX – DSP – dodatkowa funkcjonalność



Bloki funkcjonalne - DSP48e

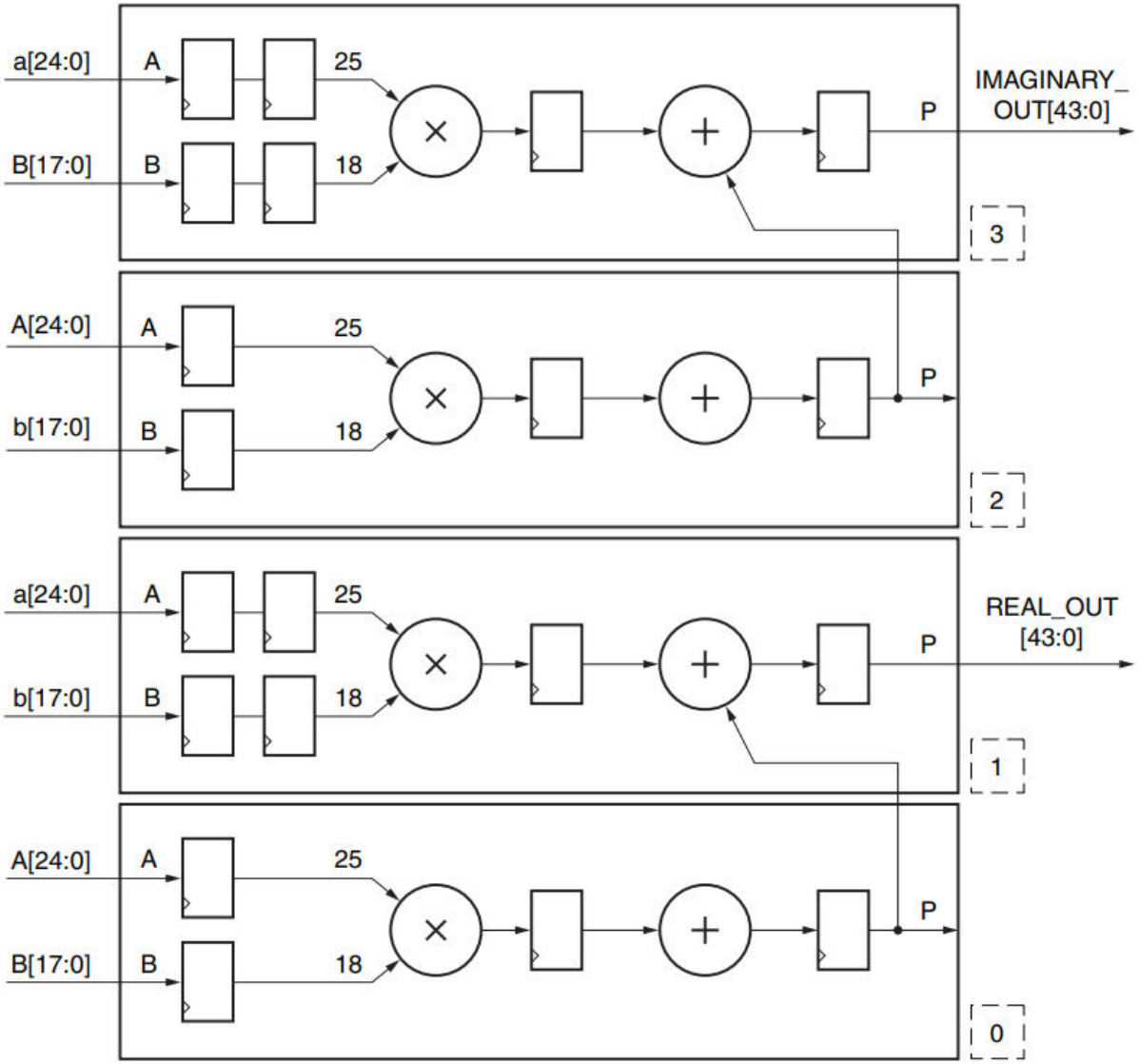


The final stages of the post addition in logic are the performance bottleneck that consume more power.



The post adders are contained wholly in dedicated silicon for highest performance and lowest power.

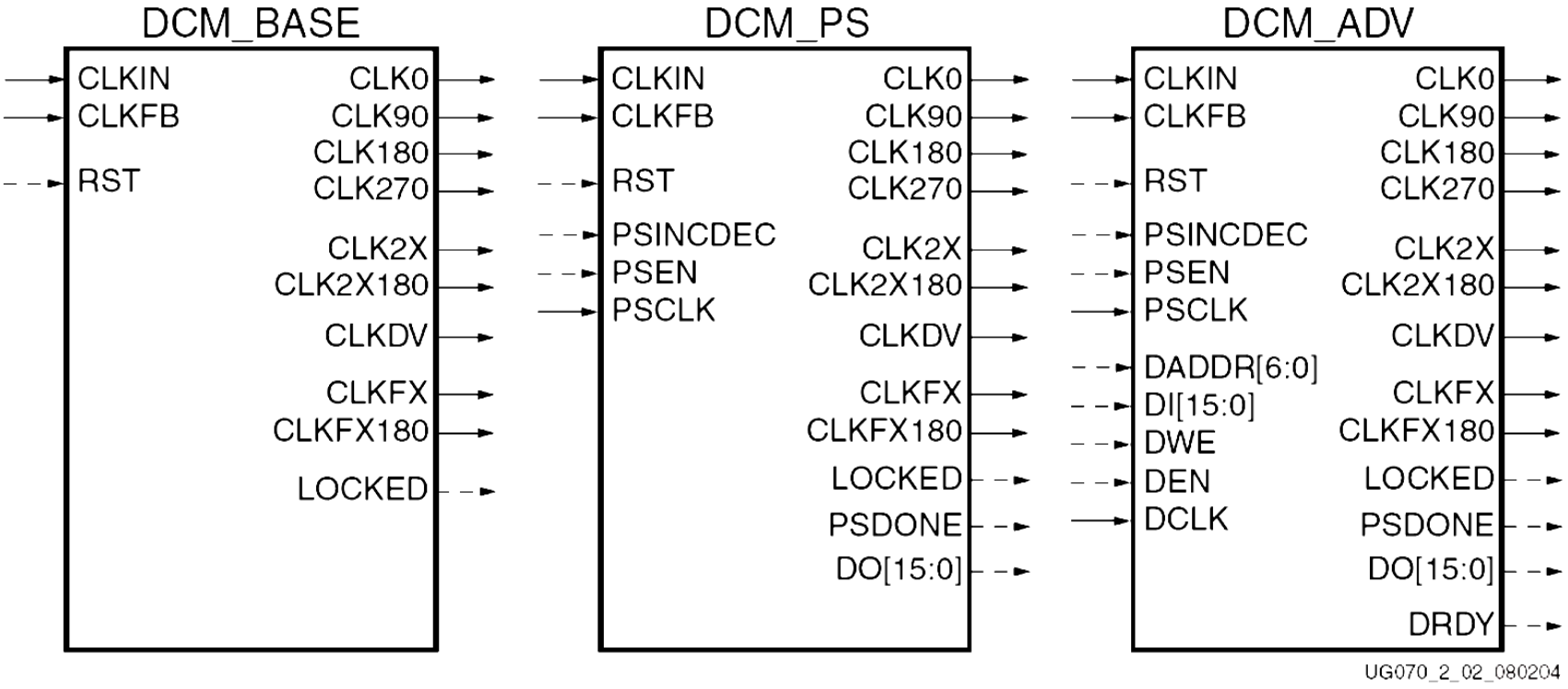
Bloki funkcjonalne - DSP48e



Zarządzanie sygnałem taktującym Digital Clock Manager (DCM)

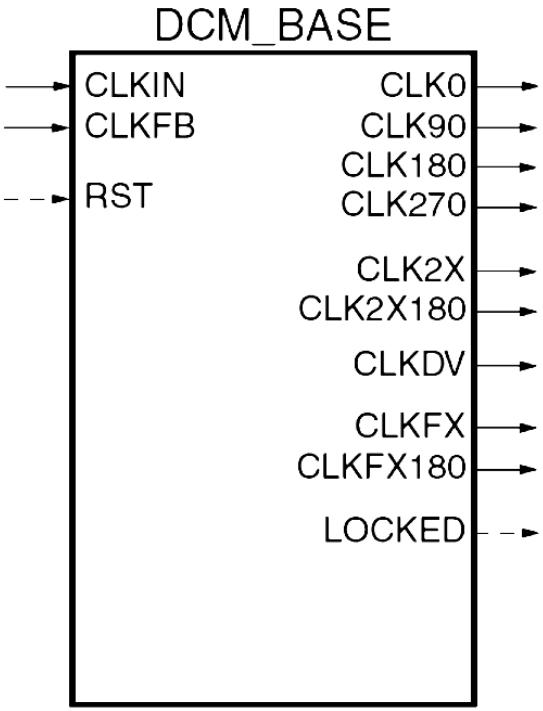
XILINX – DLL/PLL/DLL

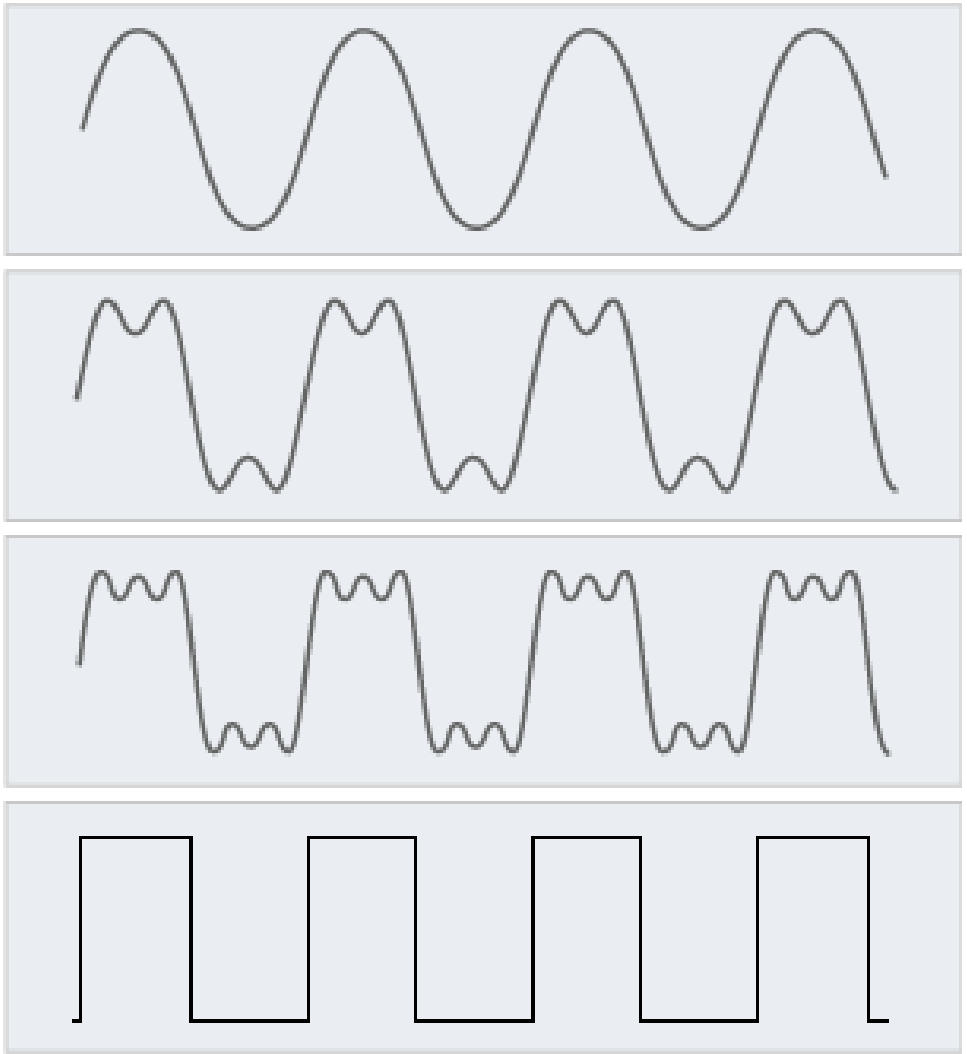
- Digital Clock Manager (DCM)
  - Phase Lock Loop (PLL)
  - Delay-Locked Loop (DLL)



XILINX – DLL/PLL/DLL

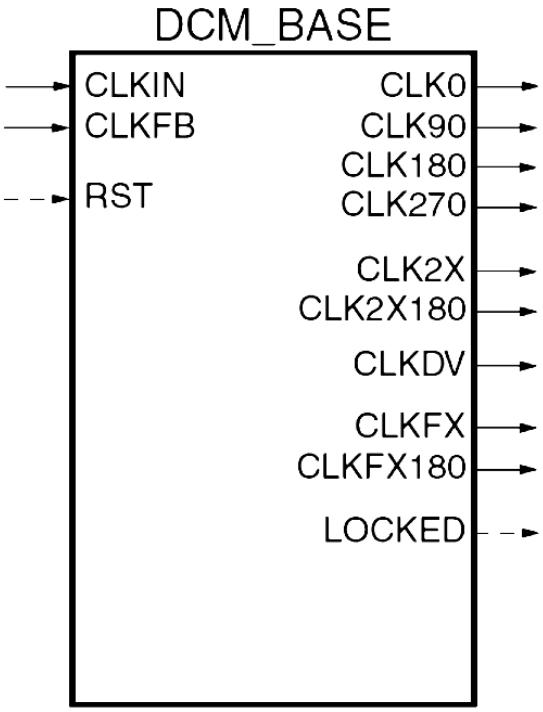
- Wejście CLKIN
  - Zniekształcony przebieg

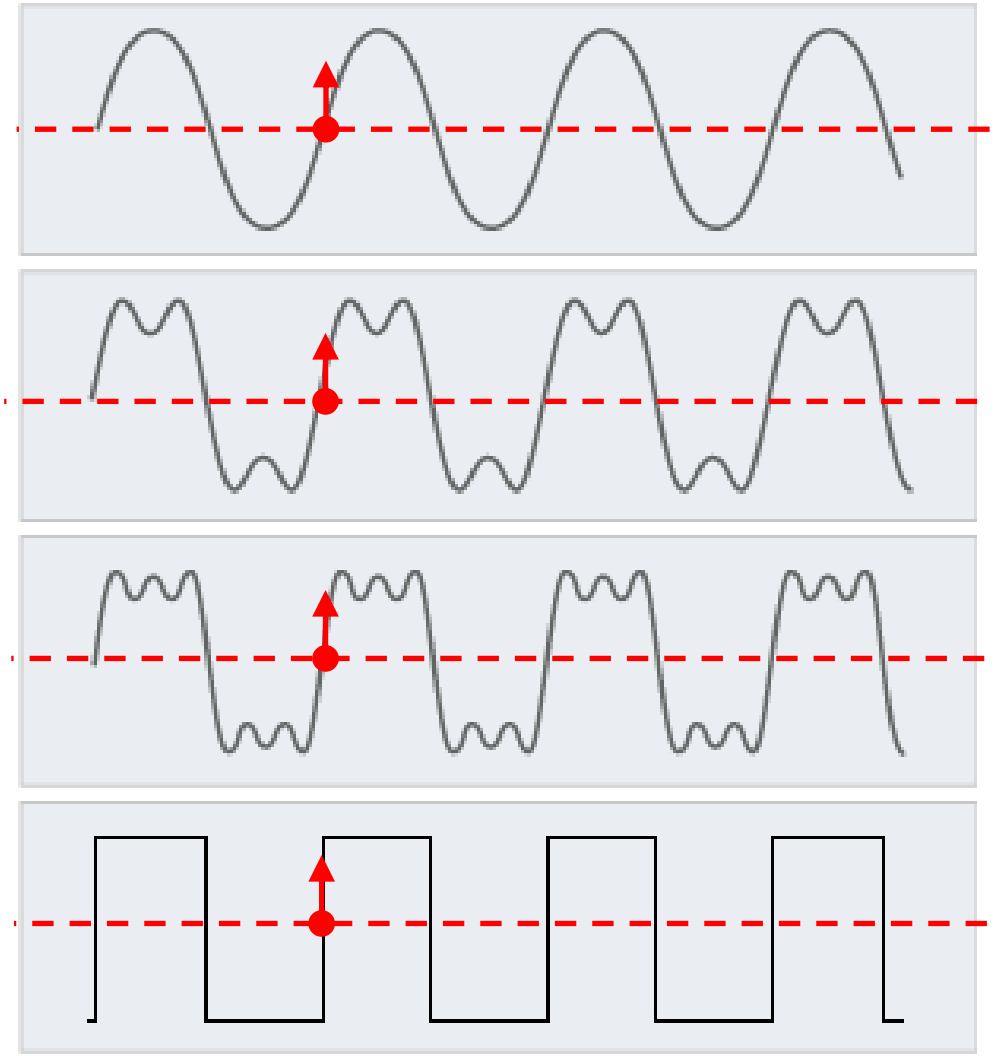




XILINX – DLL/PLL/DLL

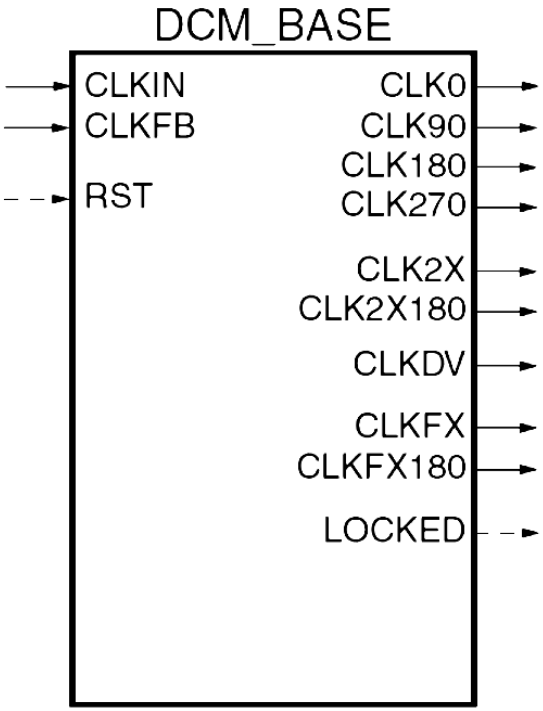
- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentu zbocza

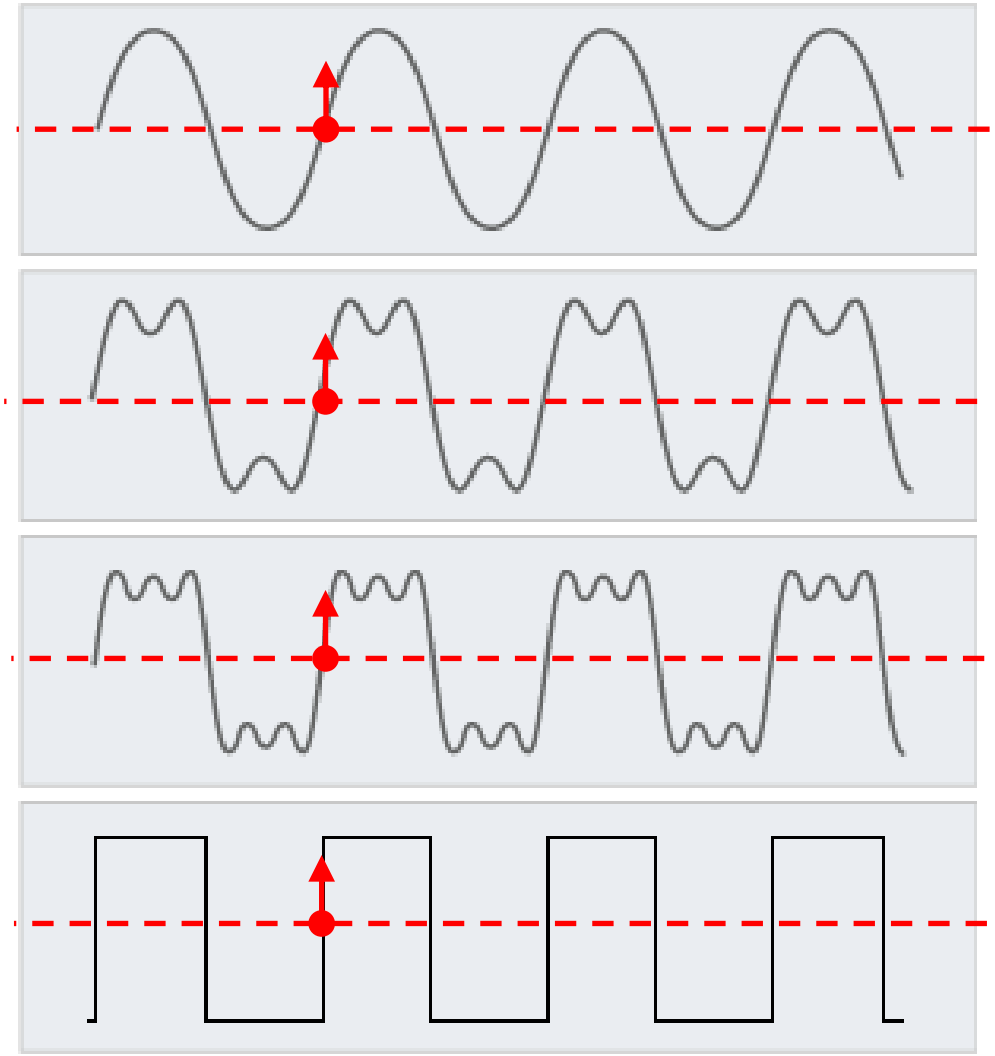




XILINX – DLL/PLL/DLL

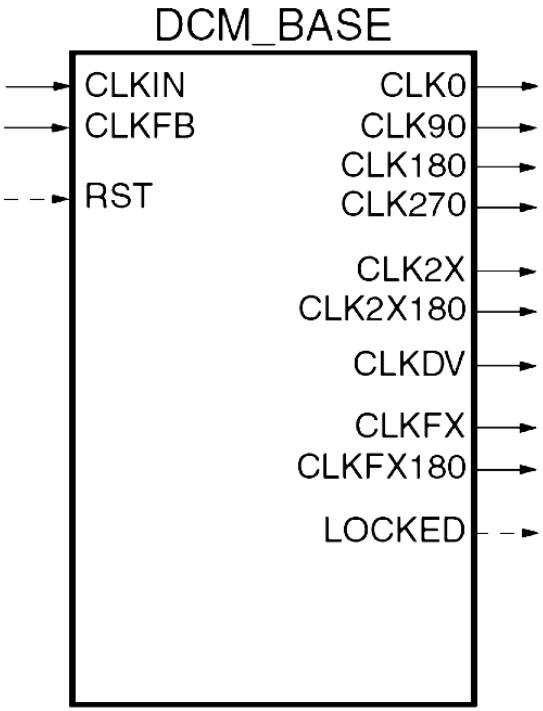
- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentu zbocza

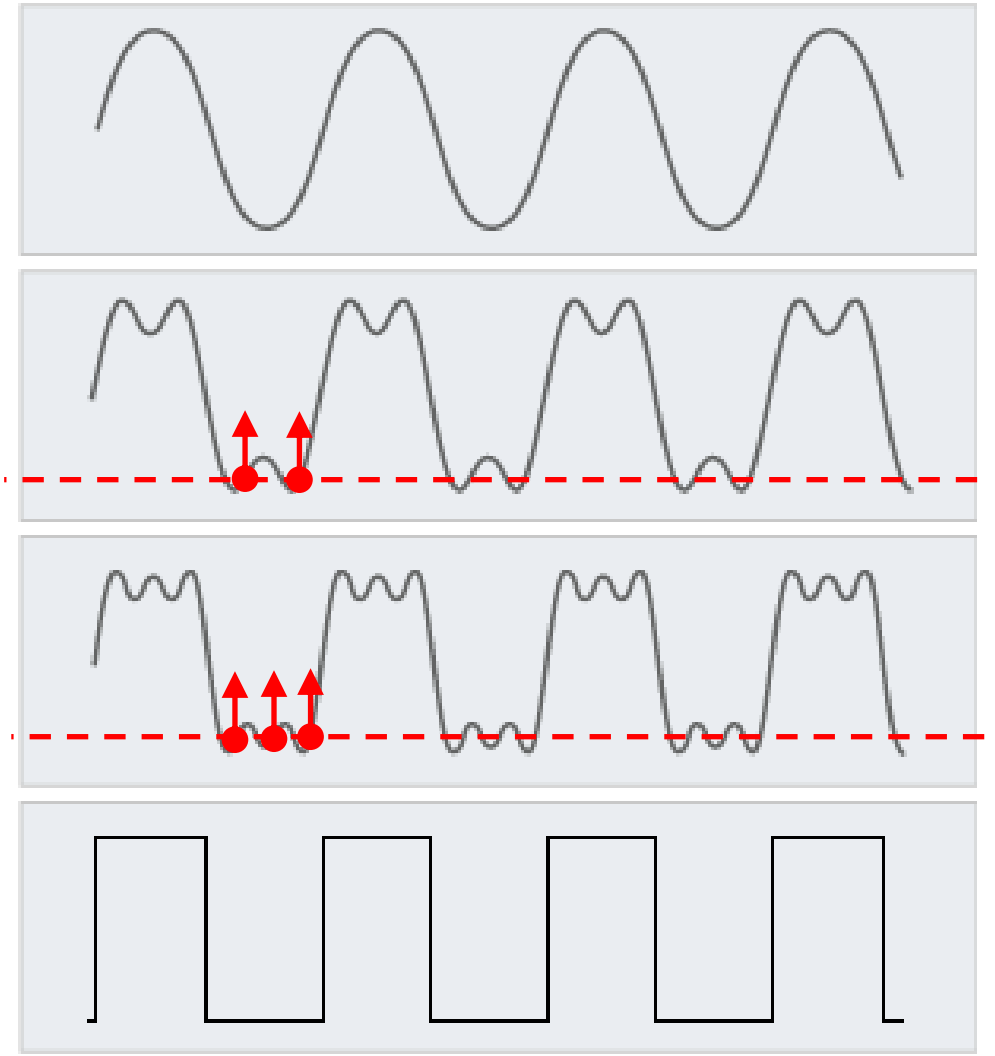




XILINX – DLL/PLL/DLL

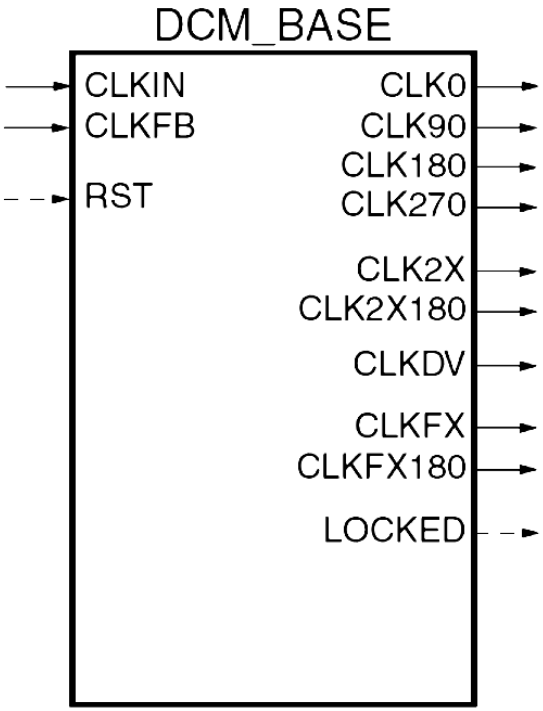
- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentów zboczy

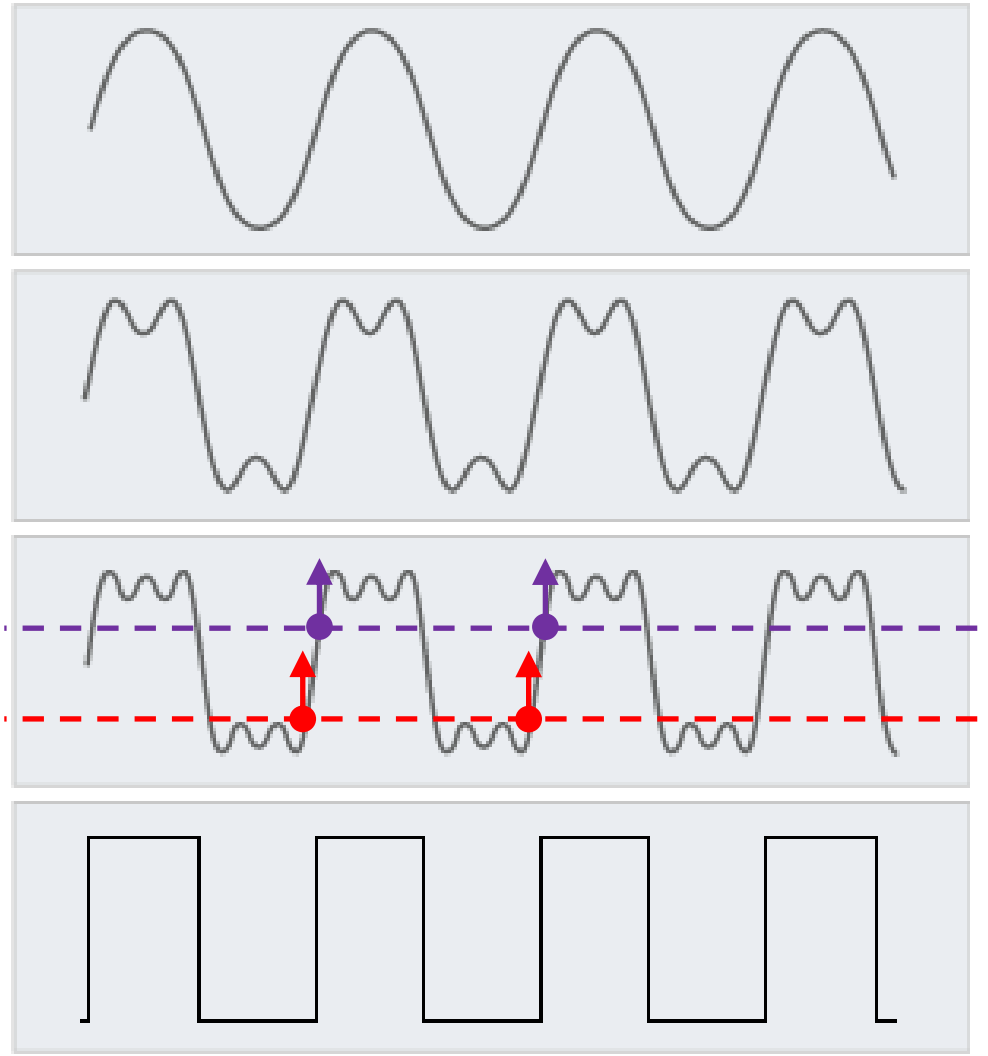




XILINX – DLL/PLL/DLL

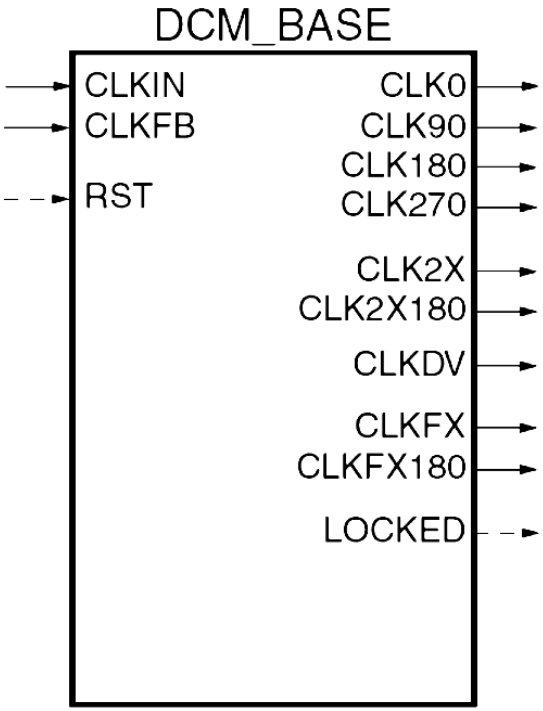
- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentów zboczy

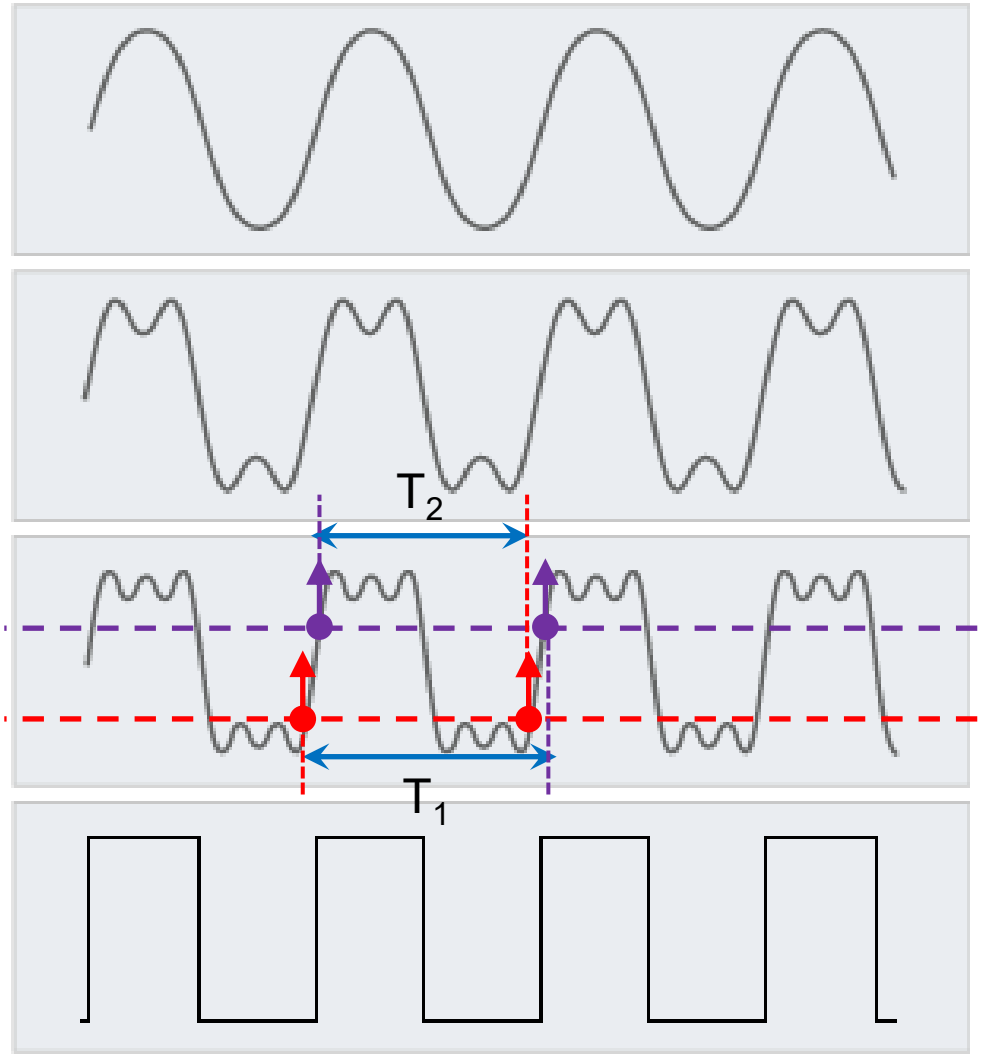




XILINX – DLL/PLL/DLL

- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentów zboczy
  - Jitter

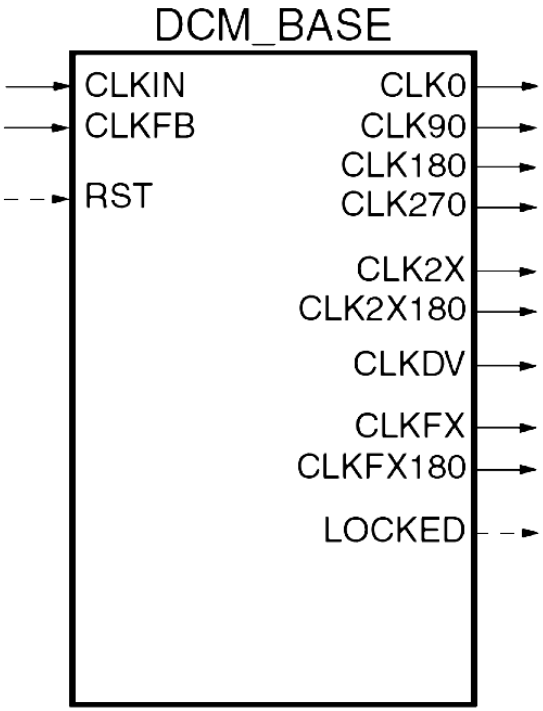


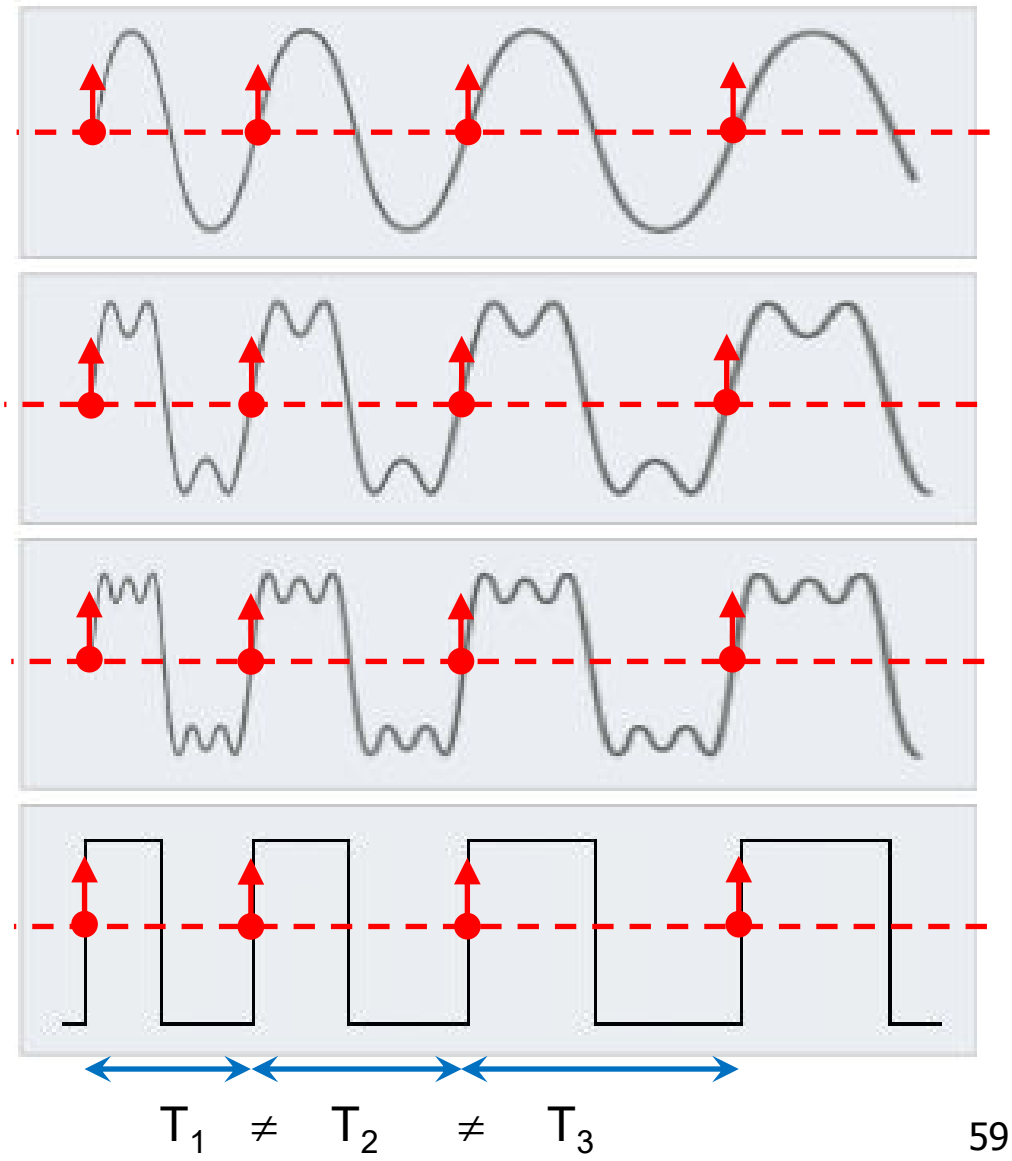


$T_1 \neq T_2$

XILINX – DLL/PLL/DLL

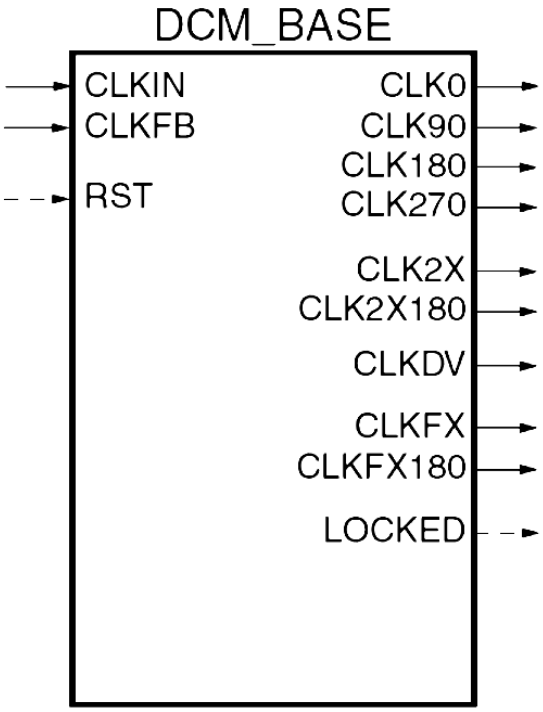
- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentów zboczy
  - Wahania częstotliwości
    - Jitter!





XILINX – DLL/PLL/DLL

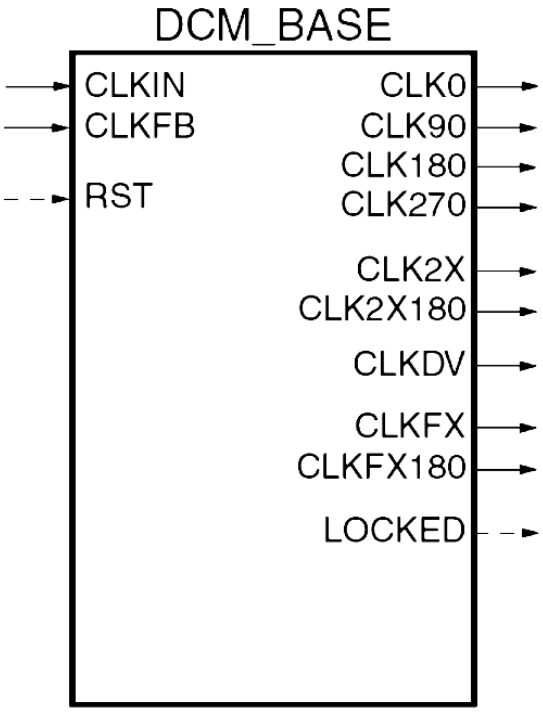
- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentów zboczy
  - Jitter



- Warunek działania układu?
  - Sygnały muszą zdążyć
    - Sygnał na ścieżce krytycznej musi zdążyć
  - Zdążyć przez okresem zegara
    - Okresem zegara z Jitterem
- Deska ratunku?
  - Uwzględniamy najdłuższą ścieżkę
  - Uwzględniamy najkrótszy możliwy okres
- Cena
  - 15-30% niższy zegar
  - Np. 70MHz zamiast 100MHz

XILINX – DLL/PLL/DLL

- Wejście CLKIN
  - Zniekształcony przebieg
  - Zła detekcja momentów zboczy
  - Jitter

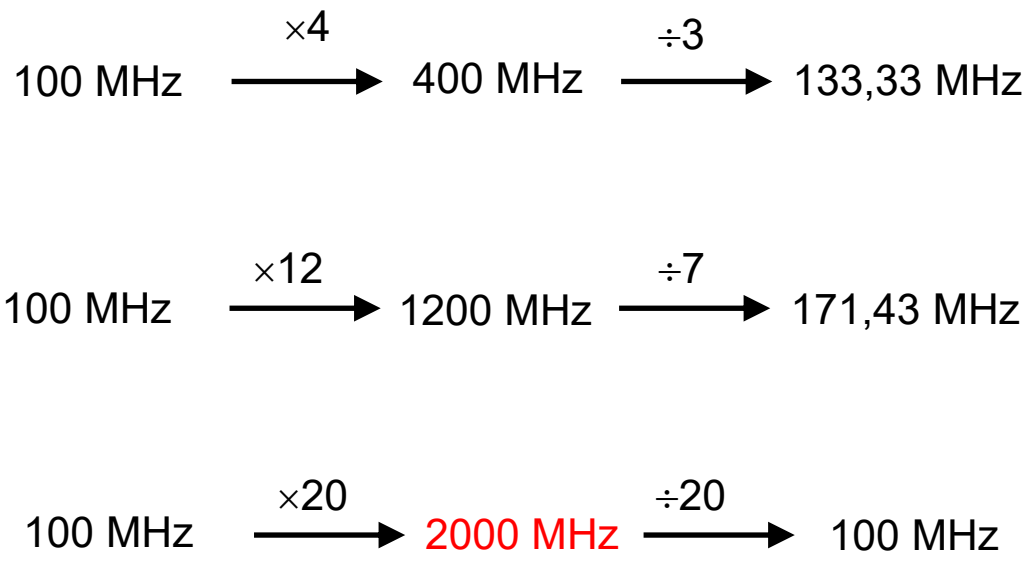


- Wyjścia
  - Czysty zegar
  - Idealny prostokąt
  - Stabilna częstotliwość
  - CLK
    - Ta sama częstotliwość
    - Fazy 0°,90°,180°,270°
  - CLK2X
    - 2x wyższa częstotliwość
  - Wyjście CLKDV
    - „Podzielona” częstotliwość
  - CLKFX
    - „Zmieniona” częstotliwość
  - LOCKED – czy zegar jest już stabilny

XILINX – DLL/PLL/DLL

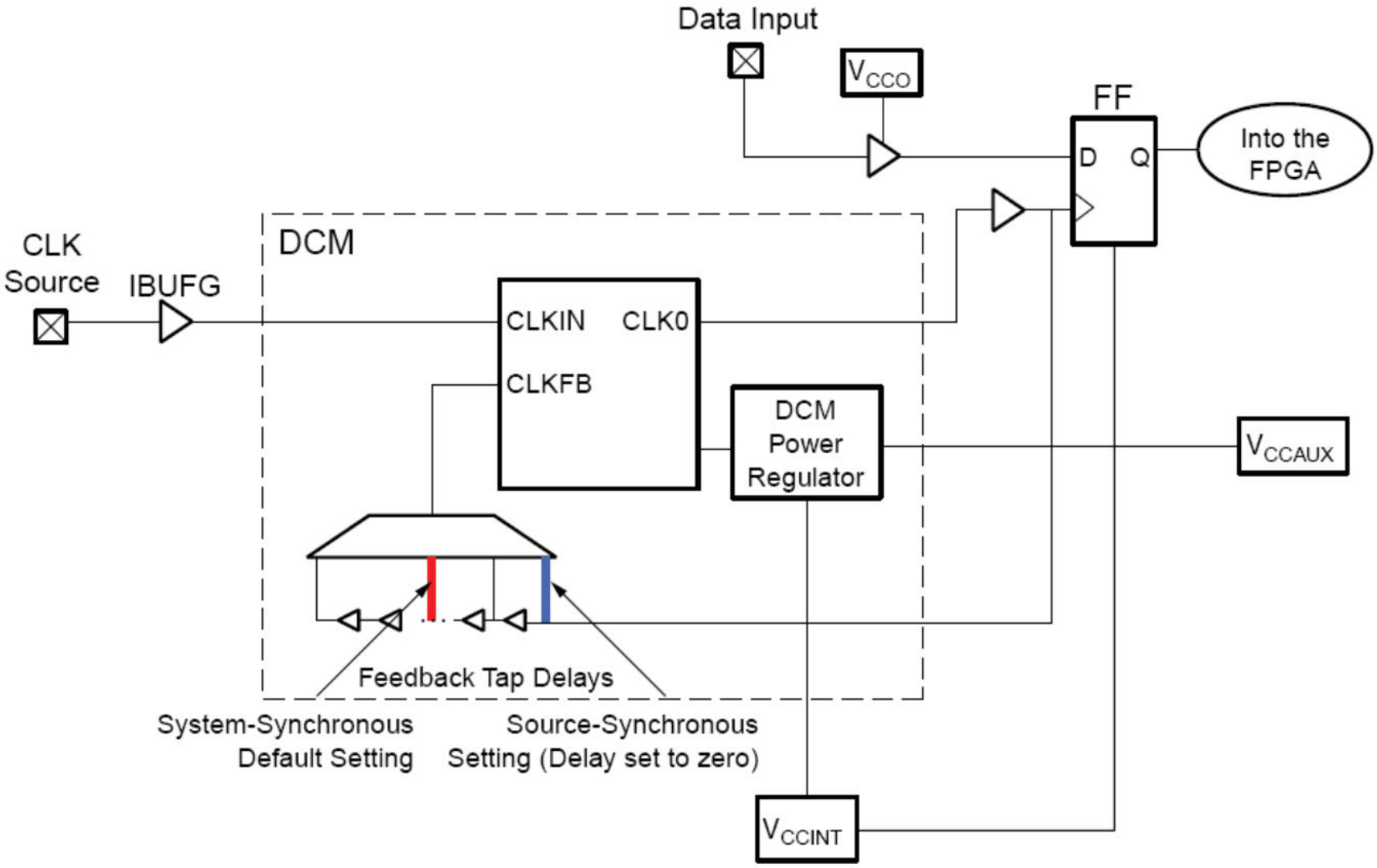
DCM Attribute Name	Description	Values	Default Value
CLKDV_DIVIDE	This attribute controls CLKDV such that the source clock is divided by N. This feature provides automatic duty cycle correction such that the CLKDV output pin has a 50/50 duty cycle always in low-frequency mode, as well as for all integer values of the division factor N in high-frequency mode.	Real: 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8, 9, 10, 11, 12, 13, 14, 15, 16	2.0
CLKFX_DIVIDE		Integer: 1 to 32	1
CLKFX_MULTIPLY		Integer: 2 to 32	4
CLKIN_PERIOD	This specifies the source clock period to help DCM adjust for optimum CLKFX/CLKFX180 outputs.	Real in ns	0.0
CLKIN_DIVIDE_BY_2	This attribute allows for the input clock frequency to be divided in half when such a reduction is necessary to meet the DCM input clock frequency requirements.	Boolean: FALSE or TRUE	FALSE

XILINX – DLL/PLL/DLL



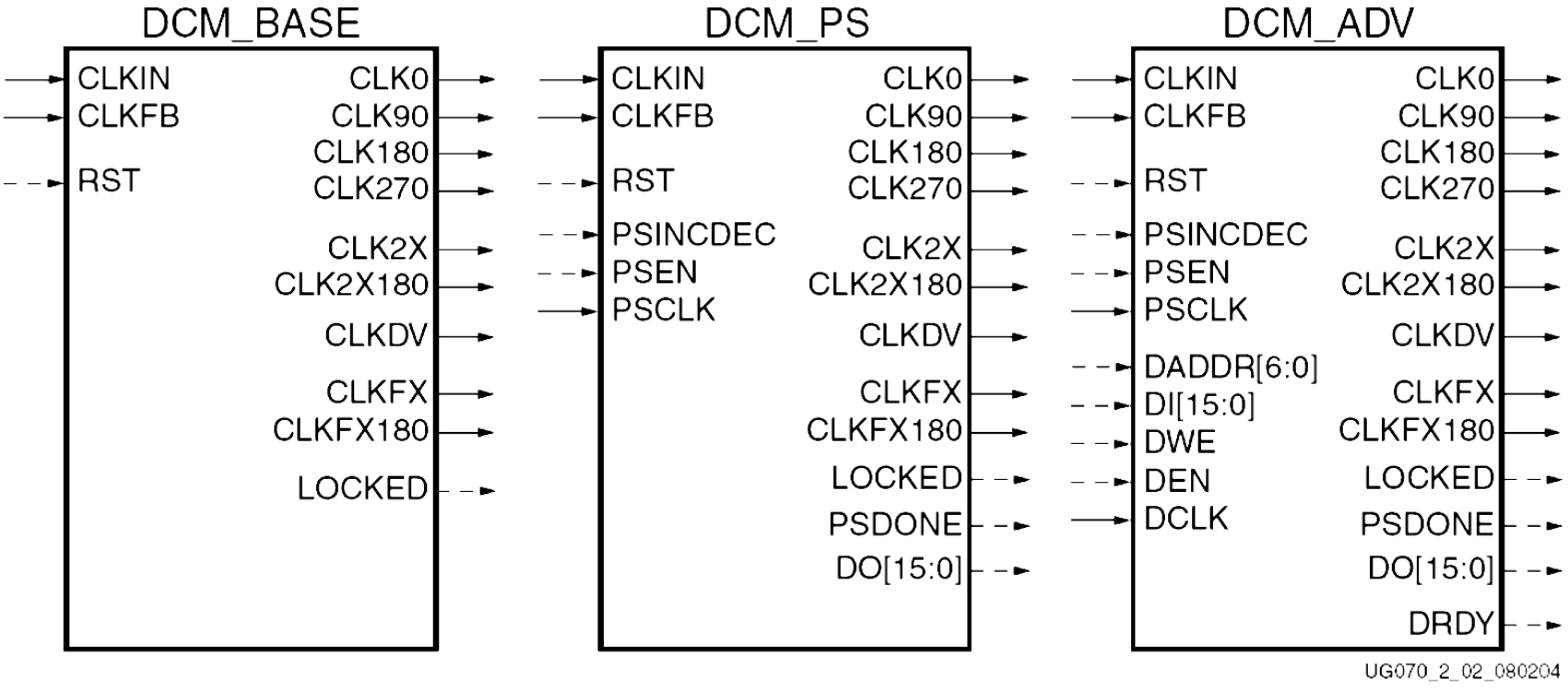
- Uwaga!
  - Częstotliwość pośrednia nie może przekroczyć limitów danego układu FPGA
  - Typowo 1 – 2 GHz
  - Nie ma to związku z wyjściową częstotliwością!

XILINX - DLL


Data Input
V_{CCO}
FF
D
Q
Into the FPGA
CLK Source
IBUFG
DCM
CLKIN
CLK0
CLKFB
DCM Power Regulator
V_{CCAUX}
Feedback Tap Delays
System-Synchronous Default Setting
Source-Synchronous Setting (Delay set to zero)
V_{CCINT}

XILINX – DLL/PLL/DLL

- DCM_PS – dynamicznie sterowanie fazą
- DCM_ADV – pełne sterowanie dynamiczne, w trakcie pracy układu



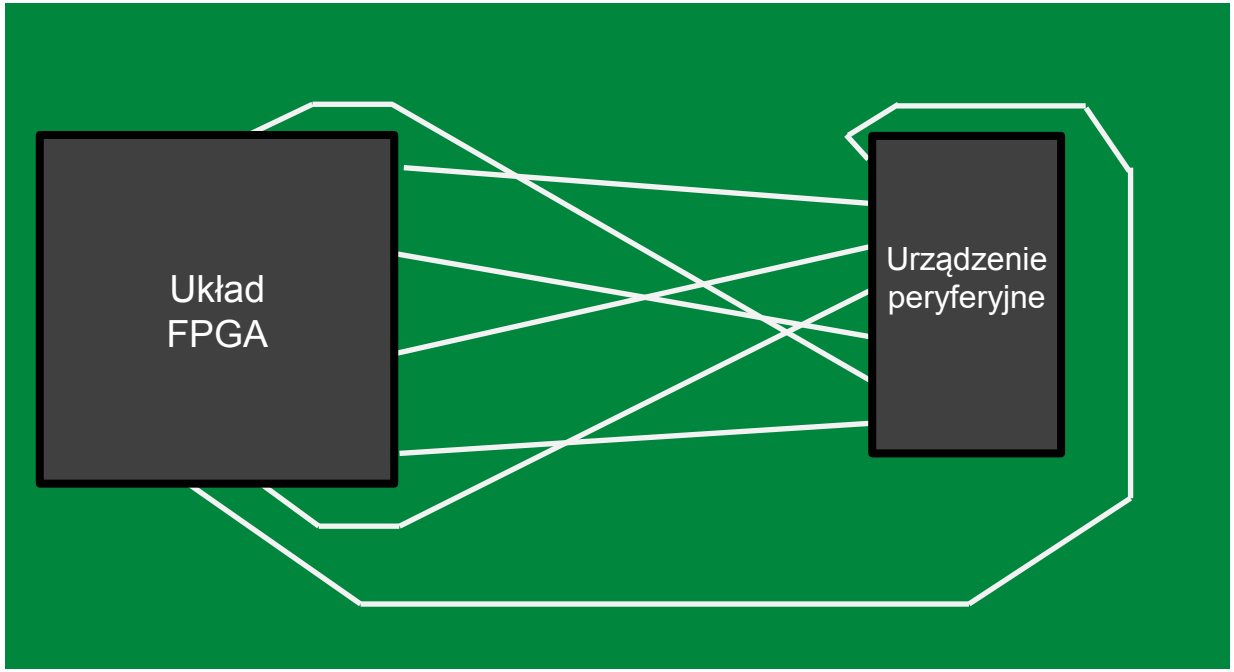
Porty wejścia / wyjścia

Porty wejścia wyjścia IOB

- Input/Output Buffer Block (IOB)
  - Pozwala konfigurować piny układu
    - Wejście
    - Wyjście
    - Wejściowo/Wyjściowe (bufor trójstanowy)
  - Częstotliwość pracy pinu taka jak matrycy
    - Sygnał zegarowy np. 100-300 MHz
    - W niektórych układach ograniczenia fizyczne
      - np. maksymalnie 150MHz
  - Podzielone na banki
    - Różne standardy napięć
      - 1.8V 2.5V, 3.3V, 5V, …
  - Upraszcza rutowanie płytek PCB (Printed Circuit Board)

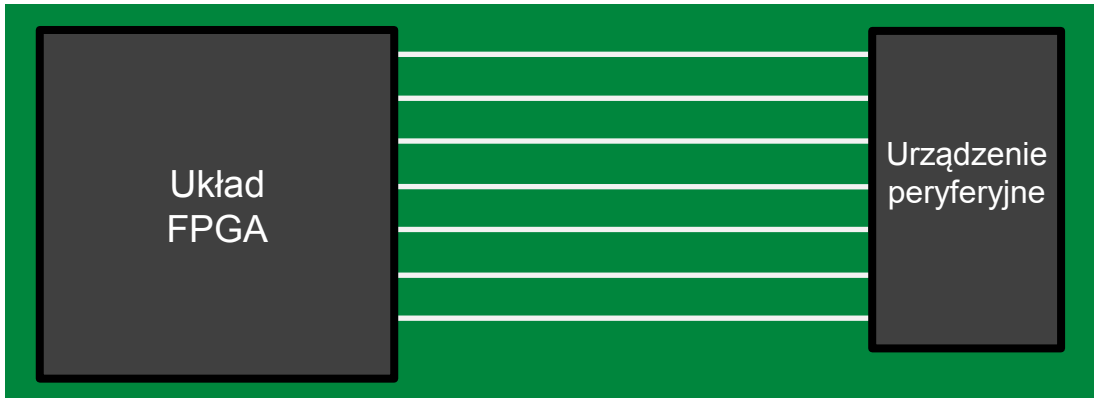
Rekonfigurowanie pinów

- Procesor/mikrokontroler
  - Sztywne przypisanie funkcji pinów układu
  - Przecinające się ścieżki – wiele warstw
  - Utrudnione rutowanie
  - Efektywnie większa i droższa płytka PCB

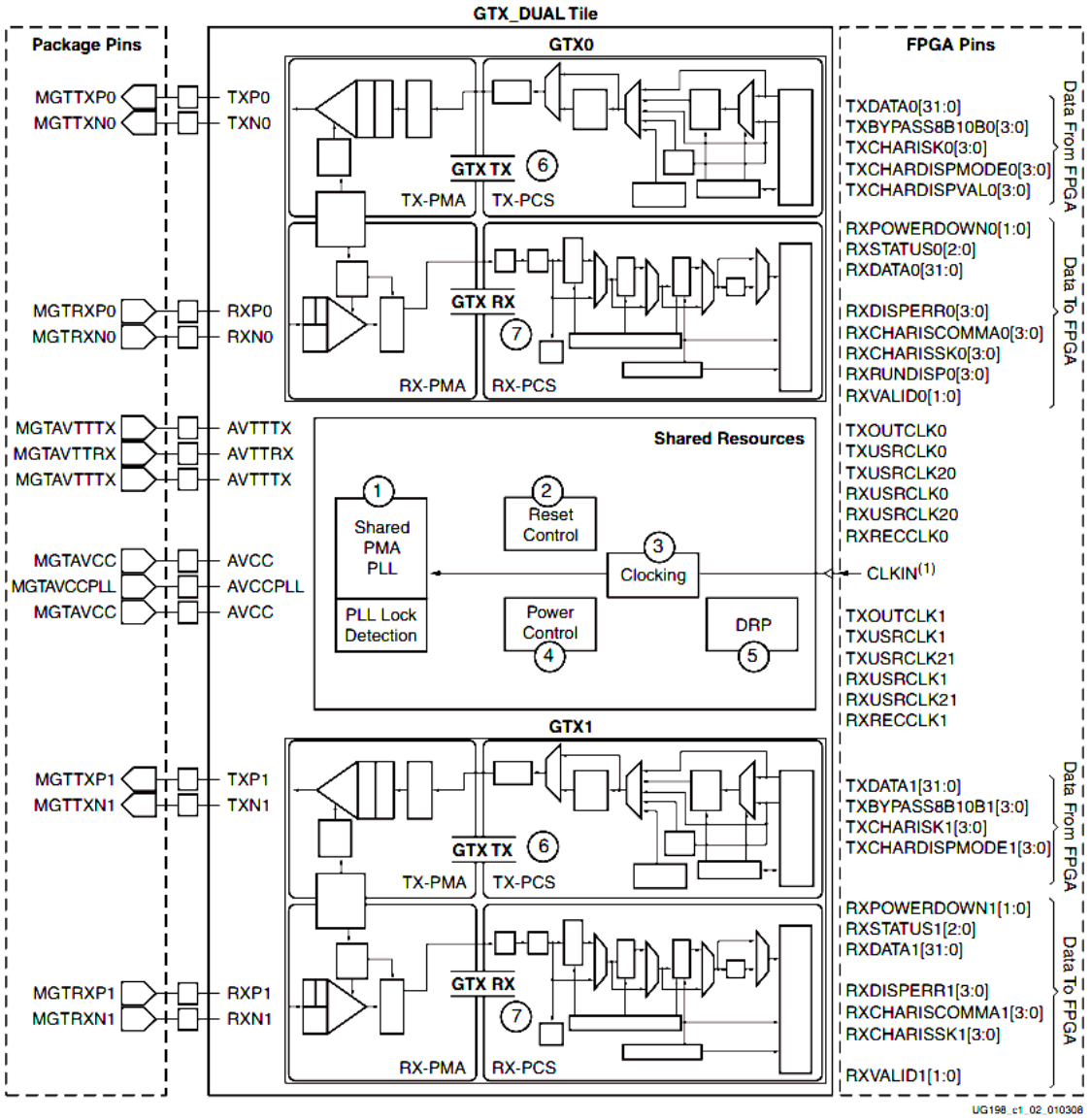


Rekonfigurowanie pinów

- Układ FPGA
  - Sami wybieramy funkcje pinów układu
  - Prowadzenie magistral
  - Efektywnie mniejsza i tańsza płytka PCB

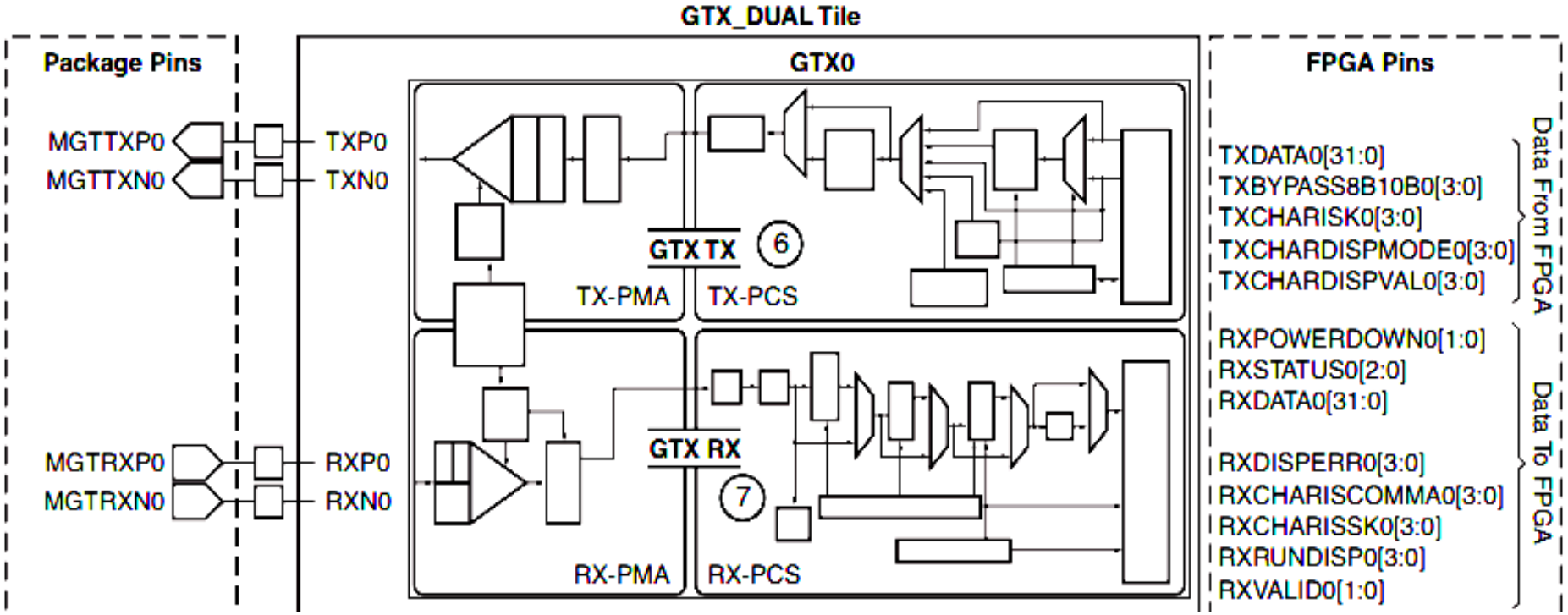


XILINX – porty GTP/GTX

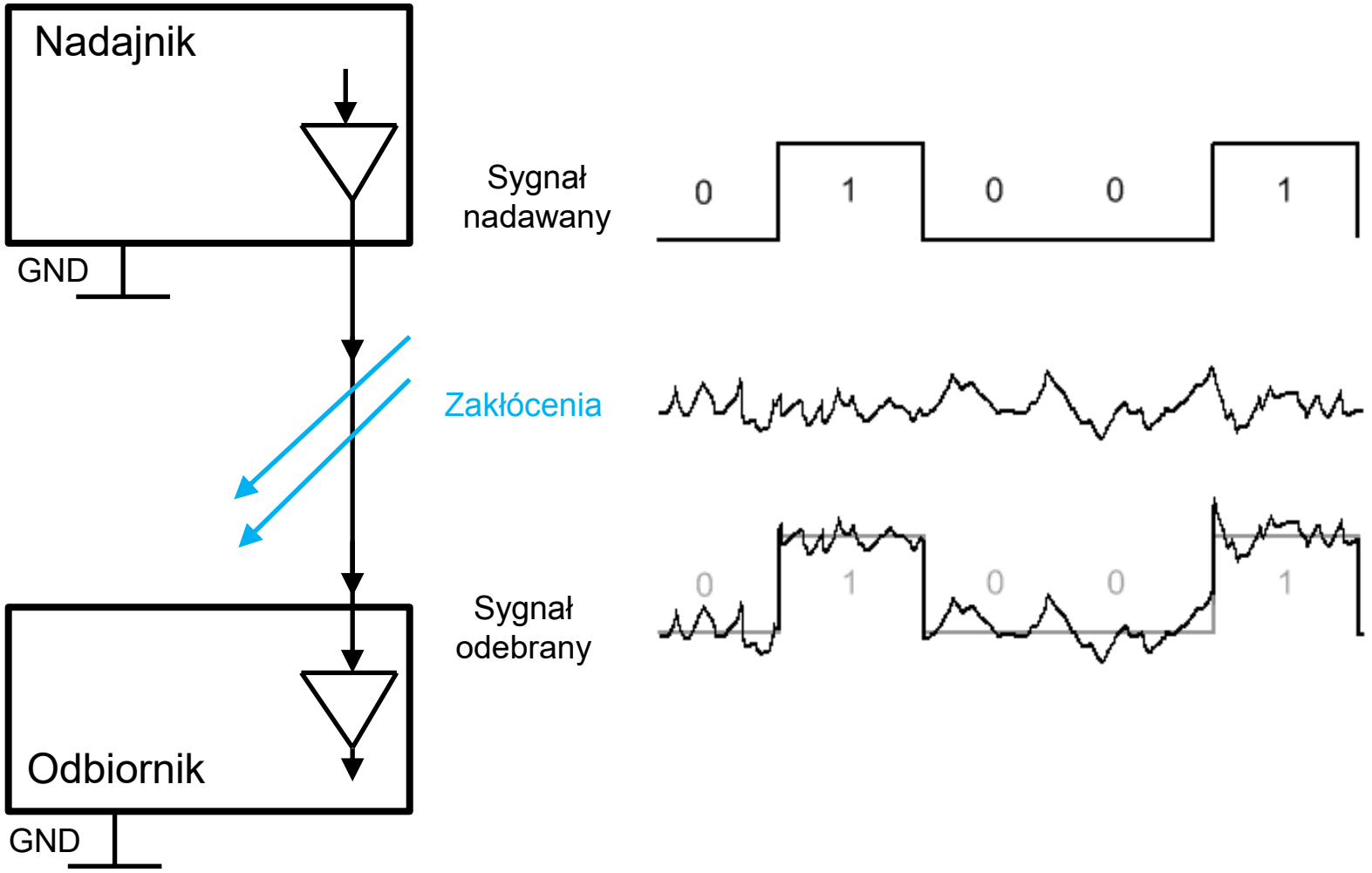


XILINX – porty GTP/GTX

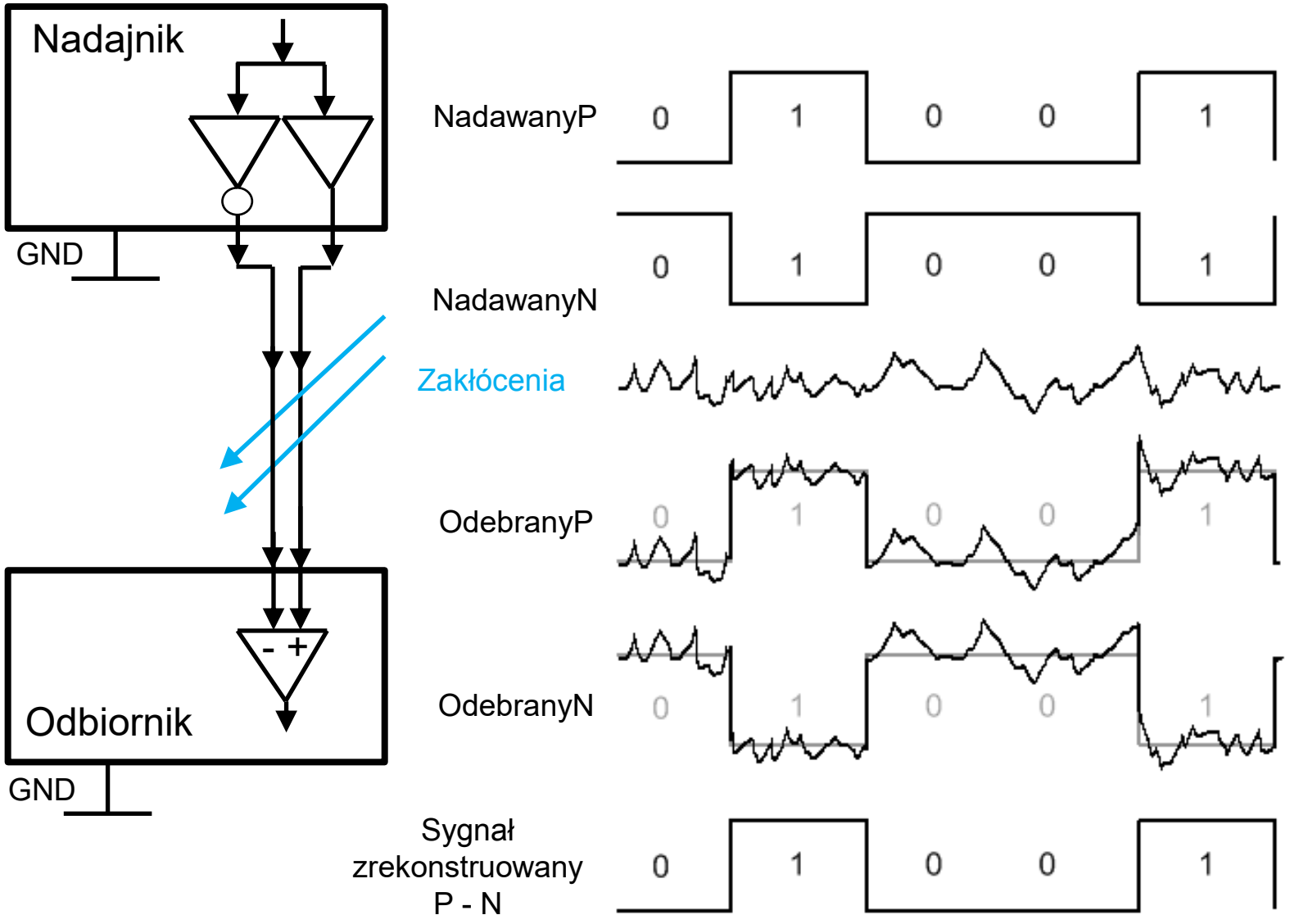
- Pary różnicowe – mniejsza stopa błędu
- Rejestry DDR – 2x, 4x wyższa prędkość
- Serializery/deserializery – np.16x wyższa prędkość



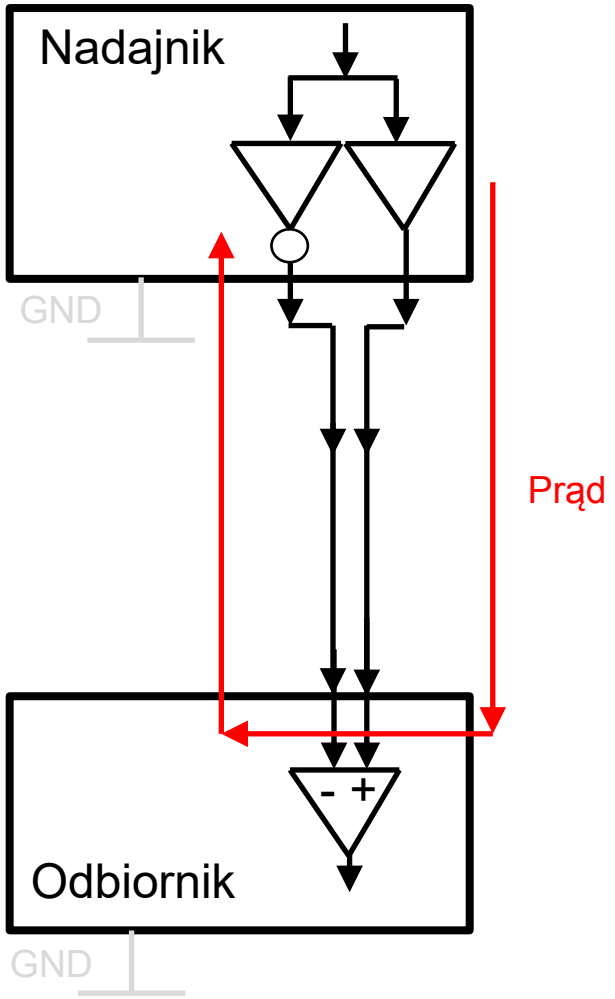
Transmisja Single-Ended (Asymetryczna)



Transmisja różnicowa (Differential)



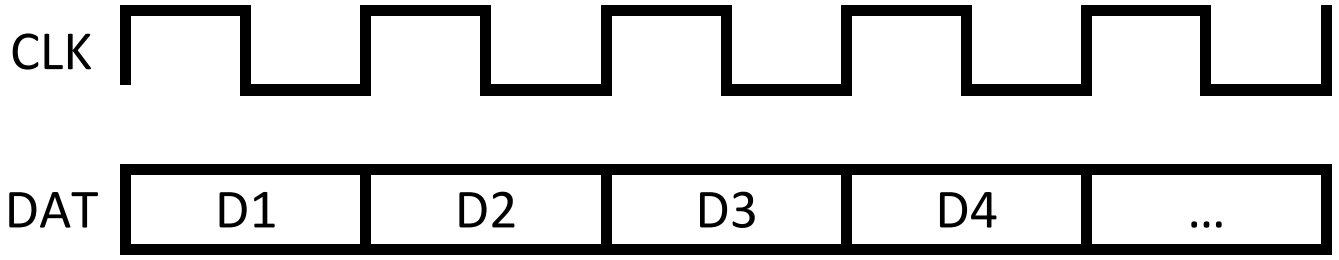
Transmisja różnicowa (Differential)



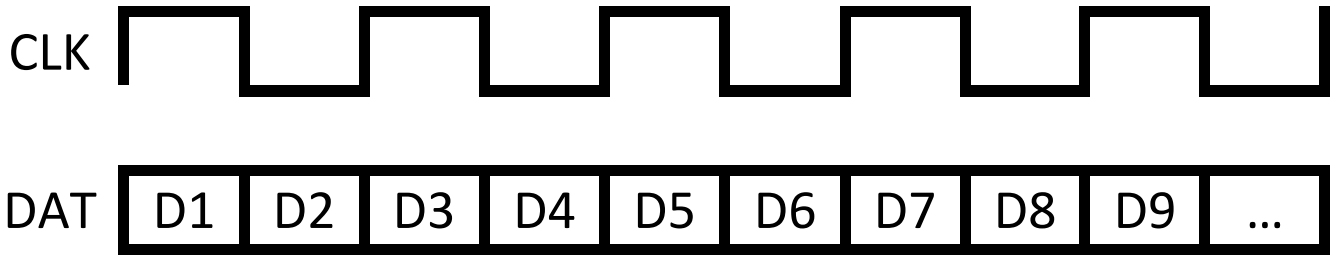
- Zalety
  - Kompensacja zakłóceń
  - Zamknięcie pętli przepływu prądu
  - Możliwe wyższe prędkości transmisji
- Wady
  - Dwa razy większa liczba linii do prowadzenia
  - Konieczność dopasowywania długości ścieżek
  - Niezbędny nadajnik sygnałów P i N

Double Data Rate (DDR)

- Single Data Rate (SDR)

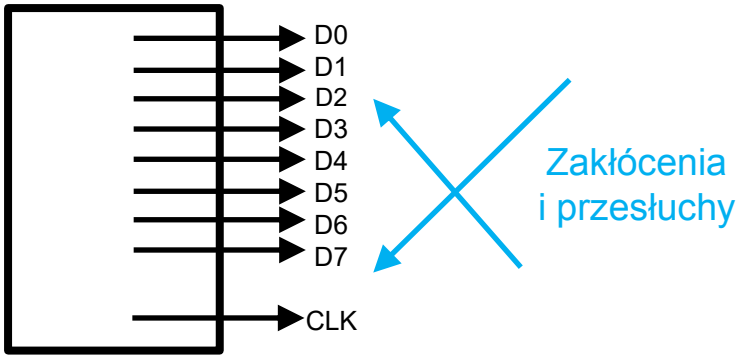


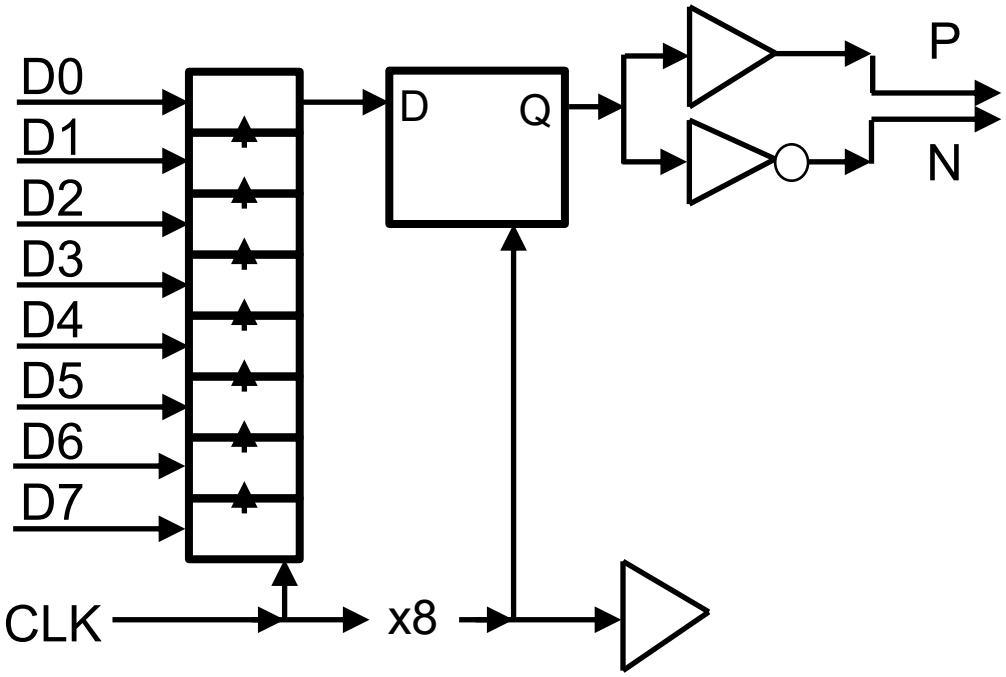
- Double Data Rate (DDR)



Serializacja/Deserializacja

- Zamiast wysyłać bity słowa **równolegle**
  - Np. słowo 8 bitowe
    - Na 8 pinach w Single-Ended
    - Na 8 parach różnicowych
  - + CLK
- Wysyłamy bity słowa **szeregowo**
  - Bit po bicie
  - Przy wyższym zegarze
  - Na pojedynczym „włóknie”
    - 1 pin w Single-Ended
    - 1 para różnicowa
  - + CLK





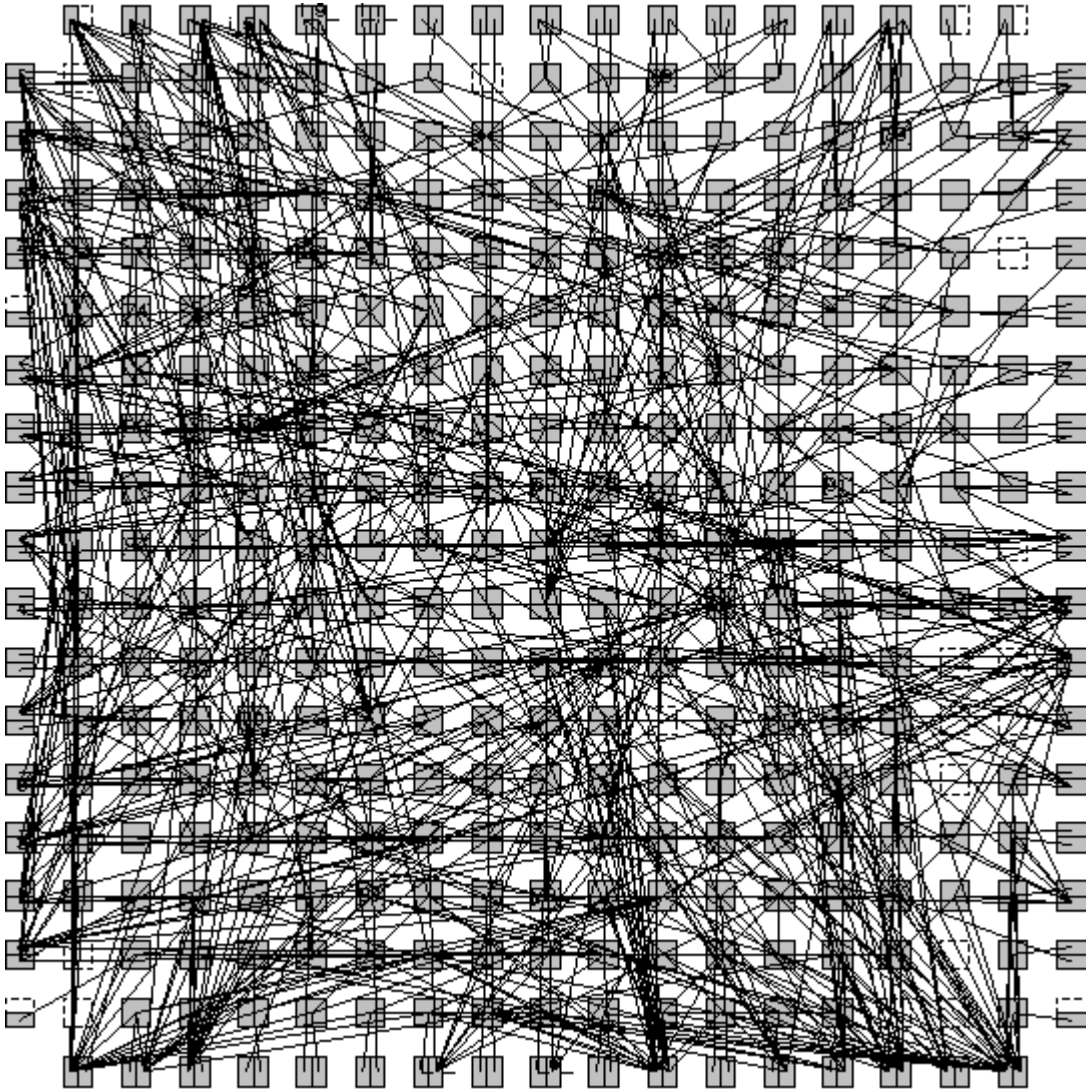
XILINX – porty GTP/GTX

Compatible Protocols	Data Rates	Miscellaneous Features
PCI Express, Rev. 1.0a PCI Express, Rev. 1.1	2.5 Gb/s	• TX receive detect • Loss of Signal (LOS)/Idle state detect • Low power states • Beacon signaling • Ground referenced termination
PCI Express, Rev. 2.0	5.0 Gb/s	
Interlaken	3.125 Gb/s, 6.25 Gb/s	Header insertion/extraction for 64B/67B
XAUI 802.3ae D5p0	3.125 Gb/s	LOS
OIF-CEI6G	6.25 Gb/s	• No Equalizer (EQ), low power mode • 4-tap adaptive DFE
OC-12/48	622.08/2488.32 Mb/s	• Allows FIFO bypassing for synchronous operation • 5x digital oversampling
FC-1, Revision 4.0	1.0625 Gb/s	Rate negotiation (allows operating the TX and RX at different speeds)
FC-2, Revision 4.0	2.125 Gb/s	
FC-4, Revision 4.0	4.25 Gb/s	
10GFC	3.1875 Gb/s	
SDI HD-SDI DVB-ASI 3G-SDI	176/270/360 Mb/s 1.485/1.4835 Gb/s 270 Mb/s 2.970 Gb/s	5x digital oversampling
10G Base-CX4 802.3ak/D4.0	3.125 Gb/s	
Gigabit Ethernet (1000BASE-CX 802.3z/D5.0)	1.25 Gb/s	
SATA Generation 1/2, Rev. 1.0a SATA Generation 2, Rev. 1.0a	1.5 Gb/s 3.0 Gb/s	• Rate negotiation for Generation 2 (entire link operates at Generation 1/Generation 2 speeds) • LOS • OOB beacon
Serial RapidIO	1.25/2.5/3.125/6.25 Gb/s	
CPRI, Version 2.0	614.4/1228.8/2457.6 Mb/s	5x digital oversampling
Infiniband, Volume 2, Release 1.1	2.5 Gb/s	
SFI-5	2.488 – 3.125 Gb/s	Synchronous clocking (bypass FIFOs)
OBSAI RP3, Spec. Issue 1.0	768/1536/3072 Mb/s	
Aurora	150 Mb/s – 6.5 Gb/s	

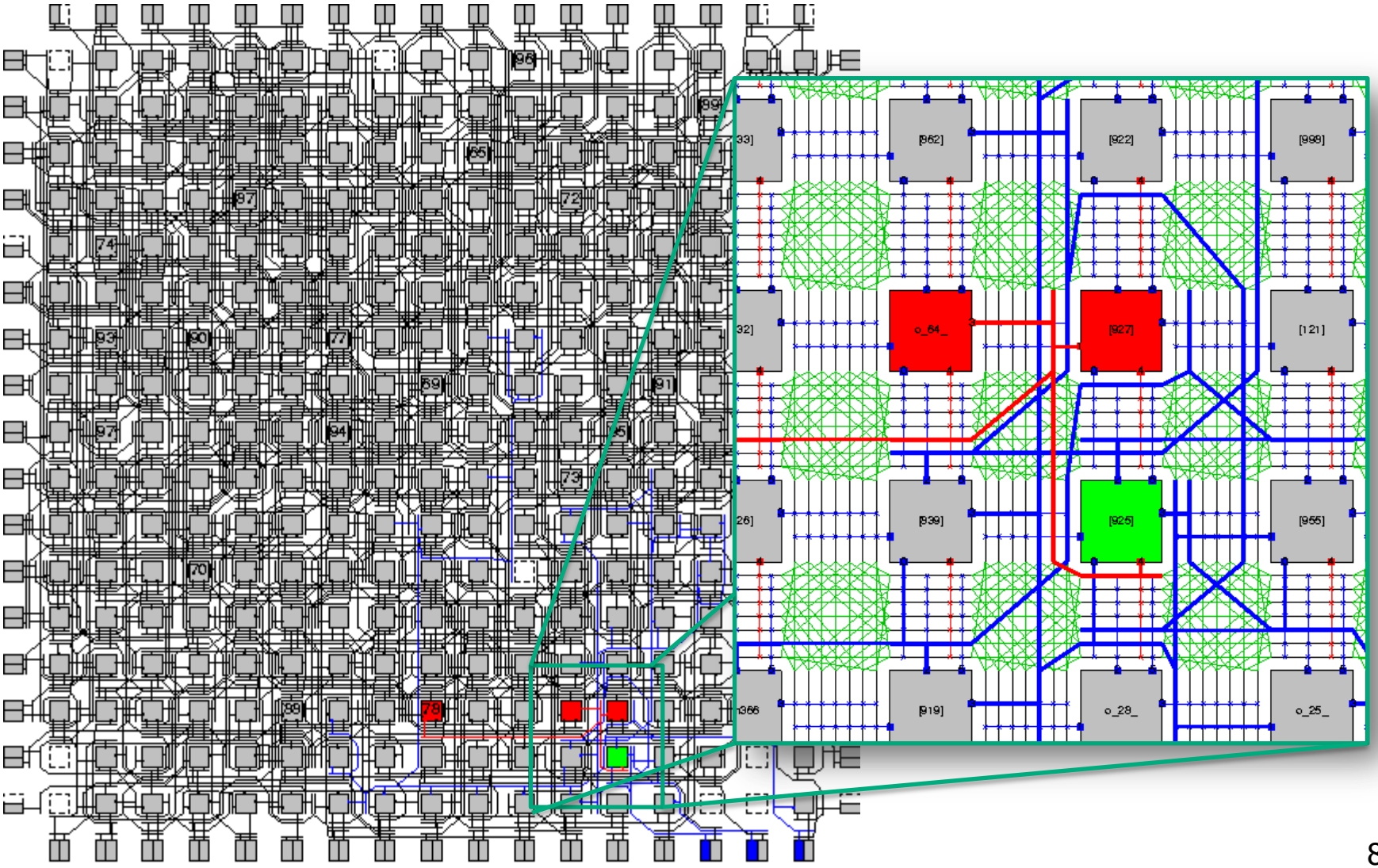
Najistotniejszy zasób w FPGA
(Very Important Resource -VIR)

Macierz rutująca

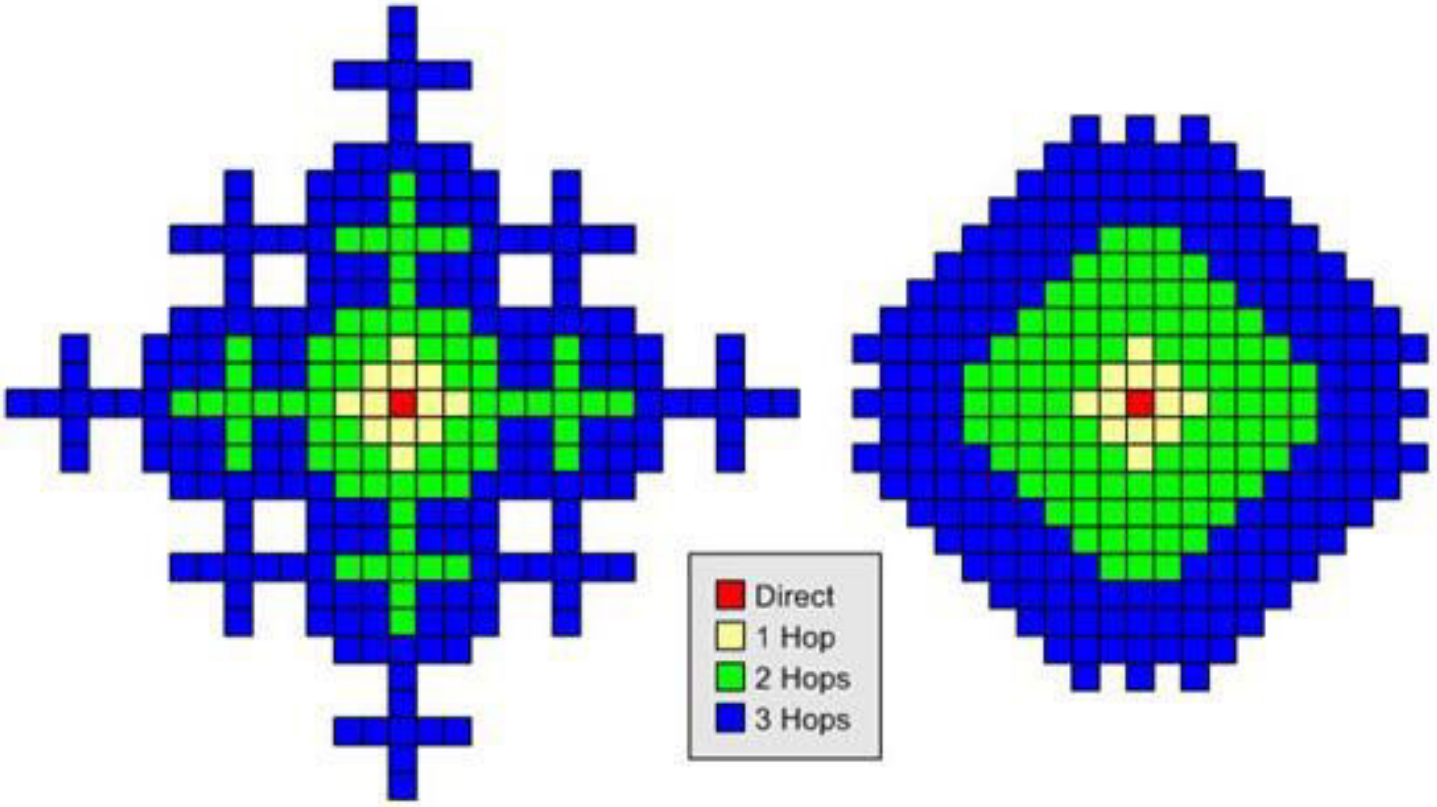
FPGA – Routing field



FPGA – Routing range



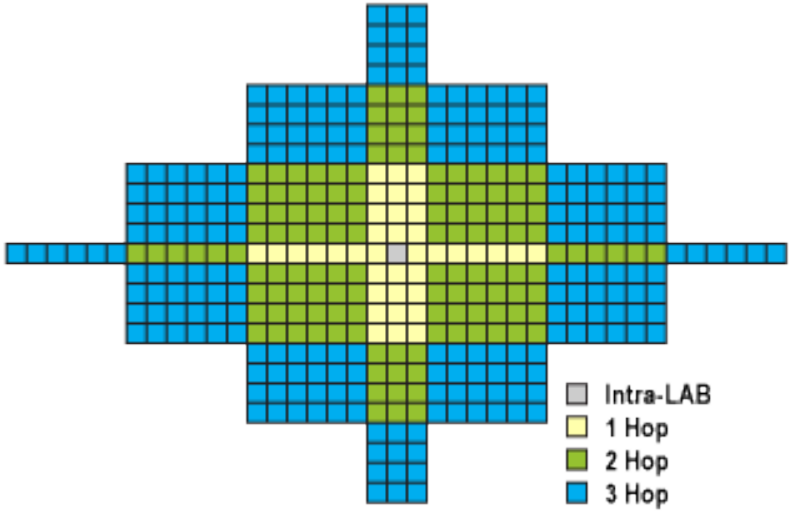
FPGA – Routing range



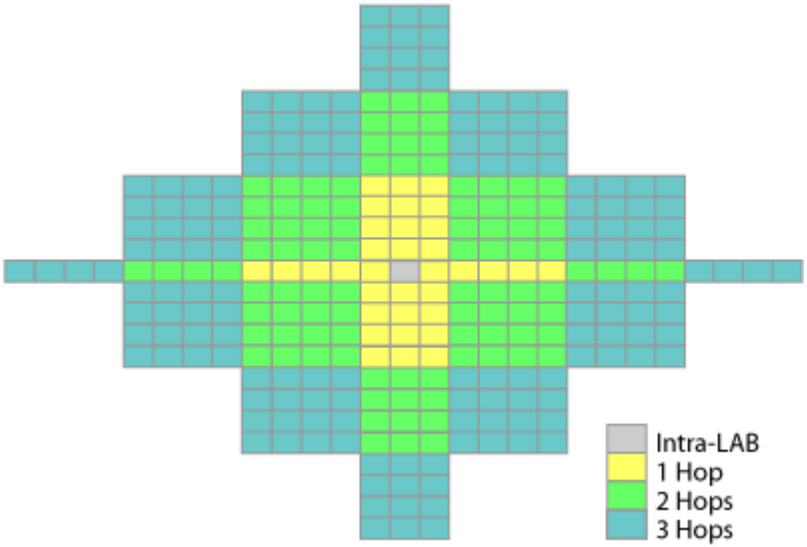
(a) Virtex-4: Traditional interconnect pattern

(b) Virtex-5: Diagonally symmetric interconnect pattern

FPGA – Routing range



STRATIX



ARRIA

Dziękuje za uwagę

- Pytania?

Układy FPGA: cz. 3

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

Różnice i podobieństwa pomiędzy różnymi rodzinami FPGA

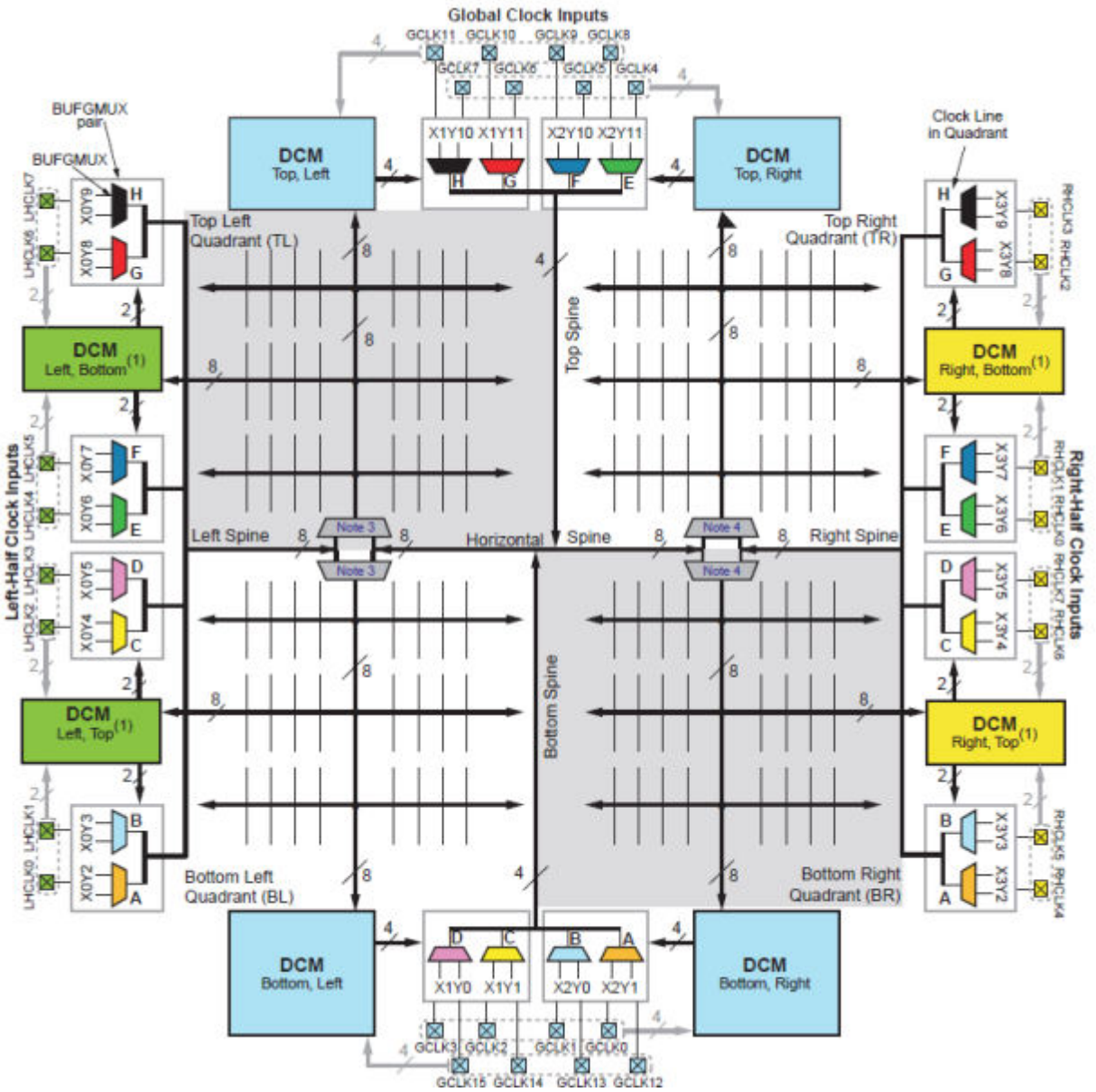

XILINX®


LATTICE
SEMICONDUCTOR.

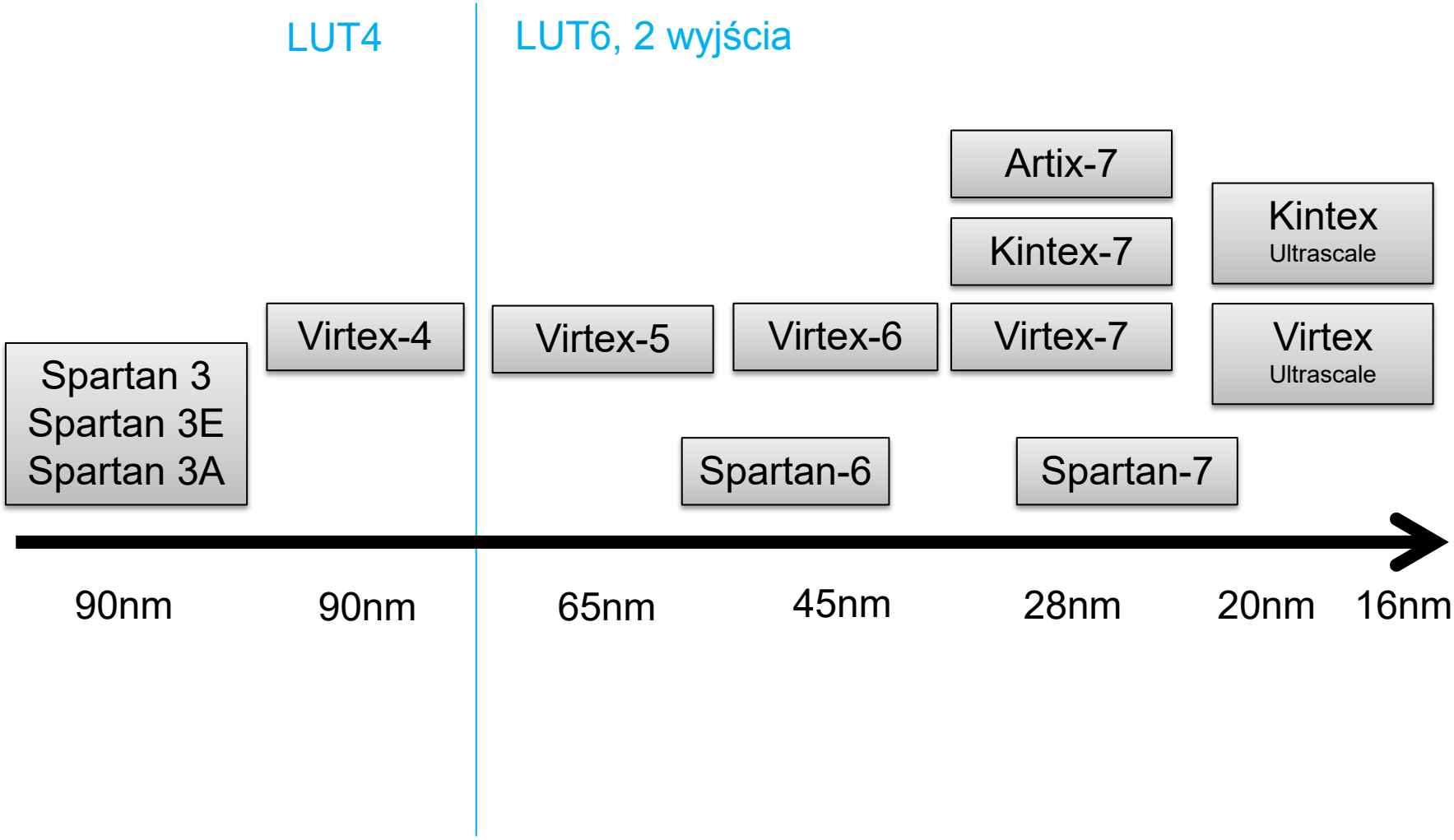

ALTERA®
MEASURABLE ADVANTAGE™


XILINX®

XILINX - struktura

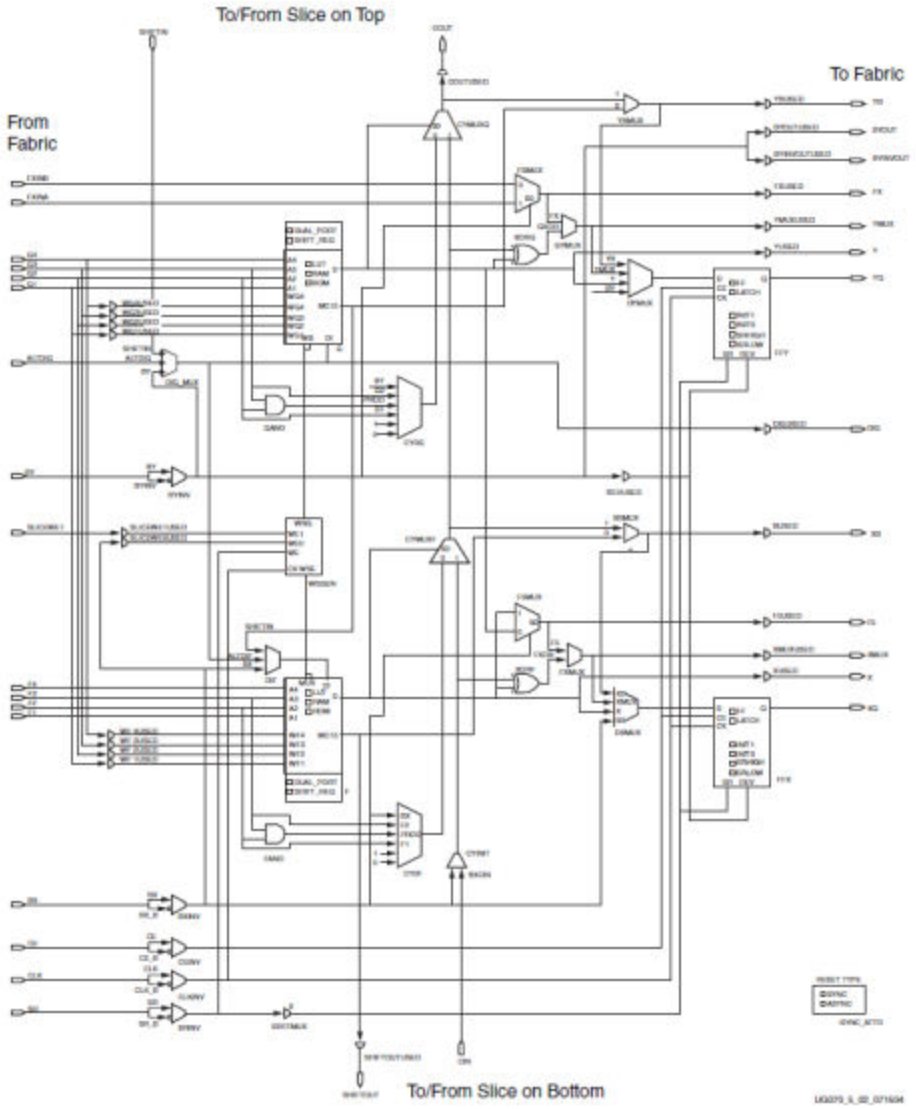


XILINX - Rodziny układów FPGA



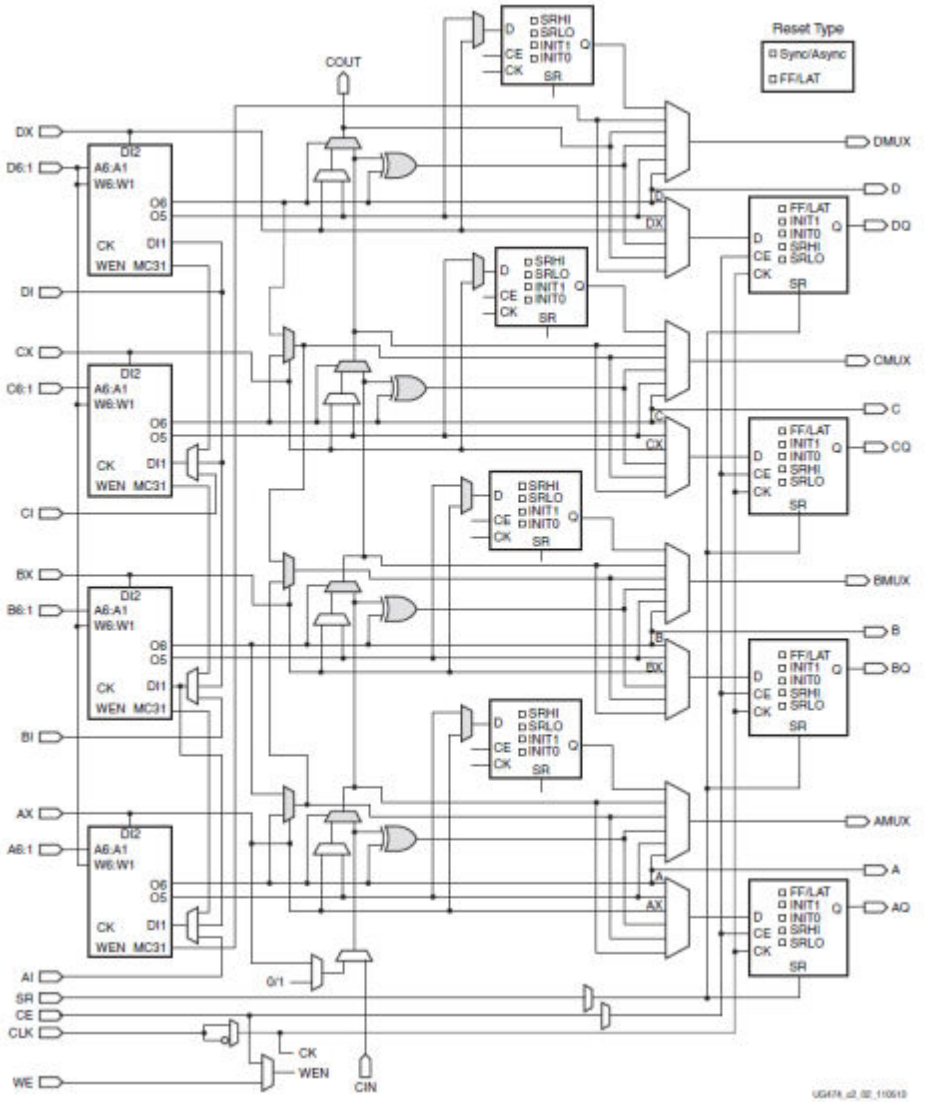
XILINX – LUT-4 (Spartan-3, Virtex-4)

- LUT4
  - 16 kombinacji wejść
  - 1 wyjście
- Pamięć rozproszona RAM/ROM (16)
- Rejestr przesuwny SRL (16)



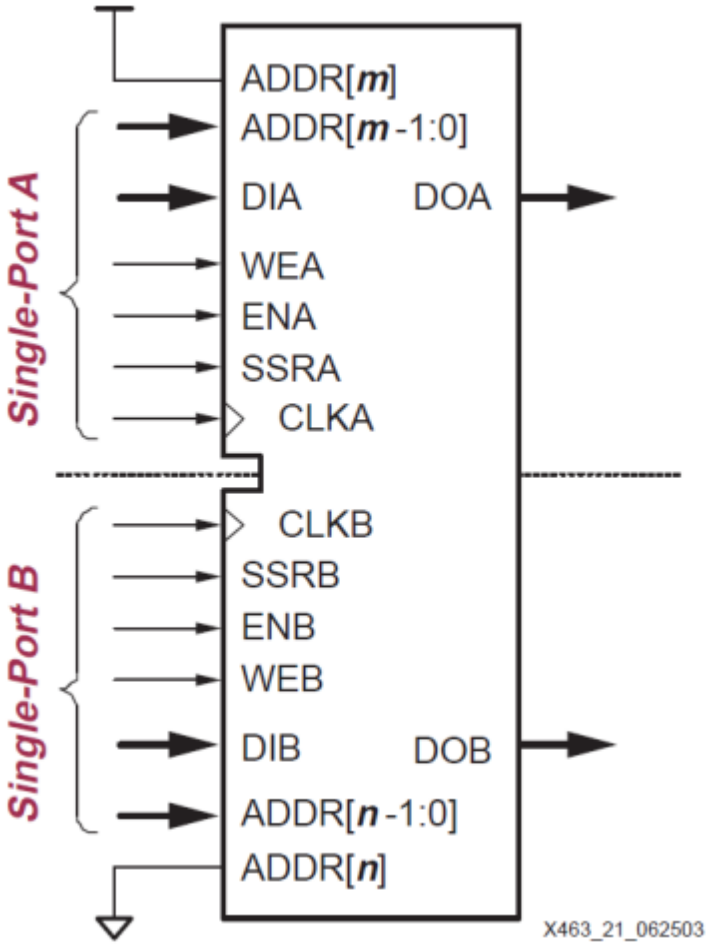
XILINX – LUT-6 (Spartan-6, inne)

- LUT6
  - 64 kombinacji wejść
  - 2 wyjścia
- Pamięć rozproszona RAM/ROM (32×4=128)
- Rejestr przesuwny SRL (32×4=128)



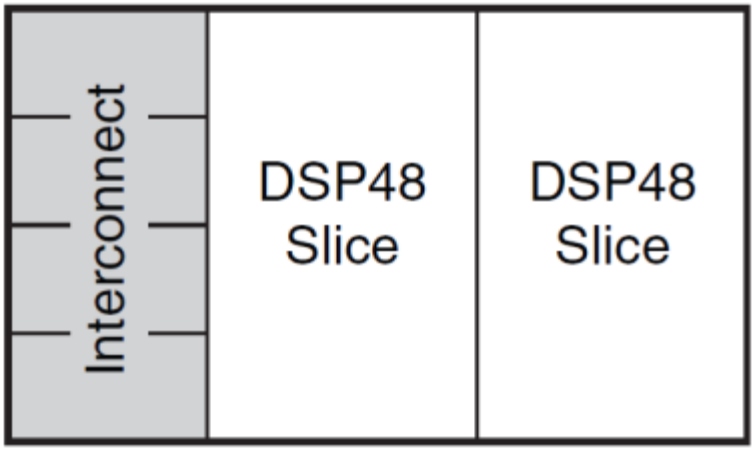
XILINX – RAMB

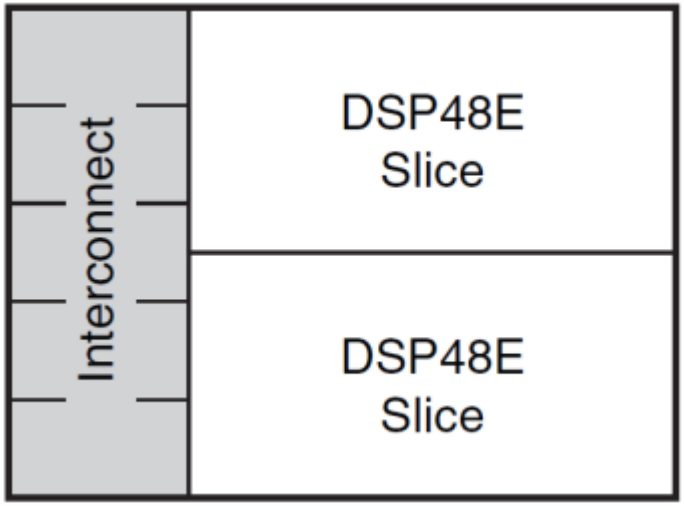
- Dwa symetryczne porty
  - Dwie domeny częstotliwościowe
- Pojemność
  - 9 kbit
  - 18 kbit
  - 36 kbit
- Szerokość słowa
  - 1 bit
  - 2 bits
  - 4 bits
  - 8/9 bitów
  - 16/18 bitów
  - 32/36 bitów
  - 64/72 bitów
- Praca jako FIFO



XILINX – Extreme DSP

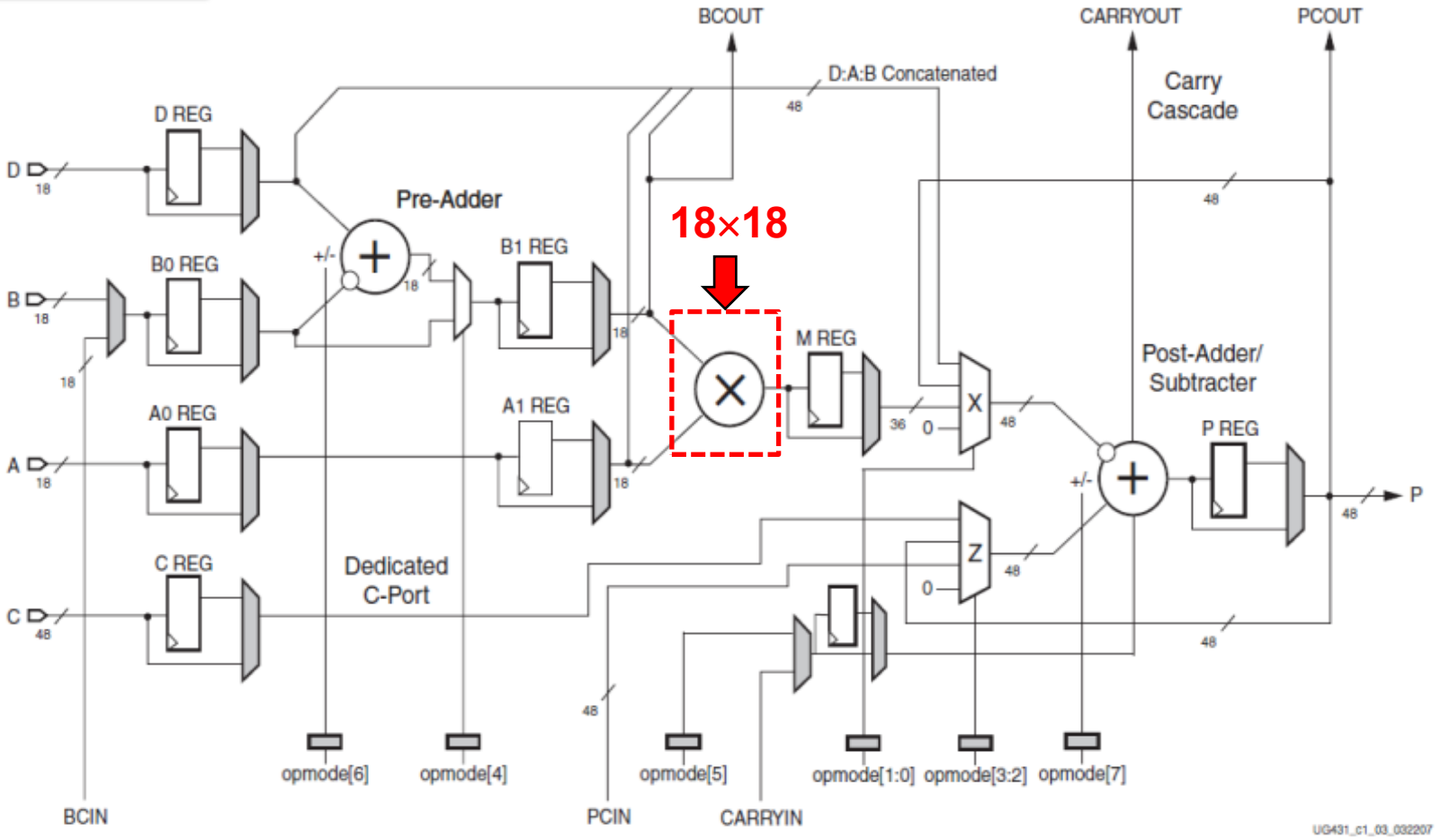
- Operacje
  - Multiply
  - Add
  - Accumulate
  - OR/AND/XOR
  - Pattern detection
- Warianty
  - 18×18
  - 25×18
  - Brak





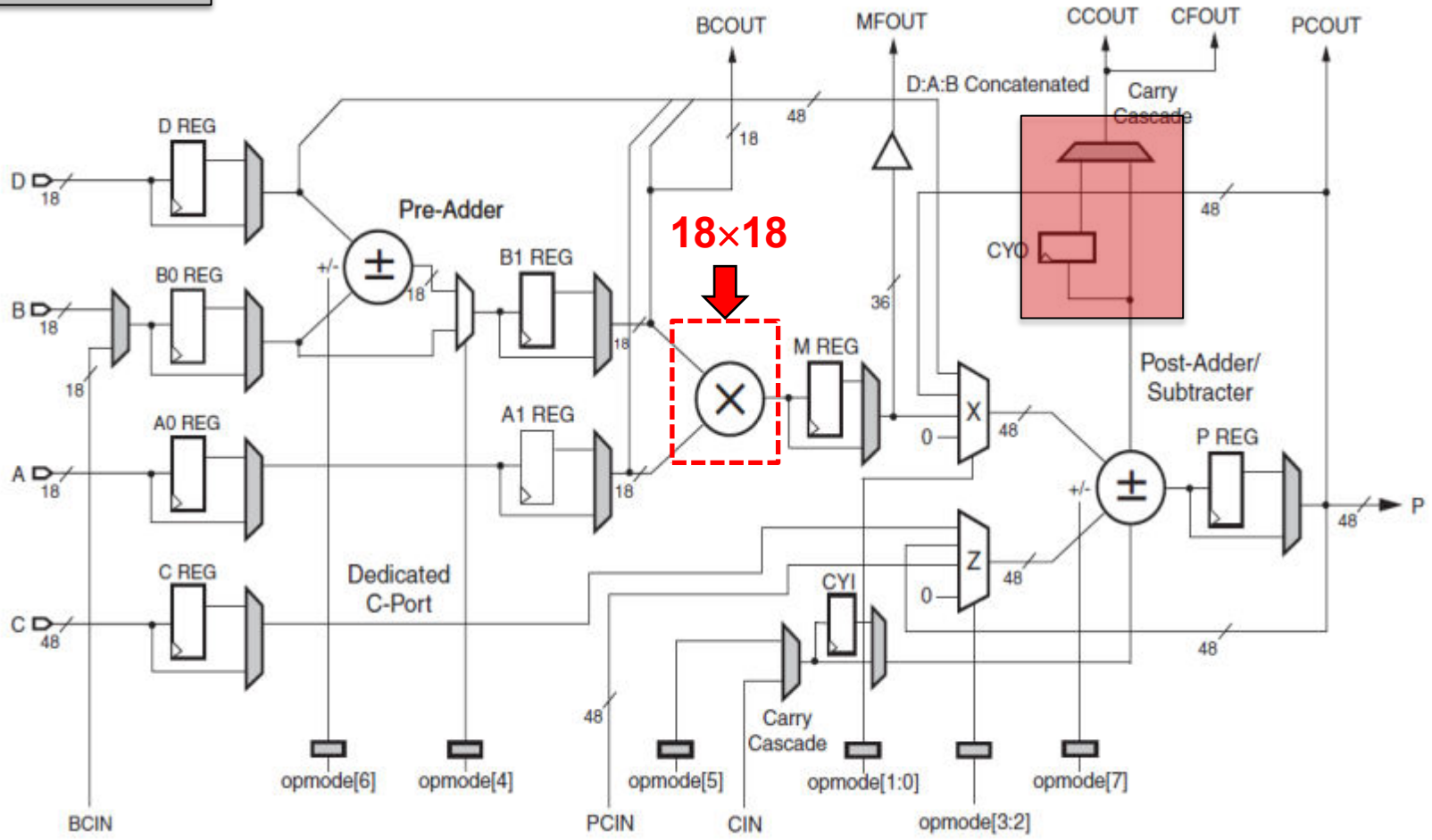
XILINX – Extreme DSP (Spartan-**3A**)

DSP48A



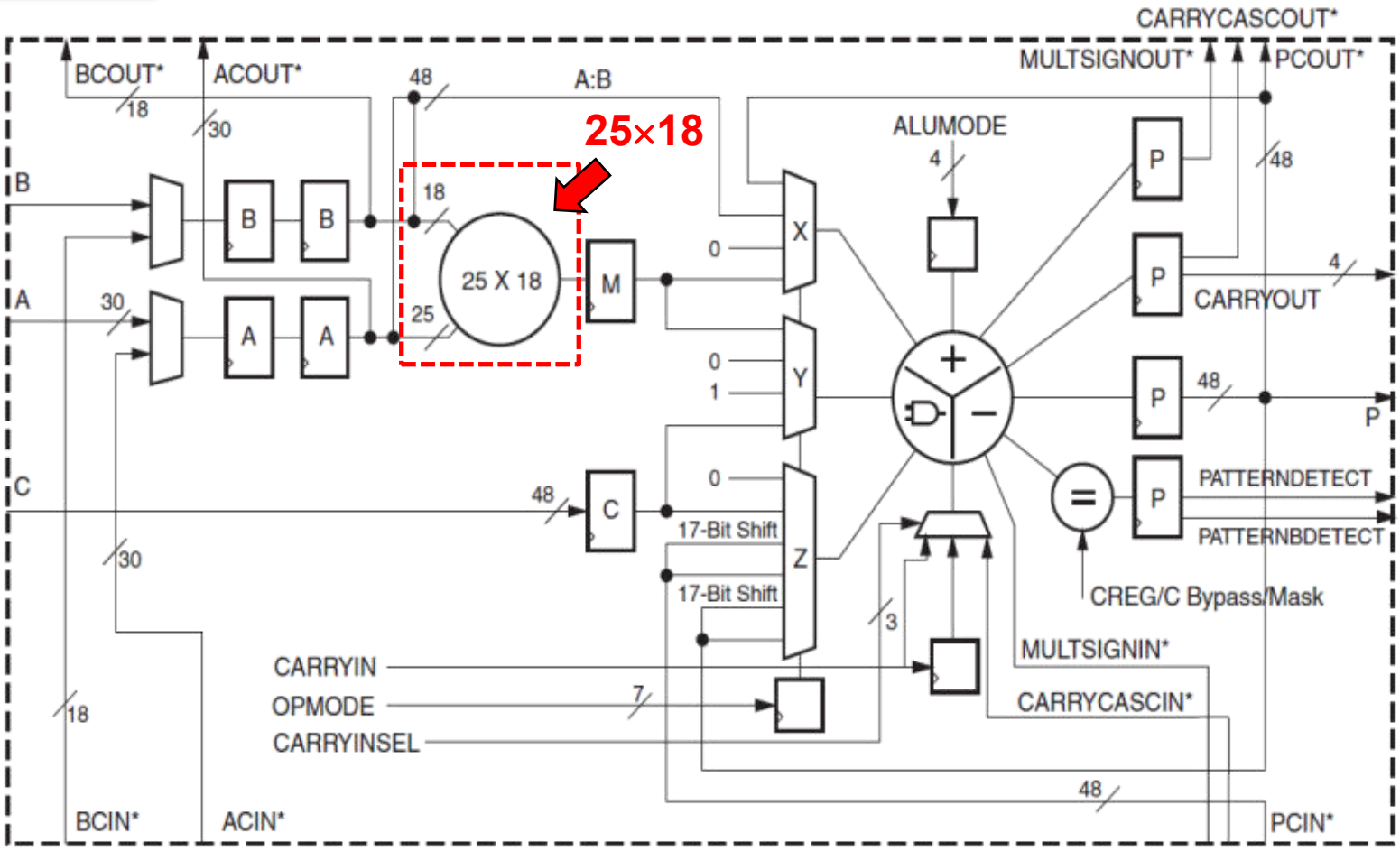
XILINX – Extreme DSP (Spartan-6)

DSP48A1



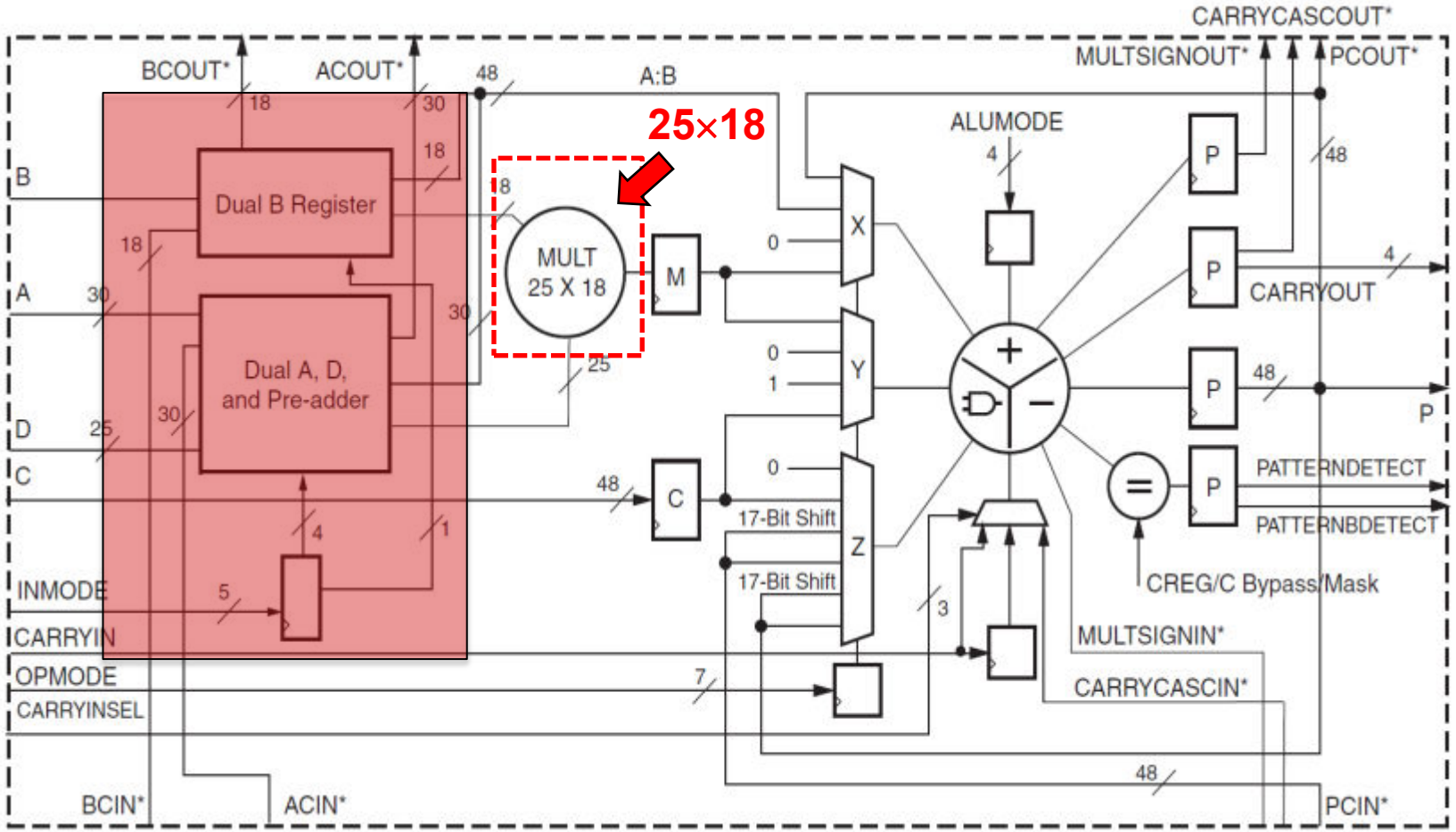
XILINX – Extreme DSP (Virtex-5)

DSP48E

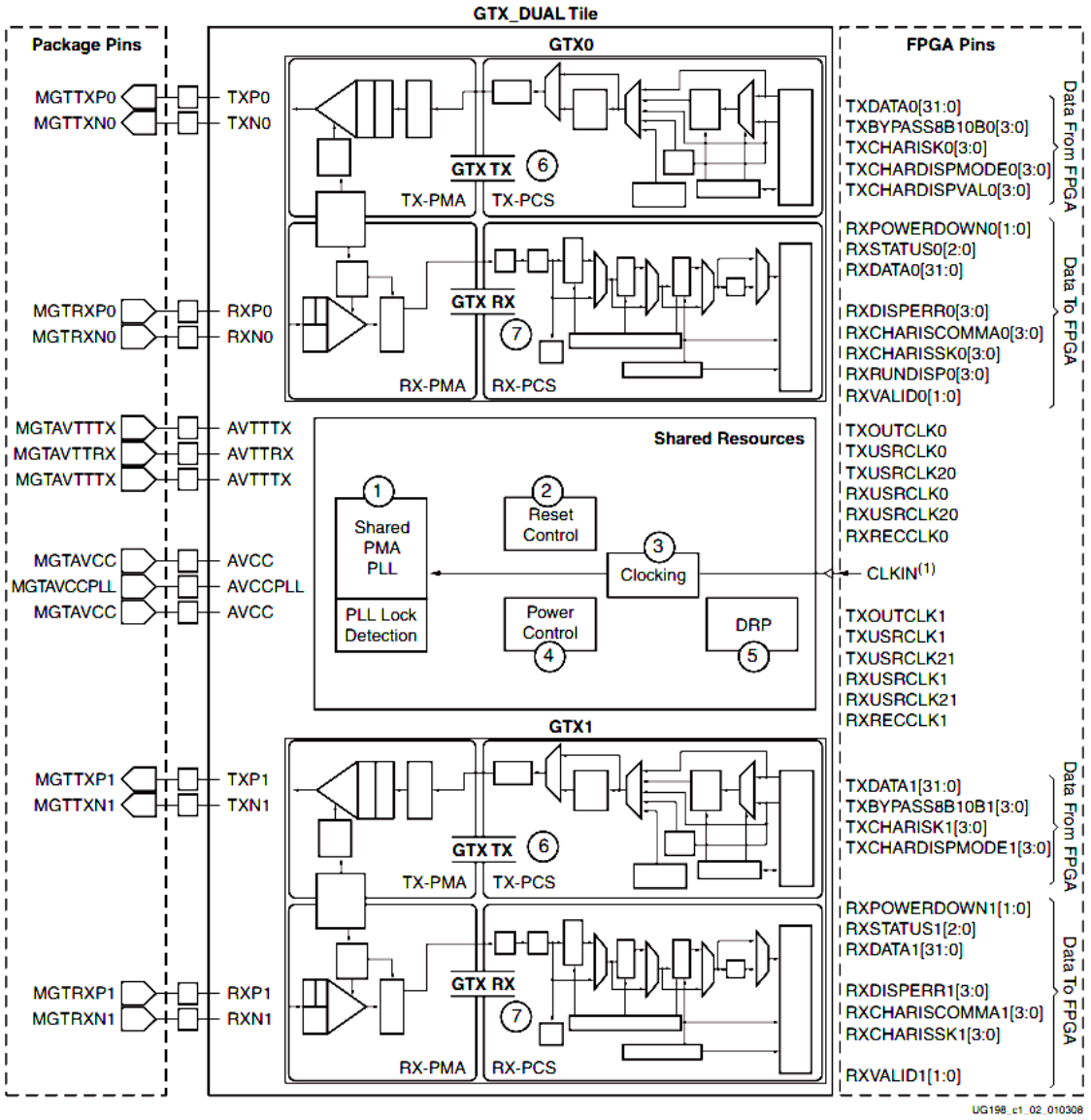


XILINX – Extreme DSP (Virtex-6/7, Kintex-7,Artix-7)

DSP48E1

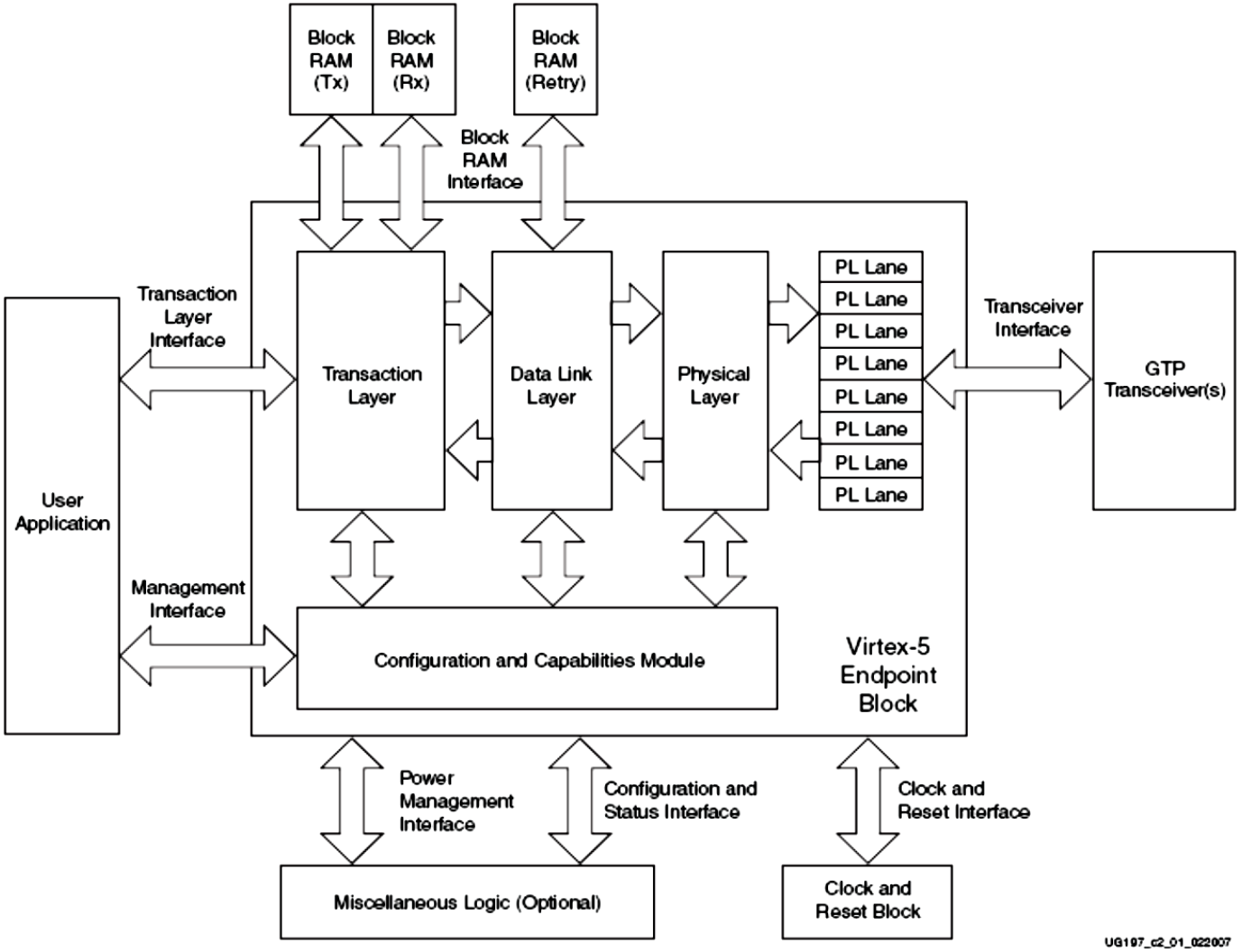


XILINX – GT* port



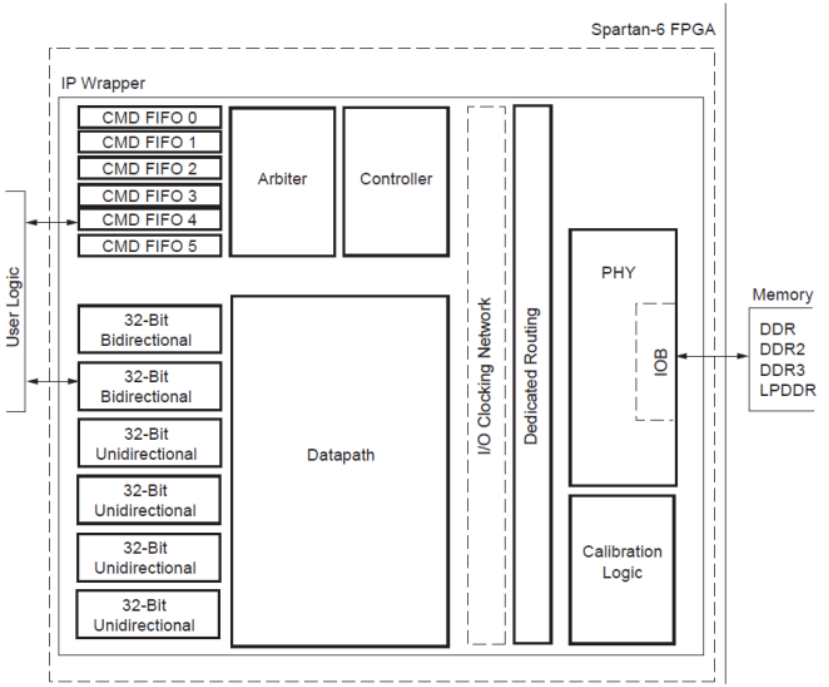
Compatible Protocols	Data Rates	Miscellaneous Features
PCI Express, Rev. 1.0a PCI Express, Rev. 1.1	2.5 Gb/s	• TX receive detect • Loss of Signal (LOS)/Idle state detect • Low power states • Beacon signaling • Ground referenced termination
PCI Express, Rev. 2.0	5.0 Gb/s	
Interlaken	3.125 Gb/s, 6.25 Gb/s	Header insertion/extraction for 64B/67B
XAUI 802.3ae D5p0	3.125 Gb/s	LOS
OIF-CEI6G	6.25 Gb/s	• No Equalizer (EQ), low power mode • 4-tap adaptive DFE
OC-12/48	622.08/2488.32 Mb/s	• Allows FIFO bypassing for synchronous operation • 5x digital oversampling
FC-1, Revision 4.0	1.0625 Gb/s	Rate negotiation (allows operating the TX and RX at different speeds)
FC-2, Revision 4.0	2.125 Gb/s	
FC-4, Revision 4.0	4.25 Gb/s	
10GFC	3.1875 Gb/s	
SDI HD-SDI DVB-ASI 3G-SDI	176/270/360 Mb/s 1.485/1.4835 Gb/s 270 Mb/s 2.970 Gb/s	5x digital oversampling
10G Base-CX4 802.3ak/D4.0	3.125 Gb/s	
Gigabit Ethernet (1000BASE-CX 802.3z/D5.0)	1.25 Gb/s	
SATA Generation 1/2, Rev. 1.0a SATA Generation 2, Rev. 1.0a	1.5 Gb/s 3.0 Gb/s	• Rate negotiation for Generation 2 (entire link operates at Generation 1/Generation 2 speeds) • LOS • OOB beacon
Serial RapidIO	1.25/2.5/3.125/6.25 Gb/s	
CPRI, Version 2.0	614.4/1228.8/2457.6 Mb/s	5x digital oversampling
Infiniband, Volume 2, Release 1.1	2.5 Gb/s	
SFI-5	2.488 – 3.125 Gb/s	Synchronous clocking (bypass FIFOs)
OBSAI RP3, Spec. Issue 1.0	768/1536/3072 Mb/s	
Aurora	150 Mb/s – 6.5 Gb/s	

XILINX – PCI-Express endpoint



XILINX – Kontroler pamięci (Spartan-6)

- W pełni funkcjonalny kontroler pamięci SDRAM
  - DDR, DDR, DDR3, LPDDR
  - Nie tylko same PHY
- 1,6 do 6,4 Gb/s



Memory Type	Data Rate: Mb/s DDR (MHz Clock)		Peak Bandwidth per MCB Interface (Gb/s)		
	Minimum	Maximum(1)	4-Bit	8-Bit	16-Bit
DDR	167 Mb/s(2) (83.3 MHz)	400 Mb/s (200 MHz)	1.6 Gb/s	3.2 Gb/s	6.4 Gb/s
DDR2	250 Mb/s(2) (125 MHz)	800 Mb/s (400 MHz)	3.2 Gb/s	6.4 Gb/s	12.8 Gb/s
DDR3	606 Mb/s(2) (303 MHz)	800 Mb/s (400 MHz)	3.2 Gb/s	6.4 Gb/s	12.8 Gb/s
LPDDR	60 Mb/s(2) (30 MHz)	400 Mb/s (200 MHz)	1.6 Gb/s	3.2 Gb/s	6.4 Gb/s

XILINX - Podsumowanie

- Niejednorodność
  - 2 architektury LUT
    - LUT4 oraz LUT6 dwuwyjściowy
  - 3 pojemności pamięci blokowej
  - 4 różne architektury bloków DSP
  - Różne prędkości i funkcjonalności portów GTX/GTP/GTH
- Od generacji 7 ujednolicenie
  - Virtex - pojemność
  - Kintex - prędkość
  - Artix - cena
    - i tak 500 do 5000 pln / układ

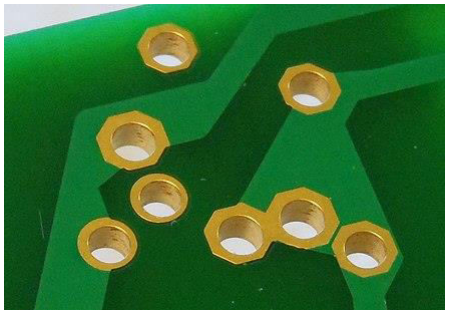
XILINX - Podsumowanie

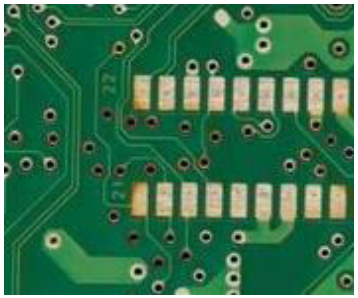
- Zalety
  - Najszybsze i największe układy
  - Popularne
- Wady
  - Cena
  - Duża praca wejścia
  - Pracochłonność projektowania PCB
  - Zaawansowane obudowy układów
    - W praktyce tylko BGA (Ball Grid Array)

Obudowy układów scalonych

- Through-hole DIL (Dual In Line)
  - Montaż przewlekany
  - Utrudniona produkcja masowa
  - Parametry sygnałowe
- SMD (Surface Mount Device)
  - Montaż powierzchniowy
  - Relatywnie duże piny
  - Mało wyprowadzeń
    - 8-16
  - Parametry sygnałowe
    - Małe anteny!









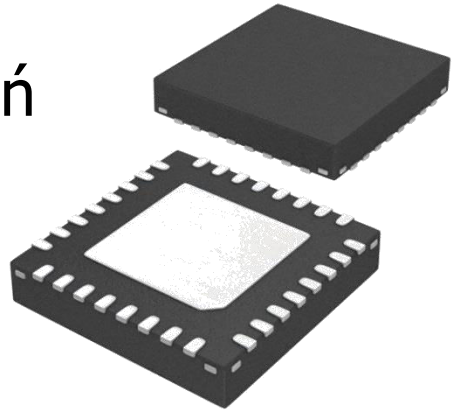


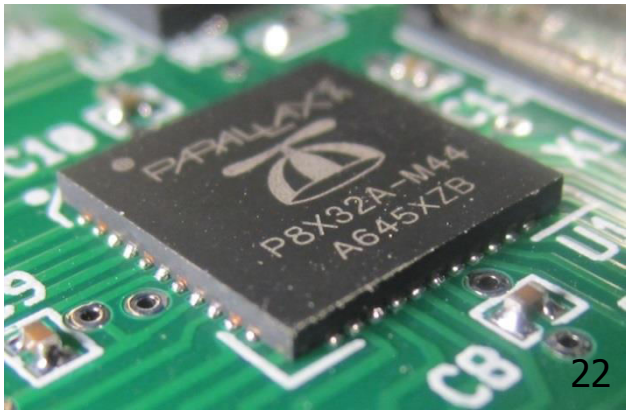
Obudowy układów scalonych

- TQFP (Thin Quad Flat Pack)
  - Montaż powierzchniowy
  - Mniejsze piny
  - Więcej pinów np. 144
  - Parametry sygnałowe



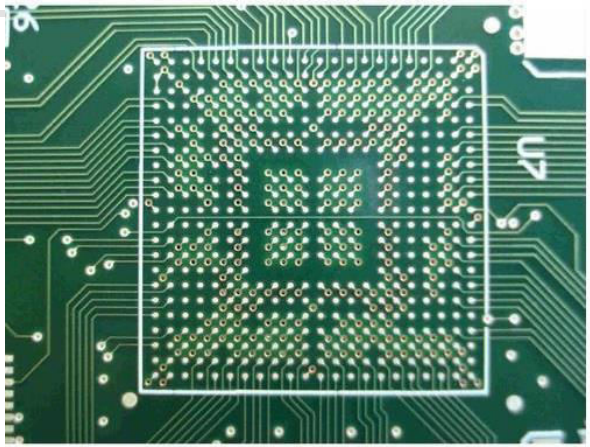
- QFN (Quad Flat No-leads)
  - Montaż powierzchniowy
  - Brak nóżek – pady!
  - Więcej wyprowadzeń np. 256
  - Lepsze parametry sygnałowe

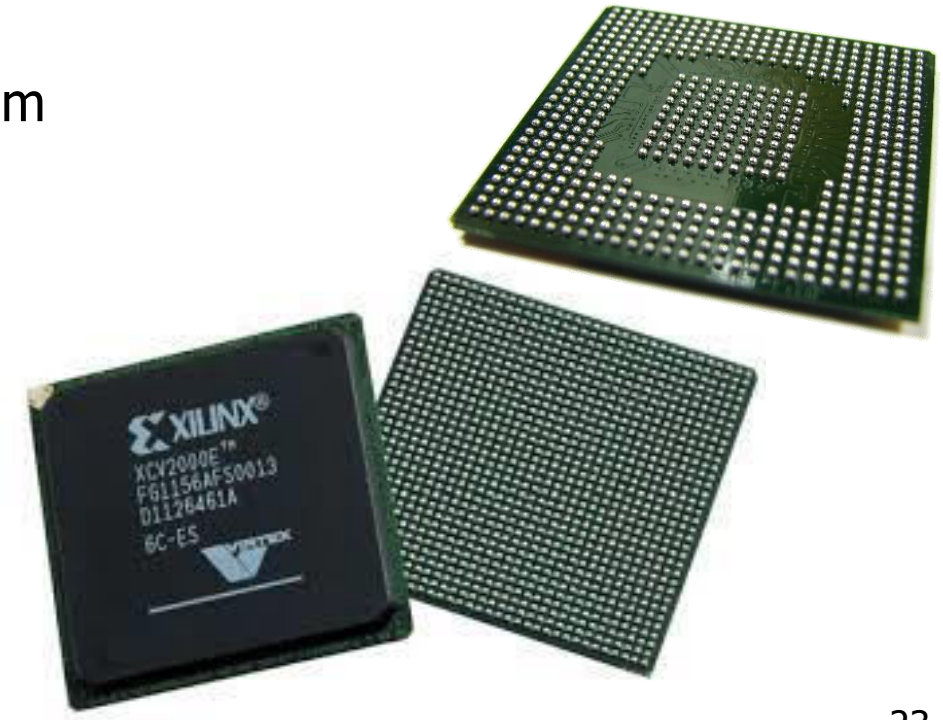




Obudowy układów scalonych

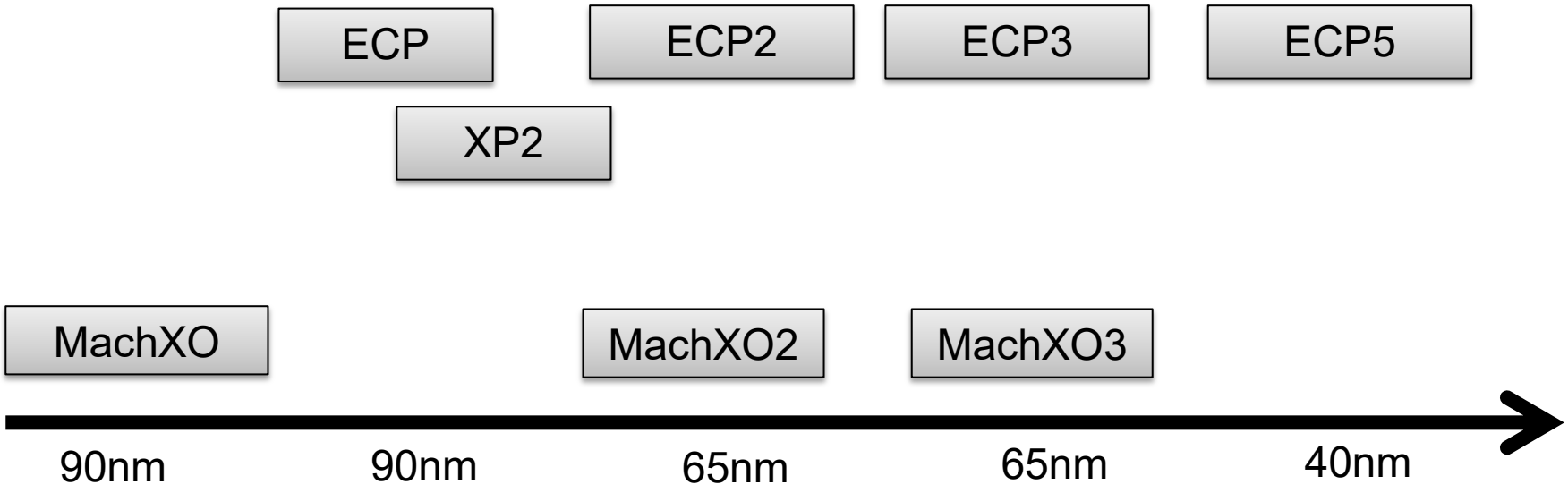
- BGA (Ball Grid Array)
  - Brak nóżek – kulki
    - Wyprowadzenia także pod powierzchnią układu
    - Np. 25×25=625
      32×32=1024
      Rozstaw kulek 0,3 – 0,8 mm
  - Skomplikowane PCB
    - Przelotki
    - Grubości ścieżek
    - Solder-maska
  - Skomplikowany montaż
    - Podgląd pod układ?






LATTICE
SEMICONDUCTOR.

LATTICE - Rodziny układów FPGA

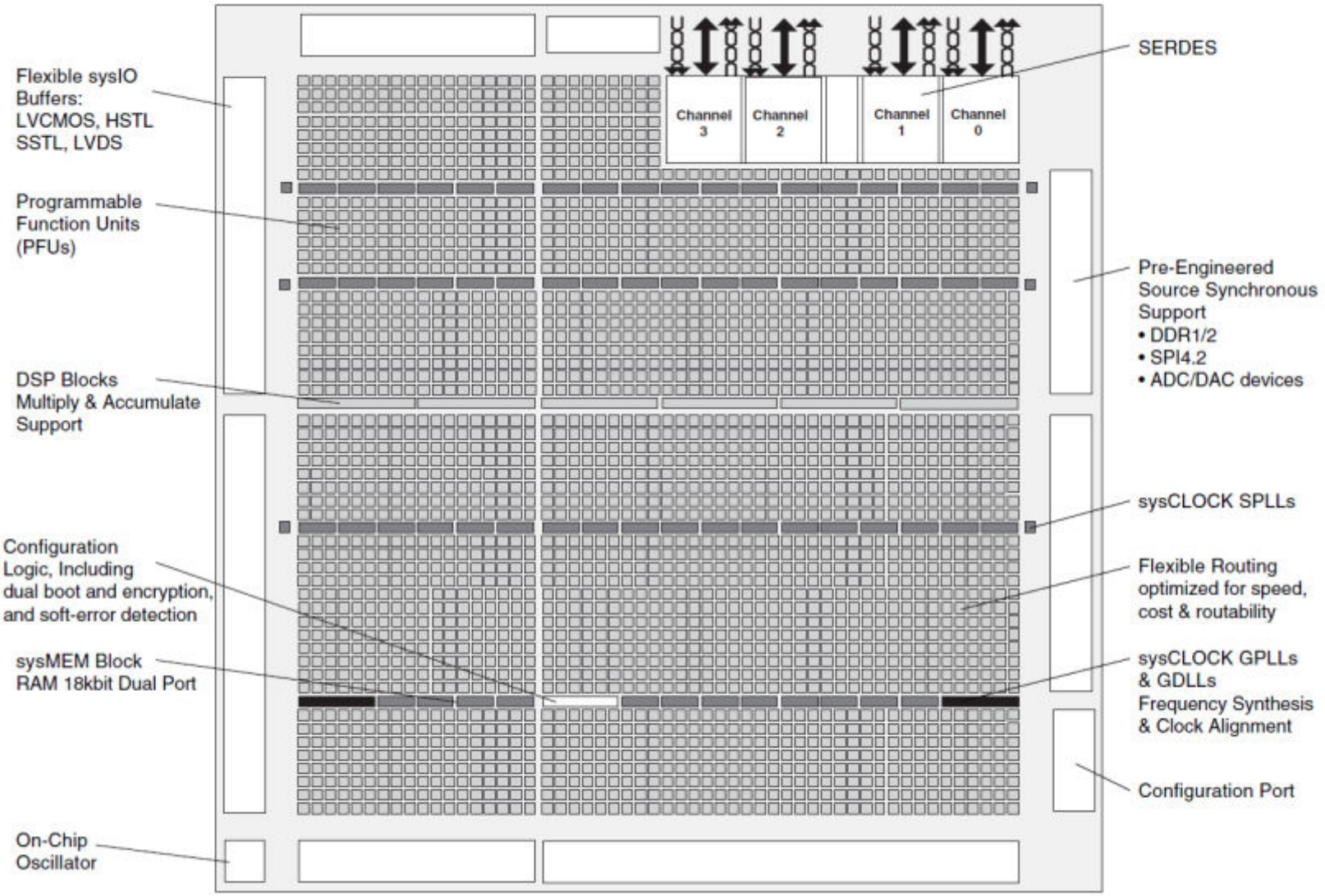


LATTICE – Prostota

- Wbudowane
  - Pamięć flash
  - Pamięci blokowa
  - PLL
  - Oscylator
- Przystępne obudowy
  - TQFP100
  - QFN32
  - BGA z dużym rozstawem kulek np. 0,6mm
- Darmowe oprogramowanie
  - Lattice Diamond
  - Aldec ActiveHDL
  - Synposys Synplify

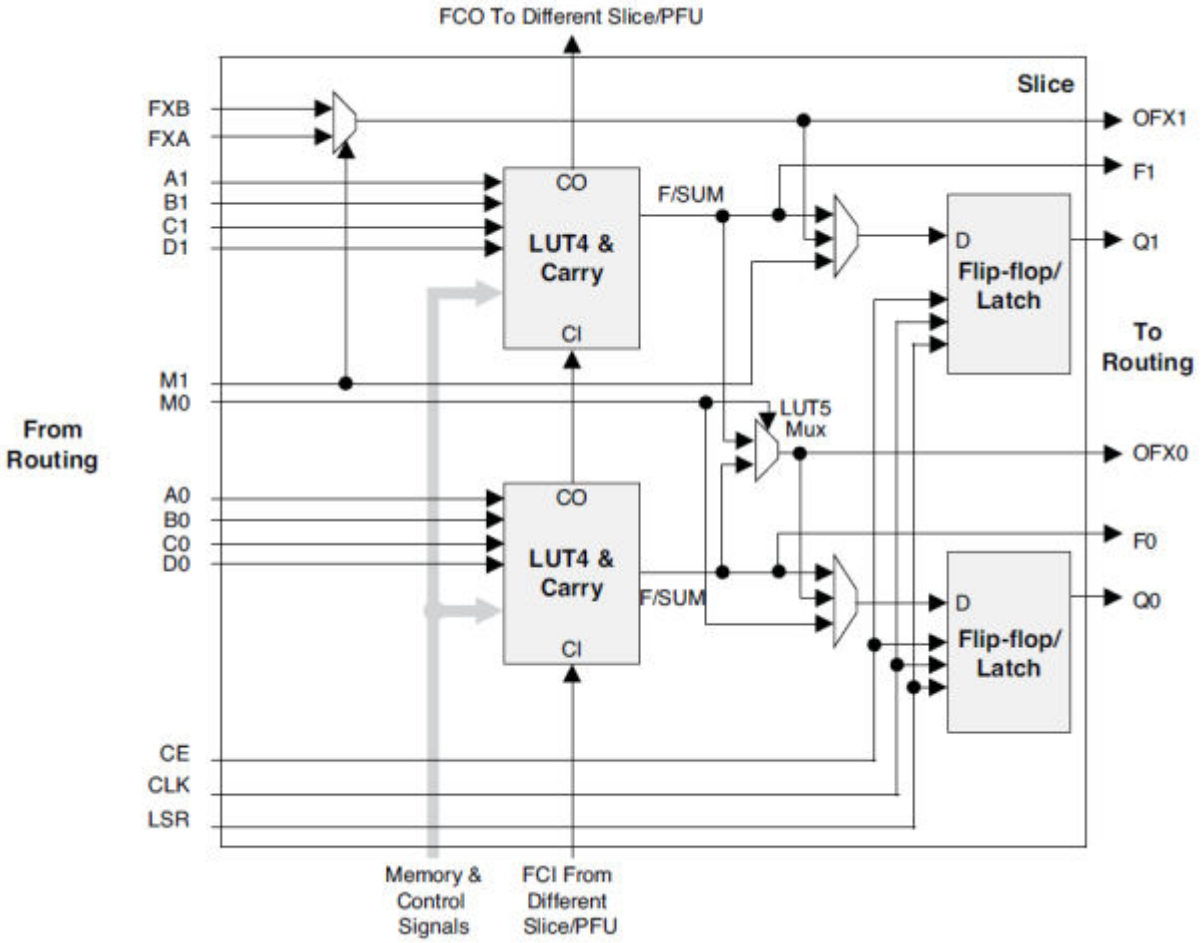


LATTICE – Struktura



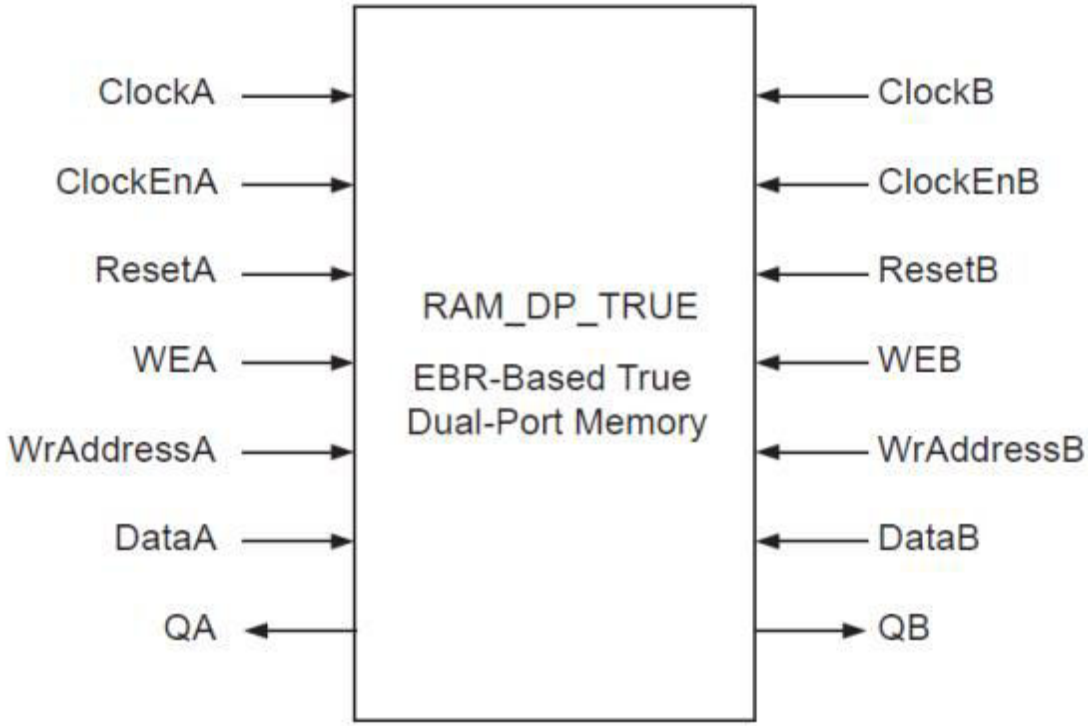
LATTICE – LUT-4

- LUT4
  - 16 kombinacji wejść
  - 1 wyjście
- Pamięć rozproszona RAM/ROM (16)
- Brak połączeń z sąsiadami poza carry-chain



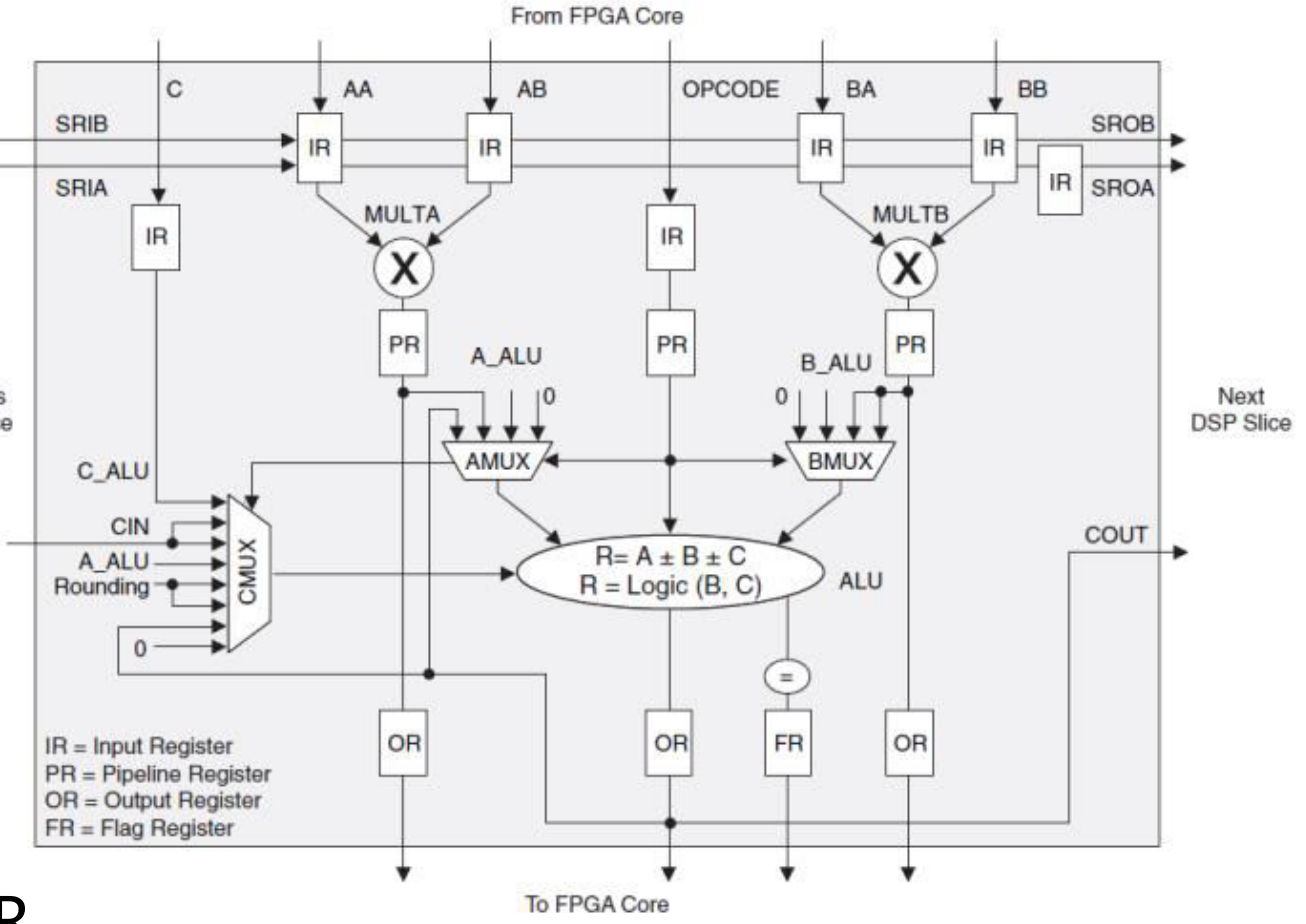
LATTICE – Block RAM

- Dwa symetryczne porty
  - Dwie domeny częstotliwościowe
- Pojemność
  - 9 kbit
- Szerokość słowa
  - 1 bit
  - 2 bits
  - 4 bits
  - 8/9 bitów
  - 16/18 bitów
  - 32/36 bitów
  - 64/72 bitów
- Różne szerokości
  - Tylko ×2 lub ÷2
- Praca jako FIFO (emulacja)

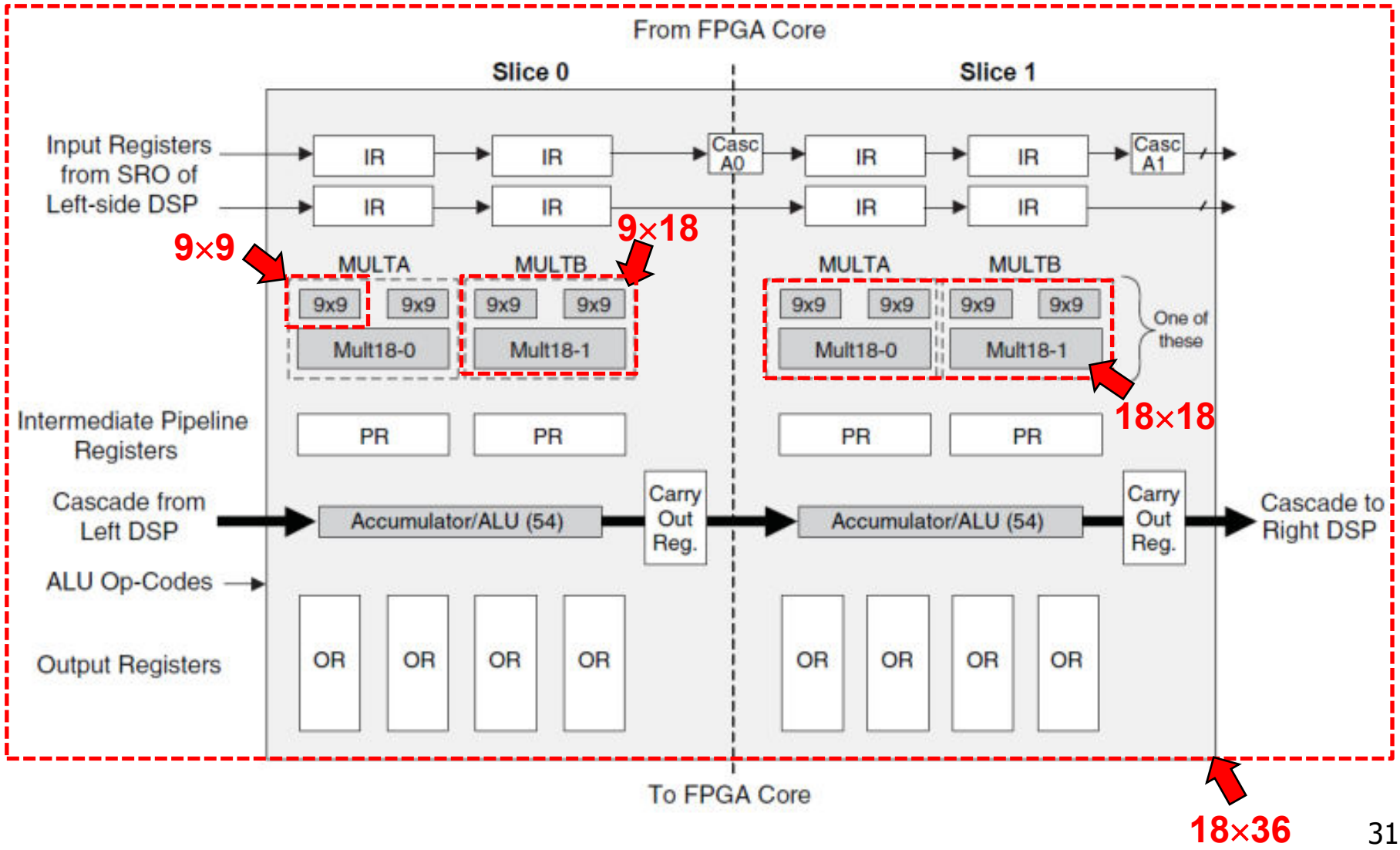


LATTICE – DSP - MULT

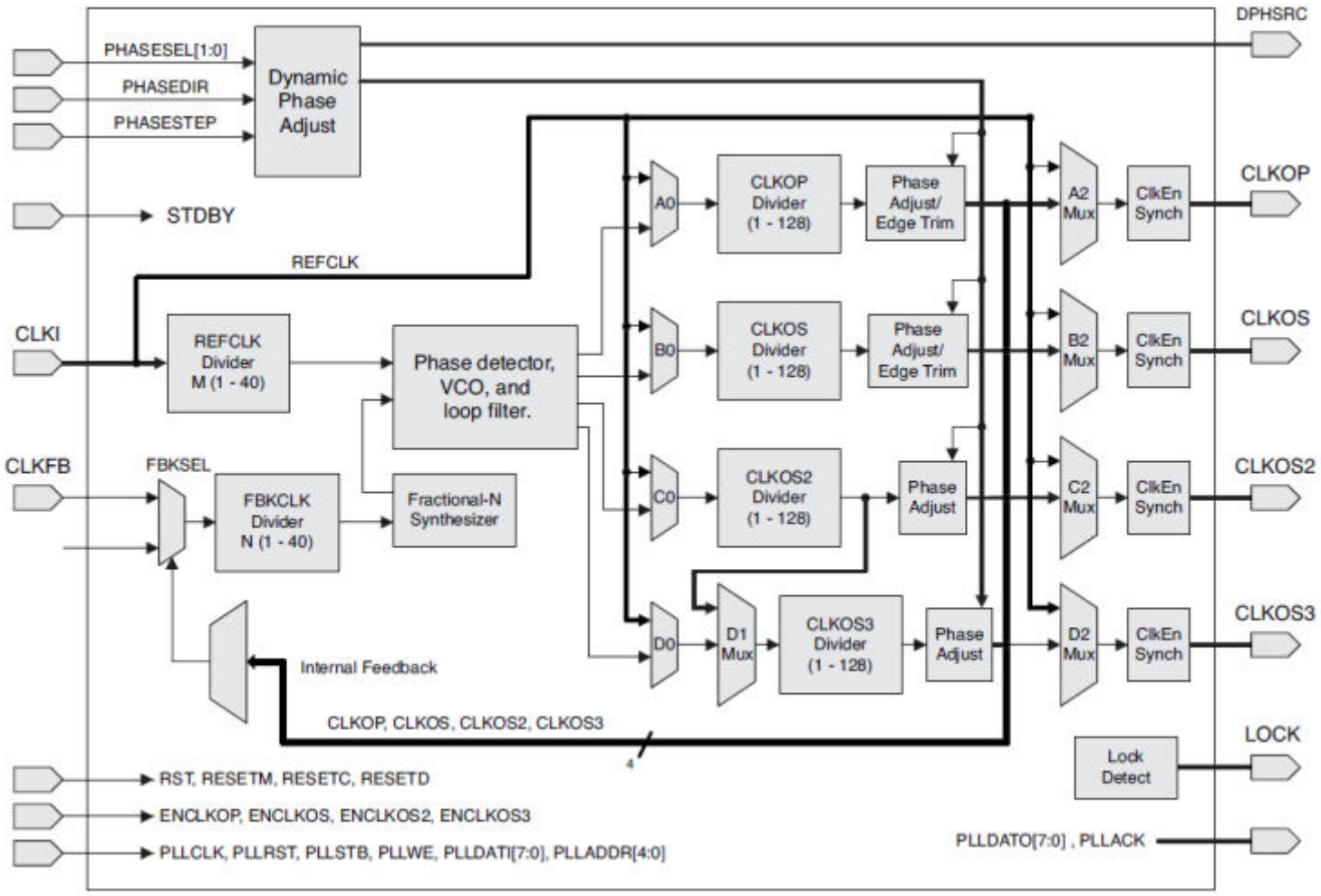
- Hierarchiczna budowa
  - 9×9 + 9×9
  - 9 × 18
  - wyżej:
    - 18×18
    - 18 × 36
    - 36 × 36
- Operacje
  - Multiply
  - Add
  - Accumulate
  - OR/AND/XOR
- Tylko w układach ECP



LATTICE – DSP - Slice

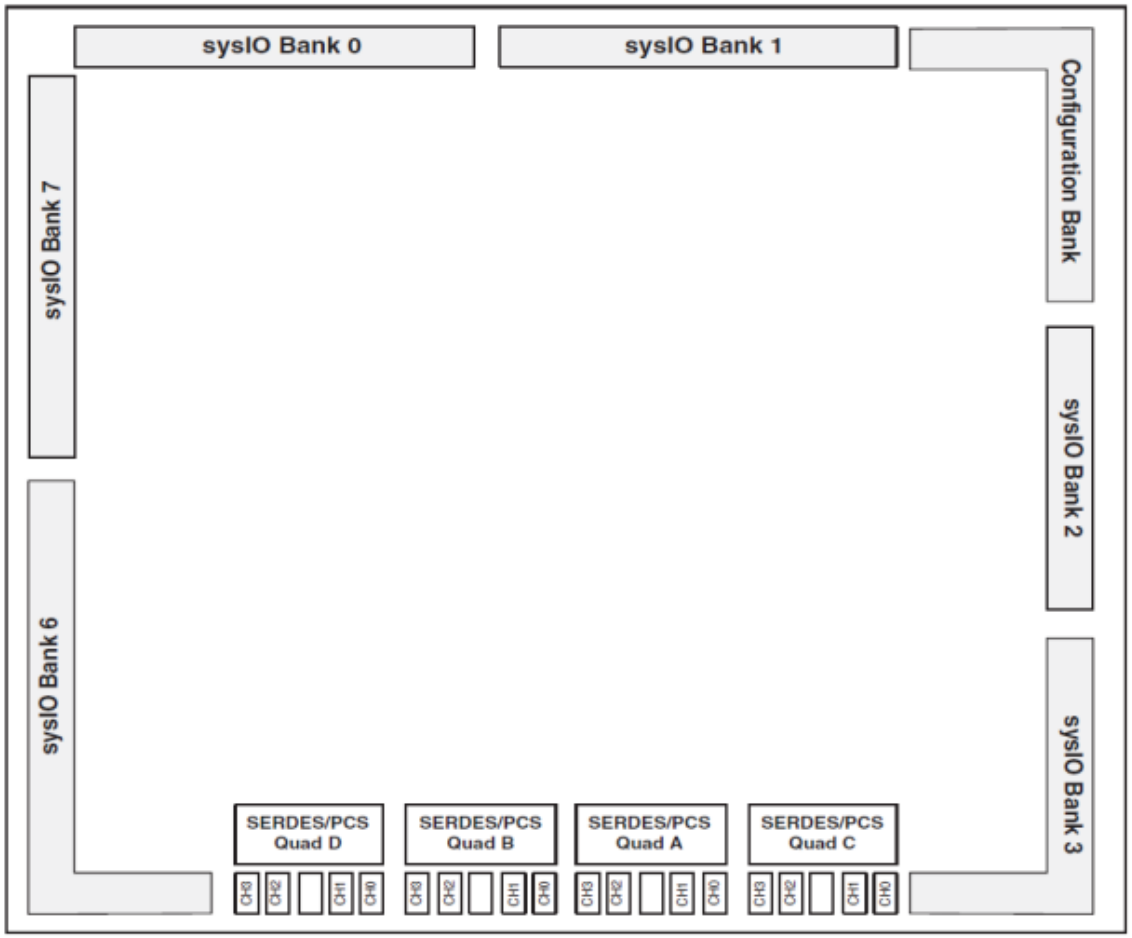


LATTICE – PLL: 4 wyjścia



LATTICE – SerDes (podobne do GT* XILINX)

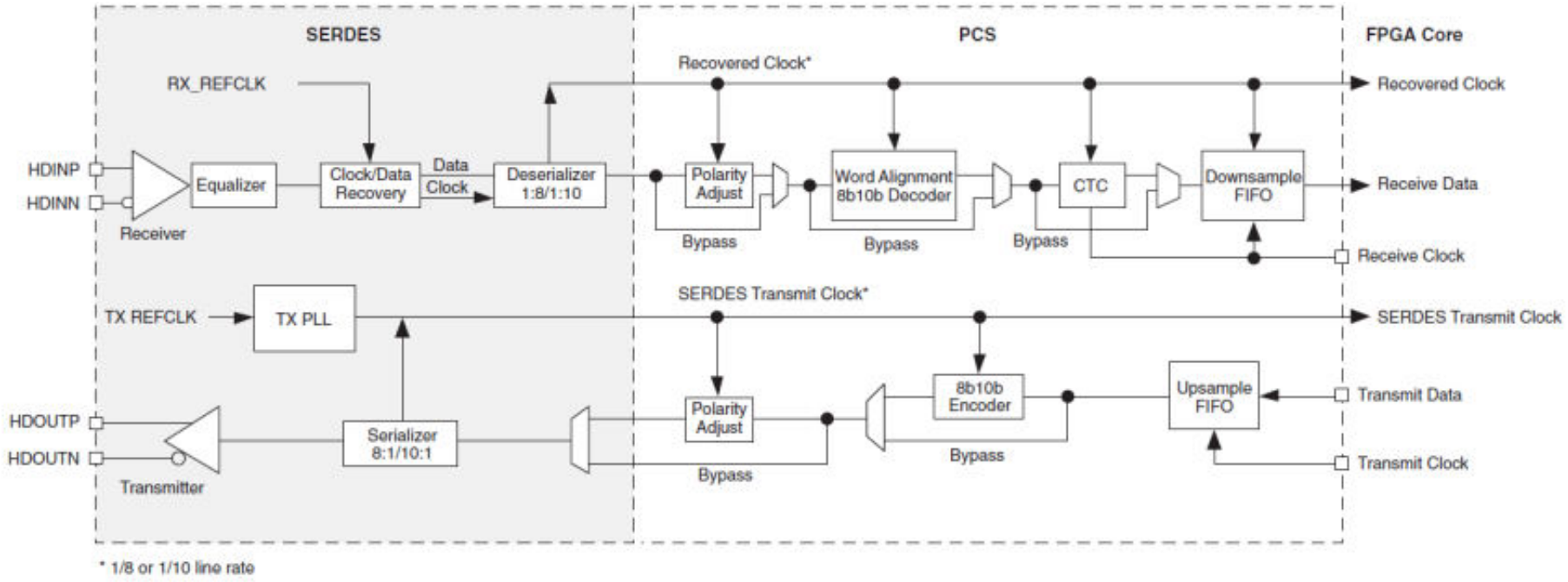
- Wszystkie banki
  - Napięcie pracy
  - Tryb par różnicowych
  - ODDR
  - IDDR
- Niektóre banki
  - IO DDR
  - DDRx4
  - DDR SDRAM
  - SerDes



- Dedykowane piny zegarowe
- P.S. Prawidłowe tłumaczenie słowa „bank” to „zestaw”

LATTICE – SerDes (podobne do GT* XILINX)

- Serializer/Deserializer
  - Dostępne w wybranych układach
  - Specjalna licencja Lattice Diamond



LATTICE – SerDes (podobne do GT* XILINX)

Standard	Data Rate (Mbps)	Number of General/Link Width	Encoding Style
PCI Express 1.1	2500	x1, x2, x4	8b10b
Gigabit Ethernet	1250, 2500	x1	8b10b
SGMII	1250	x1	8b10b
XAUI	3125	x4	8b10b
Serial RapidIO Type I, Serial RapidIO Type II, Serial RapidIO Type III	1250, 2500, 3125	x1, x4	8b10b
CPRI-1, CPRI-2, CPRI-3, CPRI-4	614.4, 1228.8, 2457.6, 3072.0	x1	8b10b
SD-SDI (259M, 344M)	143[1], 177[1], 270, 360, 540	x1	NRZI/Scrambled
HD-SDI (292M)	1483.5, 1485	x1	NRZI/Scrambled
3G-SDI (424M)	2967, 2970	x1	NRZI/Scrambled
SONET-STS-3[2]	155.52	x1	N/A
SONET-STS-12[2]	622.08	x1	N/A
SONET-STS-48[2]	2488	x1	N/A

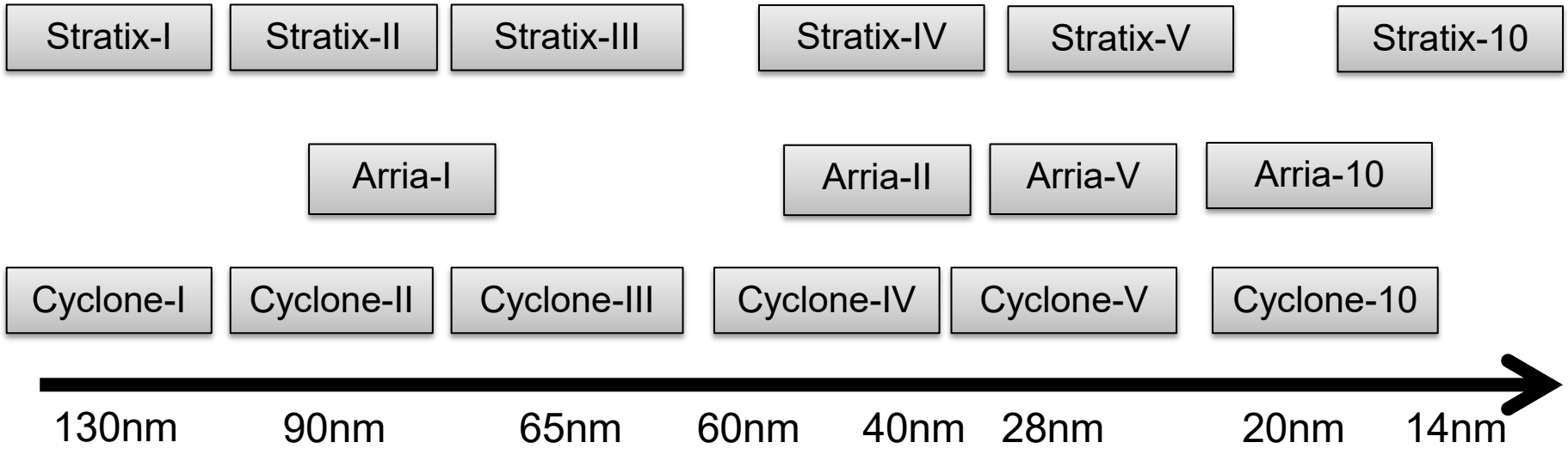
LATTICE - Podsumowanie

- Jednorodność
  - LUT4
  - 9kbitowe bloki RAM
  - DSP 9×9+
  - Jeden typ portów SerDes
- Zalety
  - Prostota
  - Cena
  - Przystępne obudowy
- Wady
  - Mniejsze możliwości
    - (wielkość, prędkość, funkcjonalność)

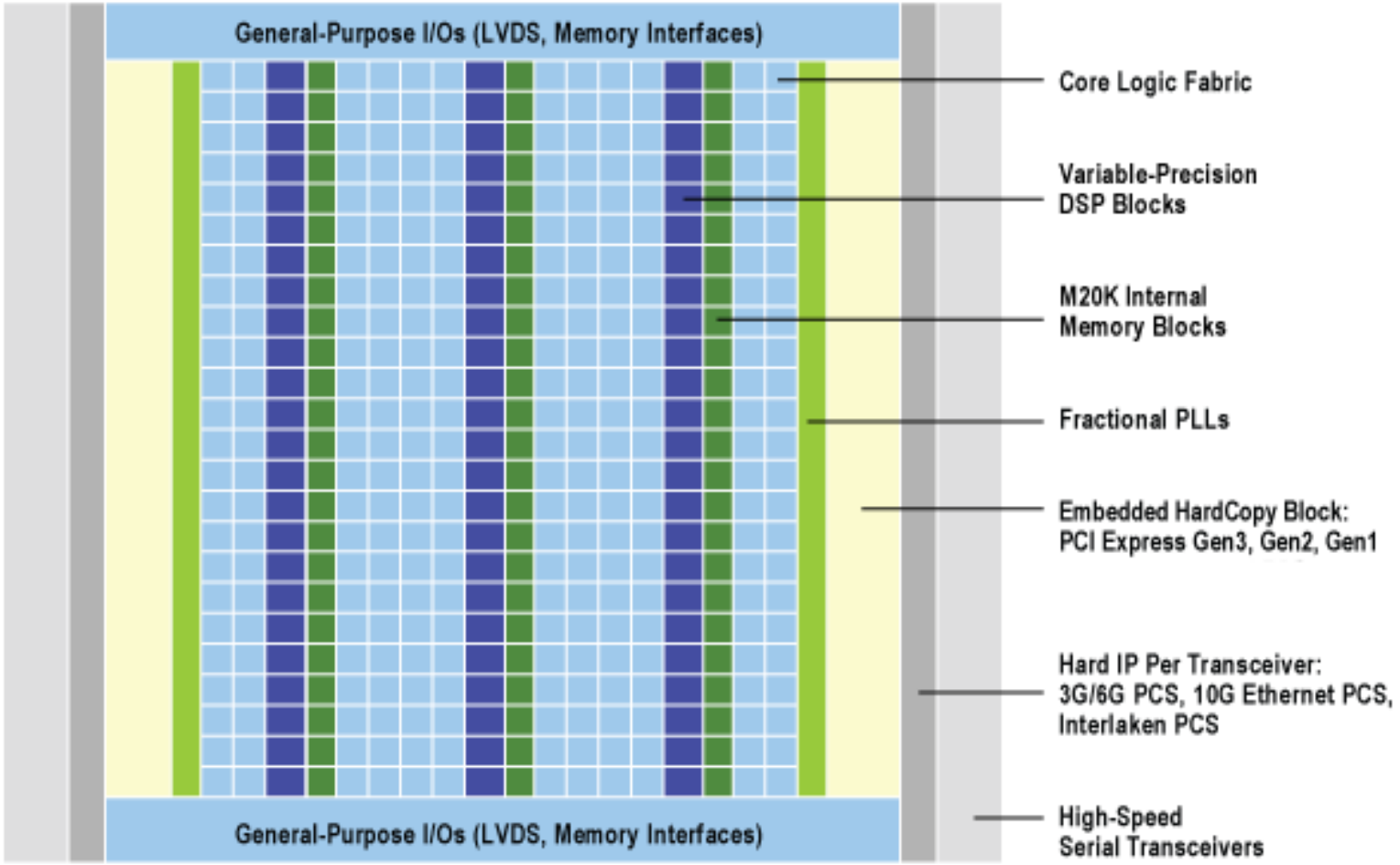

ALTERA
MEASURABLE ADVANTAGE™


intel

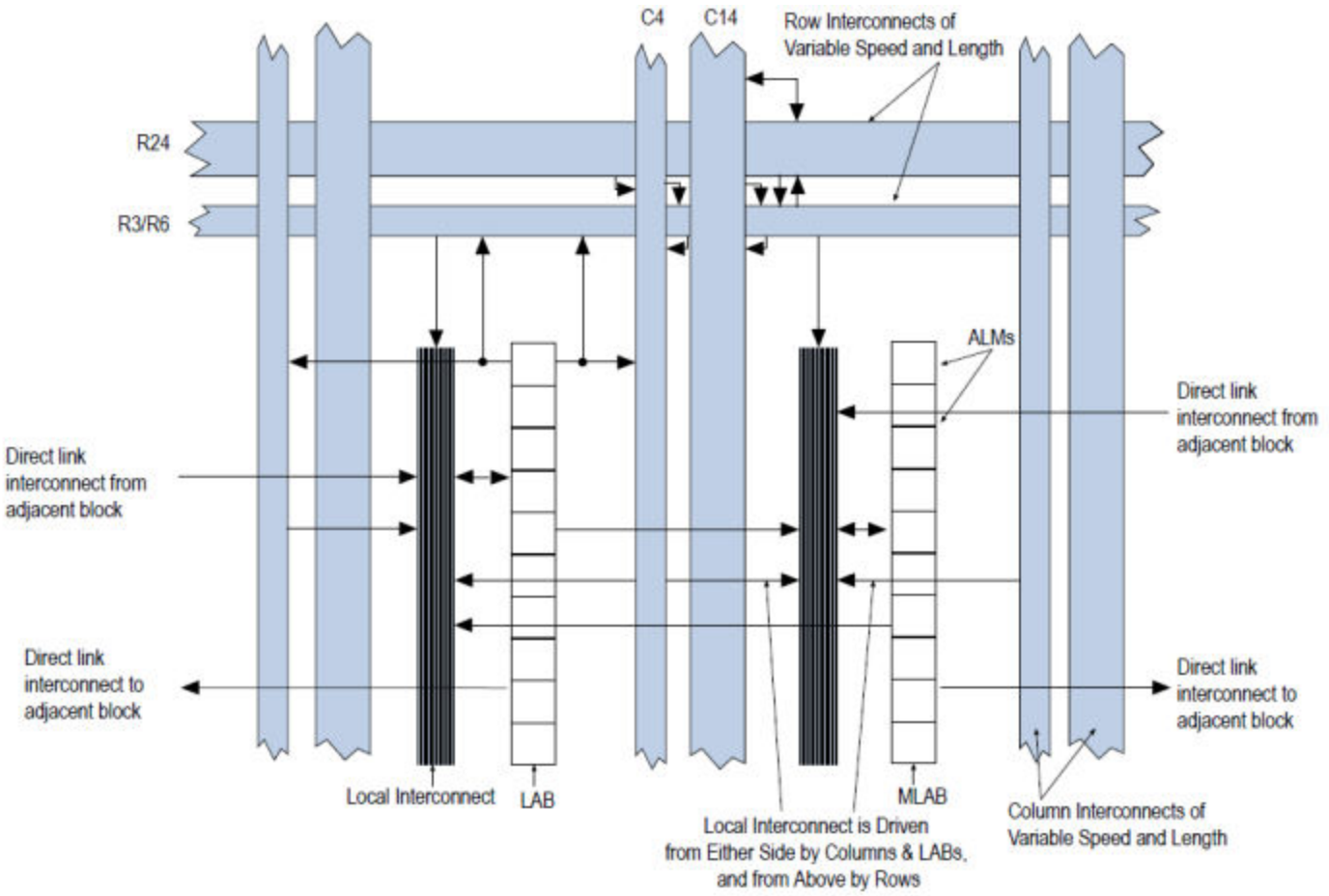
ALTERA/INTEL- Rodziny układów



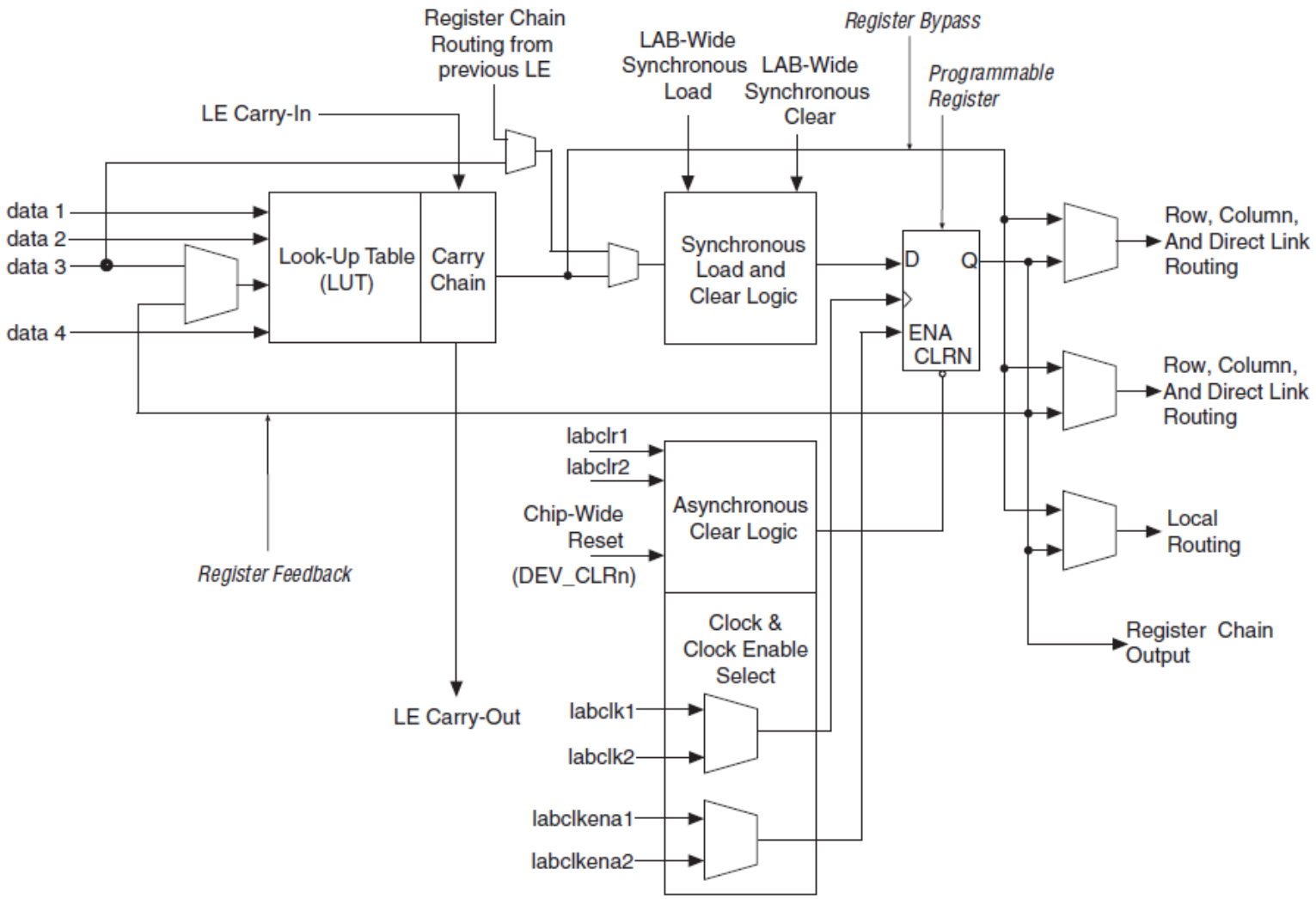
ALTERA – Ogólna struktura



ALTERA – Połączenia

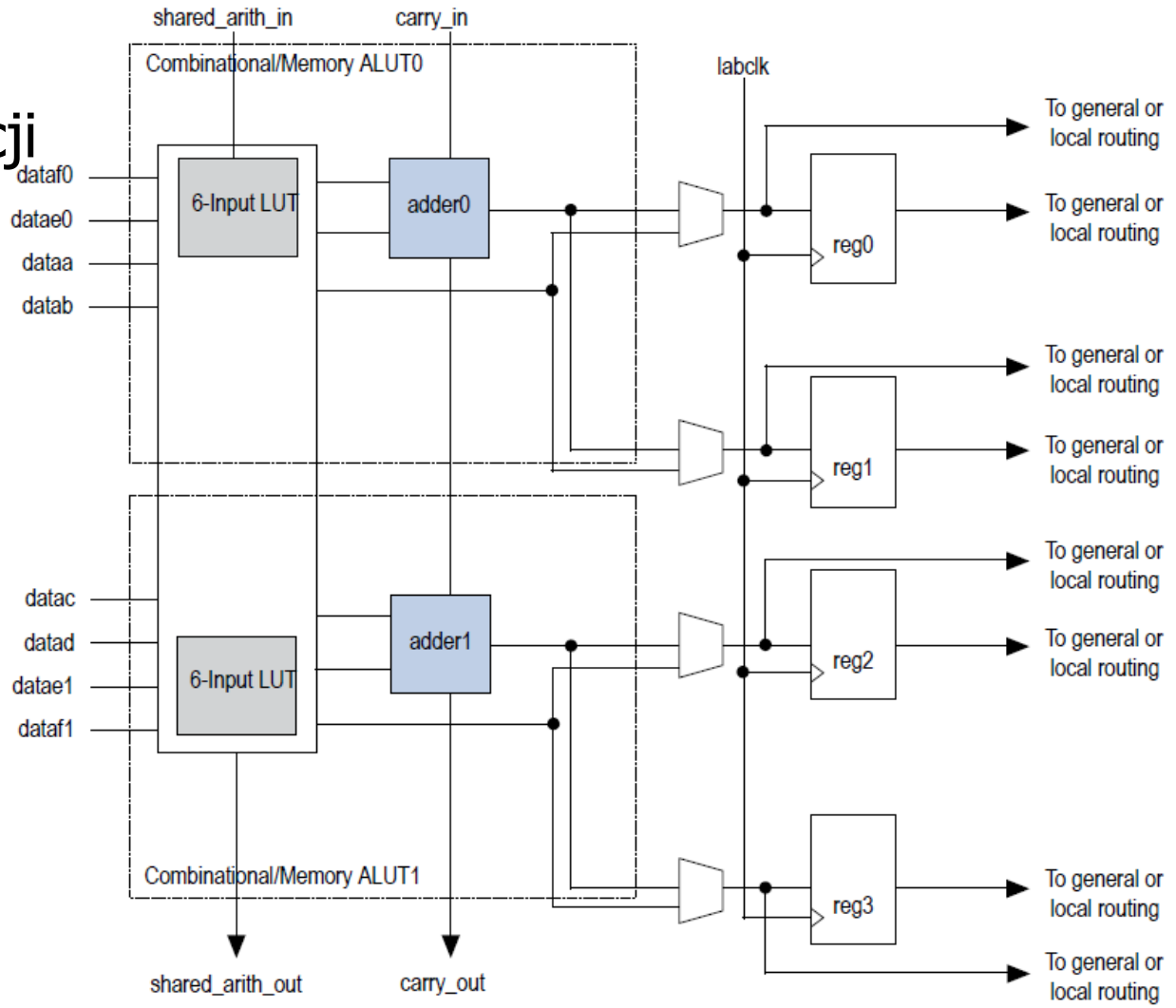


ALTERA – LUT-4


Register Chain
Routing from
previous LE
LAB-Wide
Synchronous
Load
LAB-Wide
Synchronous
Clear
Register Bypass
Programmable
Register
LE Carry-In
data 1
data 2
data 3
data 4
Look-Up Table
(LUT)
Carry
Chain
Synchronous
Load and
Clear Logic
D
Q
ENA
CLRN
Row, Column,
And Direct Link
Routing
Row, Column,
And Direct Link
Routing
labclr1
labclr2
Chip-Wide
Reset
(DEV_CLRn)
Asynchronous
Clear Logic
Local
Routing
Register Feedback
Clock &
Clock Enable
Select
Register Chain
Output
LE Carry-Out
labclk1
labclk2
labclkena1
labclkena2

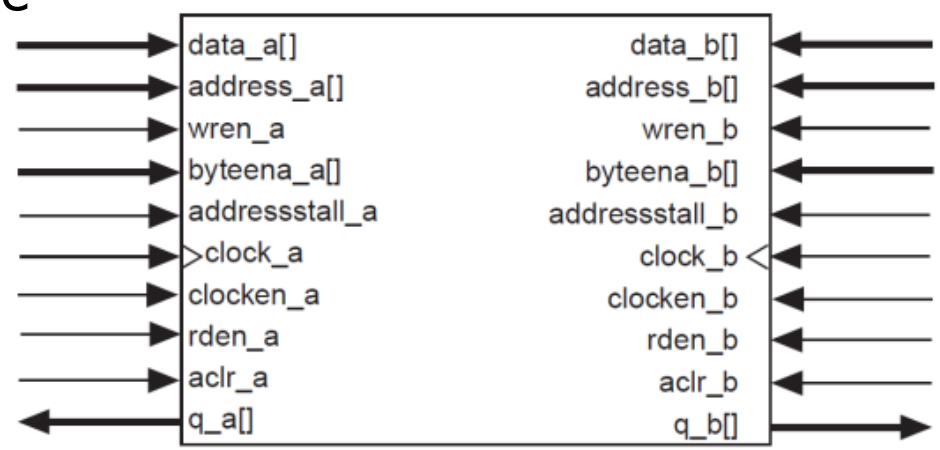
ALTERA – LUT-6

- LUT6
  - 64 kombinacji wejść
  - 2 wyjścia
  - 2 rejestry
- Dodatkowa logika (!) na sumator
- Brak pamięci rozproszonej
  - emulacja



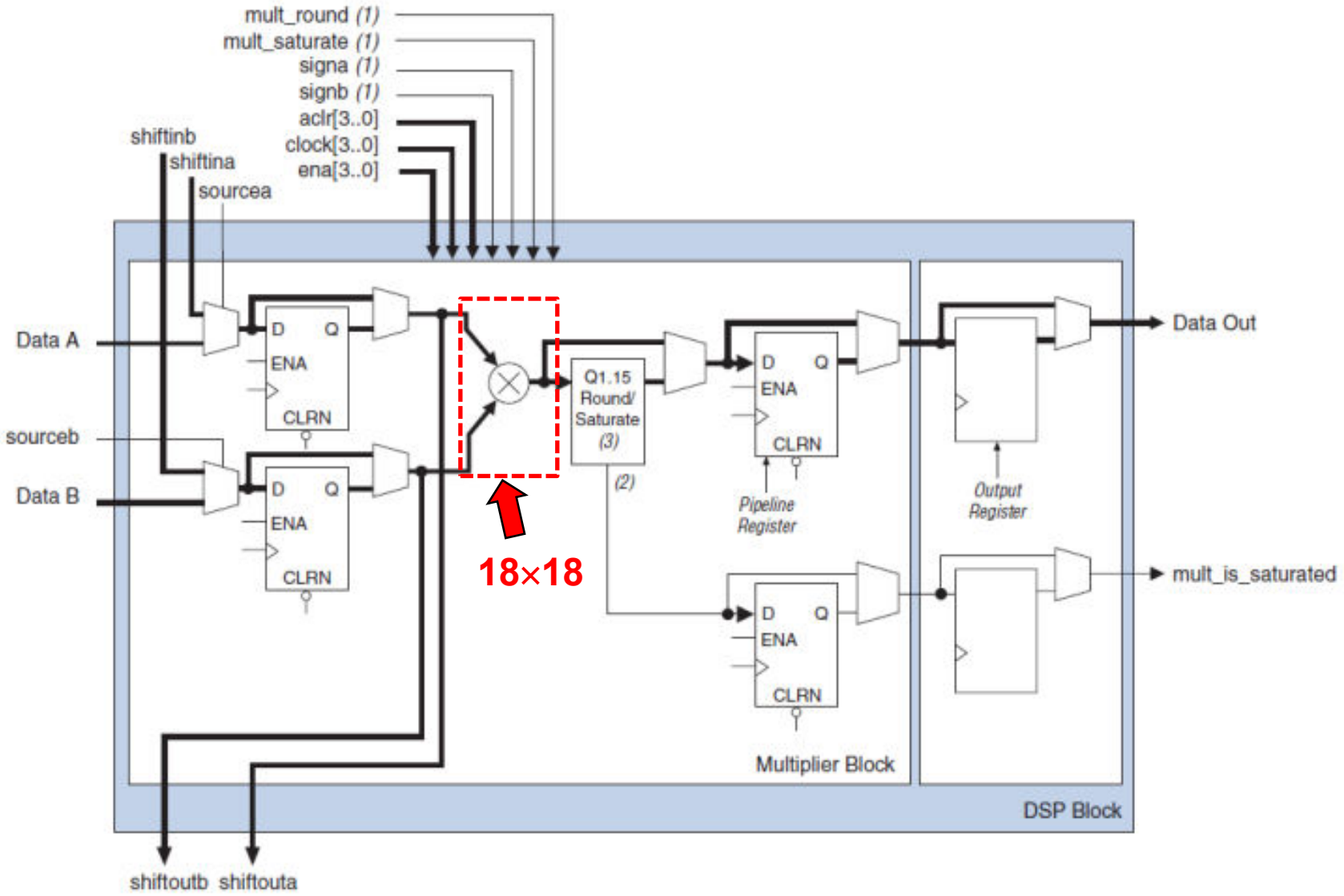
ALTERA – Block memory

- Dwa symetryczne porty
  - Dwie domeny częstotliwościowe
- Różne rozmiary bloków pamięci w jednym układzie
  - M512 – 512 bitów
  - M9K – 9 kbitów
  - M10K – 10 kbitów
  - M20K – 20 kbitów
  - M144K – 144 kbitów (16kB)
  - Uwaga na sumaryczną pojemność pamięci!
- Różne szerokości słów
  - 1 – 36 bit
- Praca jako FIFO

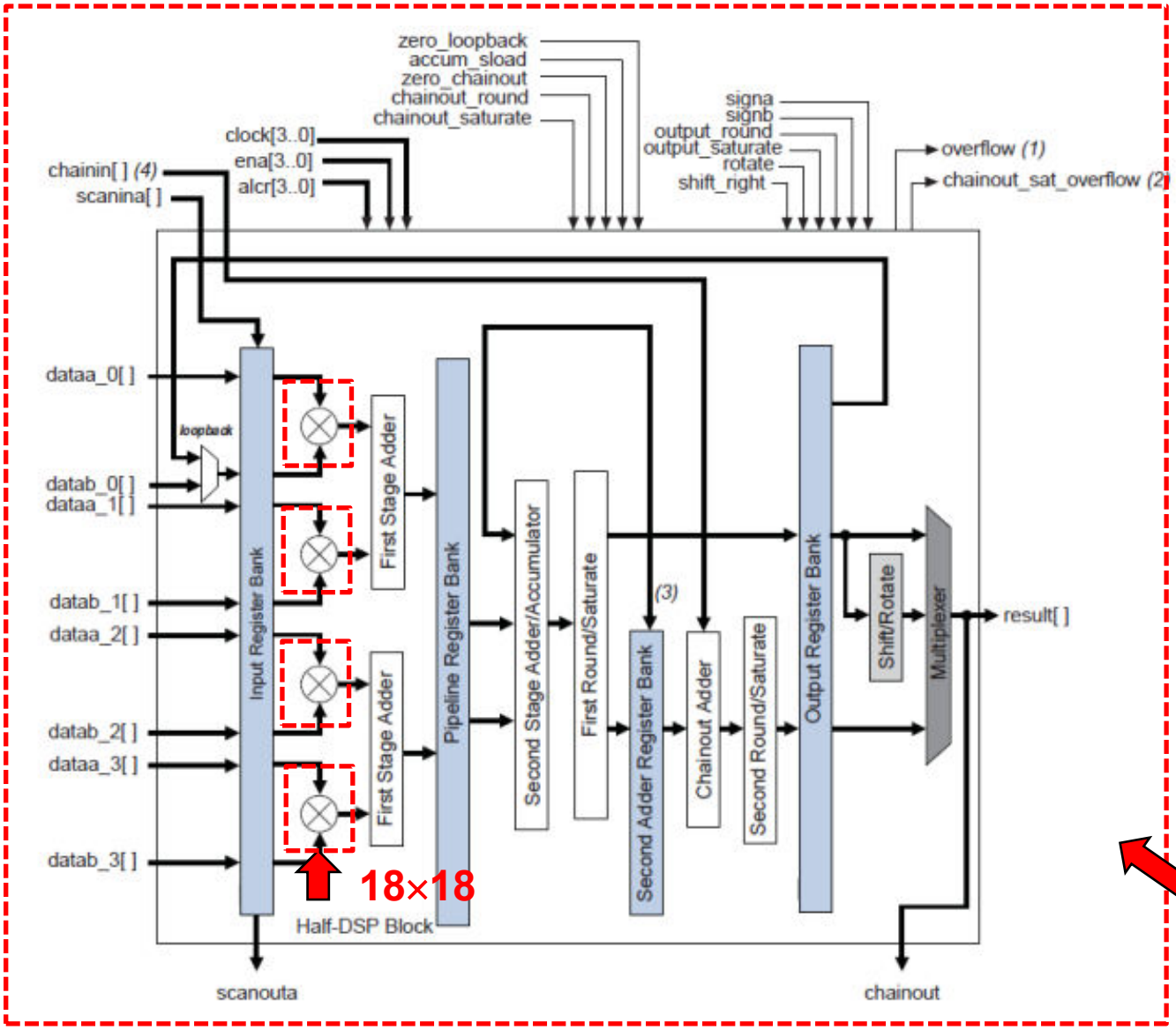


Read Port	Write Port								
	4K × 1	2K × 2	1K × 4	512 × 8	256 × 16	128 × 32	512 × 9	256 × 18	128 × 36
4K × 1	✓	✓	✓	✓	✓	✓	–	–	–
2K × 2	✓	✓	✓	✓	✓	✓	–	–	–
1K × 4	✓	✓	✓	✓	✓	✓	–	–	–
512 × 8	✓	✓	✓	✓	✓	✓	–	–	–
256 × 16	✓	✓	✓	✓	✓	✓	–	–	–
128 × 32	✓	✓	✓	✓	✓	✓	–	–	–
512 × 9	–	–	–	–	–	–	✓	✓	✓
256 × 18	–	–	–	–	–	–	✓	✓	✓
128 × 36	–	–	–	–	–	–	✓	✓	✓

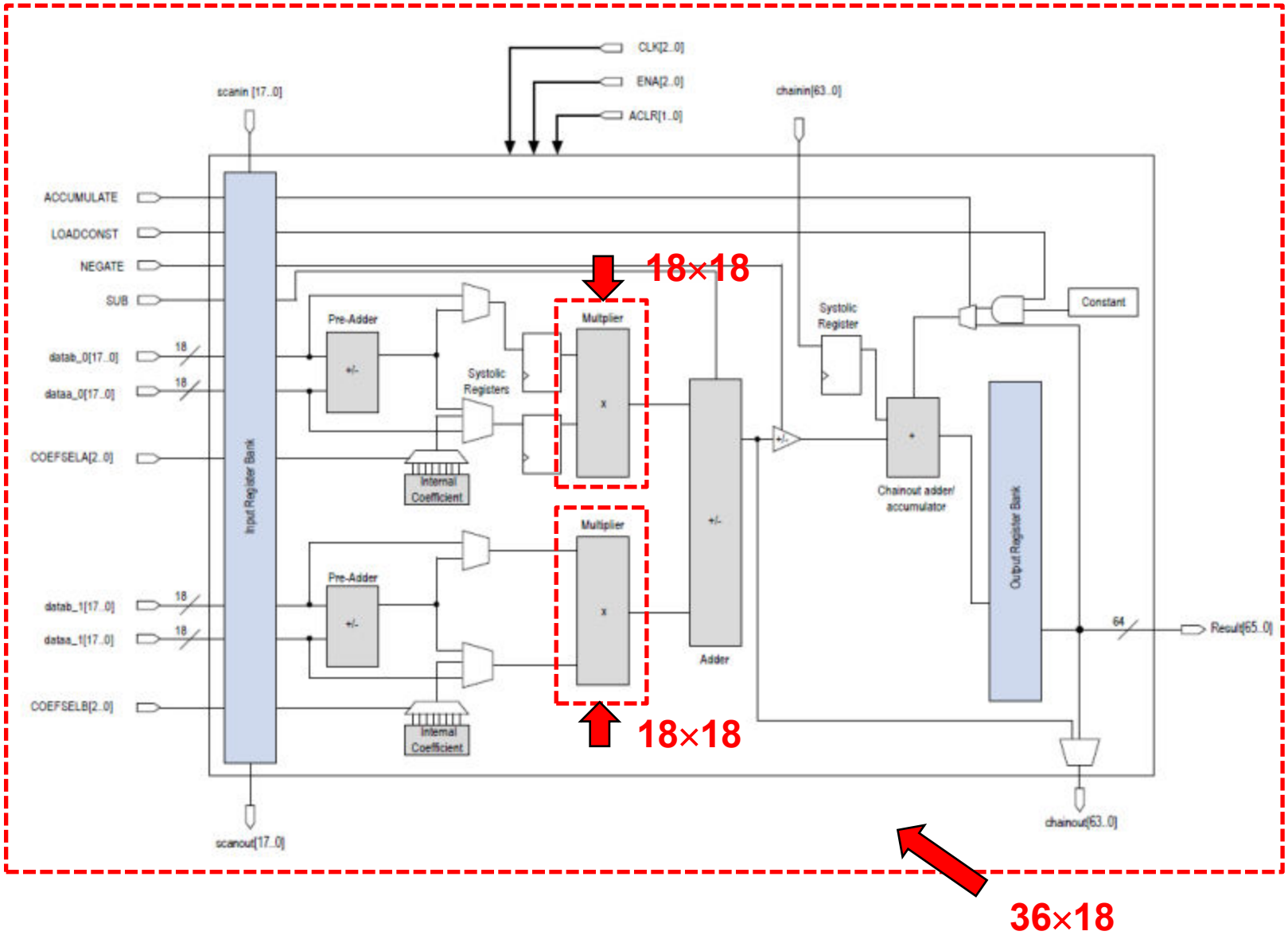
ALTERA – SimpleDSP (Cyclone II)



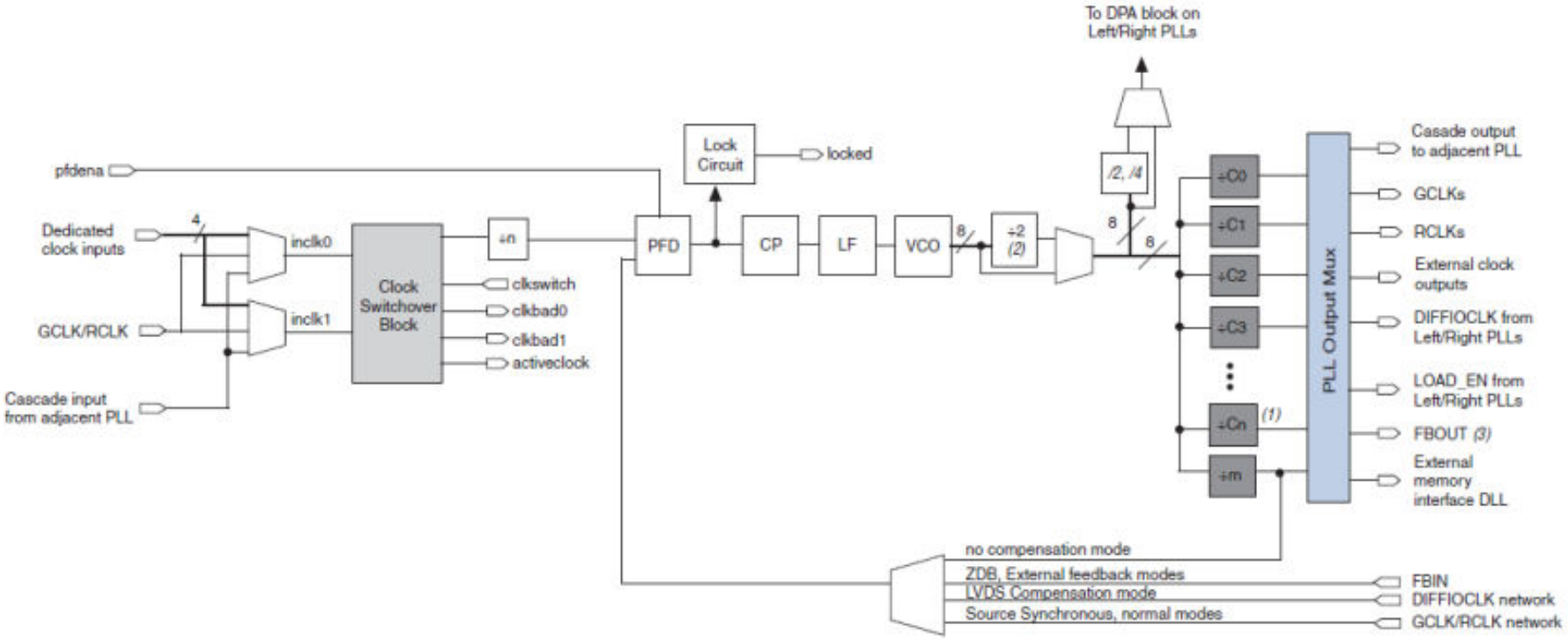
ALTERA – DSP (Stratix IV)



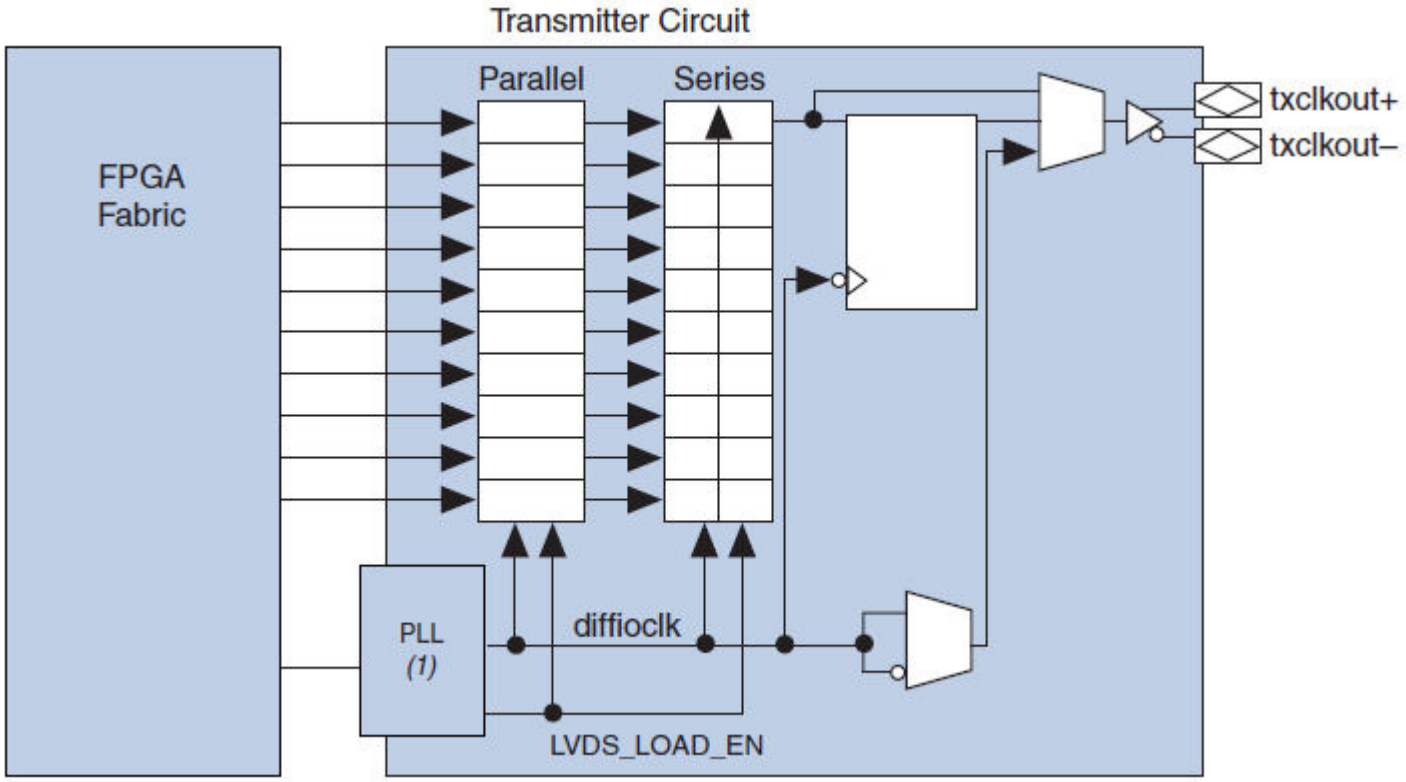
ALTERA – DSP (Stratix V)



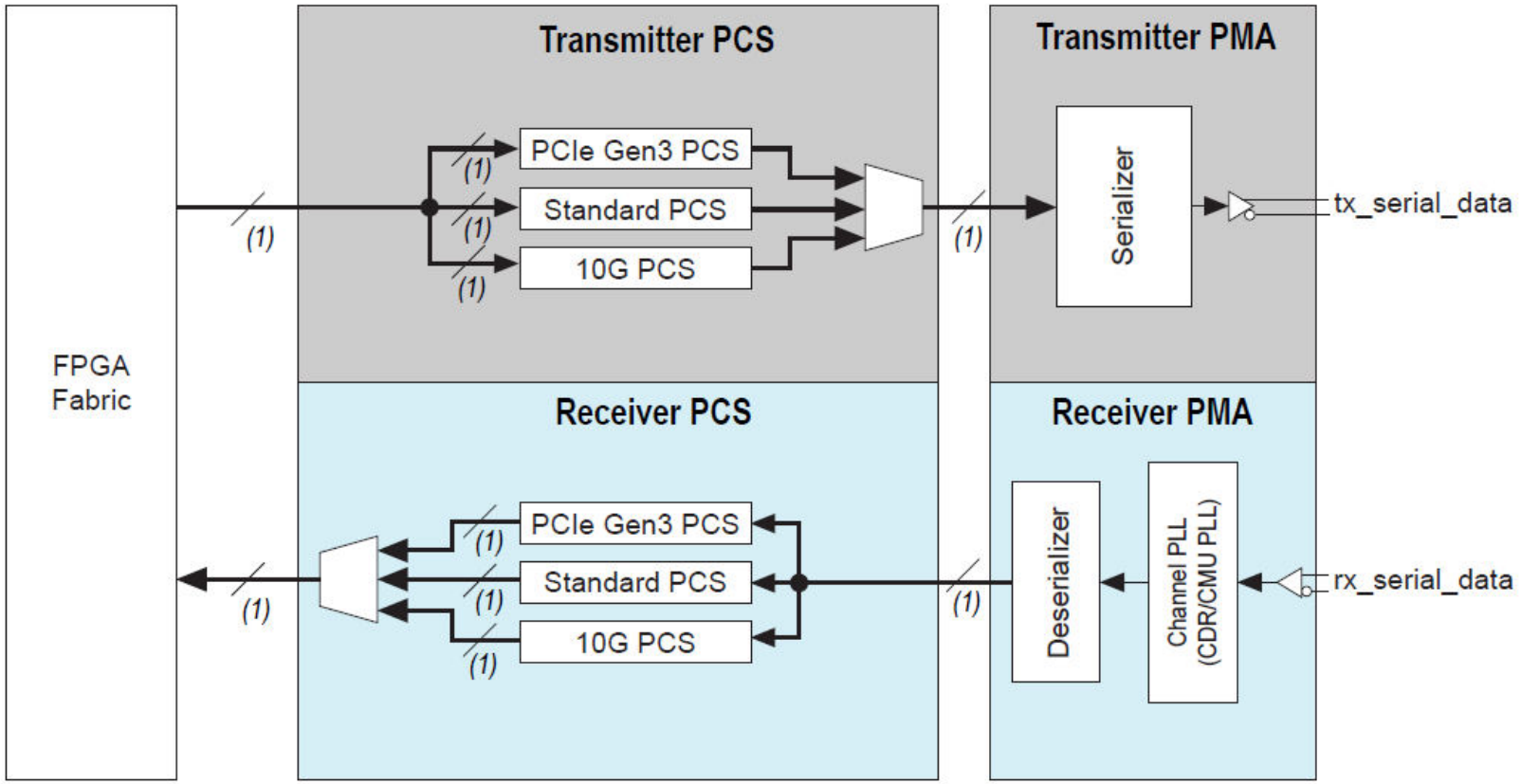
ALTERA – PLL


To DPA block on Left/Right PLLs
pfdena
Lock Circuit
locked
Dedicated clock inputs
4
inclk0
GCLK/RCLK
inclk1
Cascade input from adjacent PLL
Clock Switchover Block
÷n
clkswitch
clkbad0
clkbad1
activeclock
PFD
CP
LF
VCO
8
÷2 (2)
/2, /4
÷C0
÷C1
÷C2
÷C3
÷Cn
(1)
÷m
PLL Output Mux
Casade output to adjacent PLL
GCLKs
RCLKs
External clock outputs
DIFFIOCLK from Left/Right PLLs
LOAD_EN from Left/Right PLLs
FBOUT (3)
External memory interface DLL
no compensation mode
ZDB, External feedback modes
LVDS Compensation mode
Source Synchronous, normal modes
FBIN
DIFFIOCLK network
GCLK/RCLK network

ALTERA – SerDes



ALTERA – Multi-Gigabit Transceiver



ALTERA – Multi-Gigabit Transceiver

Protocol Solutions

PCI Express (PCIe)
PCI Express (PCIe) Hard IP
RapidIO
SerialLite II
Basic Mode

Communications

10-Gigabit Ethernet XAUI
10-Gigabit Ethernet XFI-SFI
40/100 Gigabit Ethernet
40G/100G
CEI-6G
CPRI
DisplayPort
Gigabit Ethernet
GPON
HiGig
Interlaken
OBSAI
SDH/SONET
SFI-5
SGMII
SPI-4.2
V-by-One

Computing

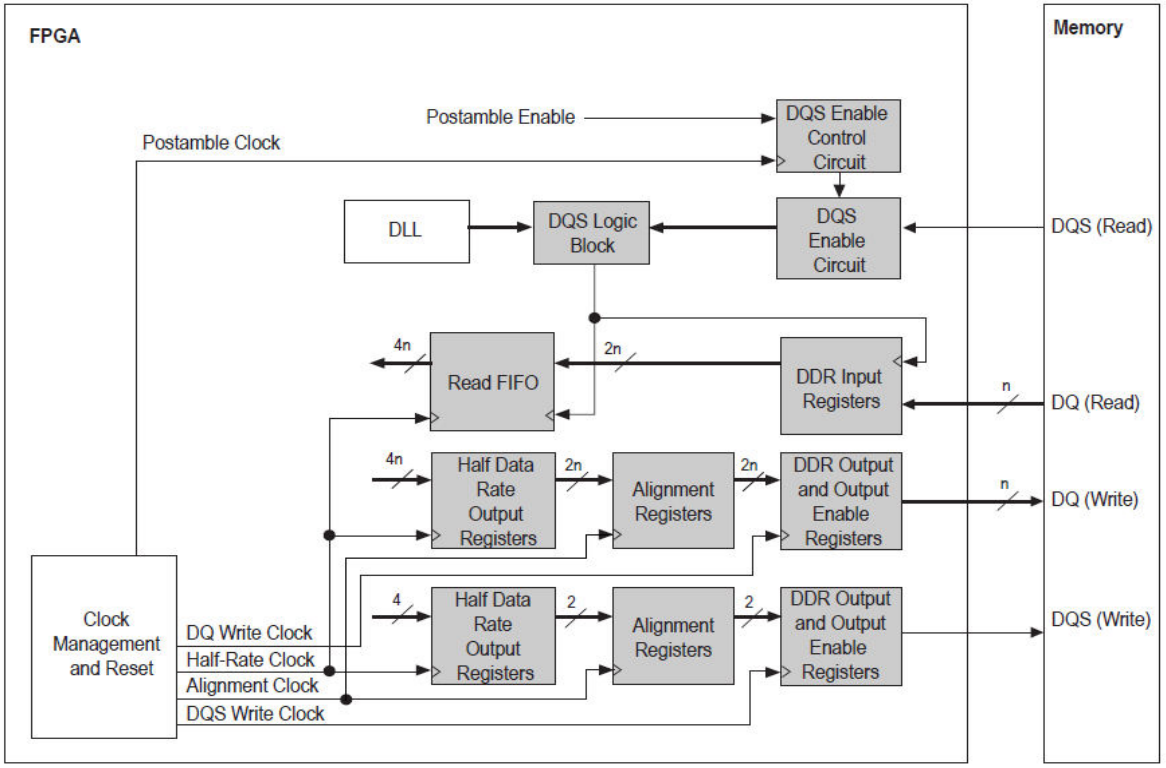
Fibre Channel
HyperTransport
SATA and SAS

Broadcast

Serial Digital Interface (SDI)

ALTERA – Pamięci

- Wsparcie PHY dla SDRAM
  - DDR2
  - DDR3
  - QDR



Memory Standard	Soft Memory Controller
DDR3 SDRAM	Half rate and quarter rate
DDR2 SDRAM	Full rate and half rate
RLDRAM 3	Half rate and quarter rate
RLDRAM II	Full rate and half rate
QDR II+ SRAM	Full rate and half rate
QDR II SRAM	Full rate and half rate

ALTERA/INTEL - Podsumowanie

- Niejednorodność
  - 2 architektury LUT
    - LUT4 oraz LUT6 dwuwyjściowy
  - 3 pojemności pamięci blokowej
    - Wiele różnych w jednym układzie
  - Wiele różnych architektur bloków DSP
  - Różne prędkości i funkcjonalności portów SerDes
- Od serii V ujednolicenie
  - Stratix - pojemność
  - Arria - prędkość
  - Cyclone - cena

ALTERA/INTEL - Podsumowanie

- Zalety
  - Najszybsze
  - Najszersze wsparcie dla protokołów komunikacji
- Wady
  - Brak pamięci rozproszonej
  - Cena
  - Pracochłonność projektowania PCB
  - Zaawansowane obudowy układów
    - W praktyce tylko BGA (Ball Grid Array)

Dziękuje za uwagę

- Pytania?

System Verilog
Język opisu sprzętu

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

System Verilog – bazujący na Verilog

- Stworzony 1984 przez Phila Moorby i Prabhu'a Goel'a w firmie Gateway Design Automation
  - Stworzony dla potrzeb symulacji
  - Składania opisująca co, kiedy ma się wydarzyć
  - Dostosowany dla potrzeb projektowania i syntezy
- W 1990 firma została kupiona przez Cadence Design Systems
- W 1995 został poddany standaryzacji przez IEEE (Verilog-95)
- W roku 2001 dodano znaczące rozszerzenia do języka (Verilog-2001)
  - Od tego momentu Verilog był już wspierany przez większość oprogramowania do projektowania sprzętu (narzędzia EDA)
- W 2005 kolejna rozszerzona wersja (Verilog-2005)

Verilog

Jakie problemy występują w języku Verilog?

Verilog

Nie zawsze jest jasne, co będzie / nie będzie się syntetyzować

- Pętle while
- Skomplikowane pętle itp.
- Operatory dzielenia / modulo
- Ukryte maszyny stanu
- …

Czasami byłoby prościej po prostu powiedzieć zamiast domniemywać

- Określić blok always jako kombinatoryczny
- Określić coś jako przerzutnik lub zatrzask
- Określić wyrażenie case jako równoległe
- Określić wyrażenie case jako priorytetyzowane
- ...

Verilog

Aby nie wpaść w problemy, należy być ostrożnym

- Domniemywane zatrzaski zamiast logiki kombinacyjnej
- Mieszanie blokujących i nieblokujących wyrażeń
- Mieszanie synchronicznych i asynchronicznych wyzwoleń w listach czułości (sensitivity list)

Niektóre rzeczy są mylące

- Czy “reg” koniecznie reprezentuje rejestr?

Verilog

Niektóre rzeczy są niewygodne

- Nie można używać wielowymiarowych tablic połączeń jako wejść/wyjść…
- Nie można używać wielowymiarowych tablic rejestrów

Wygodnie byłoby mieć

- Konstrukcje takie jak TYPEDEF, STRUCT, ENUM
- Lepsze mechanizmy synchronizacji dla pracy równoległej testbench'ów, np. semafory, kolejki FIFO
- Wbudowane elementy programowania obiektowego w testbench'ach
- Łatwiejszą integrację z programami/podfunkcjami w C/C++ względem PLI (Programming Language Interface)
- …

SystemVerilog – „Nowy” Verilog

Mniej więcej ta sama składania/semantyka

Kompatybilność wsteczna z większością kodu w Verilog’u

Wiele nowych funkcji do weryfikacji (testbench’e)

Poprawki do istniejących zapisów w Verilog’u, ułatwiające ich użycie

Nowe typy danych

C → C++
Verilog → System Verilog

SystemVerilog – „Nowy” Verilog

Zwiększenie możliwości weryfikacyjnych Verilog

Jeden język dla projektu, testbench’a i asercji

Zmniejszenie potrzeby stosowania PLI w celu uruchomienia środowiska weryfikacyjnego

Przewidywalna semantyka dla wszystkich narzędzi

- Symulacja
- Synteza
- Weryfikacja formalna

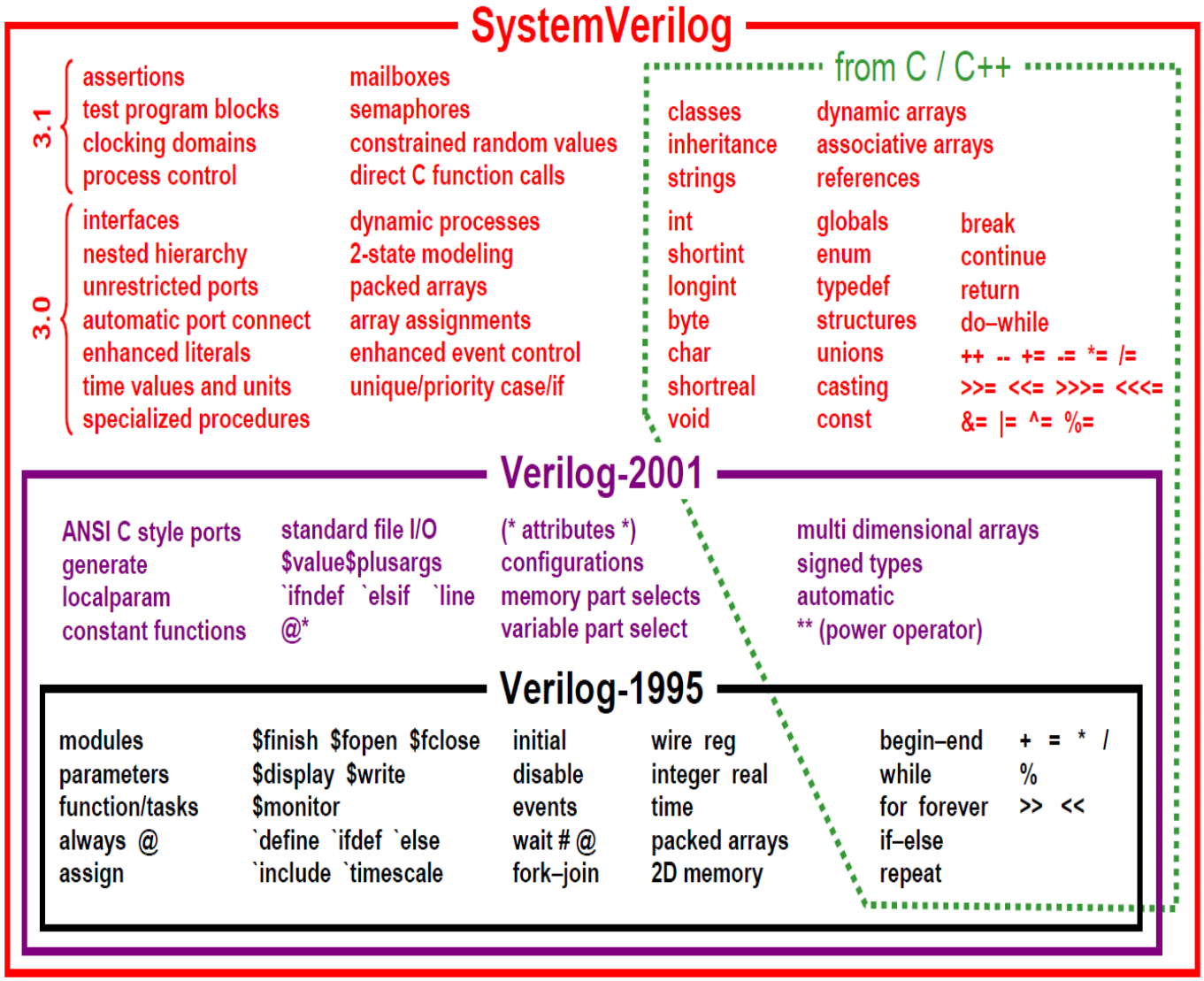
Bez pogorszenia wydajności

Kompatybilność wsteczna z Verilog IEEE 1364-2001

Historia

- SystemVerilog ma swój początek w podarowaniu konsorcjum Accelera języka Superlog w 2002
- Większość funkcji związanych z weryfikacją bazuje na języku OpenVera podarowanym przez Synopsys
- W 2005 SystemVerilog został przyjęty jako Standard IEEE 1800-2005
- In 2009 standard połączono z podstawowym Verilog'iem (IEEE 1364-2005), tworząc Standard IEEE 1800-2009
- IEEE 1800-2012
- Aktualna wersja standardu to IEEE 1800-2017
- Zestaw funkcji SystemVerilog można podzielić na dwie odrębne role:
  - SystemVerilog dla projekowania RTL , będący rozszerzeniem Verilog-2005; wszystkie funkcje tego języka są dostępne w SystemVerilog
  - SystemVerilog for weryfikacji, używający bogate techniki programowania obiektowego, bardziej zbliżony do Java'y niż do Verilog'a

Ogólny pogląd na SystemVerilog



SystemVerilog : Typy danych

Typy danych: Nowe typy abstrakcyjne

Verilog: typy 4-stanowe: `0,1,x,z` `wire` **lub** `reg`

Typy 2-stanowe: `int, shortint, longint, char, byte, bit`

shortint	- 2-stanowy, 16-bit ze znakiem
int	- 2-stanowy, 32-bit ze znakiem
longint	- 2-stanowy, 64-bit ze znakiem
byte	- 2-stanowy, 8-bit ze znakiem lub znak ASCII
bit	- 2-stanowy, szerokość definiowana przez użytkownika

Typ 4-stanowy: `logic`

Koniec zmartwień na temat wyboru `wire` vs `reg` !

Mniej mylące – nazwa nie wprowadza na manowce iż na pewno jest to rejestr, tak jak „`reg`”.

Typy specjalne: `void, shortreal`

Pozwalają na modelowanie na poziomie abstrakcji języka C

Skuteczne dla wykonywania symulacji

Typy danych: Nowe typy abstrakcyjne

Verilog: typy 4-stanowe: `0,1,x,z` `wire` **lub** `reg`

Typy 2-stanowe: `int, shortint, longint, char, byte, bit`

shortint	- 2-stanowy, 16-bit ze znakiem
int	- 2-stanowy, 32-bit ze znakiem
longint	- 2-stanowy, 64-bit ze znakiem
byte	- 2-stanowy, 8-bit ze znakiem lub znak ASCII
bit	- 2-stanowy, szerokość definiowana przez użytkownika

Typ 4-stanowy: `logic`

Koniec zmartwień na temat wyboru `wire` vs `reg` !

Mniej mylące – nazwa nie wprowadza na manowce iż na pewno jest to rejestr, tak jak „`reg`”.

Typy specjalne: `void, shortreal`

Pozwalają na modelowanie na poziomie abstrakcji języka C

Skuteczne dla wykonywania symulacji



Typy danych: typedef

typedef – definiowanie typów w stylu języka C

```
typedef pixel logic[23:0];
pixel                       one_pixel;
pixel[1920:1][1080:1]       display;
```

Dlaczego nie makro?

- **typedef** nie ma problemu ogólnej widoczności makr.
- **typedef** może być oceniony na poziomie składni, co sprawia że jest łatwiejszy do debugowania.

Typy danych: enum

Typy enumerowane, tak samo jak `enum` w C

```
typedef enum logic[2:0] {RESET, COMPUTE_A, COMPUTE_B, SEND, ERROR} states_t;

states_t current_state, next_state;
[...]
          case(current_state)
COMPUTE_A :                    next_state = COMPUTE_B;
COMPUTE_B :                    next_state = SEND;
SEND      :     if(f_ok)       next_state = COMPUTE_A;
           else if(f_err)      next_state = ERROR;
[...]
```

Opcjonalnie, typ danych może być wybrany

- Domyślnym typem danych jest **`int`**

Opcjonalnie, można określić wartości wymienionych nazw

- Domyślna wartość początkowa to **0**
- Wartości kolejnych nazw są zwiększane o 1

```
enum reg [1:0] {WAIT=2'b01, LOAD=2'b10, READY} state;
```

Typy danych : structure

`structure` jest zbiorem obiektów, które mogą być różnych typów

- Może być używany do zebrania kilku zmiennych w jeden obiekt
- Możliwość przypisania poszczególnych sygnałów w strukturze
- Możliwość przypisania struktury jako całości
- Można przekazywać struktur przez porty i do zadań lub funkcji

```
typedef struct packed {
                    logic  [2:0] packet_type;
                    logic  [3:0] dest_addr;
                    logic  [3:0] src_addr;
                    logic [47:0] data_payload;
          } router_packet;

router_packet        my_packet;

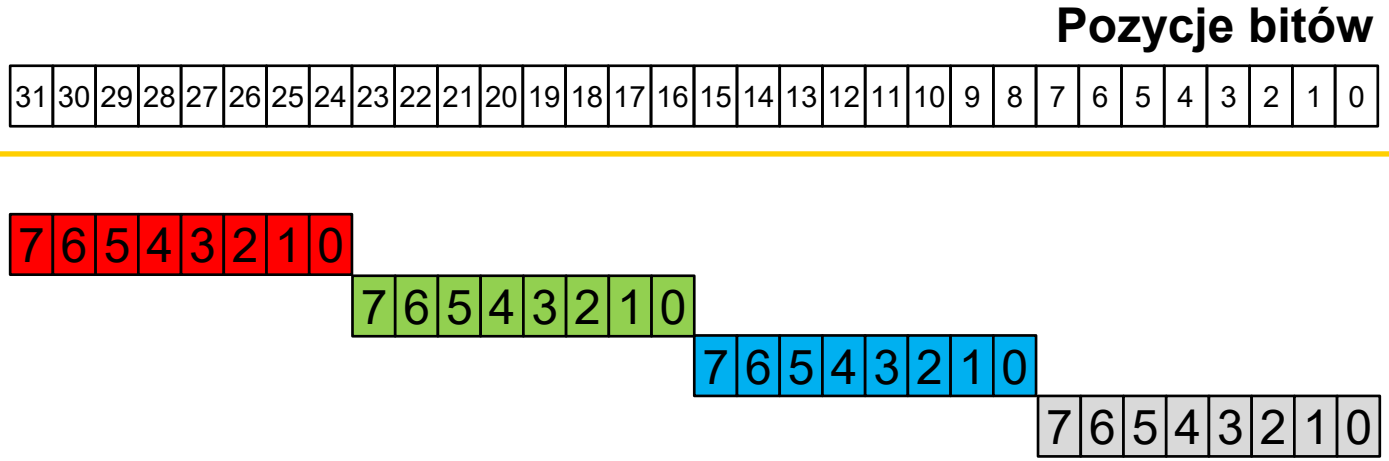
[...]
assign    my_packet.packet_type   = PT_WRITE;
assign    my_packet.dest_addr     = wr_prt;
assign    my_packet.data_payload  = 48'hAA10_2CD0_4050;
[...]
always @(posege clk)
  my_packet.src_addr <=  my_packet.src_addr + 1;
```

Typy danych : union

- Podobny do struktury
  - w danym momencie tylko jedno z pól jest ważne
  - Kolejne pola są różnymi interpretacjami tych samych, współdzielonych bitów
  - Wielkość typu union jest wielkością największego z pól

```
union
{
  // . . .  fields . . .
} name_of_union;
```

Typy danych : sturct vs union



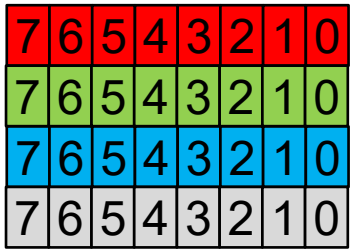

```
struct packed
{
 logic   [7:0] red;
 logic   [7:0] green;
 logic   [7:0] blue;
 logic   [7:0] alpha;
} pixel_s;
```

Typy danych : sturct vs union

Pozycje bitów

7	6	5	4	3	2	1	0

```
union
{
 logic   [7:0] red;
 logic   [7:0] green;
 logic   [7:0] blue;
 logic   [7:0] alpha;
} pixel_s;
```

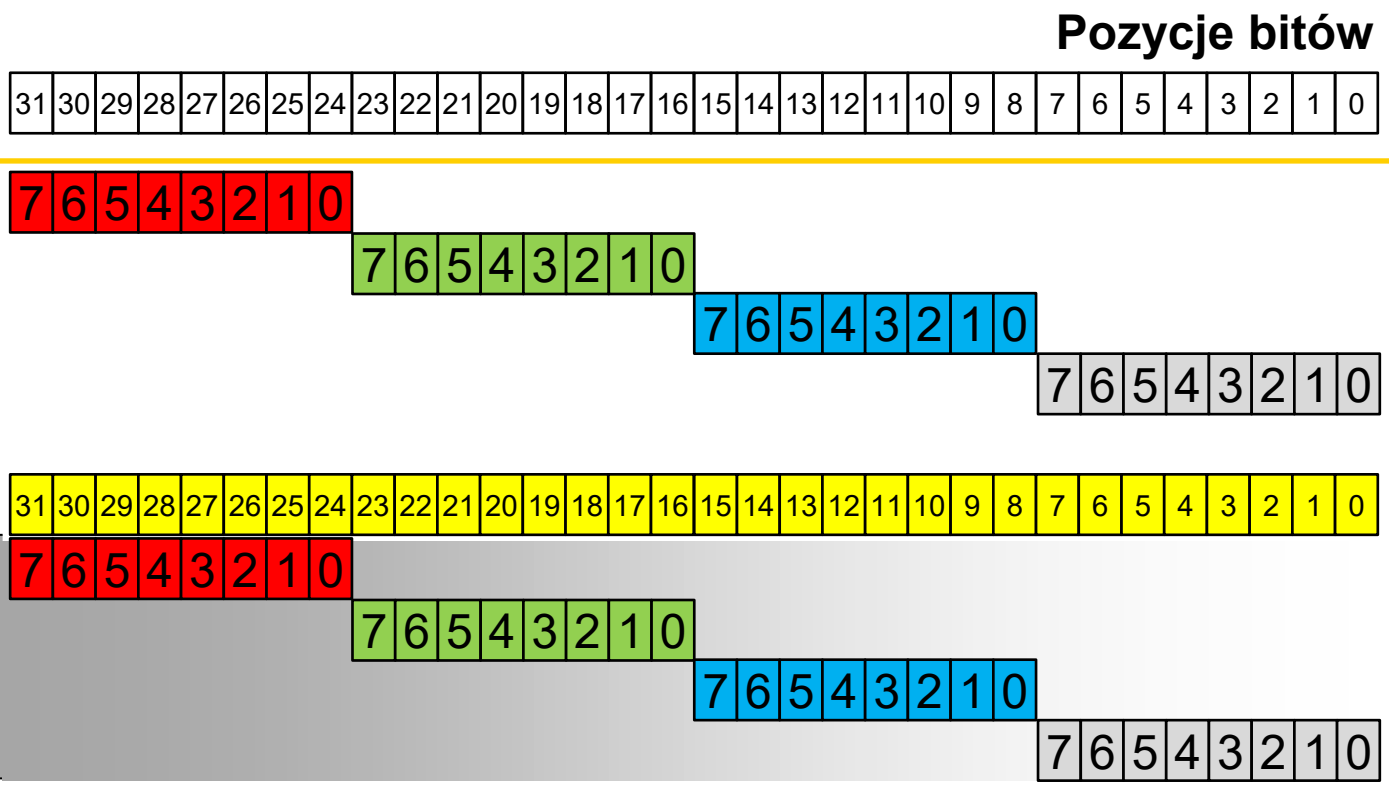


Typy danych : sturct vs union

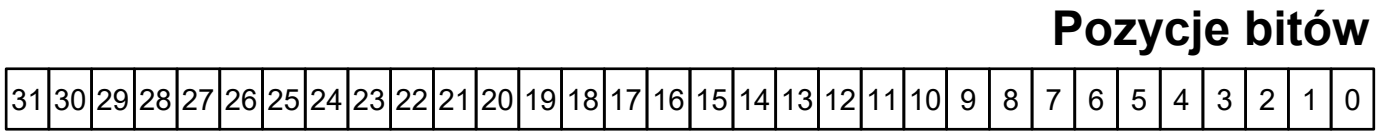
```
struct packed
{
 logic  [7:0] red;
 logic  [7:0] green;
 logic  [7:0] blue;
 logic  [7:0] alpha;
} pixel_s;

union
{
 integer  all;
 pixel_s  pixel;
} example_u;

example_u x;
```



Typy danych : sturct vs union

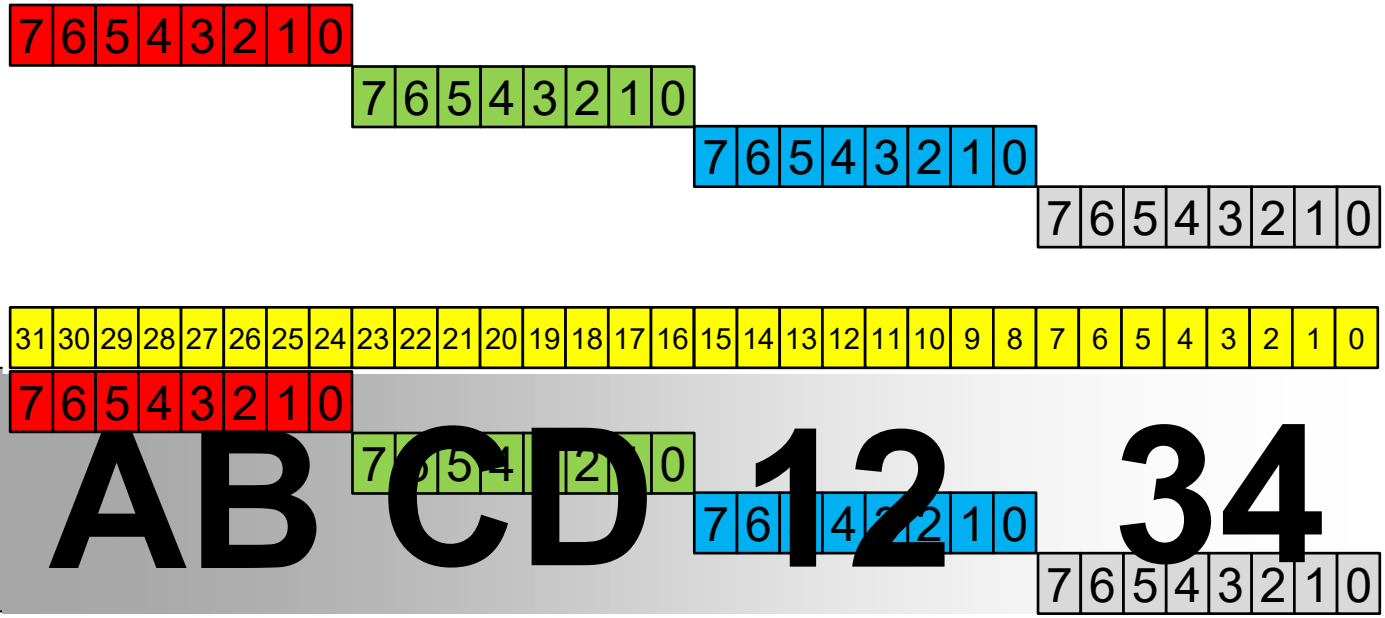



```
struct packed
{
 logic   [7:0] red;
 logic   [7:0] green;
 logic   [7:0] blue;
 logic   [7:0] alpha;
} pixel_s;

union
{
 integer  all:
 pixel_s  pixel:
} example_u;

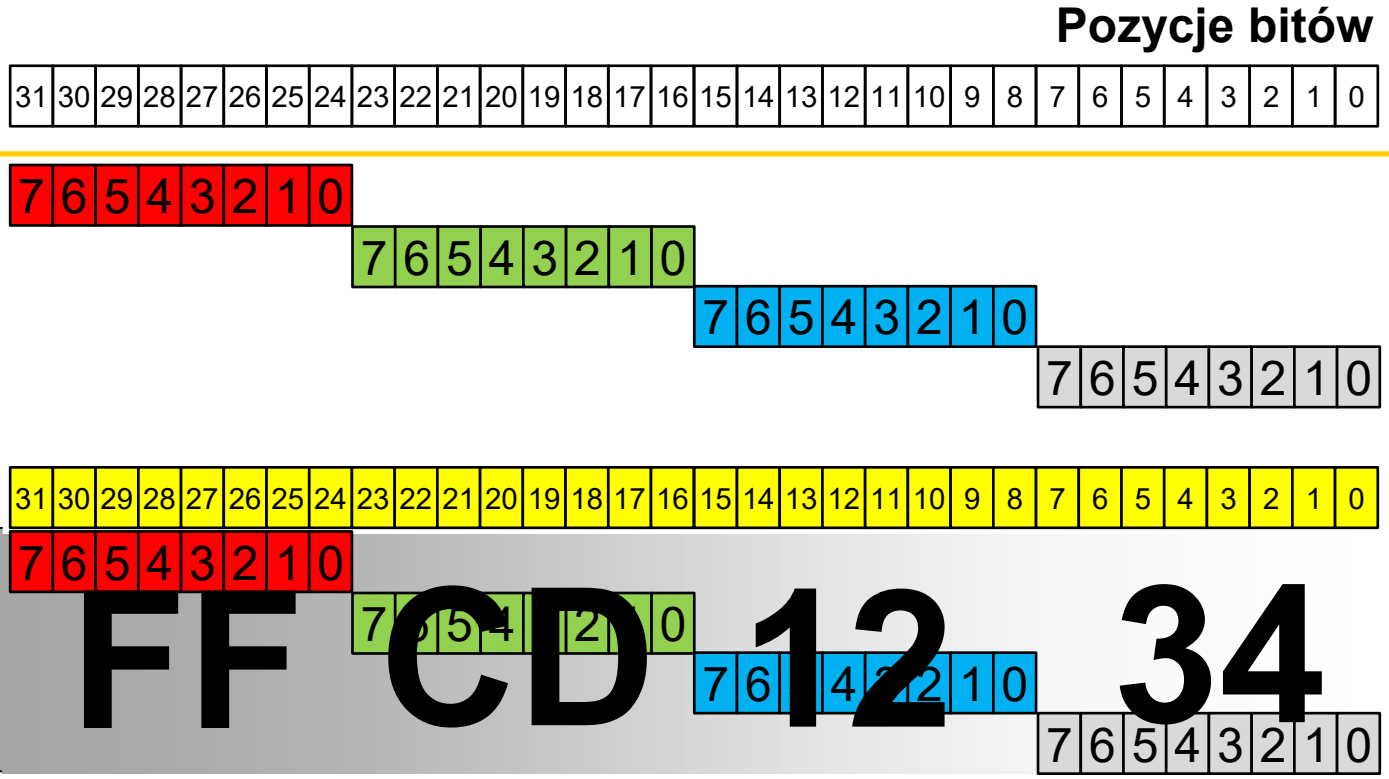
example_u x;

begin
  x.all            = 32'hABCD1234;
```




```
end
```

Typy danych : sturct vs union




```
struct packed
{
 logic   [7:0] red;
 logic   [7:0] green;
 logic   [7:0] blue;
 logic   [7:0] alpha;
} pixel_s;

union
{
 integer  all:
 pixel_s  pixel:
} example_u;

example_u x;

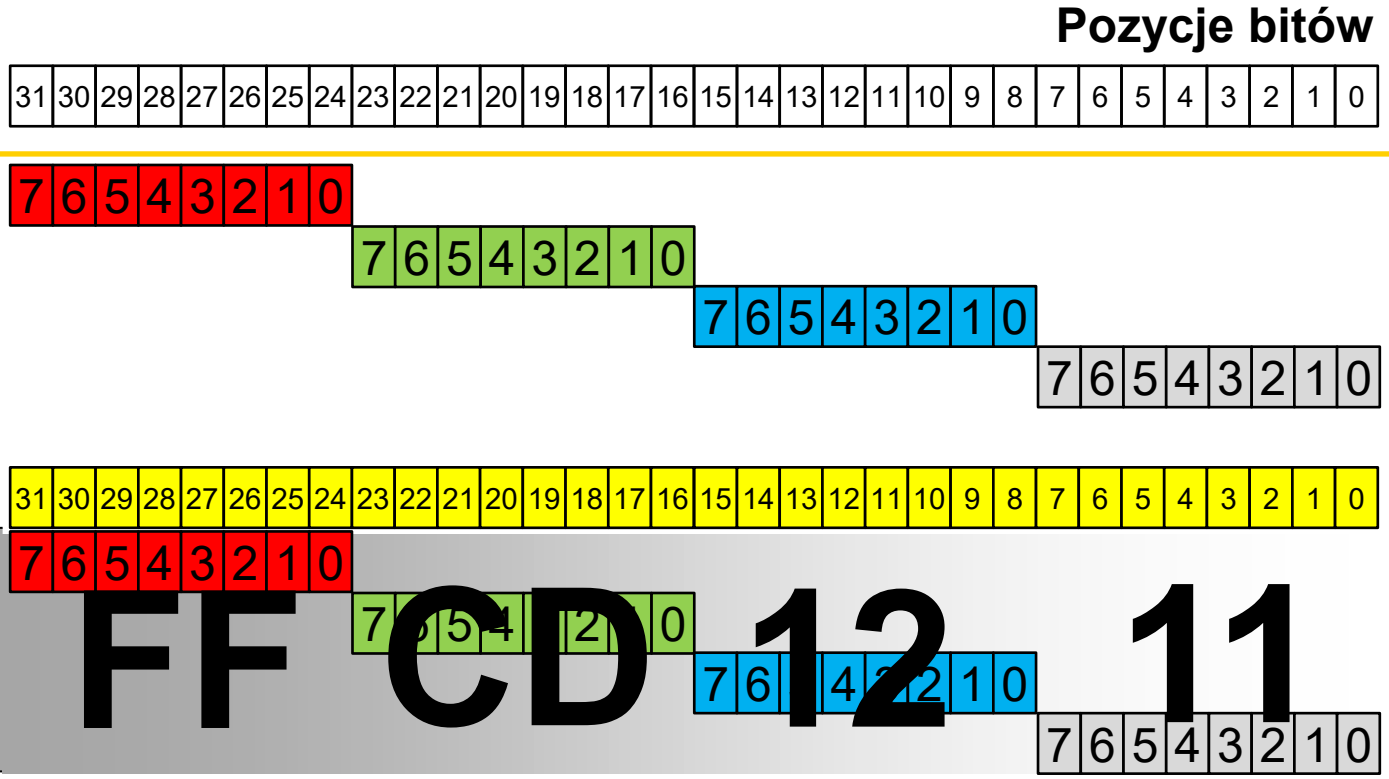
begin
  x.all           = 32'hABCD1234;

  x.pixel.red     = 8'hFF;




end
```

Typy danych : sturct vs union




```
struct packed
{
 logic  [7:0] red;
 logic  [7:0] green;
 logic  [7:0] blue;
 logic  [7:0] alpha;
} pixel_s;

union
{
 integer  all:
 pixel_s  pixel:
} example_u;

example_u x;

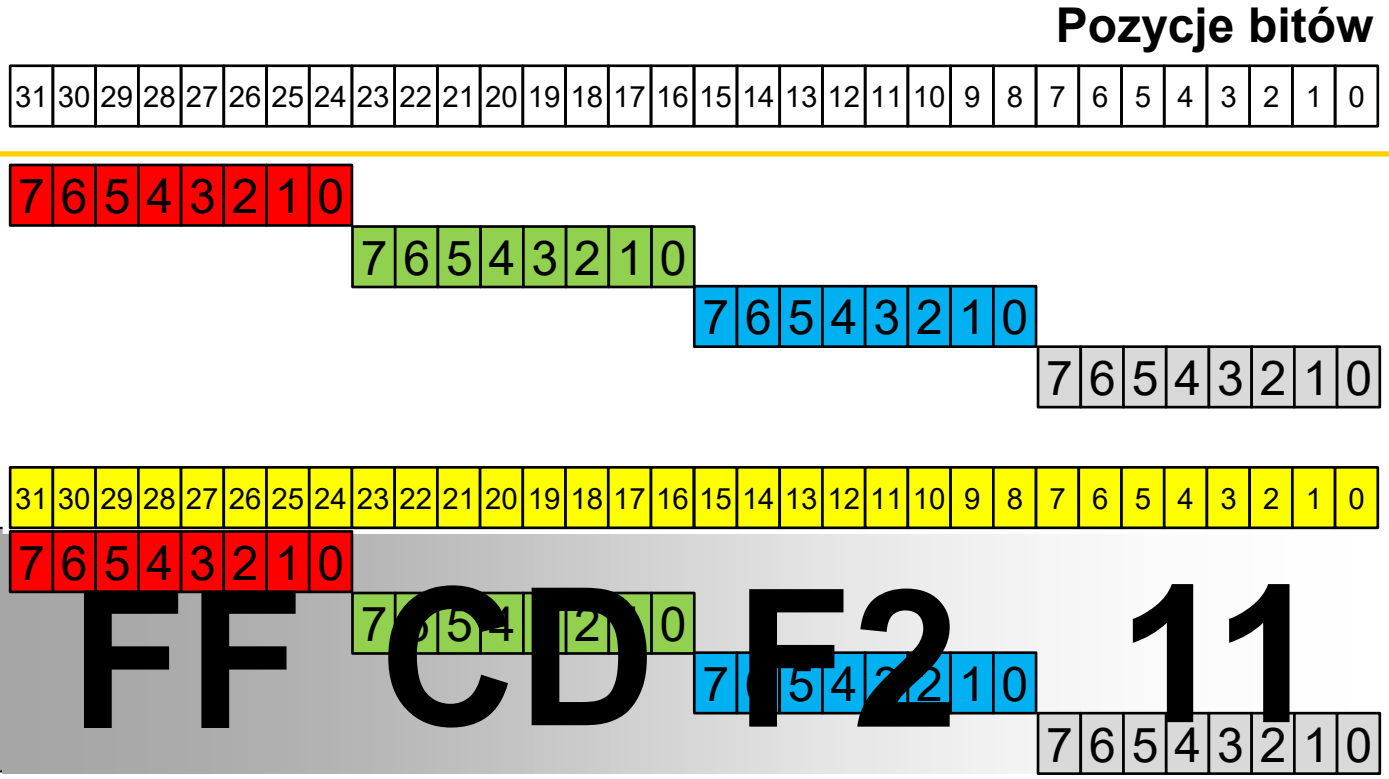
begin
  x.all           = 32'hABCD1234;

  x.pixel.red     = 8'hFF;

  x.pixel.alpha   = 8'h11;


end
```

Typy danych : sturct vs union




```
struct packed
{
 logic   [7:0] red;
 logic   [7:0] green;
 logic   [7:0] blue;
 logic   [7:0] alpha;
} pixel_s;

union
{
 integer  all:
 pixel_s  pixel:
} example_u;

example_u x;

begin
  x.all            = 32'hABCD1234;

  x.pixel.red      = 8'hFF;

  x.pixel.alpha    = 8'h11;

  x[15:12]         = 4'hF;
end
```

SystemVerilog : Pomoce syntezy (synthesis helpers)

Wyrażenia Always : Blok kombinacyjny

Lista czułości jest domniemywana na podstawie użytych wyrażeń.

Zmienne użyte po lewej stronie przypisania nie mogą być pisane przez inny proces.

Jest uruchamiany automatycznie: w czasie zero, po wszystkich blokach initial i always, dzięki czemu wyjścia są zawsze spójne z wejściami.

Styl Verilog:

```
reg [7;0] res;

always@(*)
  begin
    case (mux)
    2'd0 : res <= in0;
    2'd1 : res <= in1;
    2'd2 : res <= in2;
    2'd3 : res <= in3;
    endcase
  end
```

Styl SystemVerilog:

```
reg [7;0] res;

always_comb
  begin
    case (mux)
    2'd0 : res <= in0;
    2'd1 : res <= in1;
    2'd2 : res <= in2;
    2'd3 : res <= in3;
    endcase
  end
```

Wyrażenia Always : przerzutniki i zatrzaski

Procesura **always_latch** określa listę czułości i wykonuje się tak samo jak **always_comb** procedure.

Styl Verilog:

```
reg [7;0] res;

always @(enable, in)
  if (enable) res <= in;
```

Styl SystemVerilog:

```
reg [7;0] res;

always_latch
  if (enable) res <= in;
```

Procedura **always_ff** może być używa aby zamodelować syntezowalną logikę sekwencyjną

Styl Verilog:

```
reg [7;0] res;

always @(posedge clk)
  if (enable) res <= in;
```

Styl SystemVerilog:

```
reg [7;0] res;

always_ff @(posedge clk)
  if (enable) res <= in;
```

- Zapisy te nie mogą „zmusić” syntezera do użycia przerzutników, jeśli w programie opisano zatrzask
- Jednakże, mogą one **ostrzec** , że to co zostało opisane nie pasuje do tego co wskazano jako pożądane!

Wyrażenia sterujące: unique

`unique` mówi syntezerowi i symulatorowi, że:

tylko jeden z przypadków będzie pasował

`unique0:`

tylko jeden z przypadków będzie pasował lub żaden

Ignorowany jest priorytet, więc można zsyntezować jako jeden równoległy multiplekser.
Symulator **da ostrzeżenie** jeśli
- więcej niż jeden z przypadków pasuje
- żaden z przypadków nie pasuje (nie unique0)

Działa także z „if”: **unique if**(…) …

```
unique case (sel)
        CASE1: [...]
        CASE2: [...]
        CASE3: [...]
endcase
```

```
unique    if (sel == 1) [...]
else      if (sel == 2) [...]
else      if (sel == 3) [...]
else                    [...]
```

Wyrażenia sterujące: priority

`priority` mówi syntezerowi i symulatorowi, że:

co najmniej jeden z przypadków zawsze będzie pasował

Jeśli nie ma to miejsca, symulator **ostrzeże**.

Działa również z „**if**”.

Łatwy sposób na uniknięcie omyłkowych zatrzasków!

```
priority case (sel)
       CASE1: [...]
       CASE2: [...]
       CASE3: [...]
endcase
```

```
priority if (sel == 1) [...]
else     if (sel == 2) [...]
else     if (sel == 3) [...]
else                   [...]
```

SystemVerilog : Typy wysokiego poziomu

Typy wysokiego poziomu: tablice dynamiczne

Tablica dynamiczna jest jednowymiarową tablicą, której nie podano zakresu podczas deklaracji

Nie posiada zaalokowanego miejsca aż do użycia

Można ją zwiększać/zmniejszać podczas symulacji

Można sprawdzać wielkość w pamięci podczas symulacji

```
logic [7:0] mem [];                          // dynamic array of 8-bit vectors

initial
begin
    mem      = new [128];                    // allocate space for 128 vectors
#10 mem[62]  = 8'h8f;                        // legal assignment
#10 mem[131] = 8'h16;                        // illegal - size is currently 128
#10 mem      = new [mem.size* 2] (mem); // increase size of array by factor of 2 &
seed memory with
                                             // previous values
#10 mem[131] = 8'h16;                        // now this assignment is legal
#10 mem.delete;                              // delete all elements of array
end
```

Typy wysokiego poziomu : tablice asocjacyjne

Tablice asocjacyjne nie są alokowane aż do pierwszego użycia. Indeks adresujący nie jest ograniczony do liczb całkowitych.

Tablica asocjacyjna implementuje tabelę „look-ującą" (lookup table) elementów zadeklarowanego typu.

Typ danych użyty jako indeks używany jest jako klucz adresujący i określa uszeregowanie.

Metody:

`exists()` - sprawdza czy istnieje element pod wskazanym indeksem.
`first()` - zwraca pierwszy indeks (najmniejszy) dla którego dostępny jest element w tablicy.
`last()` - zwraca ostatni indeks (największy) dla którego dostępny jest element w tablicy.
`next()` - zwraca następny indeks (większy niż podany).
`prev()` - zwraca poprzedni indeks (mniejszy niż podany).
`num()` - zwraca liczbę wpisów w tabeli. Jeśli tabela asocjacyjna jest pusta, zwraca 0.
`delete()` - usuwa wpis o podanym indeksie.

```
int array_name [*];             //Wildcard index. can be indexed by any integral
datatype.
int array_name [ string ];      // String index
int array_name [ some_Class ]; // Class index
int array_name [ integer ];     // Integer index
typedef bit signed [4:1] Nibble;
int array_name [ Nibble ];      // Signed packed array
```

Typy wysokiego poziomu : tablice asocjacyjne

```
module assoc;
    bit [31:0] mem[*];
    int assoc[string];
    bit [31:0] i;
    initial
    begin
        mem[ 0 ] = 32'hFFAAFFAA;
        mem[ 3 ] = 32'hFFAAFFAA;
        mem[ 5 ] = 32'hFFAAFFAA;
        mem[ 7 ] = 32'hFFAAFFAA;
        $display("size of array: %d\n", mem.num());

        // find first element
        mem.first(i);
        $display ("the first index of the array is %d", i);

        // find the next element, which is now BASE_ADDR +3:
        mem.next(i);
        $display ("the next index used of the array is %d", i);

        // find the last element
        mem.last(i);
        $display ("the last index used of the array is %d", i);

        mem.prev(i);
        $display ("the previous index used of the array is %d", i);

        mem.delete();
        $display("size of array: %d\n", mem.num());
        $display(" Existence of array: %d\n", mem.exists(i));
    end
endmodule
```

Typy wysokiego poziomu : listy

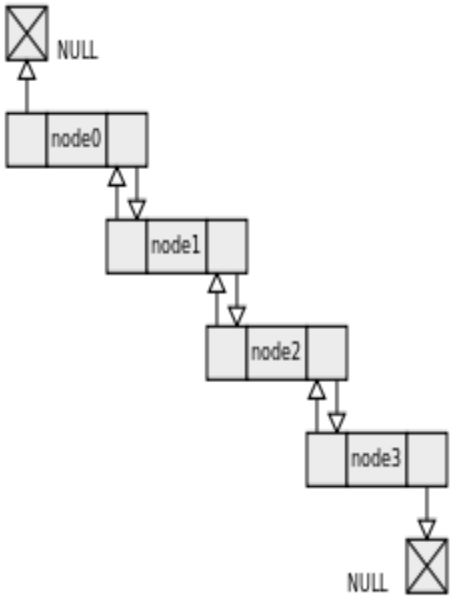
Lista implementuje klasyczną strukturę listy, analogiczną do tej z STL języka C++ (Standard Template Library).

Kontener listy zdefiniowany jest jako klasa parametryzowana w zależności od typu.

Może zawierać elementy takie jak liczba całkowita, łańcuch znaków czy obiekt klasy.

Należy najpierw zadeklarować typ listy a później pobierać jego instancje.

Język SystemVerilog oferuje wiele metod do operowania na tych instancjach.



Typy wysokiego poziomu : listy

Klasa List_Iterator oferuje metody do iterowania po elementach listy.

```
next() modyfikuje iterator tak aby wskazywał na następny element listy.
prev() modyfikuje iterator tak aby wskazywał na poprzedni element listy.

eq() sprawdza czy dwa iteratory wskazują na ten sam element listy.
neq() negacja eq().
data() zwraca element przechowywany pod lokacją wskazywaną przez
iterator.
```

Typy wysokiego poziomu : listy

Klasa listy zawiera metody pozwalajace na sprawdzenie długości listy, pobrania iteratorów do początku i końca listy, pobrania danych znajdujących się na liście, dodawanie i usuwanie elementów z listy, oraz zmieniać ich kolejność.

```
size() zwraca liczbę elementów na liście.
empty() sprawdza czy lista jest pusta.
push_front() wstawia element na początek listy.
push_back() wstawia element na koniec listy.
front() zwraca dane przechowywane na początku listy.
back() zwraca dane przechowywane na końcu listy.
pop_front() usuwa element z początku listy.
pop_back() usuwa element z końca listy.
start() zwraca iterator wskazujący na pierwszy element listy.
finish() zwraca iterator wskazujący na koniec listy.
insert() wstawia nowy element na pozycji wskazywanej przez iterator.
insert_range() wstawia nowe elementy określone zakresem ograniczonym dwoma
iteratorami.
erase() usuwa element spod wskazanej pozycji.
erase_range() usuwa elementy określone zakresem ograniczonym dwoma iteratorami.
et() przypisuje do listy elementy z zakresu pozycji ograniczonego dwoma iteratorami.
swap() zamienia miejscami elementy dwóch list o tej samej długości.
clear() czyści elementy listy, lecz nie niszczy samej listy.
purge() czyści elementy listy oraz niszczy samą listę.
```

Typy wysokiego poziomu : listy

```
List#(integer) List1;
List_Iterator#(integer) itor;
initial begin
List1 = new();
$display (" size of list is %d \n",List1.size());
List1.push_back(10);
List1.push_front(22);
$display (" size of list is %d \n",List1.size());
$display (" poping from list : %d \n",List1.front());
$display (" poping from list : %d \n",List1.front());
List1.pop_front();
List1.pop_front();
$display (" size of list is %d \n",List1.size());
List1.push_back(5);
List1.push_back(55);
List1.push_back(555);
List1.push_back(5555);
$display (" size of list is %d \n",List1.size());
```

```
itor = List1.start();
$display (" startn of list %d \n",itor.data());
itor.next();
$display (" second element of list is %d \n",itor.data());
itor.next();
$display (" third element of list is %d \n",itor.data());
itor.next();
$display (" fourth element of list is %d \n",itor.data());

itor = List1.erase(itor);
$display (" after erasing element,the itor element of list is %d \n",itor.data());
itor.prev();
$display(" prevoious element is %d \n",itor.data());
```

Typy wysokiego poziomu: skrzynka pocztowa

Skrzynka pocztowa (mailbox) jest mechanizmem komunikacji pozwalającym na wymianę wiadomości pomiędzy procesami lub wątkami.

Dane mogą być wysyłane przez jeden proces a odbierane przez drugi.

Klasa „Mailbox” posiada następujące metody:

`new()`	–	tworzy skrzynkę pocztową
`put()`	–	wrzuca wiadomość do skrzynki
`try_put()`	–	próbuje wrzucić wiadomość do skrzynki bez blokowania
`get() or peek()`	–	pobiera wiadomość ze skrzynki
`try_get() or try_peek()`	–	próbuje pobrać wiadomość ze skrzynki bez blokowania
`num()`	–	zwraca liczbę wiadomości w skrzynce

Typy wysokiego poziomu: semafor

Semafory służą do blokowania/odblokowywania wspólnych zasobów .
Jeśli zasób jest zablokowany nie może być użyty.
Jeśli zasób nie jest zablokowany to może być użyty.

Semafor w SystemVerilog posiada następujące metody:

```
Semaphore allocation : new()
Using semaphore keys : get()
Try to obtain keys without blocking: try_get()
Returing semaphore keys : put()
```

```
1  program semaphore_ex;
2
3    semaphore  semBus = new(1);
4
5    initial begin
6      fork
7        agent("AGENT 0",5);
8        agent("AGENT 1",20);
9      join
10   end
11
12   task automatic agent(string name, integer nwait);
13     integer i = 0;
14     for (i = 0 ; i < 4; i ++ ) begin
15       semBus.get(1);
16       $display("[%0d] Lock semBus for %s", $time,name);
17       #(nwait);
18       $display("[%0d] Release semBus for %s", $time,name);
19       semBus.put(1);
20       #(nwait);
21     end
22   endtask
23
24 endprogram
```

Typy wysokiego poziomu: kolejka

Kolejki (queue) i tablice dynamiczne posiadają tą samą składnię przypisywania i podawania parametrów. Dodatkowo, kolejki obsługują te same operatory.

```
int q[$] = { 2, 4, 8 };
int p[$];
int e, pos;
e = q[0];                              // read the first (leftmost) item
e = q[$];                              // read the last (rightmost) item
q[0] = e;                              // write the first item
p = q;                                 // read and write entire queue (copy)
q = { q, 6 };                          // insert '6' at the end (append 6)
q = { e, q };                          // insert 'e' at the beginning (prepend e)
q = q[1:$];                            // delete the first (leftmost) item
q = q[0:$-1];                          // delete the last (rightmost) item
q = q[1:$-1];                          // delete the first and last items
q = {};                                // clear the queue (delete all items)
q = p;                                 // copy contents again
q = { q[0:pos-1], e, q[pos,$] };       // insert 'e' at position pos
q = { q[0:pos], e, q[pos+1,$] };       // insert 'e' after position pos
```

Typy wysokiego poziomu: kolejka

W stosunku do tablic, kolejki posiadają następujące dodatkowe metody:

`size()`	–	zwraca liczbę elementów w kolejce
`insert()`	–	wstawia element na określonej pozycji
`delete()`	–	usuwa element z określonej pozycji.
`pop_front()`	–	usuwa element z początku listy.
`pop_back()`	–	usuwa element z końca listy.
`push_front()`	–	wstawia element na początek listy.
`push_back()`	–	wstawia element na koniec listy.

SystemVerilog : Operatory

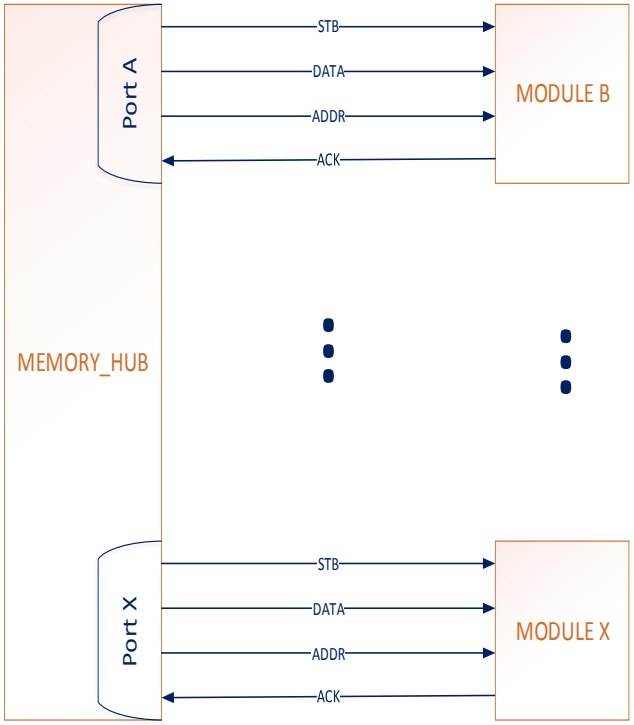
Operatory

SystemVerilog	Verilog
A += B	A = A + B
A -= B	A = A – B
A *= B	A = A * B
A /= B	A = A / B
A %= B	A = A % B
A &= B	A = A & B
A |= B	A = A |= B
A ^= B	A = A ^ B
A <<= B	A = A << B
A >>= B	A = A >> B
A <<<= B	A = A <<< B
A >>>= B	A = A >>> B
A = B++	A = B; B = B + 1;
A = ++B	B = B + 1; A = B;
A = B–-	A = B; B = B – 1;
A = -–A	B = B – 1; A = B;

SystemVerilog : Interfejsy

Verilog : połączenia portów

```
module memory_hub
(
 ...
 // port A
 output wire         pa_stb,
 output wire         pa_addr,
 output wire [71:0]  pa_data,
 input  wire  [1:0]  pa_ack,
 // port B
 output wire         pb_stb,
 output wire         pb_addr,
 output wire [71:0]  pb_data,
 input  wire  [1:0]  pb_ack,
...
 // port X
 output wire         px_stb,
 output wire         px_addr,
 output wire [71:0]  px_data,
 input  wire  [1:0]  px_ack,
 ...
);
...
endmodule
```




```
module Device
(
 ...

 input  wire         stb,
 input  wire         addr,
 input  wire [71:0]  data,
 output wire  [1:0]  ack,

 ...
);
...
endmodule
```

Verilog : połączenia portów

```
module top
(
 ...
 // port A
 wire        pa_stb,
 wire        pa_addr,
 wire [71:0] pa_data,
 wire        pa_ack,
 // port B
 wire        pb_stb,
 wire        pb_addr,
 wire [71:0] pb_data,
 wire        pb_ack,
...
 // port X
 wire        px_stb,
 wire        px_addr,
 wire [71:0] px_data,
 wire  [1:0] px_ack,
 ...
```

```
// instances

memory_hub mem_hub1
(
// port A
.pa_stb     (pa_stb),
.pa_addr    (pa_addr),
.pa_data    (pa_data),
.pa_ack     (pa_ack),
 // port B
.pb_stb     (pb_stb),
.pb_addr    (pb_addr),
.pb_data    (pb_data),
.pb_ack     (pb_ack),
...
 // port X
.px_stb     (px_stb),
.px_addr    (px_addr),
.px_data    (px_data),
.px_ack     (px_ack),
...
);
```

```
device device_A
(
.stb        (pa_stb),
.addr       (pa_addr),
.data       (pa_data),
.ack        (pa_ack),
);

device device_B
(
.stb        (pb_stb),
.addr       (pb_addr),
.data       (pb_data),
.ack        (pb_ack),
);

...

device device_X
(
.stb        (px_stb),
.addr       (px_addr),
.data       (px_data),
.ack        (px_ack),
);

endmodule // top
```

Verilog : połączenia portów

```
module top
(
 ...
 // port A
 wire         pa_stb,
 wire         pa_addr,
 wire [71:0]  pa_data,
 wire         pa_ack,
 // port B
 wire         pa_stb,
 wire         pa_addr,
 wire [71:0]  pa_data,
 wire         pa_ack,
...
 // port X
 wire         pa_stb,
 wire         pa_addr,
 wire [71:0]  pa_data,
 wire  [1:0]  pa_ack,
 ...
```

Wires!

Wires!

Wires!

```
// instances

memory_hub mem_hub1
(
// port A
.pa_stb     (pa_stb),
.pa_addr    (pa_addr),
.pa_data    (pa_data),
.pa_ack     (pa_ack),
 // port B
.pb_stb     (pb_stb),
.pb_addr    (pb_addr),
.pb_data    (pb_data),
.pb_ack     (pb_ack),
...
 // port X
.px_stb     (px_stb),
.px_addr    (px_addr),
.px_data    (px_data),
.px_ack     (px_ack),
...
);
```

Wires!

Wires!

Wires!

```
device device_A
(
.stb        (pa_stb),
.addr       (pa_addr),
.data       (pa_data),
.ack        (pa_ack),
);

device device_B
(
.stb        (pb_stb),
.addr       (pb_addr),
.data       (pb_data),
.ack        (pb_ack),
);

...

device device_X
(
.stb        (px_stb),
.addr       (px_addr),
.data       (px_data),
.ack        (px_ack),
);

endmodule // top
```

Wires!

Wires!

Wires!

Verilog : połączenia portów

```
module top
(
 ...
 // port A
 wire         pa_
 wire         pa_
 wire [71:0]  pa_
 wire         pa_
 // port B
 wire         pa_
 wire         pa_
 wire [71:0]  pa_
 wire         pa_
...
 // port X
 wire         pa_
 wire         pa_
 wire [71:0]  pa_
 wire  [1:0]  pa_
 ...
```



Verilog : połączenia portów

Co jeśli chcemy dodać jeden sygnał do portu ?

```
[...]
// port A
 output wire         pa_stb,
 output wire  [3:0] pa_tag,
 output wire         pa_addr,
 output wire [71:0] pa_data,
 input  wire  [1:0] pa_ack,
[...]
```

Musimy:

- Zmienić listę portów w każdym module
- Zmienić listę portów w module kontrolera pamięci
- Zmienić listę lokalnych linii wewnątrz modułu TOP
- Uzupełnić listę połączeń lokalnych linii do portów modułów
- Zmienić nazwy wszędzie w hierarchii

Verilog : połączenia portów

Co jeśli chcemy dodać jeden sygnał do portu ?

```
[...]
// port A
 output wire         pa_stb,
 output wire  [3:0] pa_tag,
 output wire         pa_addr,
 output wire [71:0] pa_data,
 input  wire  [1:0] pa_ack,
[...]
```



Musimy:

- Zmienić listę portów w każdym module
- Zmienić listę portów w module kontrolera pamięci
- Zmienić listę lokalnych linii wewnątrz modułu TOP
- Uzupełnić listę połączeń lokalnych linii do portów modułów
- Zmienić nazwy wszędzie w hierarchii

Verilog : połączenia portów

- Jawne „.” połączenia portów
- Niejawne „.*” połączenia portów
  - .* domniemuje podłączenie wszystkich sieci i portów o tej samej nazwie
  - Aby połączenie było domniemane, nazwa i szerokość wektora musi się zgadza
  - Typy muszą być kompatybilne

```
module top
(
 wire         clk,
 wire         rst,
 // port A
 wire         pa_stb,
 wire         pa_addr,
 wire [71:0]  pa_data,
 wire         pa_ack,
 // port B
 wire         pb_stb,
 wire         pb_addr,
 wire [71:0]  pb_data,
 wire         pb_ack,
...
```

```
 // port X
 wire         px_stb,
 wire         px_addr,
 wire [71:0]  px_data,
 wire  [1:0]  px_ack,
 ...

// instances

memory_hub mem_hub1 (.*);

device device_A (.*);
device device_B (.*);
 ...
device device_X (.*);

endmodule // top
```

Verilog : połączenia portów

- Jawne „.” połączenia portów
- Niejawne „.*” połączenia portów
  - .* domniemuje podłączenie wszystkich sieci i portów o tej samej nazwie
  - Aby połączenie było domniemane, nazwa i szerokość wektora musi się zgadza
  - Typy muszą być kompatybilne

```
module top();

wire         clk;
wire         rst;
// macro-port A
wire         pa_stb;
wire         pa_addr;
wire [71:0]  pa_data;
wire         pa_ack;
// macro-port B
wire         pb_stb;
wire         pb_addr;
wire [71:0]  pb_data;
wire         pb_ack;

...
```

```
// macro-port X
wire         px_stb;
wire         px_addr;
wire [71:0]  px_data;
wire  [1:0]  px_ack;
...

// instances

memory_hub mem_hub1 (.*);

device device_A (.*);
device device_B (.*);
 ...
device device_X (.*);

endmodule // top
```





Verilog : połączenia portów

- Niejawne „***.nazwa***” połączenia portów
  - domniemuje połączenia portów o tej samej nazwie i szerokości
  - upraszcza podłączenia instancji
  - może być łączone z połączeniami wg. nazwy

```
module top();

wire         clk;
wire         rst;
// macro-port A
wire         pa_stb;
wire         pa_addr;
wire [71:0]  pa_data;
wire         pa_ack;
// macro-port B
wire         pb_stb;
wire         pb_addr;
wire [71:0]  pb_data;
wire         pb_ack;

...
```

```
// macro-port X
wire         px_stb;
wire         px_addr;
wire [71:0]  px_data;
wire  [1:0]  px_ack;
...

// instances

memory_hub mem_hub1 (.*);

device device_A (.clk, .rst, . . . );
device device_B (.clk, .rst, . . . );
 ...
device device_X (.clk, .rst, .stb , .addr, . . . );

endmodule // top
```

Verilog : połączenia portów

- Niejawne „***.nazwa***” połączenia portów
  - domniemuje połączenia portów o tej samej nazwie i szerokości
  - upraszcza podłączenia instancji
  - może być łączone z połączeniami wg. nazwy

```
module top();

wire         clk;
wire         rst;
// macro-port A
wire         pa_stb;
wire         pa_addr;
wire [71:0]  pa_data;
wire         pa_ack;
// macro-port B
wire         pb_stb;
wire         pb_addr;
wire [71:0]  pb_data;
wire         pb_ack;

...
```

```
// macro-port X
wire         px_stb;
wire         px_addr;
wire [71:0]  px_data;
wire  [1:0]  px_ack;
...

// instances

memory_hub mem_hub1 (.*);

device device_A (.clk, .rst, . . . );
device device_B (.clk, .rst, . . . );
 ...
device device_X (.clk, .rst, .stb , .addr, . . . );

endmodule // top
```

Dobre dla podłączeń ogólnych sygnałów

Bezużyteczne dla podłączeń lokalnych indywidualnych

SystemVerilog : połączenia portów

```
module memory_hub
(
 ...
 output wire          stb  [0:N-1],
 output wire          addr [0:N-1],
 output wire [71:0]   data [0:N-1],
 input  wire          ack  [0:N-1],

 wire                 clk,
 wire                 rst,
[...]


[...]
endmodule
```

```
module device
(
 ...

 input  wire          stb,
 input  wire          addr,
 input  wire [71:0]   data,
 output wire  [1:0]   ack,

 ...
);
...
endmodule
```

```
module top();

wire         clk;
wire         rst;

// macro-ports 0..N-1

wire         stb[0:N-1];
wire         addr[0:N-1];
wire [71:0]  data[0:N-1];
wire         ack[0:N-1];

...
```

```
// instances

memory_hub mem_hub1 (.*);


device device_0 (.* );
device device_1 (.* );
 ...
device device_7 (.* );

endmodule // top
```





SystemVerilog: Interfejsy

SystemVerilog dodaje nowy, użyteczny typ portu, zwany **interfejsem (interface).**

Interfejs pozwala na zgrupowanie wielu sygnałów jako pojedynczy port.

Deklaracje sygnałów tworzących interfejs zawarte są w jednej lokalizacji kodu źródłowego.

Każdy moduł wykorzystujący interfejs, posiada jeden port a nie wiele pojedynczych sygnałów.

Wszystkie sygnały które są współdzielone pomiędzy głównymi blokami w projekcie są zebrane w jednym miejscu: w **deklaracji interfejsu**

SystemVerilog: Interfejsy

- Zalety interfejsów:
  - Osłaniają one łączność
  - Zawierają one w sobie funkcjonalność
    - parametry, stałe, zmienne, funkcje i zadania, procesy i przypisania ciągłe
  - Mogą one pomóc w tworzeniu aplikacji
  - Mogą one być używane do dostępu bezportorowego
  - Elastyczność

SystemVerilog: Interfejsy

Interfejs również może mieć pola wejściowe (input), wyjściowe (output) lub wejściowo-wyjściowe (inout).

Tylko zmienne i sieci zadeklarowane w liście portów interfejsu mogą być podłączane zewnętrznie (poprzez nazwę lub pozycję).

```
interface MyBus
(
 input wire clk
);

wire      [3:0] idx;
wire     [31:0] addr;
wire     [31:0] data;
wire            rw;

endinterface
```

```
module MyMem
(
 MyBus bus
);

reg [31:0] mem [7:0];

always @(posedge bus.clk)
  if(bus.rw) mem[bus.addr]<= bus.data

endmodule
```

```
module top;

reg   clk;

MyBus bus1(.clk(clk);

MyMem mem1(.bus(bus1);



endmodule
```

SystemVerilog: Modport

Istnieje też konstrukcja rozszerzająca definicję interfejsu: Modport.

Pozwala to dodać informację kierunkową dla interfejsu, jego funkcji i zadań. Kierunki portów są widoczne od strony modułu.

```
interface MyBus
(
 input wire clk
);

wire      [3:0] idx;
wire     [31:0] addr;
wire     [31:0] data;
wire            rw;

modport slave
(
input clk,
input addr,
input rw,
inout data
);

endinterface
```

```
module MyMem
(
 MyBus.slave bus
);

reg [31:0] mem [7:0];

always @(posedge bus.clk)
  if(bus.rw) mem[bus.addr]<= bus.data

endmodule
```

```
module top;

reg    clk;

MyBus bus1(.clk(clk);

MyMem mem1(.bus(bus1.slave);



endmodule
```

SystemVerilog: Interfejsy 1

```
1  //================================================
2  // Define the interface
3  //================================================
4  interface mem_if (input wire clk);
5     logic         reset;
6     logic         we_sys;
7     logic         cmd_valid_sys;
8     logic         ready_sys;
9     logic  [7:0]  data_sys;
10    logic  [7:0]  addr_sys;
11    logic         we_mem;
12    logic         ce_mem;
13    logic  [7:0]  datao_mem;
14    logic  [7:0]  datai_mem;
15    logic  [7:0]  addr_mem;
16    //=============================================
17    // Modport for System interface
18    //=============================================
19    modport  system (input clk,reset,we_sys, cmd_valid_sys,
20                     addr_sys, datao_mem,
21                     output we_mem, ce_mem, addr_mem,
22                     datai_mem, ready_sys, ref data_sys);
23    //=============================================
24    // Modport for memory interface
25    //=============================================
26    modport  memory (input clk,reset,we_mem, ce_mem,
27                     addr_mem, datai_mem, output datao_mem);
28    //=============================================
29    // Modport for testbench
30    //=============================================
31    modport  tb (input clk, ready_sys,
32                 output reset,we_sys, cmd_valid_sys, addr_sys,
33                ref data_sys);
34
35 endinterface
```

```
37 //================================================
38 //  Memory Model
39 //================================================
40 module memory_model (mem_if.memory mif);
41 // Memory array
42 logic [7:0] mem [0:255];
43
44 //=============================================
45 // Write Logic
46 //=============================================
47 always @ (posedge mif.clk)
48  if (mif.ce_mem && mif.we_mem) begin
49     mem[mif.addr_mem] <= mif.datai_mem;
50  end
51
52 //=============================================
53 // Read Logic
54 //=============================================
55 always @ (posedge mif.clk)
56  if (mif.ce_mem && ~mif.we_mem)  begin
57     mif.datao_mem <= mem[mif.addr_mem];
58  end
59
60 endmodule
```

SystemVerilog: Interfejsy 1

```
62 //+++++++++++++++++++++++++++++++++++++++++++++++
63 // Memory Controller
64 //+++++++++++++++++++++++++++++++++++++++++++++++
65 module memory_ctrl (mem_if.system sif);
66
67 typedef  enum {IDLE,WRITE,READ,DONE} fsm_t;
68
69 fsm_t state;
70
71 always @ (posedge sif.clk)
72   if (sif.reset) begin
73     state         <= IDLE;
74     sif.ready_sys <= 0;
75     sif.we_mem    <= 0;
76     sif.ce_mem    <= 0;
77     sif.addr_mem  <= 0;
78     sif.datai_mem <= 0;
79     sif.data_sys  <= 8'bz;
80   end else begin
81     case(state)
82       IDLE: begin
83         sif.ready_sys <= 1'b0;
84         if (sif.cmd_valid_sys && sif.we_sys) begin
85           sif.addr_mem   <= sif.addr_sys;
86           sif.datai_mem  <= sif.data_sys;
87           sif.we_mem     <= 1'b1;
88           sif.ce_mem     <= 1'b1;
89           state          <= WRITE;
90         end
91         if (sif.cmd_valid_sys && ~sif.we_sys) begin
92           sif.addr_mem   <= sif.addr_sys;
93           sif.datai_mem  <= sif.data_sys;
94           sif.we_mem     <= 1'b0;
95           sif.ce_mem     <= 1'b1;
96           state          <= READ;
97         end
98       end
```

```
99       WRITE: begin
100        sif.ready_sys  <= 1'b1;
101        if (~sif.cmd_valid_sys) begin
102          sif.addr_mem   <= 8'b0;
103          sif.datai_mem  <= 8'b0;
104          sif.we_mem     <= 1'b0;
105          sif.ce_mem     <= 1'b0;
106          state          <= IDLE;
107        end
108      end
109      READ: begin
110        sif.ready_sys  <= 1'b1;
111        sif.data_sys   <= sif.datao_mem;
112        if (~sif.cmd_valid_sys) begin
113          sif.addr_mem   <= 8'b0;
114          sif.datai_mem  <= 8'b0;
115          sif.we_mem     <= 1'b0;
116          sif.ce_mem     <= 1'b0;
117          sif.ready_sys  <= 1'b1;
118          state          <= IDLE;
119          sif.data_sys   <= 8'bz;
120        end
121      end
122    endcase
123  end
124
125 endmodule
```

SystemVerilog: Interfejsy 1

```
172 //+++++++++++++++++++++++++++++++++++++++++++++++++++
173 // Testbench
174 //+++++++++++++++++++++++++++++++++++++++++++++++++++
175 module interface_modports();
176
177 logic clk = 0;
178 always #10 clk++;
179 //=================================================
180 // Instianciate Interface and DUT
181 //=================================================
182 mem_if miff(clk);
183 memory_ctrl U_ctrl(miff);
184 memory_model U_model(miff);
185 test   U_test(miff);
186
187 endmodule
```

SystemVerilog: Interfejsy 2

```
interface Gen_Intf
  #(parameter N_slaves = 2);

  logic [7:0] Adr;
  logic slave_D   [0:N_slaves-1];
  logic slave_sel [0:N_slaves-1];
  logic Data;

  always_comb begin : selector
    int slave_ID;
    case (Adr)  // Address decode
      42: slave_ID = 0;
      50: slave_ID = 1;
    endcase
    slave_sel = 0;
    slave_sel[slave_ID] = 1'b1;
    Data = slave_D[slave_ID];
  end : selector

  modport master_mp (
    output Adr,
    input Data
  );

  generate
    genvar i;
    for (i=0; i<N_slaves; i++)
    begin : Slave
      modport slave_mp (
        input Adr,
        input .sel(slave_sel[i]),
        output .Data(slave_D[i])
      );
    end : Slave
  endgenerate

endinterface : Gen_Intf
```

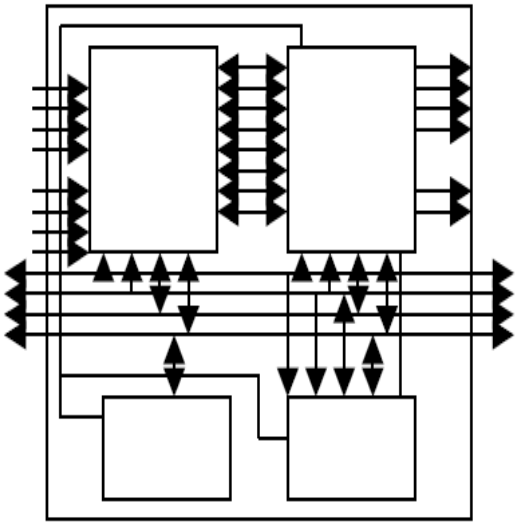
```
module Gen_Master(Gen_Intf.master_mp mi);
  initial begin
    mi.Adrs = 50;  // select first slave
    #100
    mi.Adrs = 42;  // select other slave
  end
endmodule : Gen_Master

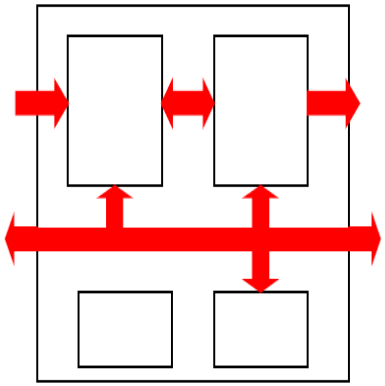
module Gen_Slave(
  interface.slave_mp si,
  input logic data
);
  assign si.Data = data;
endmodule : Gen_Slave

module Top_Fig6(input logic d42, d50);
  Gen_Intf   G ();
  Gen_Master M (G.master_mp);
  Gen_Slave S50 (G.Slave[0].slave_mp, d50)
  Gen_Slave S42 (G.Slave[1].slave_mp, d42)
endmodule : Top_Fig6
```

SystemVerilog: Interfejsy

Verilog łączy moduły używając portów
– wymaga dogłębnej wiedzy na temat połączeń
– zmiany w projekcie są trudne
– deklaracje portów są duplikowane w wielu modułach





SystemVerilog dodaje blok interfejsu
– połączenia pomiędzy modułami są zbiorcze
– definicja połączeń jest niezależna od modułów
– interfejsu mogą zawierać deklaracje asercji i testów protokołu komunikacyjnego

Dziękuje za uwagę

- Pytania?

System Verilog
Weryfikacja

dr hab. inż. Olgierd Stankiewicz
olgierd.stankiewicz@put.poznan.pl
Polanka p. 117, tel. 61 665 3840

Instytut Telekomunikacji Multimedialnej
Politechnika Poznańska

Narzędzia do weryfikacji

- System Verilog dla syntezy
  - Typy: structure, enum, union
  - Interfejsy
  - Wielowymiarowe porty
  - Pomoce (helpers) syntezy
- System Verilog dla weryfikacji
  - Kontenery (tablice dynamiczne, asocjacyjne, listy, narzędzia synchronizacji)
  - **Klasy użytkownika**
  - **Losowane zmienne (randomizowane)**
  - **Asercje**
  - **Testy pokrycia**
  - **U**niversal **V**erification **M**ethodology (**UVM**)

SystemVerilog : Klasy użytkownika

Definiowanie klas

- Podobnie jak w C++
- Zmienne trzymają referencje (uchwyty)

```
class sv_class;
  int x; // zmienna (pole)

  task set(int i); // zadania (taski)
    x = i;
    this.x = i; // zamienne z zapisem powyżej
  endtask

  function int get(); // funkcje i metody
    return x;
  endfunction
endclass

module sv_class_ex; // moduł demonstracyjny
 sv_class class_1;

  initial begin
    sv_class class_2 = new();
    class_1 = new(); // uwaga: nie podajemy typu klasy przy new

    class_1.set(10);
    class_2.set(20);
    $display("\tclass_1 :: Value of x = %0d",class_1.get());
    $display("\tclass_2 :: Value of x = %0d",class_2.get());
  end
endmodule
```

```
class_1 :: Value of x = 20
class_2 :: Value of x = 20
```

Konstruktory

```
class packet;
  bit [31:0] addr;
  bit [31:0] data;
  bit  write;
  string pkt_type;

  static int no_of_pkts_created;

  function new( bit wr);
    addr  = 32'h10;
    data  = 32'hFF;
    write = wr;
    pkt_type = "TEST";
    no_of_pkts_created++;
  endfunction // bez „return”

  function void display();
    $display("\t addr  = %0d",addr);
    $display("\t data  = %0h",data);
    $display("\t write = %0d",write);
    $display("\t pkt_type  = %0s",pkt_type);
  endfunction
endclass

module sv_construction;
  packet pkt1;
  initial begin
    packet::no_of_pkts_created = 0; // niepotrzebne, ale demonstruje użycie ::
    pkt1 = new(1); // uwaga: nie podajemy typu klasy przy new
    pkt1.display();
    pkt2 = new(0); // uwaga: nie podajemy typu klasy przy new
    pkt2.display();
  end
endmodule
```

Przypisywanie klas

- Kopiowanie uchwytu

```
packet   pkt1;
pkt1  = new(); // uwaga: nie podajemy typu klasy

packet   pkt2;
pkt2  = pkt1;
```

- Kopiowanie płytkie (shallow copy)

```
packet   pkt1;
pkt1  = new(); // uwaga: nie podajemy typu klasy

packet   pkt2;
pkt2  = new pkt_1; // płytka kopia
```

  - Kopiowanie pole po polu
  - Jeśli dane pole jest referencją do obiektu klasy, kopiowany jest tylko jego **UCHWYT** (handle)
    - Jak głęboko kopiować?

Głęboka kopia (deep copy)

```
class address_range;
  bit [31:0] start_address;
  bit [31:0] end_address  ;

  function new();
    start_address = 10;
    end_address   = 50;
  endfunction

  function address_range copy;
    copy = new();
    copy.start_address = this.start_address;
    copy.end_address   = this.end_address;
    return copy; // nadmiarowe
  endfunction
endclass
```

```
class packet;
  bit [31:0] addr;
  bit [31:0] data;
  address_range ar; //uchwyt

  function new();
    addr  = 32'h10;
    data  = 32'hFF;
    ar = new(); //tworzenie obiektu
  endfunction

  function void display();
    $display("\t addr  = %0h",addr);
    $display("\t data  = %0h",data);
    $display("\t start_address  = %0d",ar.start_address);
    $display("\t end_address  = %0d",ar.end_address);
  endfunction

  function packet copy();
    copy = new();
    copy.addr = this.addr;
    copy.data = this.data;
    copy.ar   = ar.copy;
    // wywołanie funkcji kopiującej
    return copy; // nadmiarowe
  endfunction
endclass
```

```
module class_assignment;
  packet pkt_1;
  packet pkt_2;
  initial begin
    pkt_1 = new();   //tworzenie pkt_1
    $display("\t****  pkt_1")
    pkt_1.display();
    pkt_2 = new();   //tworzenie pkt_2
    $display("\t****  pkt_2")
    pkt_2.display();
    pkt_2 = pkt_1.copy(); // wywołanie kopiowania
    //zmiana wartości w pkt_2
    pkt_2.addr = 32'h68;
    pkt_2.ar.start_address = 60;
    pkt_2.ar.end_address = 80;
    $display("\t****  pkt_1 *")
    pkt_1.display();
    $display("\t****  pkt_2 *")
    pkt_2.display();
  end
endmodule
```

```
**** pkt_1
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_2
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_1 *
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_2 *
addr = 68
data = ff
start_address = 60
end_address = 80
```

Płytka kopia (shallow copy)

```
class address_range;
  bit [31:0] start_address;
  bit [31:0] end_address   ;

  function new();
    start_address = 10;
    end_address   = 50;
  endfunction




endclass
```

```
class packet;
  bit [31:0] addr;
  bit [31:0] data;
  address_range ar; //uchwyt

  function new();
    addr  = 32'h10;
    data  = 32'hFF;
    ar = new(); //tworzenie obiektu
  endfunction

  function void display();
    $display("\t addr  = %0h",addr);
    $display("\t data  = %0h",data);
    $display("\t start_address  = %0d",ar.start_address);
    $display("\t end_address  = %0d",ar.end_address);
  endfunction




endclass
```

```
module class_assignment;
  packet pkt_1;
  packet pkt_2;
  initial begin
    pkt_1 = new();   //tworzenie pkt_1
    $display("\t****  pkt_1")
    pkt_1.display();
    pkt_2 = new();   //tworzenie pkt_2
    $display("\t****  pkt_2")
    pkt_2.display();
    pkt_2 = new pkt_1; // płytka kopia
    //zmiana wartości w pkt_2
    pkt_2.addr = 32'h68;
    pkt_2.ar.start_address = 60; // a tu jednocześnie
    pkt_2.ar.end_address = 80;   // w pkt1
    $display("\t****  pkt_1 *")
    pkt_1.display();
    $display("\t****  pkt_2 *")
    pkt_2.display();
  end
endmodule
```

```
**** pkt_1
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_2
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_1 *
addr = 10
data = ff
start_address = 60
end_address = 80
**** pkt_2 *
addr = 68
data = ff
start_address = 60
end_address = 80
```

Kopiowanie uchwytu

```
class address_range;
  bit [31:0] start_address;
  bit [31:0] end_address   ;

  function new();
    start_address = 10;
    end_address   = 50;
  endfunction




endclass
```

```
class packet;
  bit [31:0] addr;
  bit [31:0] data;
  address_range ar; //uchwyt

  function new();
    addr  = 32'h10;
    data  = 32'hFF;
    ar = new(); //tworzenie obiektu
  endfunction

  function void display();
    $display("\t addr  = %0h",addr);
    $display("\t data  = %0h",data);
    $display("\t start_address  = %0d",ar.start_address);
    $display("\t end_address  = %0d",ar.end_address);
  endfunction




endclass
```

```
module class_assignment;
  packet pkt_1;
  packet pkt_2;
  initial begin
    pkt_1 = new();   //tworzenie pkt_1
    $display("\t****  pkt_1")
    pkt_1.display();
    pkt_2 = new();   //tworzenie pkt_2
    $display("\t****  pkt_2")
    pkt_2.display();
    pkt_2 = pkt_1; // kopiowanie uchwytu
    //zmiana wartości w pkt_2 i jednoczesnie w pkt1
    pkt_2.addr = 32'h68;
    pkt_2.ar.start_address = 60;
    pkt_2.ar.end_address = 80;
    $display("\t****  pkt_1 *")
    pkt_1.display();
    $display("\t****  pkt_2 *")
    pkt_2.display();
  end
endmodule
```

```
**** pkt_1
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_2
addr = 10
data = ff
start_address = 10
end_address = 50
**** pkt_1 *
addr = 68
data = ff
start_address = 60
end_address = 80
**** pkt_2 *
addr = 68
data = ff
start_address = 60
end_address = 80
```

Parametryzacja klas

- Podobne do parametryzacji modułów w Verilogu, lub jak szablony (template) w C++

```
class packet #(parameter int ADDR_WIDTH = 32,DATA_WIDTH = 32);
  bit [ADDR_WIDTH-1:0] address;
  bit [DATA_WIDTH-1:0] data   ;

  function new();
    address = 10;
    data    = 20;
  endfunction
endclass

packet pkt;     // domyślne 10 i 20

packet #(32,64) pkt;  // podane parametry
```

```
class packet #(parameter type T = int);
  T address;
  T data   ;

  function new();
    address = 10;
    data    = 20;
  endfunction
endclass

packet pkt;                // domyślny int
packet #(bit [19:0]) pkt; // podany typ
```

Dziedzicznie klas

- Tak jak w Java

```
class parent_class;
  bit [31:0] addr;

  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class extends parent_class;
  bit [31:0] data;
  function display();
    super.display(); // metoda klasy nadrzędnej
    $display("Data = %0d",data);
  endfunction
endclass

module inheritence;
  initial begin
    child_class c=new();
    c.addr = 10;
    c.data = 20;
    c.display();
  end
endmodule
```

```
Addr = 10
Data = 20
```

Polimorfizm

- Rzutowanie obiektów

```
parent_class parent;
child_class child;
parent = child; // dozwolone
// jakiś kod
child = parent; // nielegalne – nie wiadomo czy „parent” jest typu „parent_class” czy „child_class”

parent = child;
child = parent; // to powinno działać – ale jak?

$cast(child, parent); // child = parent: sprawdzenie legalności w trakcie uruchomienia
```

- Klasy wirtualne i abstrakcyjne
  - virtual class packet; // klasa abstrakcyjna
  - virtual function display() // metody wirtualne

Ukrywanie i enkapsulacja

- local = private w c++
- protected

```
class parent_class;
  local bit [31:0] tmp_addr;

  function new(bit [31:0] r_addr);
    tmp_addr = r_addr + 10;
  endfunction

  function display();
    $display("tmp_addr = %0d",tmp_addr);
  endfunction
endclass


module encapsulation;
  initial begin
    parent_class p_c = new(5);

    p_c.tmp_addr = 20;  //niedozwolone
    p_c.display();
  end
endmodule
```

zmienna ukryta (pole ukryte)

extern / typedef

```
typedef class some_class;

class packet;
  bit [31:0] addr;
  bit [31:0] data;
  some_class ccc;
  //tylko deklaracja - definicja będzie poza ciałem klasy
  extern virtual function void display();
endclass

class some_class;
  packet p;
endclass

// definicja/implementacja na zewnątrz klasy
function void packet::display();
  $display("Addr = %0d Data = %0d",addr,data);
endfunction

module extern_method;
  initial begin
    packet p;
    p = new();
    p.addr = 10;
    p.data = 20;
    p.display();
  end
endmodule
```

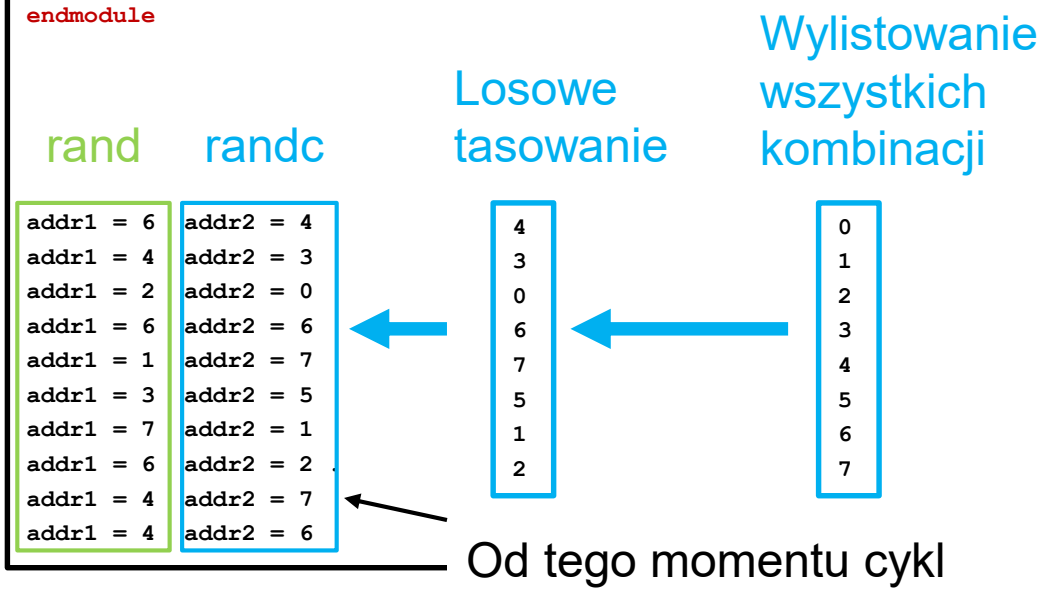
SystemVerilog : Zmienne losowane

Zmienne losowane

- Klasyczny 32-bitowy generator liczb losowych **$urandom()**
- SystemVerilog umożliwia definiowanie zmiennych losowanych (randomizowanych)
  - Zakres losowania
  - Ograniczenia losowania (constraints)
  - Wyłączenie losowania
- rand – zmienna losowana
- randc – zmienna losowana cykliczna

```
class packet;
  rand  bit [2:0] addr1; // 3 bity = 8 kombinacji 0..7
  randc bit [2:0] addr2;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();
    repeat(10) begin
      pkt.randomize(); // tutaj następuje losowanie
      $display("\taddr1 = %0d \t addr2 = %0d",pkt.addr1,pkt.addr2);
    end
  end
endmodule
```



Od tego momentu cykl losowania randc zaczyna się od nowa

pre/post_randomize

- Funkcja randomize()
  - Przetasowuje wszystkie zmienne/pola które mają włączone losowanie
  - Przed/po tym możliwe jest wykonanie dodatkowego kodu
    - Np. wyłączenie losowania zmiennej, e.g. warunkowe

```
function void pre_randomize();
  addr.rand_mode(0); // np. wyłączenie losowania
  $display("POST_RANDOMIZATION:: Addr = %0h,wr_rd = %0h", addr, wr_rd);
endfunction

function void post_randomize();
  $display("POST_RANDOMIZATION:: Addr = %0h,wr_rd = %0h", addr, wr_rd);
  while (addr==5) addr = $urandom(); // np. 5 jest zabronione
endfunction
```

Ograniczenia losowania

- Mechanizmy ograniczeń (constraints)
  - Pozwalają podać ograniczenia na dozwolone wartości wśród losowanych liczb

```
class packet;
  rand  bit [3:0] addr;
  constraint addr1_range { addr > 5; }
endclass
```

Ograniczenia losowania

- Mechanizmy ograniczeń (constraints) mogą być bardzo zaawansowane

```
class packet;
  rand  bit [3:0] addr1;
  rand  bit [3:0] addr2;
  rand bit [3:0] start_addr;
  rand bit [3:0] end_addr;
  constraint addr1_range { addr1 > 5; }
  constraint addr2_range { addr2 inside {[start_addr:end_addr]}; }
endclass

class packet2 extends packet; // przykrycie ograniczeń
  string    addr_range;
  constraint address1_range { (addr_range == "small") -> (addr1 < 8);}

  constraint address2_range { if(addr_range == "small")
                                 addr2 < 8;
                              else
                                 addr2 > 8;
                            }
  rand byte addresses [];
  constraint addresses_range { foreach(addresses[j]) addresses[j] > 4 * j; }

  rand  bit [3:0] addr1p2;
  constraint  addr1p2_range { addr1p2 == addr1 + addr2; }
endclass

pkt2.addresses_range.constraint_mode(0); // wyłączenie
pkt2.randomize();
pkt2.randomize() with { addr1 == 8;}; // constraint podany „inline”
```

Ograniczenia losowania

- Co jeśli ograniczenia są sprzeczne?
  - kompilator zwróci błąd
- Jak ominąć problem?
  - soft constraint- - stosowane, o ile nie koliduje z kolejnymi

```
class packet;
  rand bit [3:0] addr;
  constraint addr_range { soft addr > 6; }
endclass

module soft_constr;
  initial begin
    packet pkt;
    pkt = new();

    repeat(2) begin
      pkt.randomize() with { addr < 6;};
      $display("\taddr = %0d",pkt.addr);
    end
  end
endmodule
```

SystemVerilog : Asercje

SystemVerilog Assertions (SVA)

- Klasyczne programowanie
  - Nielegalne sytuacje – sprawdzanie błędów
- System Verilog
  - Występowanie określonych warunków/zdarzeń
  - Zapewnić testy pokrycia
- Dwa typy asercji
  - Natychmiastowe (Immediate)
  - Współbieżne (Concurrent)

Asercje natychmiastowe

- Muszą znajdować się w blokach proceduralnych
- Odpowiadają asercjom znanym z języka C
- Konstrukcja assert() ma dodatkowy efekt uboczny w postaci instrukcji sterowania jak if ... else

```
assert(a&&b)

assert(condition) $display("Condition is True"); else $display("Condition is False");
// if (condditon)                                 else                              ;

assert(condition) $display("Condition is True");
// if (condditon)                               ;

assert(condition) $("Condition is True"); else $fatal("Condition is False");
// if (condditon)                         else                            ;

assert(condition)
  else begin
      …….
      …….
    $fatal("Condition is False");
  end

assert(condition) else $warning("Condition is False");

label: assert(condition) else $warning("Condition is False");
```

Asercje współbieżne

- Wykonywane są przy wystąpieniu „przeskoku zegara”
- Zbliżona konstrukcja do always

```
c_assert property(@(posedge clk) not(a && b));
```

Warunki asercji

- Wyrażenia logiczne, sekwencje, właściwości

```
a&&b

sequence seq_2;
  @(posedge clk) a || b; // a lub b
endsequence

sequence seq_lib (a, b)
  a || b ;
endsequence

sequence s_lib_inst
  seq_lib(req1,req2); // również a lub b, lecz z wykorzystainem zdefniowanej parametrycznej sekwencji
endsequence

sequence seq;
  @(posedge clk) a ##2 b; // jeśli wystąpi a,  to po dwóch taktach zegara clk ma wystąpić b
endsequence

property p;
  @(posedge clk) a |-> ##1 b[*3]; // jeśli a kiedykolwiek będzie 1, to przez następne 3 takty zegara b powinno być 1
endproperty
a: assert property(p);

property p;
  @(posedge clk) a |-> ##1 b[->3] ##1 c;
  // jeśli a kiedykolwiek będzie 1, to przez następne 3 takty zegara b powinno być 1, a po tym c=1
endproperty
a: assert property(p);
```

Warunki asercji

- Dostępne funkcje

```
sequence seq_rose;
  @(posedge clk) $rose(a);
endsequence

sequence seq_fell;
  @(posedge clk) $fell(a);
endsequence

sequence seq_stable;
  @(posedge clk) $stable(a);
endsequence

property p;
  @(posedge clk) b |-> ($past(a,2) == 1); // jeśli wystąpi b, to dwa takty temu a powinno być 1
endproperty
a: assert property(p);

a_1: assert property( @(posedge clk) $onehot(state) );      // tylko jeden aktywny
a_2: assert property( @(posedge clk) $onehot0(state) );     // tylko jeden aktywny lub żaden
a_3: assert property( @(posedge clk) $isunknown(bus) ) ;    // czy x lub z
a_4: assert property( @(posedge clk) $countones(bus)> 1 ); // liczba aktywnych

property p;
  @(posedge clk)
  disable iff (reset) a |-> ##1 b[->3] ##1 c; // sekwencja przestaje być testowana jeśli wystąpi reset
endproperty
a: assert property(p);
```

SystemVerilog : Testy pokrycia (Coverage)

Test pokrycia

- Podczas projektowania testów interesuje nas czy na pewno przetestowaliśmy badany układ

```
if (a) begin
  // . . .
end else if (b) begin
  // . . . 123
end else if (c || d) begin
  // . . . 456
end
```

- Co jeśli, pomimo przetestowania miliarda wektorów testowych
  - Nigdy nie weszliśmy do gałęzi „if (b) „?
  - Do gałęzi „if (c || d)” weszliśmy zawsze bo „c” a nigdy bo „d” ?

Testy pokrycia

- Testy pokrycia kodu (Code Coverage)
  - Bloki projektowe
  - Linie
  - Warunki
  - FSM
  - Zapewniane przez narzędzie – symulator
    - Praktycznie niedostępne w darmowym oprogramowaniu… :-]
- Testy pokrycia funkcjonalności (Functional Coverage)
  - Konstrukcje języka SystemVerilog

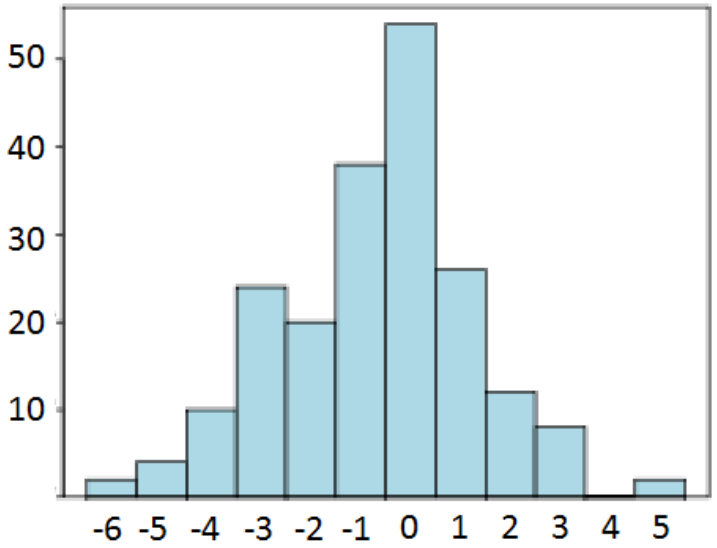
Pokrycie funkcjonalności

- Pozwalają zbierać statystyki (histogramy) pojawiania się wartości
  - Np. w randomizowanych zmiennych

```
covergroup cov_grp @(posedge clk);
  cov_p1: coverpoint a;
endgroup

cov_grp cov_inst = new();

// próbkowanie na zboczu
```




```
covergroup cov_grp;
  cov_p1: coverpoint a;
endgroup

cov_grp cov_inst = new();
@(abc) cov_inst.sample();

// próbkowanie wywoływane „ręcznie”
```

Wielkości kubełków

- Dobierane automatycznie lub definiowane przez użytkownika

```
module cov;
  logic       clk;
  logic [7:0] addr;
  logic       wr_rd;

  covergroup cg @(posedge clk);
    c1: coverpoint addr { bins b1 = {0,2,7};     //kubełek "b1" zwiększany dla addr = 0,2 or 7
                          bins b2[3] = {11:19}; //trzy kubełki b2[0],b2[1] oraz b2[3]. 9 dostępnych wartości
                                                //pogrupowanych jest tak: (11,12,13),(14,15,16) oraz (17,18,19).
                          bins b3    = {[30:40],[50:60],77};    // kubełek "b3" dla addr = 30-40 lub 50-60 lub 77
                          bins b4[] = {[79:99],[110:130],140}; //trzy kubełki b4[0],b4[1] oraz b4[3] dla wartości
                                                               //odpowiednio 79-99,110-130 oraz 140
                          bins b5[] = {160,170,180}; //trzy kubełki b5[0],b5[1] oraz b5[3] dla 160,170 oraz 180

                          bins b6    = {200:$}; //kubełek "b6" zwiększany dla addr = 200 do maksymalnej wartości, np. 255.
                          bins b7 = default;}    // wyłapuje wartości nie pasujące do innych kubełków
    c2: coverpoint wr_rd {bins wrrd};
  endgroup : cg

  cg cover_inst = new();
endmodule

covergroup cg @(posedge clk);
  c1: coverpoint addr{ illegal_bins b1 = {7,70,77};     // zabronione - ostrzeżenie podczas symulacji
                       ignore_bins b2 = (7=>70=>77);}  // przejścia wg. sekwencji.  ignore_bins - ignoruj je
endgroup : cg
```

Cross Coverage

- Iloczyn kartezjański- histogram dwuwymiarowy

```
// wg. coverpoint
bit [3:0] a, b;
covergroup cg @(posedge clk);
  c1: coverpoint a;
  c2: coverpoint b;
  c1Xc2: cross c1,c2;
endgroup : cg

// wg. nazw zmiennych
bit [3:0] a, b;
covergroup cov @(posedge clk);
  aXb : cross a, b;
endgroup

// domyślnie 16 kubełków,  więc 16x16=256 kubełków aXb.

// miesane zmienna i wyrażenie
bit [3:0] a, b, c;
covergroup cov @(posedge clk);
  BC  : coverpoint b+c;
  aXb : cross a, BC;
endgroup
```

Opcje pokrycia

- Umożliwiają zmienienie domyślnej liczby kubełków wykorzystywanej do przeprowadzenia testów pokrycia

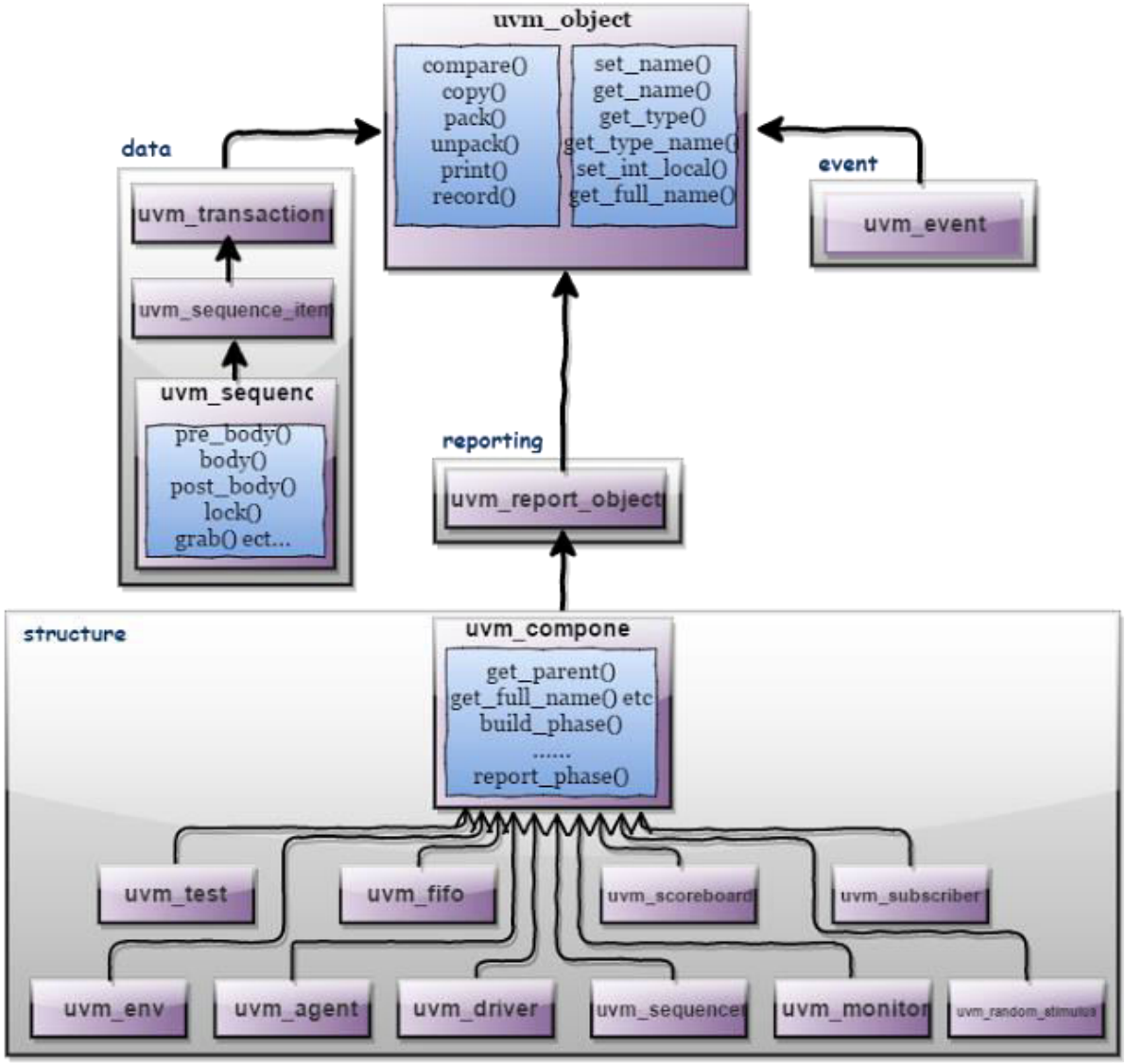
```
covergroup cg @(posedge clk);
  c1: coverpoint addr  { option.auto_bin_max = 128;}
  c2: coverpoint wr_rd { option.atleast = 2;}
  c1Xc2: cross c1, c2  { option.cross_auto_bin_max = 128;}
endgroup : cg
```

SystemVerilog:
Universal Verification Methodology (UVM)

Universal **V**erification **M**ethodology (**UVM**)

- Zestaw klas języka System Verilog
  - Dla zbudowania środowiska weryfikacyjnego
    - Dobrze zaprojektowanego
    - Wielokrotnego użytku
- Wykorzystanie wymaga wprawy i czasami dużej ilości kodu
- Użycie opłacalne przy dużych projektach

Universal Verification Methodology (UVM)



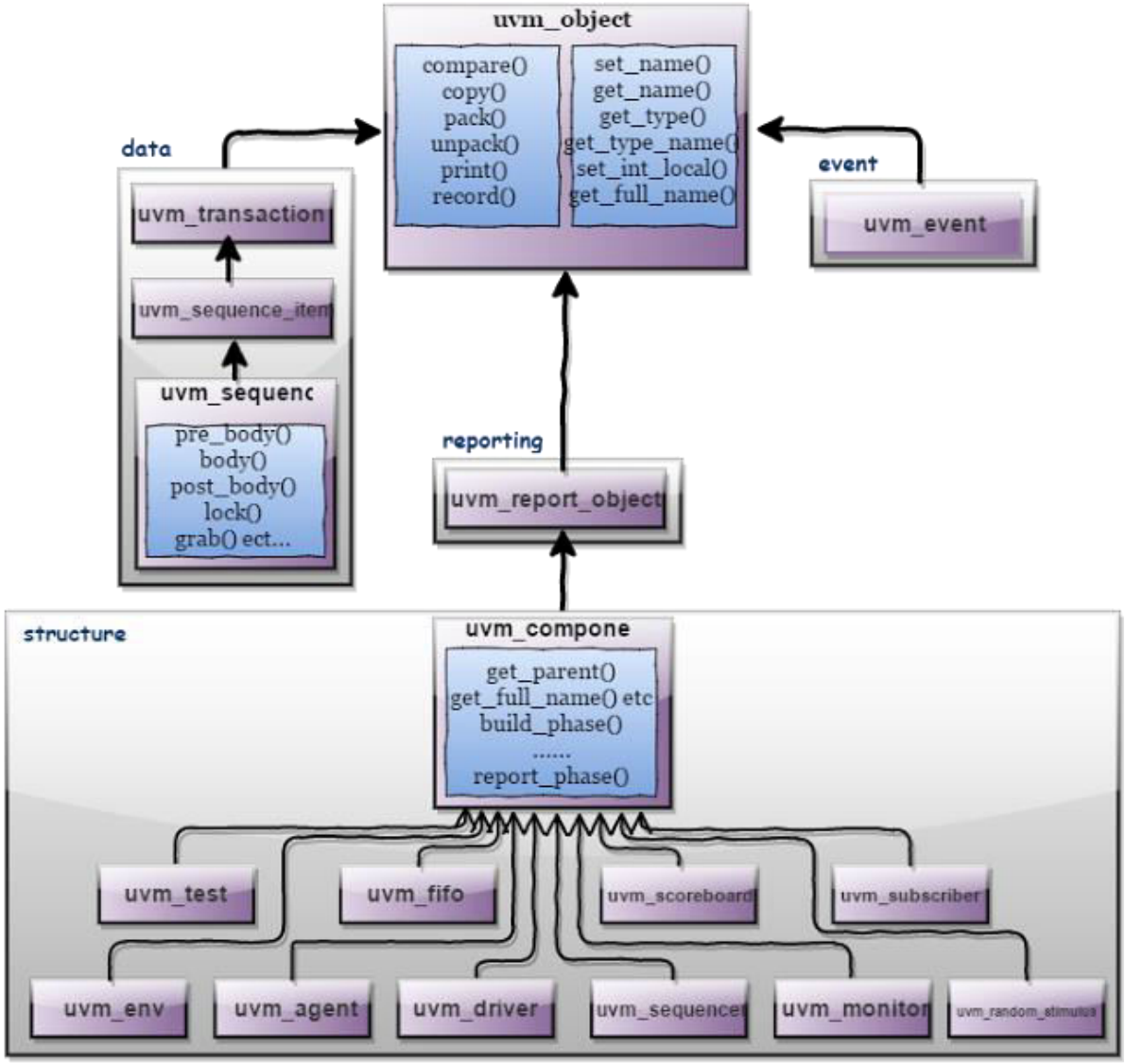
Universal Verification Methodology (UVM)

- uvm_object
  - Klasa bazowa
    - Metody operacyjne (create, copy, clone, compare, print, record, etc..)
    - Polami identyfikujące instancję (name, type name, unique id, etc.)
    - Ziarna losujące
  - Pozostałe klasy bazowe pochodzą od niej
- uvm_transaction
  - Generowanie pobudzeń i analiza
  - Wektory testowe, weryfikacja wyników

Universal Verification Methodology (UVM)

- uvm_component
  - Pseudo-statyczne obiekty istniejące w trakcie całej symulacji
  - Każdy uvm_component jest jednoznacznie adresowalny poprzez ścieżkę w hierarchii
    - np. “env.agent.driver”
  - uvm_component definiuje także przepływ faz testu
    - Każda faza (build, connect, run, …) jest zdefiniowana przez funkcję zwrotną (callback)
  - uvm_component definuje również konfigurację, raportowanie, zapisywanie transakcji i interfejsy wytwarzania obiektów – fabryka (factory)

Universal Verification Methodology (UVM)



Dziękuję za uwagę

- Pytania?